

EXPERT INSIGHT

Building Agent-Powered Applications

Your Guide to Generative AI, RAG, Fine-Tuning, and
Orchestration for Production Use



Vasyl Zvarydchuk, PhD

⟨packt⟩

Building Agent-Powered Applications

Your guide to generative AI, RAG, fine-tuning, and orchestration for production use

Vasyl Zvarydchuk, Ph.D.



Building Agent-Powered Applications

Copyright © 2026 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author nor Packt Publishing, nor its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

This book was written by Vasyl Zvarydchuk, Ph.D. Generative AI tools were used only to assist with ideation, phrasing, and diagram drafts, and all technical content and code were created, verified, and tested by the author and Packt's editorial team. Packt does not accept AI-generated content that replaces expert authorship.

Portfolio Director: Gebin George

Relationship Lead: Ali Abidi

Project Manager: Prajakta Naik

Content Engineer: Aditi Chatterjee

Technical Editor: Rahul Limbachiya

Indexer: Tejal Soni

Production Designer: Alishon Falcon

Growth Lead: Nimisha Dua

First published: April 2026

Production reference: 1270426

Published by Packt Publishing Ltd.

Grosvenor House

11 St Paul's Square

Birmingham

B3 1RB, UK

ISBN 978-1-80760-517-9

www.packtpub.com

Contributors

About the author

Vasyl Zvarydchuk is a principal AI engineer, applied data scientist, and researcher with over 15 years of experience building AI-powered and data-driven systems. With expertise across software engineering, artificial intelligence, machine learning, and data science, he brings together deep research insight and practical engineering experience. He has worked on the design and architecture of large-scale, distributed, and cloud-based systems, helping deliver intelligent solutions with real-world business impact. His experience spans both the theoretical foundations of AI and the practical challenges of building production-ready systems. Vasyl holds a Ph.D. in artificial intelligence, applied mathematics, and computer science.

Vasyl lives with his family in Washington state, where he enjoys hiking, biking, camping, and fishing whenever the Pacific Northwest weather cooperates. This book is his way of sharing what he's learned and helping engineers, data scientists, and business leaders bridge the gap between traditional development and the future of intelligent systems.

This book would not have been possible without the help, encouragement, and support of many incredible people.

First and foremost, I want to thank my family — my amazing wife and our daughter — for their unwavering support, patience, and understanding. Writing this book meant many long evenings, late nights, and stolen weekends, and I couldn't have done it without their love and encouragement every step of the way.

I'm deeply grateful to my friends who were the first reviewers and readers of this book. Your feedback, ideas, and honest critiques were invaluable. A heartfelt thank you to Alex Shyronosov and Alex Manekovskyi for your early reviews, thoughtful insights, and countless helpful suggestions. Your support and contributions played a key role in shaping and improving this book — I truly couldn't have done it without you.

A very special thanks to Egor Pushkin, Chief Architect for Data and AI at Oracle Cloud, for generously sharing his time and expertise in our in-depth conversation featured in this book. His perspectives on the state of AI, the future of large language models, and the emergence of agentic systems added depth, credibility, and inspiration to this work.

Finally, thanks to everyone who supported this journey in ways big and small — mentors, colleagues, fellow engineers, and readers who encouraged me to take on this challenge. This book is as much yours as it is mine.

— Vasyl Zvarydchuk

About the reviewer

Ashok Singamaneni is a principal data engineer and open source innovator recognized for advancing large-scale data platforms, AI-assisted engineering, and distributed systems. He co-created Brickflow and Spark Expectations, two widely adopted frameworks with more than thirteen million downloads, that modernize orchestration and data quality for Lakehouse teams. Ashok has presented at the Databricks AI Summit and has appeared on The Data Engineering Show Podcast. He is known for mentoring engineers, shaping platform strategy, and driving reliable, scalable data systems.

Subscribe for a free ebook

New frameworks, evolving architectures, research drops, production breakdowns – *AI_Distilled* filters the noise into a weekly briefing for engineers and researchers working hands-on with LLMs and generative AI systems. Subscribe now and receive a free eBook, along with weekly insights that help you stay focused and informed.

Subscribe at <https://packt.link/80z6Y> or scan the QR code below.



Table of Contents

Preface	xix
Free benefits with your book.....	xxxi
Part 1: Foundations of AI, NLP, and LLMs	1
Chapter 1: Artificial Intelligence and Natural Language Processing Fundamentals	3
Technical requirements	4
Machine learning basics	4
Training machine learning models • 11	
<i>Supervised, unsupervised, and reinforcement learning</i> • 15	
<i>Neural networks</i> • 18	
<i>Fine-tuning</i> • 22	
Core machine learning problems	23
Regression • 23	
Classification • 23	
Clustering • 25	
Ranking and recommendation • 26	
Core NLP problems and concepts	26
Tokenization • 28	
Sentence-level tokenization • 30	
Stemming, lemmatization, and stopwords • 31	
Embeddings • 34	
<i>Generating embeddings</i> • 40	

<i>Sentence-level embeddings</i> • 41	
Similarity metrics • 43	
<i>Cosine similarity</i> • 43	
<i>Euclidean distance</i> • 45	
<i>Manhattan similarity</i> • 46	
Summary	46
Chapter 2: Understanding Large Language Models	49
Technical requirements	50
Sequence-to-sequence models	50
Encoder • 52	
Decoder • 55	
Transformers • 59	
Train, run, and benchmark LLMs	63
Training objectives • 64	
LLM training datasets • 65	
Benchmark LLMs • 70	
Small language models • 74	
Multimodal models	74
Using LLMs	76
Context window • 76	
Sampling and randomness • 78	
Hallucinations • 81	
LLM families • 83	
<i>GPT models</i> • 83	
<i>Gemini</i> • 86	
<i>LLaMA</i> • 86	
<i>Claude</i> • 87	
<i>Model selection</i> • 87	
Summary	89

Chapter 3: Prompt Engineering	91
Technical requirements	92
Understanding prompt structure	92
System and user prompts • 92	
<i>System prompt structure • 95</i>	
<i>User prompt structure • 97</i>	
Examples and n-shot learning • 97	
<i>Example selection • 100</i>	
<i>Zero-shot prompting • 101</i>	
<i>Static and dynamic few-shot prompting • 102</i>	
Core prompt techniques	109
Chain of thought • 110	
Tree-of-Thought prompting • 111	
Self-consistency prompting • 112	
System role or persona-based prompting • 114	
Other techniques • 115	
Multi-prompt engineering techniques • 116	
<i>Static multi-prompt flows • 119</i>	
<i>Dynamic multi-prompt flows • 119</i>	
<i>Multi-prompt chain of thought • 121</i>	
<i>Iterative self-refinement • 124</i>	
<i>Prompt chaining • 126</i>	
Summary	127
Part 2: Building LLM Applications in Practice	129
Chapter 4: Understanding Language Tasks	131
Technical requirements	132
Text summarization	132
Extractive summarization • 133	
Abstractive summarization • 135	

Summarization parameters • 137	
<i>Summary length</i> • 137	
<i>Maximum tokens</i> • 137	
<i>Randomness settings</i> • 137	
<i>Refocus</i> • 137	
<i>Model selection</i> • 138	
Multimodal summarization • 138	
<i>Text-first multimodal summarization</i> • 139	
<i>Multimodal models</i> • 139	
<i>Summarizing tabular data</i> • 140	
<i>Large document summarization</i> • 141	
Text classification.....	144
Sentiment analysis • 147	
Intent detection • 150	
Information extraction.....	152
Named entity recognition • 153	
<i>Classical NLP pipelines</i> • 154	
<i>Transformer-based fine-tuned models</i> • 155	
<i>Large language model for NER</i> • 156	
<i>Relation and event extraction</i> • 157	
Multilanguage support.....	159
Language detection • 161	
Machine translation • 163	
Summary	165
Chapter 5: Generation, Question Answering, and Reasoning	167
Text generation	168
Text generation parameters • 171	
<i>Instruction and prompt</i> • 171	
<i>Few-shot prompting</i> • 172	
<i>Output length</i> • 173	
<i>Randomness</i> • 173	

<i>Model selection</i> • 174	
<i>Code generation</i> • 174	
<i>Synthetic data generation</i> • 178	
Question answering	179
Closed-domain and open-domain QA • 180	
Multi-hop and conversational QA • 182	
QA for structured data • 184	
Reasoning	189
LLM reasoning • 190	
<i>Commonsense reasoning</i> • 190	
<i>Arithmetic, logical, and symbolic reasoning</i> • 191	
<i>Temporal reasoning</i> • 193	
<i>Causal reasoning</i> • 193	
<i>Analogical reasoning</i> • 194	
<i>Multi-hop reasoning</i> • 195	
Reasoning models • 195	
Unknown reasoning • 197	
Summary	198
 Chapter 6: Retrieval-Augmented Generation	 201
Technical requirements	202
RAG components	202
Static retrieval and parser • 203	
Generator • 205	
Lexical retrieval • 207	
<i>Term frequency and inverse document frequency</i> • 209	
<i>BM25</i> • 213	
<i>Inverted indexes</i> • 215	
Semantic retrieval • 216	
<i>Document preprocessing</i> • 217	
<i>Select document length</i> • 218	
<i>Embedding models</i> • 220	

<i>Choose a similarity metric</i> • 220	
<i>Find the most relevant documents</i> • 221	
Approximate nearest neighbor (ANN) • 222	
<i>Tree-based methods</i> • 223	
<i>Graph-based methods</i> • 223	
<i>Quantization-based methods</i> • 225	
RAG implementations.....	225
Custom pipeline • 226	
Lexical retriever • 226	
Semantic retriever • 228	
Hybrid approach, vector databases • 232	
Vector databases • 233	
RAG as a service • 238	
Summary	239
 Chapter 7: LLM Fine-Tuning	 241
The anatomy of fine-tuning	242
How fine-tuning affects the model • 244	
Full fine-tuning • 246	
Partial fine-tuning • 248	
Parameter-efficient fine-tuning: adapters and LoRA • 250	
Supervised fine-tuning • 252	
Preference-based fine-tuning: RLHF and DPO • 254	
When and how to fine-tune	257
How to fine-tune • 258	
<i>Fine-tuning on local or on-premises hardware</i> • 258	
<i>Fine-tuning in the cloud</i> • 258	
<i>Fine-tuning as a managed feature</i> • 259	
Choosing the right model and strategy • 260	
<i>Select the model family</i> • 261	
<i>Choosing the model size</i> • 262	
<i>Selecting the fine-tuning strategy</i> • 264	

Prompting, RAG, or fine-tuning • 265	
Fine-tuning datasets	269
Hosting fine-tuned models	271
Summary	273
 Part 3: Designing, Building, and Evaluating AI Agents	 275

Chapter 8: Exploring the Architecture of AI Agents	277
---	------------

Technical requirements	278
Agent basics	278
Agent terminology • 278	
LLM agent architecture • 282	
Conversational agents • 283	
Background agents • 284	
LLM agent examples • 285	
<i>Customer support agent • 285</i>	
<i>Lead generation agent • 285</i>	
<i>Personal email filtering agent • 285</i>	
<i>Chat GPT, Gemini Chat, and Microsoft 365 Copilot • 286</i>	
Agent memory.....	286
Short-term memory • 287	
<i>LLM context window and efficiency • 289</i>	
<i>Context relevance • 293</i>	
Long-term memory • 295	
<i>User preferences • 295</i>	
<i>Profile data • 296</i>	
<i>Previous conversations • 297</i>	
Tools and orchestration	298
Agent orchestration • 299	
<i>Orchestration as a sequence of multi-label classification • 302</i>	
<i>Orchestration plan • 303</i>	
<i>Multi-plan orchestration • 304</i>	

<i>Dynamic orchestration plans</i> • 305	
Orchestration challenges • 308	
Multi-agent flows	311
Code-level multi-agent flows • 312	
Multi-agent communication patterns • 316	
<i>Worker and supervisor</i> • 317	
<i>Sequential chat</i> • 317	
<i>Roundtable discussion</i> • 318	
<i>Hierarchical task delegation model</i> • 318	
<i>Competitive agent model</i> • 318	
Agent goals and control mechanisms • 319	
Summary	320
 Chapter 9: Building AI Agents	 323

Technical requirements	324
Agent design	324
Functional requirements • 325	
System requirements • 326	
Interaction mode • 327	
Implementation approach • 327	
Skills, tools, and capabilities • 328	
Memory, state, and learning • 329	
Reasoning and control mechanisms • 329	
Safety and control boundaries • 330	
Evaluation and success metrics • 330	
Deployment and operations • 331	
Agent memory implementation	331
Custom memory development • 331	
Memory tools and frameworks • 333	
<i>MemO</i> • 333	
Agent platforms memory implementation • 335	
Memory-augmented learning • 335	

Tools and orchestration	337
Custom tools orchestration • 337	
Function calling frameworks • 338	
<i>OpenAI function calling framework • 339</i>	
<i>LangChain function calling • 341</i>	
Tools orchestration in an agentic platform • 342	
Model context protocol • 343	
Agent decision-making and planning.....	345
Agent reasoning techniques • 346	
Planning strategies • 346	
<i>Sequential decision-making • 347</i>	
<i>Hierarchical task decomposition • 347</i>	
<i>Skill chaining • 348</i>	
Error handling and recovery strategies • 349	
Human-in-the-loop • 350	
Multi-agent frameworks and protocols.....	351
AutoGen framework (Microsoft Agent Framework) • 351	
Agent-to-agent protocol • 355	
Implementing AI agents in practice.....	359
Email filtering agent – requirements • 359	
<i>Functional requirements • 359</i>	
<i>System requirements • 360</i>	
<i>Interaction mode • 360</i>	
<i>Memory, state, and learning • 361</i>	
<i>Reasoning and control mechanisms • 362</i>	
<i>Safety and control boundaries • 362</i>	
<i>Evaluation and success metrics • 363</i>	
<i>Deployment and operations • 363</i>	
Email filtering agent – design • 364	
Email filtering agent – implementation • 365	
Summary	367

Chapter 10: Evaluating LLM Applications and Agents	369
Technical requirements	370
Evaluation challenges.....	370
Evaluation strategy • 371	
<i>Multi-task AI agents' evaluation • 375</i>	
Evaluation datasets	376
Public datasets • 378	
<i>Building custom evaluation datasets • 378</i>	
Synthetic and human-generated datasets • 381	
Evaluation metrics	383
Closed-ended task evaluation • 383	
<i>Accuracy and exact match • 384</i>	
<i>Precision, recall, and F1 for binary classification • 385</i>	
<i>Multi-class classification metrics • 387</i>	
Similarity metrics for open-ended tasks • 387	
<i>Lexical similarity-based metrics • 388</i>	
<i>Semantic similarity-based metrics • 390</i>	
Human as a judge • 392	
LLM as a judge • 398	
Online vs. offline evaluations • 400	
Agent evaluation	403
RAG-based agent evaluation • 403	
<i>Retriever evaluation • 404</i>	
<i>Generator evaluation • 405</i>	
Conversational agent evaluation • 406	
<i>Agent memory evaluation • 406</i>	
<i>Tool orchestration evaluation • 408</i>	
Background agent evaluation • 411	
Responsible AI.....	411
Responsible AI risks and evaluation criteria • 413	
Evaluation datasets for safety, bias, and fairness • 414	

Jailbreak and adversarial prompt evaluation • 416	
Evaluation tools and frameworks	417
Summary	420
Chapter 11: Unlock Your Exclusive Benefits	423
Unlock this Book's Free Benefits in 3 Easy Steps.....	424
Appendix 1: Expert Insights	427
Appendix 2: Bibliography	433
Other Books You May Enjoy	442
Index	445

Preface

Throughout history, **artificial intelligence (AI)** has evolved in waves, each marked by major breakthroughs. From early theoretical concepts to the rise of **deep learning (DL)**, each wave has pushed the boundaries of what machines can do. But none have been as transformative, fast-moving, or democratizing as the current era, powered by **large language models (LLMs)**.

Each new wave of AI has arrived faster than the previous one and with greater impact [1]. In the past, it took decades to move from theoretical concepts and basic implementations in the 1950s-1960s to the rise of **machine learning (ML)** in the 1990s, and another 20 years before deep learning gained traction. But the leap from deep learning to LLMs happened in just several years, reshaping entire industries almost overnight. As breakthroughs accelerate, the time between innovation and real-world adoption continues to shrink, and we are now experiencing the fastest AI wave yet.

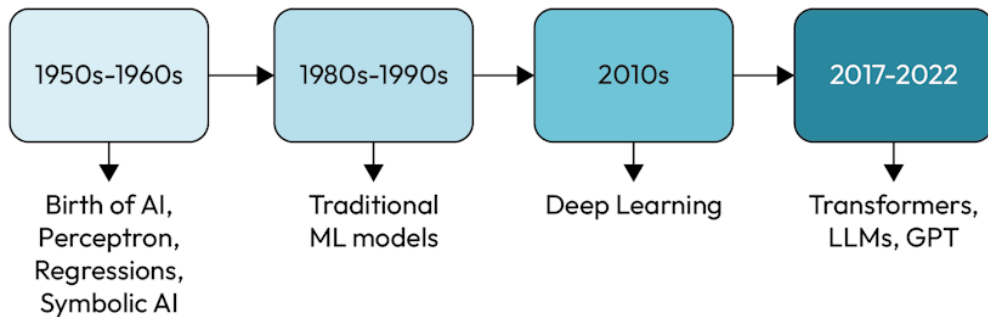


Figure 1: AI genealogy: from the early days of AI to the rise of LLMs

With the release of models like GPT-3, and the viral success of ChatGPT (chatgpt.com), AI made a dramatic leap, from solving narrow, task-specific problems to becoming a general-purpose tool capable of solving many different tasks across multiple domains. For the first time, software engineers and applied data scientists can build powerful, human-like AI systems without needing years of specialized AI experience.

It's also worth highlighting that LLMs have fundamentally changed the kind of AI applications we can build. Instead of training and maintaining a collection of separate, specialized models for each problem, we can now build intelligent systems that handle many tasks at once, all powered by a single LLM. That one model can understand language, generate content, answer questions, support decision-making, and even automate multi-step workflows.

This is not just a technological upgrade we had before, like moving from desktop applications to the cloud and mobile phones. It's a step change that opens entirely new possibilities for businesses. These possibilities range from building really human-like conversational chatbots to deploying fully autonomous AI agents. Understanding these capabilities is critical not only for technical teams but also for business leaders seeking to innovate and stay competitive.

This new era has given rise to what we now call generative AI or gen AI, a class of AI systems that can generate and understand text, code, images, and even audio and video. Unlike traditional AI, which focuses mostly on classical AI problems like classification and regression, generative AI is designed to generate content, perform reasoning, and interact in human-like ways. What once required specialized models for each task can now often be achieved with a single, well-prompted LLM, radically simplifying the development of intelligent systems.

The LLM revolution also began to reshape the roles of software engineers, applied data scientists, and ML professionals. For decades, software engineers and data scientists have worked in separate lanes, one focused on building applications, the other on developing AI models and research. But with the arrival of powerful LLMs and natural language (NL) interfaces, the line between these roles started to blur.

Powerful LLMs, combined with developer-friendly APIs, a growing ecosystem of open-source language models, modern AI frameworks, and no/low-code AI platforms, have empowered software engineers to build intelligent AI applications without needing a deep background in machine learning, AI, or **natural language processing (NLP)**. As a result, this shift has made it dramatically easier for developers to enter the AI space and contribute to modern innovation. In a September 2022 interview, Sam Altman, CEO of OpenAI, said that the biggest opportunity for the vast majority of people will be to adapt LLM for specific applications [2].

Even with all this excitement, many developers find themselves wondering how to put these tools into practice. Software engineers still face a knowledge gap when it comes to using LLMs effectively. Often this knowledge gap is not recognized because, as the saying goes, "we don't know what we don't know." Closing that gap requires learning new skills, from understanding basic NLP and AI concepts to grasping how LLMs work, designing effective prompts, structuring AI workflows, fine-tuning models, building AI agents, and evaluating their performance. These skills go beyond traditional software engineering, and that's what we will cover in this book.

The role of applied data scientists and machine learning engineers has also evolved significantly. While deep ML expertise remains important, especially for developing, fine-tuning, and supporting specialized models, LLMs have changed the way AI solutions are built and delivered. Many tasks that once required training models from scratch can now be tackled using pre-trained language models and human language instructions called prompts.

ML professionals are no longer limited to working solely with traditional AI model development. With LLMs, they can now build more dynamic, human-like interfaces that understand context, generate language, and reason across unstructured data, capabilities that were previously very hard to reach without years of NLP and AI specialization. This shift opens the door to entirely new categories of AI applications like autonomous agents and human-like chatbots.

Meanwhile, the foundational knowledge of data science and ML is more important than ever. Understanding model behavior, data quality, and evaluation techniques continues to be critical, especially as LLMs become embedded in real-world systems. Rather than replacing traditional ML work, LLMs extend what's possible, giving data scientists and ML engineers a more powerful toolkit than ever before.

The shift is already reshaping industries worldwide. In the fourth quarter of 2024, 241 S&P 500 companies mentioned "AI" in their earnings calls — the highest number in over a decade and more than double the five-year average. This surge underscores a significant shift, as businesses increasingly recognize AI's potential to drive innovation and maintain competitive advantage [3], [4].

This massive technological movement has also been seen in the explosive growth of open-source projects related to LLMs. Three AI engineering frameworks and platforms, such as AutoGPT, Stable Diffusion Web UI, and LangChain, have already surpassed the GitHub star count of the Bitcoin repository. They are now on track to rival, and potentially surpass, even the most popular web development frameworks like React and Vue.

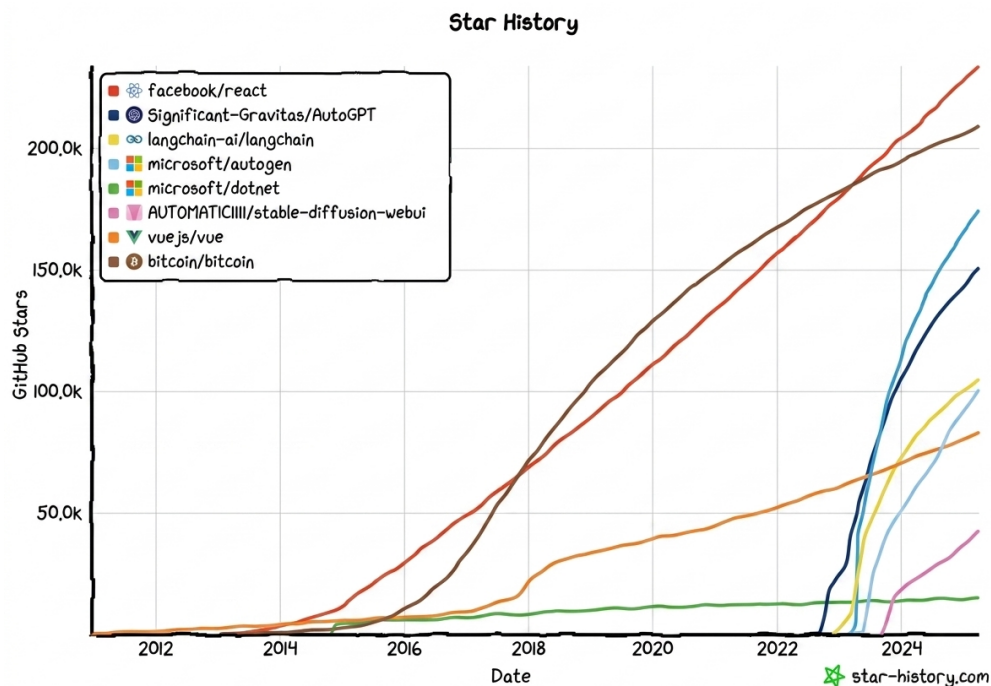


Figure 2: Growth of open-source AI frameworks compared to other popular GitHub projects (Data source: star-history.com)

So, we're standing at an inflection point. While AI has been shaping real-world products and industries for years already, the emergence of LLMs marks a new phase, one where natural language becomes the interface and building intelligent systems is no longer reserved for highly specialized professionals.

This book is your invitation to be part of that shift. Together, we'll explore how to harness the power of LLMs, design effective prompts, fine-tune, build intelligent systems, and create AI agents that matter so you can lead in this new era of AI.

What this book is about

To explain what this book is about, let me share two real-world stories.

The first story is about a software engineering team that set out to solve a complex problem using an LLM-based technology. The team did an excellent job scoping the problem, selecting tools, and trying to build a solution. But after several months of work, they hit a wall. Despite their engineering talent, they couldn't achieve the required level of quality and accuracy. With little or no background in AI, they approached the challenge as experienced developers would, but they missed one critical piece: a deeper understanding of the possibilities and limitations of the AI technology they had chosen.

The second story comes from a team of experienced data scientists, many with PhDs and deep AI expertise. They had built and fine-tuned dozens of custom models based on BERT, RoBERTa, and similar architectures to solve tasks like classification, recommendation, and summarization. This required large datasets, expensive training, and significant maintenance effort. The project started before GPT-3.5 but continued even after GPT-4 and GPT-4 Turbo were available. Despite mounting quality and maintenance challenges, the team failed to recognize the massive shift in technology. Eventually, another team replaced most of those custom models with LLM-based solutions, achieving faster development, lower costs, and even better accuracy in some cases.

These stories reflect a broader reality: even highly skilled software engineers and data scientists can struggle to navigate this new era of AI without a solid foundation in how modern LLMs work, how to use them effectively, and how to rethink the way AI solutions are built.

Trying to piece together this knowledge from blog posts, documentation, YouTube videos, web searches, one-off LLM queries, and trial and error often leads to a fragmented and incomplete understanding. These gaps can slow teams down, increase costs, and reduce the quality of solutions, at a time when AI is no longer a feature but a foundational layer of most modern software products.

This book is designed to bridge that knowledge gap. It explores how large language models are reshaping the development of AI applications, from natural language interfaces and prompt engineering to advanced techniques like **retrieval-augmented generation (RAG)**, LLM fine-tuning, and AI agents.

You will find hundreds of code examples, prompts, and schemas that show how to solve modern AI problems and build AI agents. But more importantly, this book gives you a deeper, holistic understanding of the foundations behind LLMs and how to apply them effectively in real-world systems. We believe this is the most important goal of this book and of any other foundational work in the AI space.

While countless tutorials and quick guides exist online for tackling individual tasks, very few offer a structured, end-to-end framework for understanding how LLMs truly work – how to design prompts thoughtfully, fine-tune LLMs, build robust systems, and evaluate them rigorously. That is the gap this book aims to fill.

Finally, this book examines how the AI development process itself has changed. With the rise of general-purpose models, building intelligent systems no longer requires training models from scratch or maintaining complex ML infrastructure. Instead, the focus shifts to understanding model behavior, designing effective interactions, fine-tuning existing models, and integrating AI into real-world business processes.

By the end of this book, you will not just know how to call an API; in fact, you will understand how to think about modern AI, design systems that take advantage of it, and deliver solutions that matter.

What this book doesn't cover

While this book covers the most important foundations for building modern AI applications and agents along with practical guides, code examples, prompts, and workflows, it isn't meant to be exhaustive in every area. The primary goal is to give you a strong, holistic understanding of how AI application development has changed in the era of LLMs, not to dive into every low-level technical detail.

This book does not go deep into the internals of LLM architecture or the fine details of how these models are trained and optimized. Instead, it focuses on the mental models and practical knowledge you need to use LLMs effectively in real-world applications.

While we explore a wide range of prompt engineering strategies and AI techniques, we do not attempt to catalog every possible approach – new patterns and research emerge almost weekly. Instead, you'll find proven patterns and examples that offer a strong foundation and give you the confidence to experiment and continue learning as the field evolves.

Finally, this book is not a complete reference for every framework, platform, and tool in the LLM ecosystem. Many are covered, but the intent is to provide a broad understanding of how they fit together, not to replace official documentation or full-length platform tutorials.

Think of this book as a guide that equips you to navigate the LLM landscape, speak the language of modern AI development, and confidently explore deeper topics and tools when you need them.

Who this book is for

This book is for software engineers, data scientists, applied AI practitioners, technical leads, and engineering-focused product managers who want to build production-ready applications with LLMs and AI agents. It suits readers moving from traditional software development or classical machine learning into generative AI systems. You should be comfortable with programming in Python or a similar language and understand core software engineering concepts such as APIs, data structures, and integration. Prior deep learning or LLM training experience is not required.

What this book covers

Chapter 1, Artificial Intelligence and Natural Language Processing Fundamentals, introduces the foundational concepts of AI and NLP, providing a practical overview of basics like neural networks, core ML and NLP problems, tokenization, embeddings, similarity metrics, and how information is encoded into modern AI models. It also touches on the basics of model training and fine-tuning to ground readers in the broader AI landscape. This chapter is particularly helpful for software engineers new to these concepts, while data scientists can use it as a quick refresher or skip ahead if already familiar.

Chapter 2, Understanding Large Language Models, explores the core concepts behind large language models. It walks through earlier model architectures, Seq2Seq, **recurrent neural networks (RNNs)**, and encoder-decoder models before introducing transformer architectures. The chapter explains how LLMs are designed, trained, and configured, giving both engineers and data scientists a strong foundation for understanding how LLMs work under the hood and how to use them effectively.

Chapter 3, Prompt Engineering, introduces both basic and advanced techniques for designing effective prompts. It covers prompt structure, including instructions, hints, examples, and context, and how each impacts model behavior. It then explores few-shot learning, **chain-of-thought (CoT)**, **tree-of-thought (ToT)**, self-consistency, persona-based prompting, and advanced multi-prompt strategies such as multi-step CoT and refinement loops.

Chapter 4, Understanding Language Tasks, explores core NLP capabilities of LLMs for understanding and transforming text. It covers summarization techniques, text classification, and information extraction tasks such as named entity recognition and relation extraction. The chapter also introduces multilingual capabilities, including translation, showing how LLMs can operate across different languages and contexts.

Chapter 5, Generation, Question Answering, and Reasoning, focuses on the generative and problem-solving capabilities of LLMs. It covers text generation and content creation, followed by **question answering (QA)**, including natural language to SQL (NL2SQL) for querying structured data. The chapter concludes with reasoning tasks, where LLMs perform multi-step thinking and logical inference, setting the stage for building more advanced AI systems and agents.

Chapter 6, Retrieval-Augmented Generation, introduces one of the most powerful techniques for enhancing LLMs with external knowledge. It covers lexical retrieval (BoW, TF-IDF, BM25) and then moves into semantic retrieval using embeddings, similarity metrics, and ANN search algorithms. You'll also explore popular libraries, frameworks, and vector databases for building real-world RAG pipelines.

Chapter 7, LLM Fine-Tuning, is the shortest chapter, serving as an introduction to fine-tuning. It explains what fine-tuning is, when it's appropriate, and how it compares to prompt engineering and RAG. Clear recommendations are provided for when fine-tuning is the right choice.

Chapter 8, Exploring the Architecture of AI Agents, brings together the core ideas of the book through fundamental concepts and the architecture of modern AI agents. It begins by defining what AI agents are, their types, and essential components. The chapter then dives deeper into core structural elements, such as memory, tool usage, planning mechanisms, reasoning loops, orchestration patterns, and multi-agent collaboration. By the end of the chapter, you will understand how agents are designed conceptually and architecturally, and how large language models evolve from simple text generators into goal-driven systems.

Chapter 9, Building AI Agents, is the practical culmination of the book. It takes everything we have learned, LLMs, prompting, retrieval, reasoning, tools, evaluation, and architecture, and turns it into working systems. This chapter walks step by step through designing and implementing real AI agents. You will learn how to define agent goals, select and integrate tools, implement memory strategies, structure control flows, and manage execution loops.

Chapter 10, Evaluating LLM Applications and Agents, is the final, yet foundational, chapter of the book, focusing on measuring and validating AI agent performance. It introduces metrics for closed- and open-ended tasks, covers methodologies for evaluating complex LLM applications and agents (reasoning, memory, tool orchestration), and addresses safety, fairness, and alignment. You'll learn strategies for mitigating hallucinations, reducing bias, and building trust in production systems.

How chapters fit together

While this book is designed to be read in order, you don't have to read it linearly to get value. Each chapter builds on the concepts introduced earlier, but they are also written to stand on their own, allowing you to jump to the topics most relevant to you.

The following diagram shows how the chapters relate to each other:

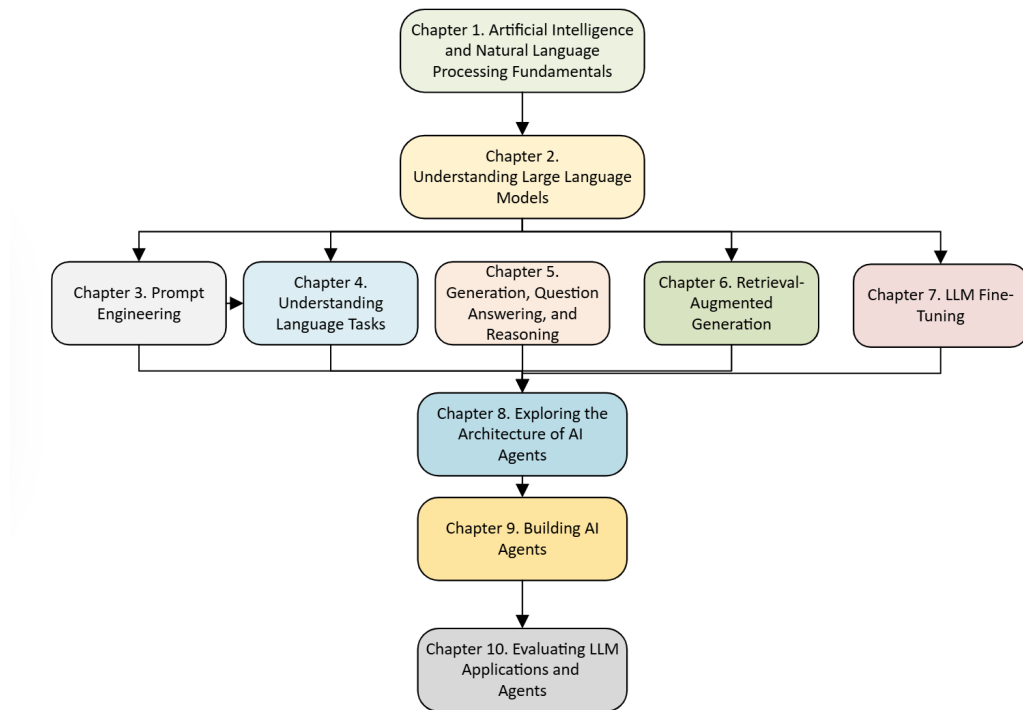


Figure 3: Roadmap of chapters and their connections

Use the preceding map as a guide when navigating the book, whether you're taking a deep dive into one topic or exploring how the pieces fit together to build end-to-end AI applications.

To get the most out of this book

This book is designed to be hands-on and implementation-focused, so having the right background and environment will help you get the most out of each chapter:

- Proficiency in Python, along with some familiarity with SQL and REST APIs, is helpful. Experience with Jupyter notebook–based workflows will also speed up experimentation and prototyping.
- Access to LLMs such as OpenAI GPT or Anthropic Claude is required. You will need an API key to run the examples and execute prompts in this book.

This book assumes familiarity with basic mathematical concepts. Understanding vectors, functions, equations, and matrices will be helpful when we discuss embeddings, similarity metrics, neural networks, and RAG. While not required, some prior exposure to AI and machine learning fundamentals will help you get the most out of this book.

Software/hardware covered in the book	Operating system requirements
Python 3.x	Windows, macOS, or Linux

You will need access to a Python development environment. A machine with at least 8 GB of RAM is recommended for running the more complex examples.

If you are using the digital version of this book, we advise you to type the code yourself or access the code from the book's GitHub repository (a link is available in the next section). Doing so will help you avoid any potential errors related to copying and pasting code.

Download the example code files

The code bundle for the book is hosted on GitHub at <https://github.com/PacktPublishing/Building-Agent-Powered-Applications>. We also have other code bundles from our rich catalog of books and videos available at <https://github.com/PacktPublishing>. Check them out!

Terminology, index, and references

Throughout the book, key terms and abbreviations are defined when they first appear. For example, you'll see **large language models (LLMs)** spelled out the first time, then used in their abbreviated form in the rest of the chapter (and repeated again in later chapters if needed). While we aim to use abbreviations consistently, we intentionally reintroduce some key terms

throughout the book to make the text more readable and accessible, especially for readers who may be jumping between chapters.

If you ever lose track of a term or concept, you can use the index at the end of the book to quickly find where it's discussed. We've included all important terms, concepts, and techniques to make navigation easier, so you can treat the book not just as a cover-to-cover read, but as a reference manual when you need it.

This book is built on a wide range of sources: research papers, books, industry reports, blog posts, YouTube videos, and personal experimentation. Because it would be distracting to cite every influence on every page, we've focused on referencing direct quotes, key statistics, and important external reports. A complete list of sources is included in the Bibliography section at the end of the book for readers who want to explore the original material in more depth.

Disclaimer on images

Some images in this title are presented for contextual purposes, and the readability of the graphic is not crucial to the discussion. Please refer to our free graphic bundle to download the images.

Download the color images

We also provide a PDF file that has color images of the screenshots/diagrams used in this book. You can download it here: <https://packt.link/gbp/9781807605179>.

Conventions used

There are a number of text conventions used throughout this book.

CodeInText: Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles. For example: "Vector similarity can be calculated by using the numpy library in Python:"

A block of code is set as follows:

```
# Training Loop
for epoch in range(num_epochs):
    optimizer.zero_grad()
    output = model(x_train)
    loss = criterion(output, y_train)
    loss.backward()
    optimizer.step()
```

Any command-line input or output is written as follows:

```
Stop words in the text:
- This
- is
- a
- that
- some
```

Bold: Indicates a new term, an important word, or words that you see on the screen. For instance, words in menus or dialog boxes appear in the text like this. For example: "**Euclidean distance**, known sometimes as **L2 distance**, calculates the straight-line distance between two points in space."

Note

Warnings or important notes appear like this.

Tip

Tips and tricks appear like this.

Get in touch

Feedback from our readers is always welcome.

General feedback: If you have questions about any aspect of this book or have any general feedback, please email us at customer@packt.com and mention the book's title in the subject of your message.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you reported this to us. Please visit <http://www.packt.com/submit-errata>, click **Submit Errata**, and fill in the form.

Piracy: If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at copyright@packt.com with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit <http://authors.packt.com/>.

Free benefits with your book

This book comes with free benefits to support your learning. Activate them now for instant access (see the "How to Unlock" section for instructions).

Here's a quick overview of what you can instantly unlock with your purchase:



DRM-Free PDF Version

Download DRM-free PDF and ePub copies of this book.



7-Day Packt Library Access

Get 7-day unlimited access to 8,000+ books and videos. No credit card required.

Available for first-time Packt+ trial users only.



Next-Gen Reader Access

Read this book on Packt Reader with progress sync, dark mode and note-taking.

How to unlock

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Keep your invoice handy. Purchases made directly from Packt don't require one.

Share your thoughts

Once you've read *Building Agent-Powered Applications*, we'd love to hear your thoughts! Scan the QR code below to go straight to the Amazon review page for this book and share your feedback.



<https://packt.link/r/1807605167>

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Join our Discord and Reddit Space

You're not the only one navigating fragmented tools, constant updates, and unclear best practices. Join a growing community of professionals exchanging insights that don't make it into the documentation.

Join our Discord at

<https://packt.link/z8ivB>

or scan the QR code below:



Follow us on Reddit at

<https://packt.link/0rExL>

or scan the QR code below:



Part 1

Foundations of AI, NLP, and LLMs

Part 1 builds the conceptual foundation for the rest of the book. It introduces the core ideas behind **artificial intelligence (AI)** and **natural language processing (NLP)**, then explains how modern **large language models (LLMs)** work and are powerful. You will learn the essential concepts needed to work effectively with LLMs, from tokens, embeddings, and transformers to inference, prompting, and model behavior in practice. The part concludes with prompt engineering techniques that show how to communicate with LLMs more reliably and steer them toward useful outputs.

This part contains the following chapters:

- *Chapter 1, Artificial Intelligence and Natural Language Processing Fundamentals*
- *Chapter 2, Understanding Large Language Models*
- *Chapter 3, Prompt Engineering*

1

Artificial Intelligence and Natural Language Processing Fundamentals

As we mentioned in the Preface, this book takes you on a journey, from understanding **large language models (LLMs)** to designing and building full-fledged AI agents. These agents are like software robots: they communicate, reason, make decisions, and perform actions. What makes them special is that they primarily interact through language.

The need to understand and generate human language gave rise to the field of **natural language processing (NLP)**. LLMs have transformed NLP and AI dramatically, enabling machines to understand and generate language with near-human-level quality. To understand how LLMs work and how to use them effectively, we first need to cover some fundamental concepts from AI and NLP.

This chapter lays the foundation for the rest of the book. The goal isn't theory for its own sake, but to give you the mental toolkit needed to understand how LLMs operate and how to use them effectively.

If you are already familiar with these fundamentals, feel free to skim or skip this chapter and return later as a refresher. But if you're a software engineer or product builder without a background in AI, this chapter will provide the vocabulary and intuition needed before we dive into building intelligent systems that actually perform useful tasks.

The following key topics are covered:

- Machine learning basics
- Core machine learning problems
- Core NLP problems and concepts
- Embeddings
- Similarity metrics

Note

Your purchase includes a free PDF copy + exclusive extras

Your purchase includes a DRM-free PDF copy of this book, a 7-day trial to the Packt+ library (no credit card required), and additional exclusive extras. See the *Free benefits with your book* section in the *Preface* to unlock them instantly and maximize your learning.

Technical requirements

The code for this chapter is available on the book's accompanying GitHub repo: <https://github.com/PacktPublishing/Building-Agent-Powered-Applications/tree/main/Chapter1>

Machine learning basics

In the early days of NLP, engineers tried to model and understand language with classical, rule-based software. They wrote programs that explicitly encoded grammar rules, dictionaries, and decision trees, attempting to capture every possible case in code. But human language is too rich and too flexible for this approach. There are simply too many variations, exceptions, and edge cases for a rules-only system to handle reliably.

The same challenge appeared in other fields, like computer vision. Engineers quickly realized that writing rules for every possible way a cat could look in a picture, or every way a sentence could be phrased, was impractical and brittle. A single misspelling, a slang phrase, or a new object in an image would break the system.

This led researchers to a new paradigm: **machine learning (ML)**. Instead of manually defining all the rules, they built algorithms that learn patterns automatically from data. In other words, rather than telling the computer exactly how to recognize a cat or parse a sentence, we feed a model many examples of what we want, and the model learns its own internal rules for mapping inputs to outputs.

There are many ways to define what an AI or *machine learning model* is. For the purposes of this book, we can think of a model as some code or an algorithm that takes input data, processes it, and produces some kind of output. In the simplest terms, it's a function that maps inputs to outputs as shown in the following figure:

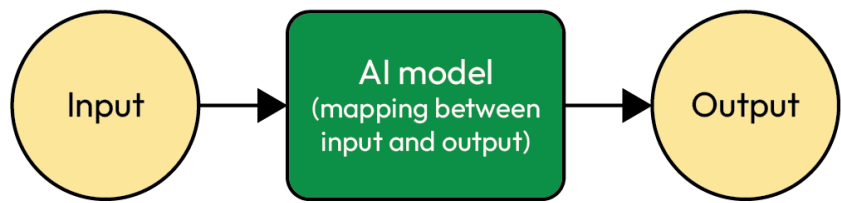


Figure 1.1: Input-output flow in a machine learning model

The following table illustrates some typical input/output pairs across various AI tasks, along with brief descriptions of each problem:

Problem type	Input	Output	Mapping
Sentiment analysis of social media posts	Social media post	Category: Positive/ Negative	text → category
Text summarization	News article	Short summary	text → text
Machine translation	Sentence in English	Sentence in French	text → text
Image capture	Image	Image description	image → text
Spam detection	Email body	Category: Spam/Not Spam	text → Boolean value
Speech-to-text recognition	Audio file (voice)	Text transcript	audio → text
Image generation	Image description	Image	text → image
Job candidate scoring and ranking	Job resume	Predicted job fit score	text → number

Table 1.1: Example AI tasks with their inputs, outputs, and mappings

These examples illustrate just how many different combinations of inputs and outputs AI models can handle. LLMs typically take text as input and generate text as output. However, many modern LLMs, or more broadly, *foundation models*, are capable of working with more than just text. When a model can process and generate content across multiple formats, such as text, images, and audio, it is known as a **multimodal model**. We will discuss multimodal models later in *Chapter 2*.

So, we can think of an AI or machine learning model as some function that maps one type of information to another.

Note

In this book, we'll use the term "function" in its mathematical and software-engineering sense: something that takes an input, applies a set of rules, and produces an output. For example, a simple mathematical function might be $y = 2x + 3$, which doubles its input and adds three. In software, a function works the same way — for instance, Python's `strip()` function takes a string as input and returns a new string with leading and trailing whitespace removed. You can think of an AI model as a much more complex function.

To make that idea more concrete, let's consider a simple example where we want to predict the price of a house based on a few factors like home square footage, number of bedrooms, location, and so on. For that, we can express our AI model as a mathematical function:

$$y = f(x)$$

where y is the predicted home price and x denotes house characteristics (square footage, location, etc.).

Now, let's try to figure out what the function f may look like. To start simple, imagine our "model" (function) just returns the average home price for every house:

$$y = w_0$$

where w_0 is the average home price (our baseline). That's a start, but clearly not very useful, as every house would get the same predicted price.

From common sense, we know larger houses usually cost more. So, let's add square footage as an input feature. For that, let's define a new variable x_1 that represents how much larger or smaller a house is compared to the average house size:

- $x_1 = 0$ if the house has an average size (e.g., 2000 sq ft)
- $x_1 < 0$ for smaller-than-average houses
- $x_1 > 0$ for larger-than-average houses

For instance, if an average house footage is 2000 square feet (or approximately 186 m²), then $x_1 = -500$ for a house with 1500 square feet (1500 – 2000), and $x_1 = 500$ for a house with 2500 square feet (2500 – 2000).

Next, let's introduce a *parameter* or *coefficient*, w_1 , that tells us how much the price changes per every square foot (unit change). Now our machine learning model has become the following:

$$y = w_0 + w_1 \cdot x_1$$

where:

- y – the predicted house price
- w_0 – the average home price
- w_1 – the price change for each square foot above or below the average
- x_1 – the difference in square footage from the average

The value w_0 is also called **bias** because it allows the model to shift the entire prediction up or down. In our case, the bias represents the average home price, serving as a baseline. The predicted prices for other homes are positioned relative to this value: some will be higher, others lower, depending on their features.

Let's plug in some real numbers. Suppose the average house in the U.S. is 2000 sq ft (about 185 m²) and costs \$400K. We also note that houses around 2500 sq ft (232 m²) will typically cost \$500K. So, a 500 sq ft increase leads to a \$100K increase in price. That means that 1 square foot for home price will cost \$200:

$$\frac{400000}{2000} = 200$$

and

$$\frac{500000}{2500} = 200$$

Now, we have got two examples for houses with 2000 and 2500 square feet. They form our very *training dataset*:

Square footage	Price
2000	400,000
2500	500,000

Table 1.2: Our training dataset

As we defined w_1 above, it will be exactly the price of one square foot and will be equal to 200, meaning every new one square foot will add \$200 to the home price. So, now our model will look like this:

$$y = 400000 + 200 \cdot x_1$$

And if we apply this to a 1500 sq ft house, where $x_1 = -500$ (see above for why), we will get:

$$y = 400000 + 200 \cdot (-500) = 300000$$

So, in our case, a house with 1,500 square feet is predicted to cost \$300K.

Of course, square footage isn't the only thing that matters. The number of bedrooms, lot size, and location also play a big role. To capture all of them, we expand our model as follows:

$$y = w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + \cdots + w_n \cdot x_n$$

where $x_1, \dots, x_n$ are all factors (such as location, number of bedrooms and bathrooms, lot size, etc) which contribute to home prices and $w_1, \dots, w_n$ represent the contribution levels (*weights*) of these factors. Variables $x_1, \dots, x_n$ are also called **model features**.

It's important to note that in this type of modeling, we're assuming that each feature contributes proportionally and independently to the final house price. In reality, that's often not the case. For example, the price difference between a 3-bedroom and a 4-bedroom house might be significant, while the difference between a 5-bedroom and a 6-bedroom home could be negligible. After a certain point, some features may stop adding value or their impact may taper off.

The values $w_1, \dots, w_n$ are also known as **weights** or **model parameters**. These parameters determine how much influence each input factor or feature has on the final output. The more complex the model, the more parameters it typically has. Modern models like LLMs can have billions or even trillions of parameters.

The type and structure of the function f determines the *model architecture*, i.e., essentially, how the model processes inputs to generate outputs. In the example above, we looked at one of the simplest forms of machine learning models: *linear regression*. It's called *linear* because the model is defined by a linear equation, and it's called *regression* because it belongs to a class of AI problems where the output is a numeric value.

In the example above, the house price prediction model takes several input features and produces a single output: the predicted price. However, in many real-world applications, the output can be a set of values rather than just one number. For example, in a medical diagnosis model, the input might be patient data, while the outputs could include blood sugar level, cholesterol level, and risk score for heart disease, among others.

So, in general, when we have several outputs, we can define a linear regression as a set of linear equations and not a single equation:

$$y_1 = w_{10} + w_{11} \cdot x_1 + w_{12} \cdot x_2 + \dots + w_{1n} \cdot x_n$$

$$y_2 = w_{20} + w_{21} \cdot x_1 + w_{22} \cdot x_2 + \dots + w_{2n} \cdot x_n$$

....

$$y_m = w_{m0} + w_{m1} \cdot x_1 + w_{m2} \cdot x_2 + \dots + w_{mn} \cdot x_n$$

where $y_1, y_2, \dots, y_m$ is the list of output features, and $w_{10}, \dots, w_{mn}$ are the weights.

The equation above can be written in matrix form (the same equation, but in a more compact form):

$$Y = W * X$$

Where the output vector is,

$$Y = [y_1, y_2, \dots, y_m]$$

The input vector is,

$$X = [x_1, \dots, x_n]$$

And this represent the weights matrix,

$$W = \begin{bmatrix} w_{10} & \dots & w_{1n} \\ \vdots & \ddots & \vdots \\ w_{m0} & \dots & w_{mn} \end{bmatrix}$$

Tip

Don't worry if all these equations and matrices feel intimidating, especially if it's been a while since your school, college, or university math classes. You don't need to be a math expert to follow along. If you're curious, try writing out a few of these equations and doing the calculations by hand; it's a great way to build intuition. But if not, just focus on the big idea: we're mapping inputs to outputs, and math is just one way to express that relationship.

So, in general, building (or training) a machine learning model means finding a weight matrix W that best maps the inputs X (input vector) to the expected outputs (output vector). Finding W typically requires performing a large number of mathematical operations, such as matrix multiplication and addition.

While linear regression is still used for practical tasks and is excellent for learning foundational concepts, modern AI systems rely on far more sophisticated architectures. These include deep neural networks and advanced models like transformers, which power today's large language models such as GPT or Claude. We will cover these advanced models more in *Chapter 2*.

The following table highlights some of the most widely used model architectures in AI, along with their typical use cases, relative complexity, and the period when they were introduced:

Model architecture	Typical use cases	Complexity	Introduced
Linear regression	Price prediction, trend analysis	Low	1805 / widely adopted in the 20th century
Logistic regression	Binary classification (e.g., spam detection)	Low	1958
Decision trees	Risk modeling, rule-based classification	Low–Medium	1960s–1980s
Random forest	Ensemble classification and regression	Medium	2001

Model architecture	Typical use cases	Complexity	Introduced
Support vector machine (SVM)	Text classification, face detection	Medium–High	1995
Feedforward neural network (FNN)	Image classification, regression tasks	Medium–High	1980s–1990s
Convolutional neural network (CNN)	Image and video recognition	High	1998
Recurrent neural network (RNN)	Time series, speech recognition	High	1980s–1990s
Transformer (GPT)	Natural language processing, LLMs	Very High	2017
Diffusion models (DALL·E, Imagen)	Image generation, generative tasks	Very High	~2020

Table 1.3: Popular machine learning model architectures

For the purpose of this book, we don't need to dive deeply into each model architecture mentioned above. Exploring them in detail is the focus of more advanced machine learning. What's important here is to understand that there are many different types of model architecture, each with its own use cases and level of complexity.

At their core, even the most advanced machine learning models, like deep neural networks or large language models, are still just functions that map inputs to outputs based on learned (calculated) parameters. A linear regression model may have only a few parameters (weights) and a very simple structure, while an LLM can have billions of parameters and a very complex architecture, allowing it to capture far more complex patterns and dependencies. The difference lies in scale and architecture, not in the fundamental idea.

Training machine learning models

In the home price model example, we demonstrated the intuition behind finding the first model weight w_1 . We looked at real examples of input (square footage) and output (house price) and estimated a weight that reflects their relationship. This hands-on, intuitive method helps illustrate how a model *learns* a pattern from data. The process of learning the model's parameters from data is called **training**.

Learning from data is a fundamental principle in machine learning and AI. This approach is very different from traditional software development, where developers must manually write rules and handle each case explicitly through code. In classical programming, the logic is handcrafted; the code does only what it's been explicitly told to do. In contrast, machine learning systems learn patterns automatically from data, allowing them to handle far more variability and complexity without needing every scenario to be hardcoded.

In our home price model example, we used historical data, such as prices of homes that have already been sold, to help the model calculate (or more accurately, *to learn*) the model parameters. By analyzing patterns in this data, the model *learns* its weights $w_1, \dots, w_n$ so it can make accurate home price predictions afterward.

To find the first weight in that model, we used just two examples with a single feature, square footage, and the corresponding actual prices. However, in practice, building real-world models typically requires large datasets with many features. The more complex the model is, the more data it needs to learn effectively.

Below is an example showing a few sample records from the housing prices dataset from Kaggle (<https://www.kaggle.com/>):

Housing.csv (29.98 kB) 📄 🗑️ ⏪

Detail Compact Column 10 of 13 columns ▼

# price	# area	# bedrooms	# bathrooms	# stories	✓ mainroad	✓ guestroom	✓ basement	✓ hotwaterh...
13300000	7420	4	2	3	yes	no	no	no
12250000	8960	4	4	4	yes	no	no	no
12250000	9960	3	2	2	yes	no	yes	no
12215000	7500	4	2	2	yes	no	yes	no
11410000	7420	4	1	2	yes	yes	yes	no
10850000	7500	3	3	1	yes	no	yes	no
10150000	8580	4	3	4	yes	no	no	no

Figure 1.2: Housing prices dataset from Kaggle

Today, thousands of open-source datasets are available online, playing a fundamental role in the advancement of AI. As discussed earlier, a dataset is a collection of inputs often paired with their expected outputs, used to train and evaluate models. These resources are shared by researchers, companies, and communities to support training, testing, and benchmarking of AI systems. Platforms like Kaggle (<https://www.kaggle.com/>), Hugging Face (<https://huggingface.co/>), and Google Dataset Search (datasetsearch.research.google.com) make it easy to explore and experiment with real-world data.

Coming back to our model, we can say that the weights $w_1, \dots, w_n$ incorporate or *encode* the knowledge the model has learned from the training data. In our home price model example, all

the patterns and relationships observed from sold houses are captured and encoded in these weights. That knowledge is then used to predict the price of new, unsold houses in the future. A similar principle applies to all machine learning models.

In other words, the model's parameters *store* the knowledge it has learned from data. You can think of the model (very roughly) as a kind of *knowledge database*. But unlike traditional databases, where information is stored in specific, structured locations (like tables and rows in SQL databases), knowledge in an AI model is distributed across all weights. What's more, the model doesn't just memorize the training data; it also *learns* how to apply that knowledge to new, unseen inputs. This ability to perform well on unfamiliar data is known as **generalization**, and it's one of the most powerful aspects of machine learning.

Sometimes, a model can perform extremely well on the data it was trained on, yet struggle or fail when it's given new, unseen data. This issue is known as **overfitting** in machine learning. It means the model has learned the training data too well, including noise or random fluctuations, rather than learning the general patterns. As a result, it can't generalize its knowledge to make accurate predictions on real-world data. Overfitting is one of the key challenges in building robust and reliable AI models.

An analogy from everyday life would be a mother teaching her child how to solve a few specific problems or behave in certain situations. If the child simply memorizes what to do in those exact cases, but struggles when the situation changes even slightly, the child hasn't really learned the underlying principle, only the examples. In the same way, an overfitted model performs well on its training data but fails to generalize to new situations.

Today, modern machine learning frameworks like PyTorch and TensorFlow automate much of the training process, making it easier to build models without manually implementing all the math.

Note

Historical note

Before 2015, data scientists and researchers relied heavily on tools like MATLAB, R, Torch, and other numerical computing libraries for machine learning research. The release of **TensorFlow** (Google, 2015) and **PyTorch** (Facebook AI Research, 2016) marked a major turning point. These open-source frameworks quickly became the de facto standard for building and training ML and deep learning models. Their intuitive APIs, strong community support, and GPU/TPU integration made it dramatically easier to prototype, train, and deploy models at scale. Together, they have significantly accelerated the development of modern ML and AI, including today's large language models.

To see what training looks like in practice, let's walk through this code example:

```
import torch
import torch.nn as nn
import torch.optim as optim

# Sample training data (square footage → house price)
# x: square footage difference from average, y: house price
x_train = torch.tensor([[ -500.0], [ 0.0], [ 500.0]])
y_train = torch.tensor([[300000.0], [400000.0], [500000.0]])

# define hyperparameters
learning_rate = 0.000001
num_epochs = 100

# Define a simple linear model:  $y = w * x + b$ 
# w - is a weight (slope), b - is a bias (intercept)
model = nn.Linear(in_features=1, out_features=1)

# Loss function and optimizer
criterion = nn.MSELoss()
optimizer = optim.SGD(model.parameters(), lr=learning_rate)

# Training Loop
for epoch in range(num_epochs):
    optimizer.zero_grad()
    output = model(x_train)
    loss = criterion(output, y_train)
    loss.backward()
    optimizer.step()

# After training, print learned weights
print("Trained model: y =", model.weight.item(), "* x +", model.bias.item())
```

This simple example demonstrates how a linear regression model can learn (or calculate) the relationship between square footage and house price by adjusting its internal weights during training. With modern libraries like PyTorch, it's possible to build and train such models using just a few lines of code.

In this code, first, we create a simple training dataset with three examples. Then, we define two learning parameters, the learning rate and the number of epochs, known as **hyperparameters**.

After that, we define the structure of our linear regression model. We also set up an optimizer, which is responsible for updating the model parameters w and b during training. Finally, we train the model, and at the end, we print the learned values of w and b .

Note

In PyTorch and TensorFlow, the term *tensor* refers to a generalized, multidimensional array of numbers. Tensor extends the concept of vectors and matrices. A 1D tensor is simply a vector (a list of numbers), while a 2D tensor is a matrix. A 3D tensor can be thought of as a stack of matrices, forming a cube-like structure. This pattern continues with higher dimensions, making tensors a flexible way to represent data of varying shapes and complexities in machine learning models.

For the purpose of this book, we don't need to dive deeply into how every part of the code above works or how to train models from scratch, as that's more the focus of classical machine learning. What's important here is to grasp the big picture: how models learn from data, what terms like training, datasets, and parameters mean, and how all of these pieces come together.

After training, we can use the model. The process of using a trained model to generate outputs or predictions on new data is called **inference**. In practice, this means providing the model with new, unseen input, and letting it produce an output based on what it has learned during training. In our example, inference means predicting the price of a house that hasn't been sold yet.

Here, again, we should also emphasize one of the fundamental concepts in AI and machine learning: we train our model on one dataset and then use it to make predictions on new, unseen data. This contrasts with traditional software engineering, where we typically handle specific cases explicitly and expect our code to work for the same or very similar scenarios. In AI, the goal is to generalize and handle a much broader and more complex range of situations.

Supervised, unsupervised, and reinforcement learning

There are several types of machine learning approaches:

- **Supervised learning**
- **Unsupervised learning**
- **Reinforcement learning**

Supervised learning is a type of learning where we know both the inputs and the corresponding expected outputs. The house price model we discussed earlier is an example of supervised learning because we provide the model with both the characteristics of the houses (inputs) and their actual prices (outputs). In some tasks, such as classification, these expected

outputs are often referred to as **labels**. The process of providing the correct answers or outputs for supervised learning is called **annotation** or **labeling**.

Unsupervised learning is a type of learning where we don't have the correct outputs or labels for the data. Instead, the model tries to find patterns, structures, or relationships within the input data on its own. A common task in unsupervised learning is **clustering**: grouping similar items together. For example, a model might analyze customer behavior data and group similar customers together without being told what those groups should be.

Let's illustrate the difference between supervised and unsupervised learning with a simple example. A mother teaches her child to recognize animals by pointing to a dog and saying, "This is a dog," then to a cat and saying, "This is a cat." Over time, the child learns to identify and distinguish between dogs and cats, even unfamiliar breeds, based on what they've learned. This is an example of supervised learning, where labeled examples ("This is a dog," "This is a cat") guide the learning process.

Now imagine an alien visiting Earth with no idea what names humans give to different animals. It observes many creatures and begins grouping them based on similarities, placing certain animals together because they look alike, without knowing their names. This is an example of unsupervised learning, where the model discovers patterns and structures in the data without any labels.

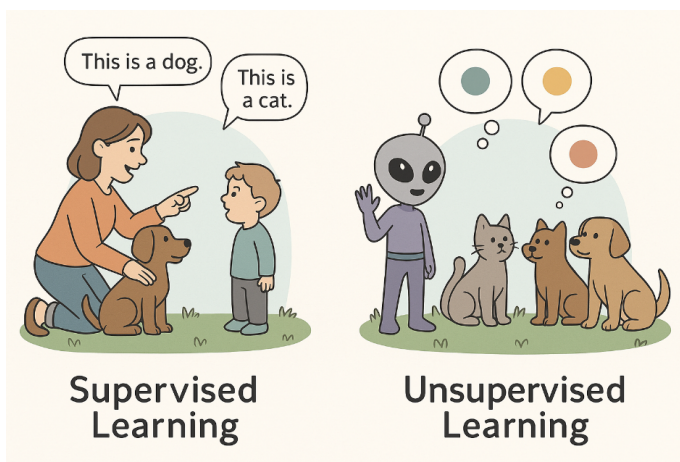


Figure 1.3: An illustration of supervised and unsupervised learning

And finally, **reinforcement learning (RL)** is a type of machine learning where the model learns by interacting and receiving feedback in the form of rewards or penalties. In other words, the model "trains itself" through experience. Instead of learning from labeled examples like supervised learning, it learns through trial and error, trying different actions and learning from the outcomes. Over time, the model figures out which actions lead to better results.

The ultimate goal is to learn a strategy, called a **policy**, that helps it make the best decisions to maximize rewards in the long run. This policy is itself a model, just like in supervised learning, but it is trained based on interactions rather than static datasets.

A common example of reinforcement learning is teaching a robot to walk: it tries different movements and receives a penalty when it falls, as if it's "getting hurt." Over time, it learns to adjust its actions to walk more effectively and avoid those penalties. Another example is training an AI to play video games. The model explores the game environment, learning which strategies improve its score and lead to success. Reinforcement learning is widely used in robotics, game AI, and even in real-world applications such as recommendation systems and automated stock trading.

One of the most iconic moments in the history of AI was the 2016 showdown between reinforcement learning model AlphaGo, developed by DeepMind (now part of Google), and Lee Sedol, one of the world's top professional Go players. Go had long been considered one of the most complex board games for AI to master due to its enormous number of possible moves and deep strategic elements. AlphaGo shocked the world by defeating Lee Sedol with a score of 4–1, demonstrating the power of reinforcement learning combined with deep neural networks. This victory marked a turning point in AI development, showcasing that machines could now master intuition-driven tasks once thought to be uniquely human.

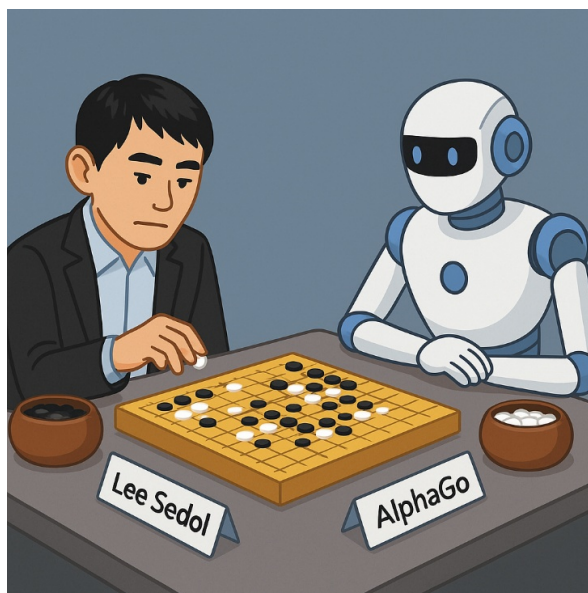


Figure 1.4: Go tournament between Lee Sedol and DeepMind AlphaGo

In another major milestone for AI, a model called OpenAI Five showcased the power of large-scale reinforcement learning in the complex, team-based video game Dota 2. Unlike board games, Dota 2 demands real-time decision-making, long-term strategy, coordination, and adaptability, all within a highly dynamic and partially observable environment. In 2019, OpenAI Five played against the reigning world champion team, OG, and won convincingly, becoming the first AI system to defeat top-tier human players in such a game. This achievement demonstrated how reinforcement learning can scale using massive self-play training and deep neural networks, enabling AI to operate effectively in chaotic, strategic, multi-agent environments, and pushing the boundaries of what AI systems can achieve.

Neural networks

Once you understand the basic concepts of machine learning and see how linear regression works, it becomes easier to grasp how **neural networks (NNs)** evolved. At their core, neural networks are simply a more flexible and powerful extension of the same idea: they take input data, perform some transformation, and produce output. The key difference is that, instead of using a single linear equation like in linear regression, neural networks stack multiple layers of such equations. Since most real-world relationships are non-linear and can't be fully captured by straight lines, neural networks introduce non-linear transformations, and this is where the concept of a neuron comes in.

Note

A **linear** relationship means that when the input changes, the output changes by a constant amount. For example, if vehicle fuel consumption increases by the same 1 liter for every 30 kilometers driven, the relationship between driven distance and fuel used is linear. A non-linear relationship means the change is not constant; it may curve, accelerate, or level off. For example, the speed of a car vs. its fuel efficiency: after a certain speed, every extra speed causes disproportionately higher fuel consumption.

Basically, a *neural network* is a collection of interconnected units called **neurons**. Each neuron receives input from other neurons, processes it, and passes the result forward through the network.

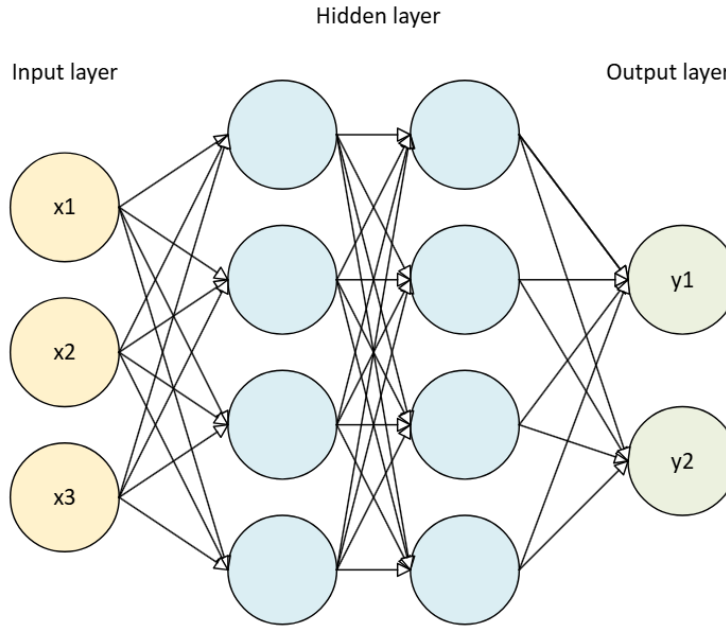


Figure 1.5: Neural network with hidden layers

In the image above, we have an input layer consisting of input features (x_1, x_2, x_3) and an output layer representing the predicted outputs (y_1, y_2) . The circles in the middle (blue color) represent neurons. A neuron is a linear equation followed by a transformation. In simple terms, a neuron can be defined as follows:

$$neuron = a(w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n)$$

where a is some non-linear function. This function is often called an **activation function**. Here are some of the most commonly used activation functions in modern neural networks:

- **Rectified linear unit (ReLU):**

$$a(x) = \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases}$$

– simple, efficient, and commonly used in neural networks

- **Sigmoid:**

$$a(x) = \frac{1}{1 + e^{-x}}$$

– outputs values between 0 and 1, making it useful for binary classification tasks

There are also other activation functions, such as **Gaussian error linear unit (GeLU)**, Tanh, and Swish, which are used in modern architectures like transformers.

A **layer** is a group of neurons that work at the same stage of a neural network. In *Figure 1.5*, we have one input layer, one output layer, and two intermediate (hidden) layers in between. Each neuron in a given layer processes the input it receives from the neurons in the preceding layer. You can think of a layer as a row of workers (*neurons*): they each do their job in parallel, then pass their results forward to the next group of workers (*layers*). The more layers a model has, the more complex it is and the more complex processes it can model.

Layers between the input and output layers are called **hidden layers**. The more hidden layers a model has, the more complex and expressive it becomes. Each hidden layer performs a different transformation. For example, in an image recognition model that detects people, animals, or objects in photos, the layers might operate as follows:

- The first layer detects basic patterns like edges, lines, and textures
- The second layer combines those low-level features into shapes and more complex patterns
- The third layer identifies entire objects, such as faces, animals, or vehicles

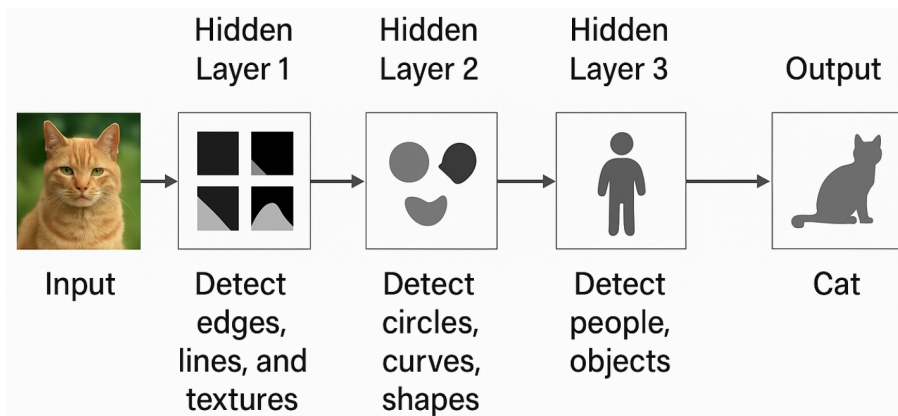


Figure 1.6: From pixels to concepts: feature extraction across layers in neural networks

A neural network where all neurons are connected to each other is called a **fully connected** or **dense neural network**. In this architecture, each neuron in one layer is connected to every neuron in the next layer. Or every neuron in one layer gets information from all neurons from a previous layer. Fully connected layers are commonly used in *feedforward* neural networks.

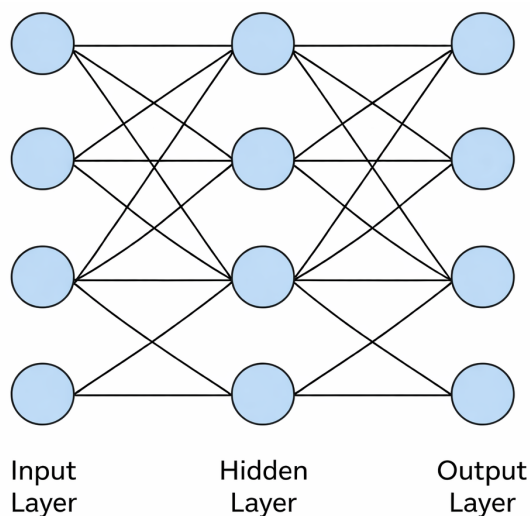


Figure 1.7: Fully connected neural network

The linear regression model we explored earlier can actually be seen as a very simplified form of a neural network: one that has only a single layer and no activation function (or $y = x$ *activation function*). While basic, this structure lays the foundation for understanding more complex neural networks, which build upon this idea by stacking layers and introducing activation functions to model non-linear patterns in data.

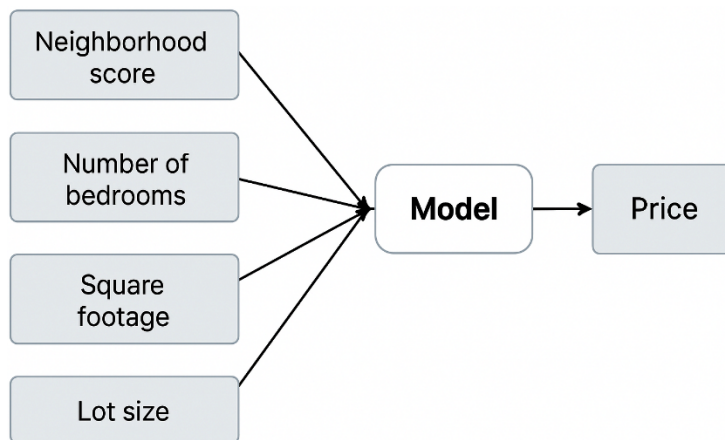


Figure 1.8: Linear regression is a simple home price neural network

So far, we've built an intuition for linear regression, seen how neural networks extend this idea, and introduced key concepts such as training, inference, weights, and layers. Together, these

fundamentals form the backbone of modern AI systems. They give you the mental framework needed to understand more advanced topics we'll explore next, including machine learning architecture, deep learning techniques, and ultimately, LLMs.

Fine-tuning

Once you've trained a model, you can use it to solve a specific task, like predicting house prices in our earlier example. In that case, we built a housing price model from scratch: we defined the architecture (*a simple linear regression*) and learned (*calculated*) its parameters directly from data.

In many real-world scenarios, especially for complex tasks like natural language processing or image recognition, training a model completely from scratch isn't efficient and sometimes not even feasible. Training modern models like LLMs requires massive datasets, powerful hardware, and significant time.

This is where fine-tuning comes in. **Fine-tuning** is a technique where you start with a **pretrained model**, one that has already been trained on a large, general-purpose dataset, and continue training it on a smaller, task-specific dataset. Instead of starting from zero, you're building on top of what the model already knows. This allows it to adapt to a new domain, audience, or goal while retaining the general knowledge it learned during pretraining.

Note

The term **fine-tuning** is most commonly used to describe the process of retraining a model on new data. However, in some contexts, especially outside of machine learning research, it's also used more loosely to mean adjusting or refining something. Be mindful of the context in which it's used.

For example:

- A language model trained on internet-scale data can be fine-tuned to perform better on legal documents, customer service chat logs, or medical records
- An image recognition model trained on millions of everyday images can be fine-tuned to detect manufacturing defects, medical anomalies in X-rays, or specific products in a store inventory

Fine-tuning is usually faster and less resource-intensive than training from scratch, and it often results in better performance for the specialized task.

As we've learned, all of a model's "knowledge," i.e., the patterns it has discovered during training, is stored in its parameters. Fine-tuning works by updating those parameters with new examples, incorporating additional knowledge while typically preserving most of the

original training. Depending on the approach, we may update all of the parameters or just a small subset (this is sometimes called **partial fine-tuning** or **parameter-efficient fine-tuning**).

We'll explore fine-tuning in more detail in *Chapter 2*, revisit it when we discuss how LLMs are built, and dive even deeper into its practical applications in *Chapter 7*.

Core machine learning problems

Armed with the core concepts of AI, we're ready to explore the types of problems these technologies are designed to solve. Advanced AI models, especially LLMs, have transformed how we approach many of these problems, but the problems themselves haven't changed. We still categorize tasks into familiar groups like prediction, classification, and generation; what has changed are the tools and techniques we use to solve them.

In this section, we'll look at the classic problem categories in machine learning, the building blocks that underpin countless real-world applications.

Regression

At the beginning of this chapter, we explored one of the earliest and fundamental problems in AI: regression. **Regression** is a core task in machine learning where the goal is to predict a continuous numeric value based on a given set of input parameters. The house price model is a classic regression task. Other common examples include sales estimation, forecasting weather temperatures, or predicting stock prices. In each case, the output is a real number, such as the temperature, revenue figure, or predicted share price.

Formally, a regression model can be defined by a function:

$$y = f(x)$$

where x is a set of input features (a vector), and y is a single number or a vector of numbers. The model is trained in a supervised manner using labeled examples, with the goal of finding a function that captures the relationship between inputs and outputs.

One of the simplest regressions is a **linear regression**, which assumes the relationship can be modeled as a straight line, i.e., a linear relationship. Despite its simplicity, linear regression is interpretable and still applied in practice and science.

Classification

Classification is another fundamental problem in machine learning, where the goal is to assign input data to one of several predefined categories or classes. Unlike regression, which predicts continuous values, classification deals with discrete labels (or categories). For example, determining whether an email is "spam" or "not spam" (ham), so two categories.

Other examples include classifying customer reviews as "*positive*", "*neutral*", or "*negative*", or recognizing handwritten digits from 0 to 9. The following figure illustrates a binary classification problem:

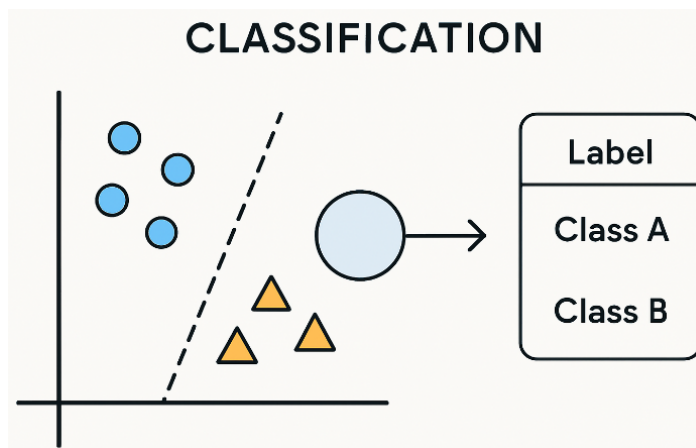


Figure 1.9: Illustration of a binary classification problem

Formally, a classification model, similar to regression, can be defined with a function:

$$y = f(x)$$

where x is a vector of input features, but, in contrast to regression, y is a categorical value representing a class. The function in classification is learned in a *supervised manner* as well, using a dataset where each input is paired with the correct label (class). The model's goal is to accurately map new, unseen inputs to the correct class. Common architectures for classification include logistic regression, decision trees, random forests, and neural networks.

There are three common types of classification problems:

- **Binary classification:** The model predicts one of two possible outcomes
Example: Spam detection, where the goal is to classify an email as either spam or not spam.
- **Multi-class classification:** The model selects one outcome from more than two possible classes
Example: Sentiment analysis of social media posts with possible outputs like *Positive*, *Neutral*, or *Negative* (three classes)
- **Multi-labeled classification:** The model can assign multiple labels (classes) to the same input

Example: Blog post tagging, where a single post might receive several tags such as "AI," "LLM," and "Data Science"

Classification problems are widely used in real-world applications such as fraud detection, medical diagnosis, image recognition, and language processing. For example, in NLP, classification is used for tasks like intent detection in chatbots or topic categorization. Large language models also perform classification tasks effectively, especially when the inputs are in natural language form, making them a powerful tool for text-based classification. We will learn text classification problems in *Chapter 4*.

Clustering

Clustering is an unsupervised learning problem where the goal is to group similar data points together based on their features, without predefined labels. Unlike supervised learning, where a model is trained on examples with known outputs, clustering algorithms try to discover natural structure or patterns in the data on their own.

The result is a set of clusters, with each cluster containing data points that are more similar to one another than to points in other clusters (see *Figure 1.10*):

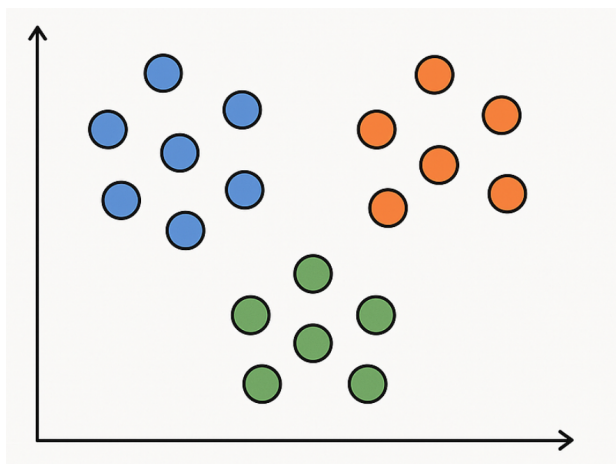


Figure 1.10: Clustering similar objects

Clustering is commonly used in exploratory data analysis, customer segmentation, document organization, image grouping, and anomaly detection. For example, a marketing team might use clustering to segment customers into groups based on their purchasing behavior, even if those groups weren't defined in advance. Popular clustering algorithms include K-Means, DBSCAN, and hierarchical clustering.

Ranking and recommendation

Ranking is a machine learning problem where the goal is to order a set of items based on their relevance to a specific query or user. Unlike classification or regression, which predict values for individual items, ranking models focus on comparing items relative to one another. A classic example is a web search engine like Google Search or Microsoft Bing, which must return web pages ranked by how relevant they are to a user's query. The model must decide which results are most relevant and in what order they should appear.

There are different approaches to ranking. **Pointwise** models score each item independently, **pairwise** models compare items in pairs to decide which is better, and **listwise** models evaluate the whole list at once. While pointwise ranking can be loosely viewed as applying regression or classification to individual items, true ranking tasks require making relative judgments between items, which is why pairwise and listwise methods are often more effective. We will come back to pointwise, pairwise, and listwise approaches in *Chapter 8*.

Recommendation is closely related to ranking but focuses on suggesting items a user might like. For example, streaming platforms like Netflix recommend movies or shows based on your viewing history and preferences, and e-commerce sites like Amazon suggest products based on your past purchases or browsing behavior. The key challenge is to personalize results by modeling user preferences and item characteristics.

Most recommendation systems are built using traditional models such as *collaborative filtering*, *matrix factorization*, or *gradient boosting*. However, LLMs can also be applied to recommendation tasks, especially in natural language interfaces like chatbots or when working with unstructured data such as user reviews or product descriptions.

Core NLP problems and concepts

As mentioned earlier, NLP focuses on solving problems related to human language, primarily through text and speech. Over time, researchers and engineers have defined a set of core NLP tasks that form the backbone of many modern language-based applications. These include summarization, text generation, machine translation, question answering, information extraction, reasoning, and others. These tasks remain highly relevant today and continue to evolve with the rise of large language models.

In this section, we'll provide a brief overview of these key problem types to build a foundation for the material ahead. We'll explore them in more depth, along with practical approaches to solving them, in *Chapter 4*. Let's now look at problem types:

- **Text classification:** Assigns one or more predefined categories to a given text. Common use cases include spam detection, intent classification, topic labeling, and

sentiment analysis. It serves as a building block for many downstream NLP applications.

- **Summarization:** Generates a shorter version of a longer text while preserving its key information. It's a fundamental NLP task with many variations, including summarization of long or heterogeneous documents, focused, and multi-document summarization.
- **Information extraction:** Identifies structured information such as names, dates, quantities, or relationships from unstructured text. It's used in document analysis, knowledge base construction, and automated tagging. A well-known subtask is **named entity recognition (NER)**, which we'll cover more in *Chapter 4*.
- **Text generation:** Produces coherent and meaningful text based on a user prompt or context. Applications include writing assistants, creative content generation, conversational agents, and more. A notable subdomain is software code generation, where models produce executable code snippets based on natural language instructions.
- **Question answering (QA):** Provides direct answers to user questions based on a given document or external knowledge. QA is widely used in virtual assistants, customer support chatbots, and intelligent search systems.
- **Reasoning:** Involves drawing logical conclusions, making inferences, or solving multi-step problems based on input information. Reasoning tasks may include commonsense reasoning, mathematical reasoning, logical deduction, or multi-hop question answering. While LLMs have shown promising results in some types of reasoning, performance can vary depending on task complexity and structure, and it's an area of active research and rapid improvement.

All these problems we will cover in this book, especially in *Chapter 4*, and they form the foundation for more complex systems like AI agents and virtual assistants.

Now let's turn to some of the core building blocks of NLP, the techniques that make it possible for machines to work with human language. From a computer's perspective, human language is just raw text, essentially a sequence of bytes or characters. But AI models can't directly understand text in this raw form. They need a numerical representation that can be processed mathematically.

This is where preprocessing steps like tokenization, stemming, lemmatization, and stop-word removal come in. These techniques transform messy, human-readable text into a structured format that models can work with, effectively turning words into numbers, normalizing variations, and filtering out noise.

Tokenization

One of the first steps in any NLP pipeline is **tokenization**, which is a process of breaking down text into smaller components called **tokens**. These tokens can represent various text elements such as words, sub-words, combinations of words (n-grams), etc. Usually, we deal with tokens as individual words or sub-words. However, tokenization can also manage other text units, like sentences, phrases or even characters, depending on the context and needs.

A **tokenizer** is the tool or a software component responsible for performing tokenization. It converts raw text into tokens (text parts) and, in many NLP applications, further maps those tokens to numerical representations called **token IDs**. This numerical representation is important for language models because, as we said before, ML models cannot work with text directly, only with numbers. Token IDs serve as input for AI models.

Many libraries, such as SpaCy and NLTK, offer tokenization tools for various NLP tasks. Additionally, most language models, like BERT or GPT, have their own specialized tokenizers optimized for their architecture. Here is an example of Python code to tokenize a sentence by using the BERT model tokenizer from Hugging Face:

```
# using Hugging Face transformers Library
from transformers import AutoTokenizer

# Load a tokenizer from BERT model
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
sentence = "Tell me a joke"

# tokenizing the sentence
tokens = tokenizer.tokenize(sentence)

# Get token IDs
token_ids = tokenizer.convert_tokens_to_ids(tokens)

# Output results
print("Tokens:", tokens)
print("Token IDs:", token_ids)
```

This code produces the following:

```
Tokens: ['tell', 'me', 'a', 'joke']
Token Ids: [2425, 2033, 1037, 8257]
```

In other words, a tokenizer maps text to a list of words and then to a list of numbers (token IDs):

"Tell me a joke" → [Tell, me, a, joke] → [2425, 2033, 1037, 8257]

Most word-based tokenizers have a *vocabulary*, a predefined list of commonly used words or sub-words. This vocabulary is often implemented as a hash-table (dictionary, map), where the key is the token (such as a word or sub-word), and the value is its corresponding token ID. Here is an example of a portion of the vocabulary for the BERT tokenizer:

```
{ 'keep': 2562, 'z': 1062, '##time': 7292, 'bordeaux': 16384, 'them': 2068, ... }
```

The size of the vocabulary defines the complexity of language models, particularly memory usage and compute resource requirements. To balance complexity and flexibility, most LLMs use a relatively medium-sized vocabulary, typically around 20K–100K words.

However, natural languages like English or Spanish contain far more unique words than that. Words outside of the vocabulary are called **out-of-vocabulary (OOV)** words. They are usually handled through algorithms that break words down into smaller subwords.

One of the most popular methods to resolve out-of-vocabulary tokens is **byte pair encoding (BPE)**, which enables the model to split rare or unseen words into known sub-words from the vocabulary. Here is an example of how the T5 model tokenizer splits the "Microsoft Copilot" phrase into tokens:

```
from transformers import AutoTokenizer

# Load a tokenizer from the T5 model
tokenizer = AutoTokenizer.from_pretrained("t5-small")

# Input sentence
sentence = "Microsoft Copilot"

# Tokenize the sentence
tokens = tokenizer.tokenize(sentence)
token_ids = tokenizer.convert_tokens_to_ids(tokens)

# Output results
print("Tokens:", tokens)
print("Token IDs:", token_ids)
```

This code returns the following:

```
Tokens: ['_Microsoft', '_Co', 'pilot']  
Token IDs: [2803, 638, 24650]
```

We can see that the tokenizer has the word "Microsoft" in the vocabulary (token Id = 2803) but does not have "Copilot". It splits "Copilot" into two parts: "Co" and "pilot". So, the word "Copilot" is mapped to two tokens - [638, 24650]. That's why typically the number of words does not equal the number of tokens in text.

It's important to remember that symbols like '.' (dot), ',' (comma), numbers, and other non-alphabetical characters are typically treated as tokens. These symbols are important for models to understand language structure, such as punctuation and sentence boundaries. The newline character ('\n') is also often included in the tokenizer's vocabulary, allowing LLMs to recognize and process new lines and paragraphs.

Symbols also contribute to higher token usage. On average, the token count in text can be roughly estimated as 1.3-1.5 times the word count, though this varies depending on the tokenizer and text characteristics. Data formats, like JSON, can consume more tokens due to heavy use of symbols and structural elements. To optimize token usage, simpler formats such as YAML, CSV, or plain text are often preferred. However, the actual savings depend on the content and structure of the data.

Sentence-level tokenization

Sentence-level tokenization involves splitting a block of text into individual sentences. The simplest approach uses punctuation marks like dots (.), question marks (?), and exclamation points (!) as indicators of sentence boundaries. However, this method is far from perfect. It can easily be misled by abbreviations such as "e.g.," "i.e.," or names like "Dr.," where the dot doesn't indicate the end of a sentence.

To handle these cases more reliably, modern NLP libraries like spaCy use a combination of statistical models and built-in linguistic rules to distinguish true sentence boundaries from exceptions like abbreviations and titles. Below is an example demonstrating how to perform sentence-level tokenization using the spaCy library:

```
import spacy  
  
# Load spaCy English Language model  
nlp = spacy.load("en_core_web_sm")  
  
# Sample text
```

```
text = "Dr. Smith arrived at 3 p.m. he was tired. However, he continued working.  
Amazing, isn't it?"  
  
# Process the text with spaCy  
doc = nlp(text)  
  
# Iterate through detected sentences  
print("Detected sentences:")  
for sent in doc.sents:  
    print("-", sent.text)
```

This code produces the following:

```
Detected sentences:  
- Dr. Smith arrived at 3 p.m. he was tired.  
- However, he continued working.  
- Amazing, isn't it?
```

Here, we can see that spaCy correctly identifies three sentences, avoiding incorrect splits that would occur if we simply split the text by periods. A naive approach might have incorrectly returned six or more sentences due to abbreviations like "Dr." or "p.m." that can be treated as sentence boundaries.

Stemming, lemmatization, and stopwords

Stemming is a technique used to reduce words to their root or base form, typically by chopping off suffixes. For example, words like "walking", "walker", and "walk" might all be reduced to the stem "walk". This helps group different variations of a word under the same base for simpler analysis. However, stemming is usually rule-based and can produce non-dictionary forms (e.g., "better" → "bet"). Stemming can be fine for some basic search and indexing tasks, but it can reduce interpretability and accuracy in more advanced NLP tasks.

Lemmatization, on the other hand, uses linguistic knowledge to reduce a word to its base or dictionary form called a **lemma**. It takes into account a word's part of speech and grammatical context. For example, "better" becomes "good", and "running" becomes "run". The result is cleaner and more accurate text, which is why lemmatization is preferred in modern NLP workflows, especially with LLMs.

Let's consider a simple Python example that demonstrates the difference between stemming and lemmatization:

```
import spacy
from nltk.stem import PorterStemmer
nlp = spacy.load("en_core_web_sm")
stemmer = PorterStemmer()
# Sample words
words = ["running", "fairly", "better", "flies", "wolves"]
# Process with spaCy
doc = nlp(" ".join(words))
# Apply stemming and lemmatization
for token in doc:
    print(f"Word: {token.text:<10} | Stem: {stemmer.stem(token.text):<10} | Lemma: {token.lemma_}")
```

This code returns the following:

Word: running	Stem: run	Lemma: run
Word: fairly	Stem: fairli	Lemma: fairly
Word: better	Stem: better	Lemma: well
Word: flies	Stem: fli	Lemma: fly
Word: wolves	Stem: wolv	Lemma: wolf

In the preceding code example, we can see the difference between stemming and lemmatization. Lemmatization produces better results but requires more computational resources to execute.

In many NLP tasks, certain common words like "the", "is", or "and", known as **stop words**, are often removed during preprocessing because they typically carry little *semantic* meaning on their own. These words are so frequent and predictable that they rarely help in distinguishing between different texts or topics. Removing them can improve the efficiency of models.

Note

In the context of AI and language models, the word **semantic** refers to the meaning of words, phrases, sentences, or some text. When we talk about semantic similarity, we mean how different words or sentences relate in meaning, even if they use different vocabulary. For example, "I'm happy" and "I feel great" are semantically similar, even though they share no exact words.

However, removing stop words is highly context dependent. In some applications, such as sentiment analysis, dialog modeling, or question answering stop words may still carry important cues. For example, consider the sentences "I am happy" and "I am not happy". The only difference is the word "not", which is often treated as a stop word, yet it completely flips the sentiment of the sentence. These small words can carry important relational or contextual meaning. That's why many modern NLP pipelines treat stop word removal as an optional and task-specific step, rather than a default.

Let's consider a simple Python example that identifies stop words in a sentence:

```
import spacy
# Load spaCy English Language model
nlp = spacy.load("en_core_web_sm")
# Sample text
text = "This is a simple sentence that contains some stop words."
# Process the text
doc = nlp(text)
# Print stop words
print("Stop words in the text:")
for token in doc:
    if token.is_stop:
        print("-", token.text)
```

This code produces the following:

```
Stop words in the text:
- This
- is
- a
- that
- some
```

The preceding code example shows that words like "this", "is", "a", "that", and "some" are defined as stop words.

Now let's look at how LLMs actually process text tokens. AI models can only work with numbers, so text must first be converted into a numerical representation that the model can understand. This is where **embeddings** come in.

Embeddings

As we saw in the previous section, tokens are first converted into numerical IDs; for example, "cat" might become the number 4521, and "dog" might become 9827. These IDs are just lookup keys: they tell the model which token we're referring to, but they don't carry any information about the meaning or similarity between tokens. To the model at this stage, "cat" and "dog" are no closer together than "cat" and "refrigerator."

This creates a challenge because language is full of relationships and patterns, and words with similar meanings or roles should be represented in a way that reflects those similarities. For example, "king" and "queen" are closely related (one could be seen as the male/female version of the other), and "Paris" and "London" are similar because they are both cities. In contrast, "king" and "refrigerator" or "city" and "lesson" have little in common and should be far apart in meaning.

To capture these kinds of relationships, and more generally, to give tokens a representation that reflects their meaning, embeddings were introduced.

Basically, **embeddings** are a mapping between tokens (or token IDs) and n-dimensional vectors (points or arrays of numbers) in an n-dimensional space. These vectors encode *semantic* and syntactic information about the tokens, allowing the model to understand relationships such as similarity, context, and meaning. For example, tokens with similar meanings or usage, like "king" and "queen," might have vectors that are close to each other in this space, while unrelated tokens will be farther apart.

Note

In mathematics, a vector is often defined by two points: its origin (or starting point) and its endpoint. For simplicity, in the context of embeddings and many applications in machine learning, we generally represent vectors by their endpoint coordinates, assuming the origin is fixed at the zero point $(0, 0, \dots, 0)$ in the respective dimensional space.

So, if we apply a tokenizer to the sentence, *Tell me a joke*, and then pass it through an embedding, we get a mapping like this:

$$\text{"Tell me a joke"} \rightarrow [2341, 573, 10234, 98] \rightarrow \begin{bmatrix} 0.12 & -0.04 & \dots & 0.01 \\ 0.34 & 0.67 & \dots & 0.2 \\ 0.11 & -0.34 & \dots & -0.03 \\ -0.05 & 0.91 & \dots & -0.61 \end{bmatrix}$$

Here, the text is mapped to an array of numbers (token Ids) and next every number is mapped to a vector (one row in the matrix above). Like the word "Tell" mapped to a vector $[0.12, -0.04, \dots, 0.01]$ (first row in the matrix), the next word, "me", is mapped to the vector $[0.34, 0.67, \dots, 0.2]$, and so on.

Embeddings work on the principle that tokens with similar meanings or with strong relationships are represented by vectors placed close to each other in the *embedding space*. In contrast, tokens with different meanings or no correlation are represented by vectors positioned far apart.

For example, the embeddings for "man" and "woman" would be close in the space, as they are semantically related. The embeddings for "man" and "graph," however, would be far apart, reflecting their lack of semantic or contextual similarity.

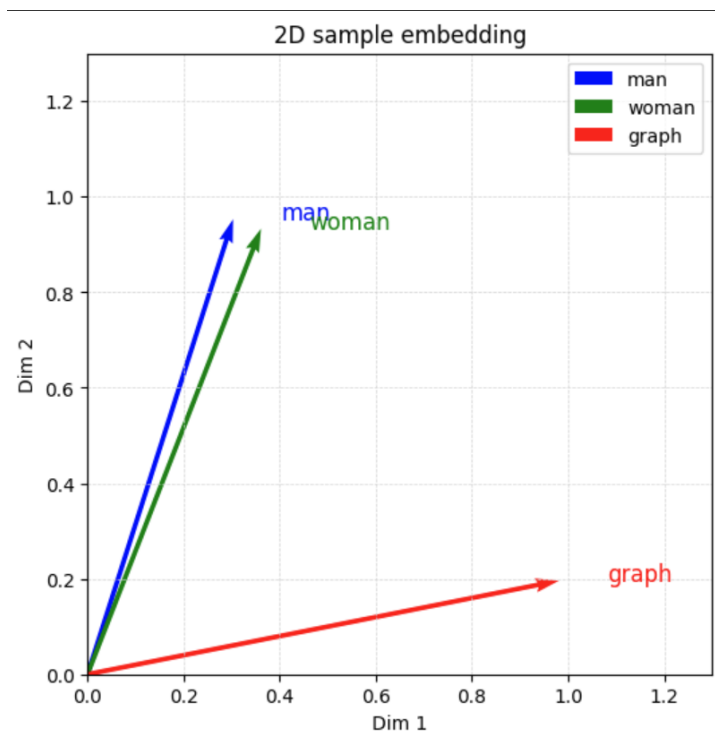


Figure 1.11: Relationship between semantically similar and opposite words

Note

The concept of embedding emerged around the 1970s and shares some similarities with the hashing concept in software engineering, where objects are mapped to a fixed-size byte array (or a number). But a key difference is that in hashing, even a slight change in the object should result in a significantly different hash (high sensitivity). In contrast, embeddings are designed to reflect semantic or contextual relationships, so minor changes in the input result in small changes in the embedding.

Why are n -dimensional spaces used for embeddings? To understand this, let's consider the limitations of using only one dimension ($n = 1$). In this case, each token is represented by a single number (e.g., its token ID). For instance, if the token "apple" is represented by the number 100, we can place only two tokens at a fixed distance of 10 from "apple": at 90 and 110. This setup severely restricts how many tokens can be positioned close to the "apple" word.

Now, if we use 2-dimensional embeddings, we gain significantly more flexibility. Tokens can be arranged anywhere on a circle around "apple" with the same distance to it, allowing for many more neighbors. In 3 dimensions, this flexibility increases further, as tokens can now be positioned on the surface of a sphere. As the number of dimensions (n) increases, the embedding space expands exponentially, providing more opportunities to position related tokens closer together while maintaining meaningful distances from unrelated ones. This higher dimensionality allows embeddings to effectively capture complex relationships and nuances in the data.

To illustrate this idea, consider how words related to "apple" might be arranged in one-dimensional and two-dimensional spaces.

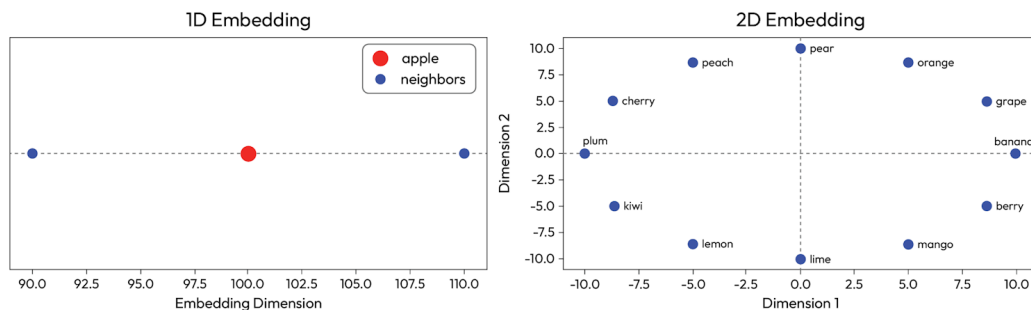


Figure 1.12: Embedding representation in 1D and 2D space

In a 1-dimensional space, only two words, such as "orange" and "mango", can be placed close to "apple". In contrast, in a 2-dimensional space, many more related words can be positioned around "apple" at similar distances.

Typical embedding dimensions range from 100 to several thousand, depending on the complexity and purpose of the model. For example, GPT uses a dimensionality of 768 for its embeddings in the smaller versions and up to 12,288 dimensions in its largest configurations at the time of writing this book. These dimensions allow the model to capture semantic (meaning) and contextual relationships between tokens. The choice of dimensionality is managed by a balance between computational efficiency and the richness of information encoded in the embeddings.

Now, let's look at how many tokens we can locate in an n -dimensional space. If we use **32-bit floating-point numbers (FP32)** to represent each number of coordinates in the embeddings, we can have roughly 2^{32} possible values per dimension. Consequently, an n -dimensional space can hold $2^{32 \times n}$ possible combinations. For instance, with $n=100$, this equates to 2^{230} or approximately 10^{96} possible tokens, which is a huge number.

Tokenizers typically utilize around 20,000 to 100,000 tokens in their vocabulary as we mentioned before. This means the embedding space is *extremely sparse*, leaving vast amounts of unused space; see Figure 1.13:

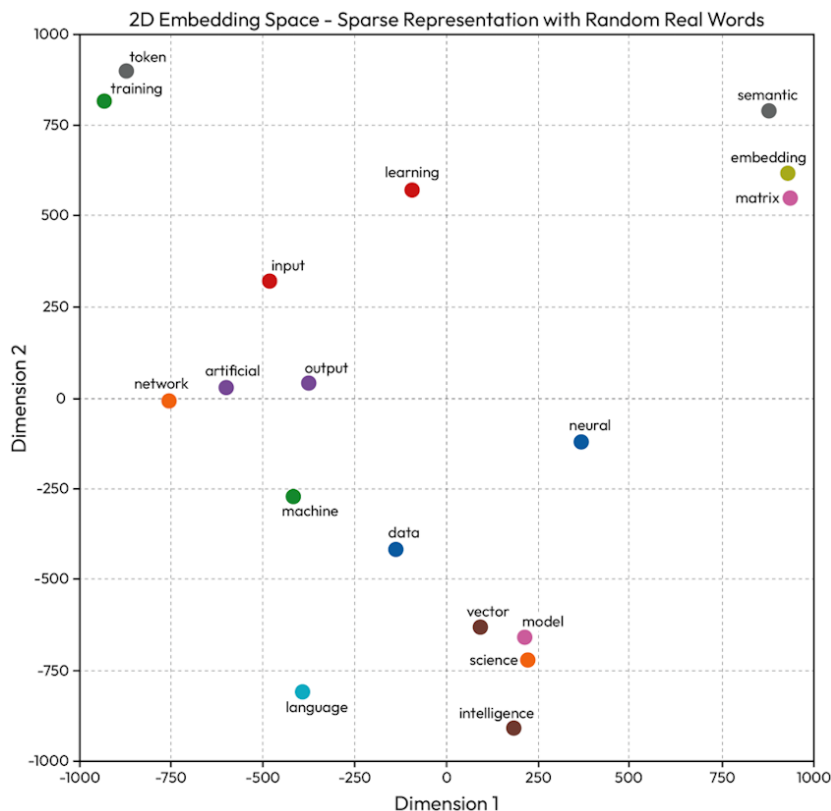


Figure 1.13: Sparse representation of 2-dimensional embedding space

This sparsity is intentional, allowing the model to combine tokens effectively. For example, when two token embeddings are added together, the resulting embedding represents a point or a vector that is "somewhere in between" the two original tokens. This property enables embeddings to capture complex relationships and interactions between tokens, such as contextual meanings and semantic blending, which are essential for tasks like language understanding and generation.

In practice, the effective embeddings space capacity is actually a little bit smaller than the theoretical maximum described earlier. This is because embeddings are typically *normalized*, meaning that each embedding vector is scaled to have 1 unit length (or magnitude of 1). For instance, this vector $[0.8, 0.6]$ is normalized because it's

$$\text{length} = \sqrt{0.8^2 + 0.6^2} = \sqrt{0.64 + 0.36} = \sqrt{1} = 1$$

Note

The length (or magnitude) of a vector is the distance from its starting point to its ending point. As we noted earlier, the starting point is usually the origin — $(0,0, \dots, 0)$.. Therefore, the length of the vector $v = (v_1, v_2, \dots, v_n)$ is calculated using the Euclidean distance formula $||v|| = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}$.

So, an n-dimensional sphere where each dimension is represented by a **32-bit floating-point number (FP32)** will contain approximately $2^{32 \times (n-1)}$ points (similar to an n-1 dimensional space).

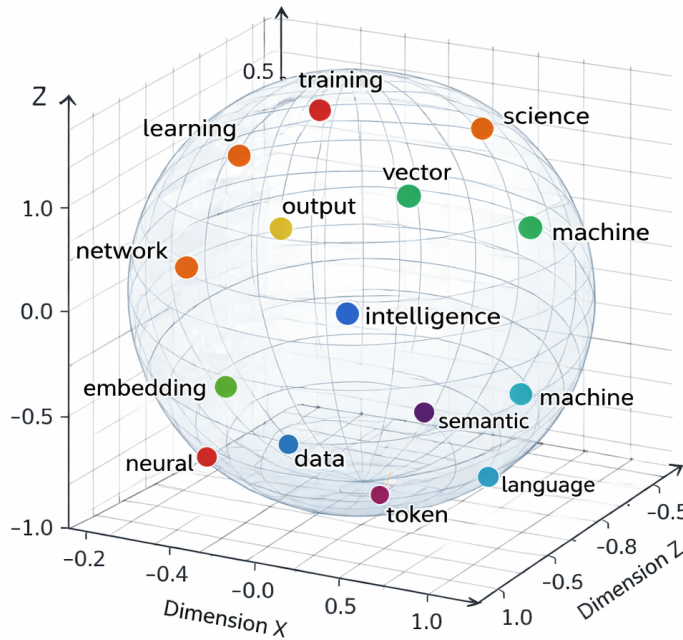


Figure 1.14: The representation of normalized embeddings in 3D embedding space

To normalize a vector $v = [v_1, v_2, \dots, v_n]$, we divide each component (each number in vector) by the vector's magnitude (or length):

$$v_{\text{normalized}} = \frac{v}{||v||}$$

Here $||v|| = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}$ is the length of the vector (L2 norm or Euclidean length).

For example, if we have a vector $\mathbf{v} = [3, 4]$, then the length (magnitude) will be 5 because $\sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$ and the normalized vector will be:

$$\mathbf{v}_{\text{normalized}} = [3/5, 4/5] = [0.6, 0.8]$$

Normalization is applied to embeddings for reasons such as ensuring stability in training, simplifying similarity measures, efficiently using space, and improving model generalization.

In most cases throughout this book, when we talk about embeddings, we'll be referring to normalized embeddings.

Generating embeddings

There are a few ways embeddings can be created. Let's explore some of the most widely used approaches:

One-hot encoding represents each token as a vector whose length equals the size of the vocabulary. Exactly one element in this vector is set to 1, indicating the presence of that specific token, and all the other elements are set to 0. For example, if we have a vocabulary ["cat", "dog", "snake"], then we can represent tokens in that vocabulary in this way: "cat" -> (1, 0, 0), "dog" -> (0, 1, 0), and "snake" -> (0, 0, 1). This way, each token has a unique vector representation, and no two tokens share the same position of 1s. One-hot embedding is very simple but has obvious limitations: the larger the vocabulary, the larger the embedding dimensions. It also doesn't capture relationships between tokens. Like in the example above, "cat" and "dog" have a stronger relationship than "dog" and "snake," but their embedding vectors have the same distance and direction.

Another approach is called **static embeddings**. In this method, embeddings are precomputed for every word or token in the vocabulary before the model is used. These embeddings are fixed, meaning the same word always maps to the same vector, no matter where it appears or what the surrounding words are. For example, the word "bank" would have a single embedding whether it appears in "river bank" or "bank account," even though the meaning is different in each case.

Popular methods for generating static embeddings include **Word to Vector (Word2Vec)** and **Global Vectors (GloVe)**, which learn these vectors by analyzing large text *corpora* and capturing statistical relationships between words.

Note

A **corpus** (*plural: corpora*) is simply a large collection of text that is used for linguistic analysis or training AI models. For example, a corpus might consist of millions of news articles, Wikipedia pages, or social media posts.

Static embeddings capture relationships between words based on their co-occurrence in a large text corpus (Wikipedia or Common Crawl). Words that frequently appear in similar contexts (e.g., "king" and "queen") will have vectors that are closer in the embedding space.

Static embeddings are computationally efficient because they can be precalculated in advance. They are stored as look-up tables (maps, hash tables, dictionaries). However, they do not account for the context. For example, "bass" (the fish) and "bass" (the musical instrument) share the same vector, even though they have different meanings and can be used in different contexts.

To overcome the limitations of static embeddings, **contextual embeddings** were introduced. Unlike static embeddings, these types of embeddings are generated dynamically at runtime. The representation of each word or token depends on its surrounding context in a sentence or document. This means that the same word can have different embeddings depending on how it is used; for example, "bank" in "river bank" will have a different embedding vector than "bank" in "open a bank account".

Typically, the contextual embeddings are part of the language model parameters and are trained during the training process. They are optimized along with other parts of the model. We will see that more in *Chapter 2*.

Sentence-level embeddings

So far, we have discussed embeddings for dedicated words or sub-words: tokens. These types of embeddings are called **word-level** or **token-level embeddings**. They are fundamental building blocks for LLMs, capturing the meaning of individual words or sub-words. However, many practical tasks require *sentence-level* or *text-level* embeddings to represent the meaning of larger text units. These higher-level embeddings are important for tasks like semantic similarity comparisons, classification of related sentences or documents, semantic search, RAG (see *Chapter 5*), and so on.

Sentence- and text-level embeddings capture the combined semantic context of a phrase, sentence, or entire document. Therefore, we can say that sentence-level embeddings are a mapping between text and a fixed-size vector, like here:

"In 1939, Enzo Ferrari founded a car company." $\rightarrow$ $[0.45, -0.32, \dots, -0.34]$

So, we mapped the entire sentence "In 1939, Enzo Ferrari ..." to one vector instead of splitting it into words (tokens) and mapping each word to a vector as we did for word-level embeddings.

Tip

Contextual embedding can be extremely useful in real-world applications. For example, imagine you're extracting action items from meeting transcripts by using applications like Google Meet, Zoom or Microsoft Teams and want to check whether similar tasks already exist in your database. If you rely on basic string matching, you may end up with many near-duplicates; items that are worded slightly differently but refer to the same task. This is where sentence-level embeddings can help. By storing the embeddings of existing action items in the database, you can compare them with the embeddings of new items from the transcript. This allows you to detect semantic similarity, even when the wording isn't identical.

There are a few ways to get sentence-level or arbitrary text-level embeddings:

- Pooling word embeddings: Aggregating word-level embeddings to represent the entire sentence or text
- Pretrained embeddings sentence models: Using models specifically trained for sentence embeddings, such as **Sentence-BERT (SBERT)** or text-embedding-3 from OpenAI

For example, consider the phrase "Red Ferrari" represented by two embedding vectors: [0.6, 0.8] for "Red" and [0.3, 0.9] for "Ferrari." A basic approach to obtain a sentence-level embedding is to sum up these vectors and compute their average.

$$\left[\frac{0.6 + 0.3}{2}, \frac{0.8 + 0.9}{2} \right] = [0.45, 0.85]$$

So, the [0.45, 0.85] can be taken as a sentence-level embedding for the "Red Ferrari" phrase.

Pooling word embeddings is a simple and intuitive approach to generate sentence-level representations, but it's often less effective compared to using pre-trained sentence embedding models.

On the other hand, sentence embedding models are trained specifically to map an entire sentence (or longer piece of text) into a single embedding vector. In other words, the input is an arbitrary piece of text, such as a sentence or paragraph, and the output is one vector that represents its overall meaning.

Similarity metrics

Once we have embeddings that represent words, phrases, or sentences as vectors, the next step is often to compare them. This is where *similarity metrics* come into play. They are essential tools for comparing embeddings by quantifying the degree of similarity between vectors. Common metrics include cosine similarity, Euclidean distance, dot product, and Manhattan similarity. Each offers a slightly different way of measuring closeness.

Cosine similarity

Cosine similarity measures the cosine of the angle between two vectors, focusing on their direction. It ranges from -1 (completely opposite) to 1 (identical) and is widely used for text embeddings. It is defined by the formula:

$$\text{Cosine similarity} = \frac{u \cdot v}{||u|| ||v||}$$

where $u \cdot v$ is the *dot product* of two vectors defined by the formula:

$$u \cdot v = \sum_{i=1}^n u_i v_i$$

and $||u||$ is a vector norm (L2 norm or Euclidean norm) defined by the formula:

$$||u|| = \sqrt{\sum_{i=1}^n u_i^2}$$

As an example, let's take two vectors $[0.8, 0.6]$ and $[0.79, 0.61]$. The dot product will be:

$$u \cdot v = (0.8 \times 0.79) + (0.6 \times 0.61) = 0.632 + 0.366 = 0.998$$

Next norm for the first and the second vectors will be:

$$||u|| = \sqrt{0.8^2 + 0.6^2} = \sqrt{0.64 + 0.36} = \sqrt{1} = 1$$

$$||v|| = \sqrt{0.79^2 + 0.61^2} = \sqrt{0.6241 + 0.3721} = \sqrt{0.9962} \approx 1$$

Then, the cosine similarity for $[0.8, 0.6]$ and $[0.79, 0.61]$ vectors will be approximately 0.998. These vectors are nearly aligned, in fact, almost identical in direction, which means very high semantic similarity.

Vector similarity can be calculated by using the numpy library in Python:

```
import numpy as np
from numpy.linalg import norm
# Define vectors
u = np.array([0.8, 0.6])
v = np.array([0.79, 0.61])
# Cosine similarity formula
cos_sim = np.dot(u, v) / (norm(u) * norm(v))
print(f"Cosine Similarity: {cos_sim:.4f}")
```

In the graphic, similarity metrics can be visualized in the following way: the blue and green vectors point in similar directions, indicating high cosine similarity, while the red vector points in the opposite direction, resulting in a negative cosine similarity:

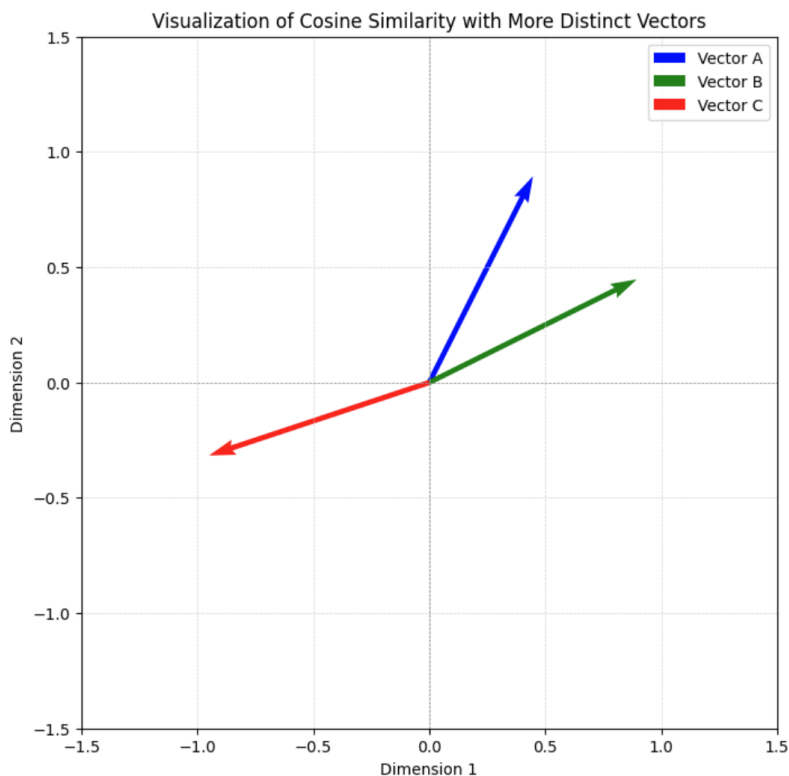


Figure 1.15: Representation of similar and opposite vectors in 2D space

One of the disadvantages of cosine similarity is that it measures only the direction (*angle*) between two vectors and does not account for their length (*magnitude*). This means that two

vectors with different lengths can still have the same cosine similarity if they share the same direction. For example, the 2D vectors $[3, 6]$ and $[1.5, 3]$ are different in magnitude but have the same cosine similarity equal to 1 because their direction is identical (angle = 0).

In the context of LLMs, this limitation is mitigated by using normalized embeddings, where all vectors are scaled to have length equal to 1 (*magnitude = 1*). With normalized embeddings, the direction of the vectors entirely determines their cosine similarity, ensuring that the measure reflects true semantic relationships without being influenced by differences in magnitude.

To compute cosine similarity, we need approximately $6 * n$ operations, where n is the dimensionality of the embeddings. This makes cosine similarity computationally more expensive compared to simpler metrics like Euclidean or Manhattan distance (see below). However, when working with normalized embeddings, the cosine similarity formula simplifies to the dot product between vectors since the norms of the vectors are equal to 1:

$$\text{Cosine Similarity} = \mathbf{u} \cdot \mathbf{v}$$

In this case, the calculation requires approximately $2 * n$ operations, which is faster than both Euclidean and Manhattan metrics. This optimization not only reduces computational cost but also preserves the focus on the semantic relationships between the embeddings, making cosine similarity a preferred choice in applications using normalized vectors.

Notice that the majority of pretrained embedding models, like text-embedding-3 (OpenAI), provide normalized embeddings.

Euclidean distance

Euclidean distance, known sometimes as **L2 distance**, calculates the straight-line distance between two points in space. It is useful for measuring absolute differences but may be sensitive to the scale of the embeddings. It is defined by the formula:

$$\text{Euclidean Distance} = \sqrt{\sum_{i=1}^n (u_i - v_i)^2}$$

To compute the Euclidean distance, we need approximately $3 * n$ operations, where n is the dimensionality of the embeddings. It is faster and more accurate when we work with un-normalized embeddings.

Manhattan similarity

Manhattan distance, also known as **L1 distance**, calculates the sum of the absolute differences between corresponding components of two vectors. It's often considered more robust to *outliers* compared to other distance metrics. It is defined by the formula:

$$\text{Manhattan Distance} = \sum_{i=1}^n |u_i - v_i|$$

where $|a|$ is the absolute value of number a .

Note

An **outlier** is a term frequently used in statistics to describe a data point that is significantly different from the rest of the data. For example, if most house prices in a neighborhood are between \$300K and \$500K, but one house sells for \$1.5M, that price would be considered an outlier.

Manhattan distance requires approximately $3 * n$ operations, making it slightly faster than Euclidean distance because it avoids the computationally expensive square root operation. The main drawback of Manhattan distance compared to Euclidean distance is that it lacks sensitivity to diagonal or straight-line relationships. It depends heavily on axis alignment and can be less intuitive in geometric interpretations, especially in high-dimensional spaces.

Summary

In this chapter, we explored the foundational ideas that power modern AI systems, from the basics of machine learning to the core building blocks of NLP. We covered what it means for a model to "learn" from data, introduced supervised, unsupervised, and reinforcement learning, and showed how training and inference work. You saw how neural networks extend simple models like linear regression, stacking layers of neurons to capture complex, non-linear patterns. We also introduced the idea of fine-tuning, where pre-trained models are adapted to new tasks, which is a key technique behind today's LLMs.

We then explored the key types of problems that AI systems are designed to solve, including regression, classification, clustering, ranking, recommendation, summarization, text generation, information extraction, and question answering. These tasks form the backbone of nearly every real-world AI application, from search engines to chatbots and recommendation systems.

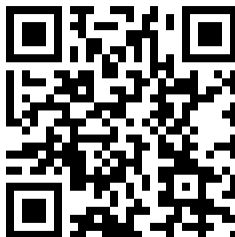
Finally, we introduced the core building blocks of NLP: tokenization, sentence segmentation, stemming, lemmatization, and stop words, followed by embeddings. You learned how words, sentences, and even entire documents can be converted into numerical representations that models can process and how similarity metrics like cosine similarity allow us to compare them meaningfully.

This chapter was meant as a top-level overview and refresher, enough to build intuition and a shared vocabulary, without getting lost in the details. If you're new to AI, this should give you the confidence to follow the rest of the book. And if you've worked with AI before, this section may serve as a quick but valuable refresher.

Armed with these fundamentals, you're ready to move to the next stage of the journey. In the next chapter, we'll dive into large language models and explore how they're built, how they work, and why they've revolutionized the field of AI. This is where the building blocks you've just learned come together to create systems that can reason, converse, and act in ways that were unimaginable just a few years ago.

Get this book's PDF version and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.

2

Understanding Large Language Models

The rise and rapid spread of **large language models (LLMs)** have sparked a revolution in AI. Practically overnight, these models became part of our daily lives: powering chatbots, productivity tools, and creative applications. This sudden shift unleashed a wave of curiosity and excitement: What exactly are LLMs? How do they work? Why do they seem so intelligent, and where are their limits?

The global response was enormous. News outlets, blogs, YouTube channels, and research communities lit up with content explaining, analyzing, and experimenting with these new capabilities. Academic interest surged as well; according to ArXiv Trends, the number of papers mentioning "*large language models*" and "*LLM*" skyrocketed after the release of ChatGPT in late 2022 [6]. It quickly became clear that we weren't just witnessing a breakthrough; we had entered a new era of AI.

In the previous chapter, we explored the core foundations of AI and NLP, from machine learning models and training methods to embeddings and similarity metrics. This chapter builds on that foundation by focusing on how LLMs actually work. We'll start by looking at the key predecessors to LLMs, such as sequence-to-sequence models, which laid the groundwork for modern generative AI. Then we'll explore transformers, the architecture that revolutionized NLP and made LLMs possible. Finally, we'll cover how LLMs are trained, run, and evaluated, giving you a practical understanding of their inner workings. By the end of this chapter, you'll have the intuition needed to design more effective prompts, configure LLMs for your applications, fine-tune LLMs, anticipate their behavior, and build smarter, more reliable AI systems.

We will cover the following key topics:

- Sequence-to-sequence models
- Train, run, and benchmark LLMs
- Multimodal models
- Using LLMs
- LLM families

One quick heads-up: this chapter may feel like a deeper dive, or even a little hacky at times, because we're tackling one of the most complex (and fascinating!) topics in the book: how LLMs work under the hood. But don't worry; we'll take it step by step, keeping explanations as clear and intuitive as possible. And if something doesn't fully click right away, that's okay as future chapters will revisit and reinforce many of these ideas as we move from theory into practical applications.

Technical requirements

The code for this chapter is available on the book's accompanying GitHub repo: <https://github.com/PacktPublishing/Building-Agent-Powered-Applications/tree/main/Chapter2>

Sequence-to-sequence models

If you're not an AI professional with years of experience, diving straight into the architecture of modern LLMs can be overwhelming, and it's easy to get lost. These models are incredibly complex under the hood. To make the learning curve more approachable, we'll start by exploring some of the foundational models that paved the way for today's LLMs. One of these key building blocks was the sequence-to-sequence, or Seq2Seq, model.

Sequence-to-sequence (Seq2Seq) models were among the first architectures in deep learning, designed for tasks that involve mapping one sequence to another. This architecture was used for tasks like machine translation, text summarization, and conversational AI. Seq2Seq models became the foundation for many of the next developments in NLP and are particularly important in understanding the evolution of modern architectures like transformers used for LLMs. For instance, Google Translate used the Seq2Seq architecture for translation tasks from 2016 to around 2020.

Seq2Seq models are designed specifically for *sequential data*, that is, data where the order of elements matters because each element may depend on those that came before. In natural language, for example, the meaning of a word often relies on the words that preceded it, making text inherently sequential. Other common examples of sequential data include time series like stock prices, where each value is influenced by previous values.

One more example of sequential data is the sequence of image frames in a video. These frames appear in a specific order, and the content of each frame is closely related to the ones before and after it. This kind of time-ordered dependency is a defining characteristic of sequential data. Many other types of data, like speech and sensor signals, also follow a sequence where order matters. To analyze such data effectively, we need models that can learn from these patterns over time.

In contrast, many types of data are not sequential and don't rely on any specific order. For example, imagine we want to predict the prices of 10 houses. Each house is independent of the others, so we can predict the price for the first house, then the second, and so on. Each prediction can be made separately. In fact, we could start with the 5th house, then move to the 1st, and it wouldn't affect the results. The same idea applies if we need to identify animals in several different pictures; each image can be analyzed on its own, in any order, without impacting the outcome.

Today, Seq2Seq models are not as widely used, having been largely replaced by transformer-based architectures (see next sections). However, they remain a valuable learning tool. Understanding Seq2Seq models helps build intuition about key concepts in language models, such as encoder-decoder architectures, auto-regressive generation, and attention mechanisms. Seq2Seq models consist of two main components: an encoder and a decoder.

Note

Historical note

Statistical models such as **Hidden Markov Models (HMMs)** (1960s–1980s), **Autoregressive Integrated Moving Average (ARIMA)**, and others were among the first models designed for sequential data. These models processed data sequentially, making predictions based on patterns and dependencies in the sequence. These models have built a foundation for the *auto-regressive generation* commonly used today in LLMs (see "Encoder and Decoder" below).

Encoder

In Seq2Seq models, the **encoder** processes the input sequence and compresses it into a fixed-size vector called a **context vector**. The input for the encoder is the sequential data. For text data, it will be the list of token Ids obtained after text tokenization (see *Chapter 1*).

For instance, if we need to translate a text from English to Spanish, the encoder will transform the input tokens (token Ids) into an embedding vector, such as:

$$\text{"Enjoy the show"} \xrightarrow{\text{tokenizer}} [1503, 234, 5465] \xrightarrow{\text{encoder}} [0.25, -0.06, 0.04, \dots]$$

This process might remind you of sentence-level embeddings, which we covered in *Chapter 1*. Both approaches produce a single vector to represent a piece of text. However, the key difference lies in their purpose. Sentence-level embeddings are designed to help compare pieces of text. For example, to check if two paragraphs discuss similar topics. In contrast, the encoder's context vector is created to support downstream tasks, such as translation or summarization. Its goal is not just to capture meaning, but to provide all the necessary information for the decoder to generate the correct output.

To get the context vector, the encoder first takes the embeddings of each token in the input ([1503, 234, 5465] in the case above). Then it applies a *transformation* to combine them into a single vector. This transformation is usually complex, but to keep it simple, imagine the encoder just adds all the token embeddings together. The result is a single vector that represents the whole input text, similar to how we created sentence-level embeddings in *Chapter 1*.

For example, consider the sentence "Enjoy the show." The context vector in this case would be a transformation that combines the embeddings of all three tokens: "Enjoy", "the", and "show". The resulting vector represents the meaning of the entire sentence and occupies its own position in the embedding space, just like individual word embeddings.

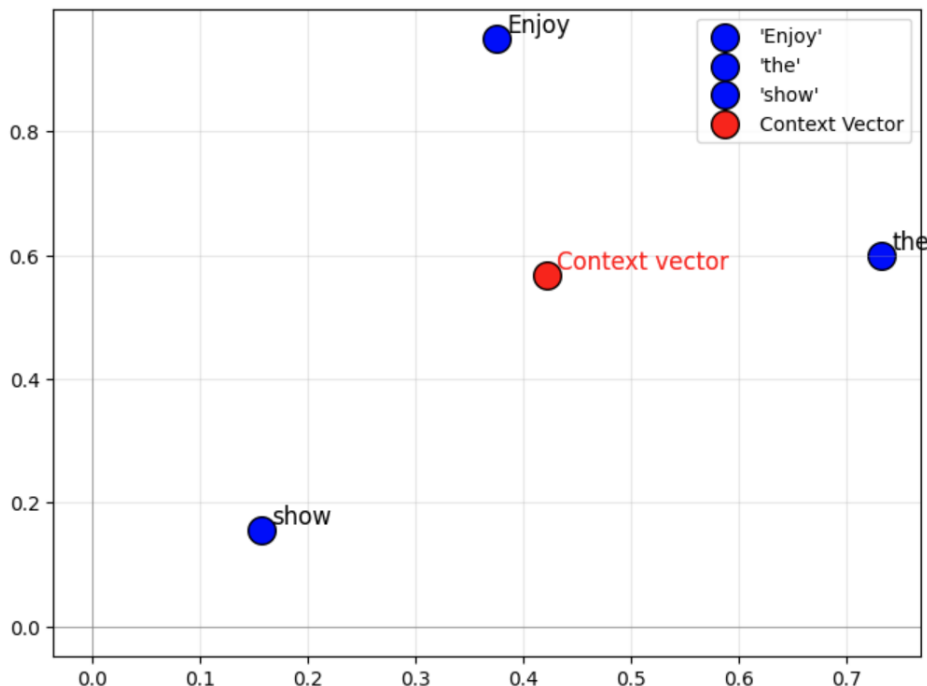


Figure 2.1: An example of a context vector created from token embeddings

In a Seq2Seq model, the encoder is typically built using a special type of neural network called a **recurrent neural network (RNN)**. Unlike standard neural networks that process inputs independently, RNNs are specifically designed to handle sequential data, where the order of elements matters. They do this by maintaining a kind of "memory" of previous inputs as they process each new element in the sequence.

To illustrate how an RNN works, let's consider a simple example: a video of a boy throwing a stone. Our goal is to predict where the stone will land, using only the first part of the video, up to the point where the stone is still in the air. As mentioned earlier, a video is made up of a sequence of images (frames), ordered over time. Each frame gives us a snapshot of the scene at a specific moment. So, we can treat the video as sequential data.

We start by feeding the first frame into the model. From that, we might detect the initial position of the stone. Then we input the second frame, which shows the stone beginning to move. Now, by comparing the two frames, we can estimate the direction of the stone's motion. At this point, we've learned something useful: the stone's trajectory. We store that information (trajectory) in what we call a **state**, which is essentially the model's *memory* of what it has seen so far. As we feed in more frames, the RNN continues to update this state (trajectory) based on

the new information and everything it has already learned. Eventually, it uses that accumulated knowledge about the trajectory to predict where the stone will fall.

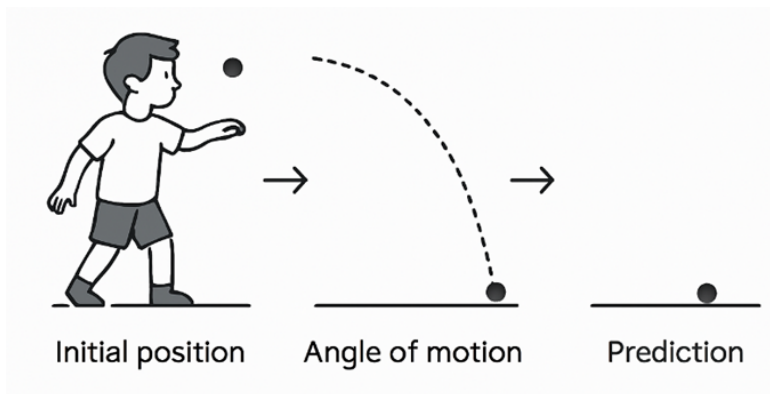


Figure 2.2: Illustration of an RNN to predict object movement

This is the key idea behind RNNs: they process one item in the sequence at a time, and at each step, they remember what came before. This memory, or state, helps them make better predictions for tasks where earlier context matters.

For text data, we work with a sequence of tokens obtained after tokenization. The RNN model processes these tokens one at a time. It starts with the first token and generates an initial state. Then, for each subsequent token, it combines the current token with the previous state to produce a new state. This process continues through the entire sequence, allowing the RNN to build up a context that reflects the information from all tokens. The final state can be seen as a summary or *context vector* of the entire input sequence.

Today, building models like Seq2Seq with RNNs is much easier due to the capabilities provided by modern machine learning frameworks. Below is a simplified example of how to implement an encoder using PyTorch (intended for demonstration purposes only):

```
import torch.nn as nn
class Encoder(nn.Module):
    def __init__(self, input_size: int, embedding_dim: int, hidden_dim: int):
        """
        input_size: vocabulary size (e.g., 10000)
        embedding_dim: size of each embedding vector (e.g., 256)
        hidden_dim: size of hidden layer in GRU (Gated Recurrent Unit) (e.g., 512)
        """
        super(Encoder, self).__init__()
        self.embedding = nn.Embedding(input_size, embedding_dim)
```

```
self.rnn = nn.GRU(embedding_dim, hidden_dim)

def forward(self, token_ids):
    embeddings = self.embedding(token_ids)
    _, state = self.rnn(embeddings)
    return state # Context vector is the hidden state of the last layer
```

For the purpose of this book, we don't need to dive into the low-level details of how this code works. What matters is developing a high-level understanding and intuition for how such models are structured. In this example, the encoder consists of two main components: an embedding layer (`self.embedding`) and a recurrent layer (`self.rnn`), implemented using a **gated recurrent unit (GRU)**. At this point, it's enough to know that a GRU is a type of RNN, a variant designed to handle sequential data efficiently.

The *forward* method above takes a sequence of token IDs as input, converts them into embeddings using the embedding layer, and then feeds those embeddings into the GRU. The method returns the hidden state from the RNN, which serves as the context vector. This vector captures the semantic meaning of the entire input sequence (tokens) and will be passed to the decoder for further processing.

Note

In PyTorch and most deep learning frameworks, the `forward()` method defines the forward pass of the model, how input data moves forward through the network's layers to produce an output. It's called forward because it represents the first phase of computation (as opposed to the backward pass).

So, roughly, the encoder makes some transformations of input encodings, creating another encoding (*context*) vector in an n-dimensional embedding space. Since the embedding space is extremely sparse (see *Chapter 1*), a new context vector presents a new "entity" there.

Decoder

Now let's look at the second main component of a Seq2Seq model: the decoder. The **decoder** takes the context vector produced by the encoder and generates the output sequence one token at a time. At each step, it predicts the next token based on the context vector and the tokens it has already generated.

This step-by-step process is known as **auto-regressive generation**. In *auto-regressive models*, each token is predicted using the outputs from the previous steps. This allows the model to capture dependencies within the sequence effectively, making it well-suited for tasks involving ordered data. Auto-regressive generation is a core technique in modern language models like

GPT or Claude, where text is produced token by token, with each new word conditioned on the ones that came before it.

Note

This explanation is intentionally simplified. In real LLM implementations, autoregressive generation involves additional optimizations and complexities, such as batching, advanced parallelism, and caching of intermediate results, that make it far more efficient than the basic description here. For the purposes of this book, we'll focus on the core idea, which is enough to build a solid understanding and intuition of modern LLMs.

For instance, in translation tasks, the decoder predicts the first word in the target language, then uses that prediction to generate the next word, continuing until an end-of-sequence is produced. This sequential process allows the decoder to maintain logical structure and meaning in the output.

Below is a simplified implementation of a decoder using PyTorch (intended for demonstration purposes only):

```
import torch.nn as nn
class Decoder(nn.Module):
    def __init__(self, output_size: int, embedding_dim: int, hidden_dim: int):
        """
        output_size: size of the output vocabulary
        embedding_dim: size of the embedding vectors
        hidden_dim: size of the hidden state in the GRU
        """
        super(Decoder, self).__init__()
        self.embedding = nn.Embedding(output_size, embedding_dim)
        self.rnn = nn.GRU(embedding_dim + hidden_dim, hidden_dim)
        self.fc_out = nn.Linear(hidden_dim, output_size)

    def forward(self, input_token, state):
        embedded = self.embedding(input_token)
        output, state = self.rnn(embedded, state)
        output = self.fc_out(output)
        return output, state
```

We can see that the decoder also has embeddings (for outputs), an RNN, and additionally a linear fully connected (dense) layer (`self.fc_out`) to produce the final output. The forward method first gets embeddings from inputs (while training or inference), next passes inputs to the RNN (similarly to the encoder), and finally retrieves output that is a probability distribution vector for output tokens. Such an output vector is also called **logits**.

To illustrate encoder and decoder work, let's consider a very simplified case when our input and output vocabulary have only items to translate the English phrase "Enjoy the show" into Spanish. Note that the values in the example below are not actual calculated results; they are provided solely for demonstration purposes:

```
# SOS - start of sentence, EOS - end of sentence
input_vocabulary = ["<SOS>", "<EOS>", "Enjoy", "the", "show"] # English - tokens
output_vocabulary = ["<SOS>", "<EOS>", "Disfruta", "del", "espectáculo"] #
Spanish

input_size = len(input_vocabulary) # 5
output_size = len(output_vocabulary) # 5
# Initialize encoder and decoder
encoder = Encoder(input_size, embedding_dim=3, hidden_dim=3)
decoder = Decoder(output_size, embedding_dim=3, hidden_dim=3)
# Tokenized input: <SOS> Enjoy the show <EOS>
token_ids = [0, 2, 3, 4, 1]
# Encode the input sequence
context_vector = encoder(token_ids) # output: [0.27, -0.95, 0.15]
# <SOS> token is used to start decoding the output sequence
sos_token_id = 0
# Get the first output token
logits, context_vector = decoder(sos_token_id, hidden=context_vector)
# Logits=[0.73,0.23,0.15,0.13,0.28], context_vector=[0.38,-0.92,0.03]
first_output_token_id = logits.argmax(1) # output: 0
# Get the second output token
logits, context_vector = decoder(first_output_token_id, context_vector)
# Logits=[0.12,0.29,0.81,0.05,0.21], context_vector=[-0.26,-0.81,0.17]
second_output_token_id = logits.argmax(1) # output: 2 (Disfruta)
# Get the third output token
logits, context_vector = decoder(second_output_token_id, context_vector)
# Logits=[0.01,0.31,0.25,0.79,0.11], context_vector=[0.17,0.77,-0.52]
third_output_token_id = logits.argmax(1) # output: 3 (del)
# Final output sequence (token IDs)
output = [0, 2, 3, 4, 1] # <SOS> Disfruta del espectáculo <EOS>
```


In this example, we first define the input and output vocabularies (lines 2–3), including special tokens: <SOS> (start of sentence) and <EOS> (end of sentence). These tokens help the model understand where the sentence begins and ends.

We then initialize the encoder and decoder (lines 7–8), setting both the input and output vocabulary sizes to 5. The embedding and hidden dimensions are set to 3 purely for demonstration purposes; they're intentionally small to keep the example simple and easy to follow. We use the indices of the vocabulary as token IDs. For example, the input token IDs [0, 2, 3, 4, 1] correspond to the English sentence <SOS> Enjoy the show <EOS>, where <SOS> is index 0, "Enjoy" is 2, "the" is 3, "show" is 4, and <EOS> is 1.

Next, we compute the context vector (line 12) by passing the input token IDs into the encoder. This vector summarizes the meaning of the entire input sequence and serves as the starting point for decoding. Next, we begin decoding the output sequence on line 16. The decoder takes the <SOS> token ID (serving as an initial state) and the context vector as input and returns a set of *logits*, i.e., values that represent the likelihood of each possible output token. The token with the highest score is selected. In this case, the highest logit value is 0.73, corresponding to token ID 0 (<SOS>), so that becomes the first predicted output token.

We then repeat the process for the next token (line 20), this time passing the previously predicted token (<SOS>) and the updated context vector back into the decoder. The model now incorporates what it has already generated. This time, the highest logit is 0.81, which corresponds to the token with ID equal to 2, or the "*Disfruta*" word in the output vocabulary. This step-by-step process continues for the remaining tokens. Eventually, the full output sequence [0, 2, 3, 4, 1] is generated, which translates to <SOS> Disfruta del espectáculo <EOS>.

Note

Note that the decoder, and more generally, any language model, doesn't directly return the ID of the next token. Instead, it returns a probability distribution over all possible tokens, typically in the form of an array of *logits*, such as `logits = [0.73, 0.23, 0.15, 0.13, 0.28]` for our case. This highlights a fundamental difference between AI models and traditional deterministic code: AI models produce probabilistic outputs. As a result, their predictions can vary.

In real-world applications, vocabulary is typically much larger, ranging from 10,000 to 200,000 tokens or more, and embedding and hidden dimensions are significantly bigger (usually between 256 and 1024 or higher). These larger sizes help the model capture more complex semantic and contextual relationships.

Now we can summarize that in Seq2Seq models, the encoder and decoder play the following roles:

- **Encoder:** Converts the input sequence into a context vector that captures the overall input meaning
- **Decoder:** Uses this context vector, along with its own evolving hidden state, to generate the output sequence token by token

Note

Note that further development of Seq2Seq models extended the encoder beyond producing a single context vector. This change addressed the fundamental limitation of compressing a large amount of information into one fixed-size representation. Instead of generating only one summary vector, newer Seq2Seq architectures produce a sequence of context vectors, one for each input token.

Transformers

Seq2Seq models had several significant limitations:

- **Difficult to scale and parallelize:** They rely on sequential RNNs, which process tokens one at a time. This makes both training and inference (usage) slower and less efficient, especially when working with large datasets or long texts.
- **Struggled with long sequences:** RNN-based models often have trouble handling longer sequences, such as texts with 500 to 1000+ tokens. It's because of limitations in preserving context over time.

Transformer architecture was introduced to overcome the limitations of Seq2Seq models, particularly their reliance on RNNs. Unlike RNNs, which process tokens one at a time in sequence, transformers can analyze the entire input simultaneously using a powerful mechanism called *self-attention*.

Self-attention is a natural evolution of the attention mechanism first introduced in Seq2Seq models. In traditional attention, the decoder learns to selectively focus on parts of the input sequence while generating each token. Self-attention takes this idea further: every token in a sequence can attend (focus) to every other token, including itself. This allows the model to capture relationships between all parts of the sequence, regardless of their position.

In simple terms, self-attention generates a table (matrix) that shows how much each token in a sequence should pay attention to every other token, including itself.

For example, for the sentence, "The cat sat on the mat," such a matrix can look like the following (not real weights, for demo purposes only):

Token	The	cat	sat	on	The	mat
The	0.4	0.3	0.1	0.1	0.05	0.05
cat	0.2	0.5	0.2	0.05	0.025	0.025
sat	0.1	0.3	0.3	0.2	0.05	0.05
on	0.05	0.1	0.2	0.4	0.15	0.1
the	0.1	0.05	0.1	0.2	0.4	0.15
mat	0.05	0.05	0.1	0.2	0.2	0.4

Table 2.1: Example of a self-attention matrix

The preceding table shows how each word attends to others in the sentence. For example, "cat" places a high attention value on itself, as well as on the words "The" and "sat", which are closely related in meaning and structure. Similarly, "mat" assigns more attention to "on" and "the", reflecting their connection in the phrase 'on the mat'.

And because self-attention doesn't rely on step-by-step processing, it can be fully parallelized, making transformers dramatically faster and more scalable than RNN-based models. This ability to handle entire sequences in parallel is one of the key reasons transformers became the foundation of today's large-scale language models.

To illustrate the self-attention mechanism in action, consider another example with the following sentence: "The cat sat on the mat because it was warm."

Now ask yourself: What does "it" refer to? Similarly, as we showed in the table above, self-attention helps the model figure that out. When processing the word "it", the model doesn't just look at the word before it. Instead, self-attention allows the model to look at all the other words in the sentence and decide which ones are most relevant.

In this case, the model learns that "mat" is strongly related to "it", while words like "cat" or "sat" are less relevant. The attention mechanism assigns higher *attention scores* to the important words. So instead of reading left to right, one word at a time (like an RNN), the transformer sees the whole sentence at once and decides what to focus on.

So, by resolving scaling challenges, transformers enabled the development of much larger models, which significantly improved their ability to solve a wide range of generic tasks. This scalability was one of the key factors behind the transformer's revolution in NLP.

Note

In the case of transformers, the "*law of large models*" proved true once again. Before the rise of deep learning, AI researchers often focused on designing sophisticated and problem-specific models. Success depended heavily on clever architecture, handcrafted features, and domain-specific heuristics. But deep learning introduced a new paradigm: scale. It showed that with enough data, compute, and model capacity, even relatively simple architectures could outperform complex, handcrafted solutions. The transformer architecture amplified this trend, demonstrating that scaling up model size, training data, and compute leads to dramatically better performance across a wide range of tasks.

The size of large transformers today is hundreds of billions of parameters and even trillions. For instance, GPT-3 has around 175B (billion) parameters, and GPT-4 around 1.3 trillion (not officially confirmed).

Transformers are significantly more complex and less intuitive compared to Seq2Seq models. Some transformer models, such as BERT, include only the encoder component, while others, like GPT, have only the decoder. Models like T5 incorporate both encoder and decoder components. Despite these variations, we can notice that transformer architecture shares several similarities with Seq2Seq models: encoder, decoder, autoregressive generation (token-by-token), and an attention mechanism (specifically self-attention).

Let's look at a high-level overview of the original Transformer architecture introduced in one of the most popular AI papers, "Attention Is All You Need" [7]. For the purposes of this book, we've simplified the diagram by omitting some components to keep things clear and focused.

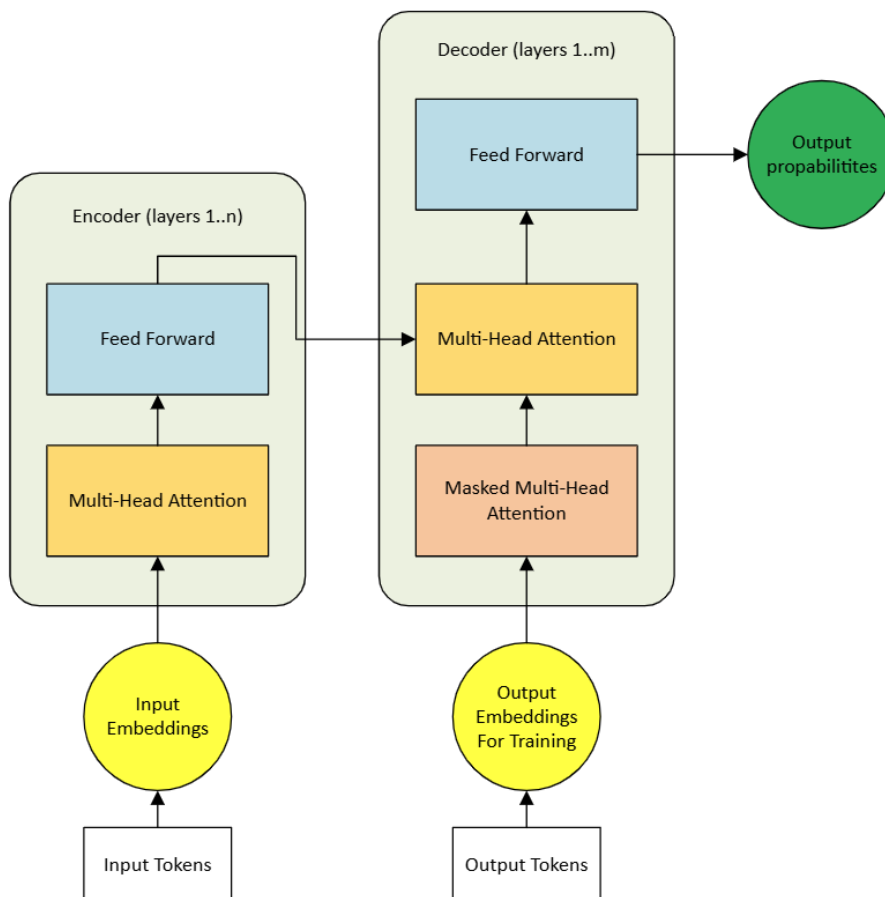


Figure 2.3: Transformer architecture (simplified)

In the preceding figure, we can see that both the encoder and the decoder consist of multiple stacked layers. Each layer contains a *multi-head attention* block followed by a *feed-forward network*. In addition, the decoder includes a *masked multi-head attention* mechanism.

The input sequence begins as token IDs, which are first converted into embeddings. Then these embeddings are passed into the encoder. Once processed by the encoder, the enriched representations (contextual embeddings) are passed to the decoder. Finally, the decoder produces a probability distribution over the vocabulary, representing the likelihood of each possible next token.

For the training pipeline, the expected output tokens are also used. These target tokens are fed into the decoder as input embeddings. The masked multi-head attention ensures that each position can attend only to previous tokens and not future ones. This allows the model to learn how to generate the next token step by step, while still processing the entire sequence in parallel during training.

So, we can observe that the high-level transformer architecture is an evolution of earlier models and shares some conceptual similarities with Seq2Seq architectures. Like Seq2Seq models, transformers begin with tokenization and embeddings, perform transformations on those embeddings, and ultimately produce output token probabilities in an auto-regressive manner, predicting one token at a time.

However, the key difference is that transformers are designed to process all tokens in a sequence simultaneously using self-attention, allowing for parallelization during training. In contrast, RNNs (used in Seq2Seq models) process tokens sequentially, which limits scalability and training speed.

Additionally, transformers apply much deeper and more complex transformations to embeddings through stacked attention layers and feed-forward networks, whereas RNNs rely on simpler recurrence mechanisms to pass information through the sequence.

Train, run, and benchmark LLMs

As we learned in *Chapter 1*, the capabilities of any machine learning model, including large language models, are shaped by the data they're trained on. When writing traditional software, we define behavior manually, rule by rule, and case by case. In machine learning, it's the opposite: data defines behavior. The patterns and relationships found in the training data determine what the model learns and how it performs.

LLMs are no exception. These models don't understand language in the way humans do; they learn by processing massive amounts of text and identifying statistical patterns across billions of examples. The quality, scale, and diversity of this training data directly impact everything that LLMs can do.

In this section, we'll explore how LLMs are trained, what data goes into them, and how their performance is measured through benchmarks. We'll also cover the basics of how language models are run and used. Understanding this process is key to interpreting what these models can and cannot do.

Broadly, LLMs possess two core capabilities: *reasoning* (e.g., solving problems, logical inference) and recalling *factual knowledge* (remembering facts). Both capabilities are acquired during the training process. The model learns patterns, relationships, and facts from vast amounts of text data.

Typically, large language models are trained in multiple stages. The first stage is self-supervised pretraining on massive text datasets. During this phase, the model is exposed to vast amounts of text, such as web pages, books, articles, and other publicly available sources, and learns to predict missing or next tokens. Through this objective, the model captures patterns, relationships, and structural properties of language without requiring manually labeled data. Next, usually LLMs are fine-tuned on task-specific datasets to perform well in tasks like question answering, content generation, summarization, sentiment analysis, and so on. Finally, some LLMs like GPT-3.5 and GPT-4 use **reinforcement learning from human feedback (RLHF)** to align LLMs with human preferences.

Training objectives

To train a language model, we need to define what exactly it should learn; that's where the *training objectives* come in. The objectives guide the model during training, telling it what kind of predictions to make. In the case of LLMs, the most common objectives are **causal language modeling (CLM)** and **masked language modeling (MLM)**. These two approaches shape how the model learns from data and what it becomes good at.

In CLM, the goal is to predict the next token in a sequence based solely on the tokens that preceded it. Formally, given a sequence of tokens $[t_1, t_2, \dots, t_n]$, the model learns to predict the next token t_{n+1} .

For example, let's consider the sentence: "*Michael Jackson was a singer*". We provide the model the first part of the sentence: "*Michael Jackson was a*", and the model needs to predict that the next token is "*singer*", not any other word. The full sentence, including the word "*singer*", serves as the correct answer or expected output (*ground truth*), while the partial sentence without the word "*singer*" serves as the model input.

This approach aligns with the natural flow of language, making causal language modeling suitable for tasks like text generation. We can also see that in this way, the model can be self-supervised without active human labeling and data annotations (creating correct answers).

Models trained with CLM often use a transformer *decoder-only* architecture like GPT, LLaMA, and Mistral. In these models, masked self-attention (see above) ensures that each token can attend (correlate) only to the tokens that precede it, creating a *unidirectional context*. This means the model can "look" backward to incorporate context from all previous tokens but cannot "look" forward.

Causal language modeling is particularly well-suited for tasks like text generation, including storytelling, creative writing, dialog systems (chatbots), code generation, and summarization. Because it trains models to predict the next token in a sequence, it aligns naturally with generative use cases. CLM has also become the most widely used training objective for modern large language models.

Another training objective used in LLMs is **masked language modeling (MLM)**. In MLM, the goal is to predict tokens masked (hidden) in text based on the surrounding context. For example, let's have a sentence:

Angelina Jolie is an actress known for her humanitarian work and starring roles in movies like Maleficent and Tomb Raider.

Now we randomly mask some words (15% of tokens) like this:

```
"Angelina Jolie is an [MASK] known for her [MASK] work and starring roles in  
[MASK] like Maleficent and Tomb Raider."
```

where [MASK] is a hidden token. So, the model needs to predict that the first [MASK] = "actress", the second [MASK] is "humanitarian", and the third is "movies".

Masked language modeling has traditionally been considered more suitable for tasks that require language understanding rather than generation. It has been widely used in models like BERT, which worked relatively well at tasks such as text classification, **named entity recognition (NER)**, and semantic similarity. However, recent LLMs trained with causal language modeling have demonstrated comparable or even superior performance on many of these understanding tasks, largely due to their increased scale and generalization capabilities.

LLM training datasets

LLMs are trained on diverse datasets containing gigabytes to terabytes of text data, sourced from web crawls (web pages), books, encyclopedias (Wikipedia), scientific papers (arXiv), code repositories (GitHub), and more. Below is a list of some of the most widely used datasets:

Dataset	Released	License	Size	Used In *	Description
Common Crawl	2008	Public Domain	~250 TB	GPT-3, GPT-4*, LLaMA	Web crawl data contains over 250 billion web pages, metadata, and text extracts spanning 17 years. Collected by automated crawlers on a monthly basis, it captures web pages in over 40 languages.
Books Corpus	2015	MIT	~6 GB	BERT, GPT-2, GPT-3	A collection of free novel books written by self-published authors (via Smashwords), which contains 11,038 books (around 74M sentences and 1G words) in 16 different sub-genres
Wikipedia	2001	CC BY-SA 3.0	~20 GB	GPT models*, BERT, LLaMA	Comprehensive factual text in multiple languages from the Wikipedia encyclopedia.
GitHub Code Datasets	Varies	Mixed Licenses	~1TB	Codex, GPT models*	The dataset consists of 115M code files from GitHub in 32 programming languages with 60 extensions, totaling 1TB of data. The dataset was created from the public GitHub dataset

Table 2.2: The list of the most popular datasets used to train LLMs

Note

* The datasets listed here are commonly cited in research papers and model documentation as sources for training language models. However, many LLM developers, including OpenAI for GPT-4 and later models, have not publicly disclosed their exact training datasets. The information in *Table 2.2* should therefore be treated as *reasonable assumptions* rather than confirmed facts. In practice, datasets are often filtered, deduplicated, and preprocessed before training, which may significantly change their final composition.

Training LLMs on large text datasets gives them a strong foundation in language. This first stage is called **pretraining**, and the models produced are known as **pretrained** models. However, pretraining alone is often not enough to perform specific tasks well. To improve performance, these models are usually *fine-tuned* on task-specific datasets for things like question answering, content generation, summarization, sentiment analysis, and more.

As we learned in *Chapter 1*, **fine-tuning** refers to the process of taking a pretrained model and continuing its training on a smaller, specialized dataset tailored to specific tasks or domains. Unlike pretraining, which aims to build broad language understanding from large, diverse data, fine-tuning helps the model adapt to targeted use cases, improving its performance in specific applications.

Here is the list of some common tasks solved by fine-tuning and their datasets:

Task	Dataset	Dataset description	Training objective	Used in *
Summarization	CNN/Daily Mail	News articles paired with multi-sentence summaries (~300K examples, ~1GB)	Generate concise summaries of news articles	GPT-3.5*, GPT-4*, BART, T5, Pegasus

Task	Dataset	Dataset description	Training objective	Used in *
Question answering	Stanford QA Dataset (SQuAD)	Over 100K questions and answers annotated on Wikipedia passages (~50 MB)	Predict answers to questions given a context passage	GPT-3.5*, GPT-4*, BERT, RoBERTa, T5
Sentiment analysis	IMDB reviews	50,000 movie reviews labeled as positive or negative (~80 MB).	Classify the sentiment (positive/negative) of movie text reviews	GPT-3.5, BERT, RoBERTa, DistilBERT
Text classification	AG News	120K categorized news articles across 4 classes (~20 MB).	Classify text into predefined categories such as news topics (e.g., sports, politics)	GPT-3.5, BERT, DistilBERT, XLNet
Machine translation	WMT (e.g., WMT16 English-German)	Bilingual text pairs for machine translation; varies by year and language (~10M sentence pairs, ~10GB).	Translate text from one language to another (e.g., English to German)	GPT-3.5, GPT-4, MarianMT, T5

Task	Dataset	Dataset description	Training objective	Used in *
Code generation	GitHub code datasets	Millions of publicly available GitHub repositories (~1TB+).	Generate code snippets from text descriptions or complete code	OpenAI Codex, GPT-3.5, GPT-4

Table 2.3: Datasets used for LLMs fine-tuning

Note

* Similarly, as for pretraining datasets we must note that LLM developers often do not disclose the exact datasets used to fine-tune their models. So, this dataset list should be viewed as *reasonable assumptions* — not complete or confirmed sources.

Another important technique often used to improve LLMs is **reinforcement learning from human feedback (RLHF)**. As we learned in *Chapter 1*, reinforcement learning is based on the idea of learning through model interaction and reward. RLHF applies the same principle. It uses feedback from humans.

Roughly, RLHF makes LLM "more human-like". LLM pretraining on large text datasets (first stage) often can train models to generate outputs that are irrelevant, unsafe, or inconsistent with user intent. RLHF addresses this gap by incorporating human preferences into the training process. Human annotators (regular people) provide feedback by ranking model outputs for specific prompts (questions for LLMs), creating a dataset of preferences. This dataset is then used to train a *reward model*, which guides the LLM to produce outputs that align with human expectations.

So, at a high level, the process of training an LLM can be described by the following schema:

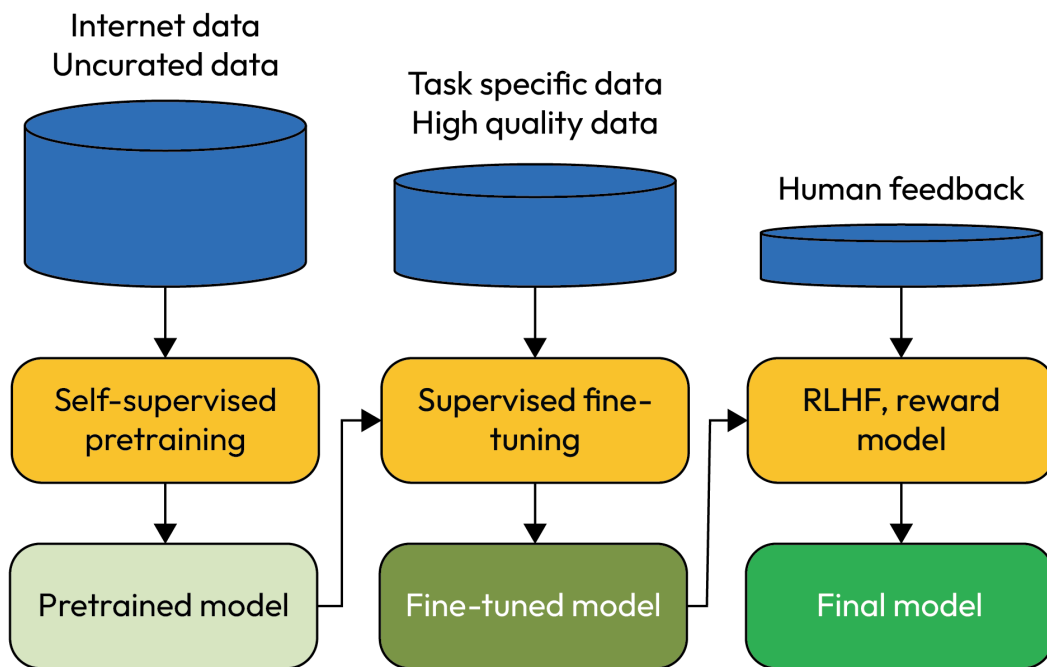


Figure 2.4: Typical three-stage training process for LLMs: pretraining, supervised fine-tuning, and RLHF

Note that the initial pretraining stage requires the most computational resources and extremely large datasets, which are typically not curated due to their scale. Fine-tuning is less resource-intensive but depends on high-quality, task-specific data. The final stage, human feedback, requires manual annotation, making it the most labor-intensive.

Benchmark LLMs

Once a large language model is trained, the next critical step is to evaluate how well it performs. This is where *benchmarking* comes in. Just like we use tests to measure a student's performance, benchmarks are standardized tasks designed to assess a model's capabilities. They help compare different models, track progress over time, and reveal strengths and weaknesses.

For classical machine learning problems like regressions and classifications, the benchmarking process is usually straightforward. Developers create a dataset or use an existing one and split it into two parts: a *training set* (typically 80–90%) and a *test set* (usually 10–20%). The training set is used to train the model, while the test set is used to evaluate how well the model performs on unseen data.

Note

As we highlighted in *Chapter 1*, testing a trained model on data dedicated and separated from training data is a fundamental principle in machine learning and AI. This practice helps to prevent **overfitting**, a situation where the model performs well on the data it saw during training but fails to generalize to new, unseen examples.

For example, imagine we are building a model to predict house prices, a problem we explored earlier in *Chapter 1*. Let's say we have a dataset containing information on 1,000 houses that have already been sold. We randomly select 800 houses (80%) for training and use the remaining 200 houses (20%) for evaluation. After training, we test the model on the 200 unseen houses. Suppose the model predicted prices accurately for 180 houses and failed to predict accurate prices for 20. In this case, we can say the model's accuracy is $\frac{180}{200} = 90\%$.

Here, accuracy is our evaluation metric; it tells us how often the model made correct predictions on the test set. With 90% accuracy, we can expect that, on average, the model will produce 9 correct predictions out of 10 and fail on about 1 out of every 10 cases. This brings us back to a fundamental characteristic of machine learning models that we briefly discussed in *Chapter 1*: their probabilistic nature.

For LLMs, benchmarking is especially important because these models are expected to perform well across a wide range of tasks, such as reasoning, coding, summarization, and question answering. However, it's also important to understand that benchmarking LLMs is a very complex challenge in itself. Unlike traditional models that are evaluated on narrow, well-defined tasks, LLMs are general-purpose systems with capabilities that are difficult to fully capture in a single test.

Despite the growing number of benchmarks available today, they often reflect narrow task definitions, static test sets, and sometimes artificial constraints. As a result, benchmark scores don't always translate into reliable performance in real-world use cases.

Two models might perform similarly on a benchmark but behave very differently when integrated into actual applications. Yet, benchmarks remain the best available method for evaluating and comparing LLMs today. So, while they offer valuable signals, they should be interpreted with caution and complemented by task-specific testing whenever possible.

Let's take a look at some GPT models' benchmarks as of April 2025, released by OpenAI [9]:

Dataset Model	MMLU	GPQA	MATH	HumanEval	MGSM	DROP	Simple QA
GPT o1	91.8	75.7	96.4	n/a	89.3	90.2	42.6
GPT-4o (11/20/2024)	85.7	46.0	68.5	90.2	90.3	81.5	38.8
GPT-4-turbo (4/9/2024)	86.7	49.3	73.4	88.2	89.6	86.0	24.2
Llama 3.1 405b	88.6	50.7	73.8	89.0	91.6	84.8	
Claude 3.5 Sonnet	88.3	59.4	71.1	92.0	91.6	87.1	28.9

Table 2.4: LLMs' benchmarks table as of April 2025

In the right columns, we can see the list of benchmarks along with the performance for different LLMs. The values are shown as percentages, indicating the proportion of correct answers: higher values mean better performance. As we can see, different models perform slightly differently across tasks. For example, LLaMA achieves the highest score on the MGSM dataset (91.6%), but performs less strongly on benchmarks like GPQA and MATH.

We can also observe that newer models don't always outperform earlier ones on every task or benchmark. For example, the earlier version GPT-4-turbo performed slightly better than the newer GPT-4o on benchmarks such as MMLU, GPQA, MATH, and DROP.

Now, let's take a look at some of these benchmark datasets from *Table 2.4* in more detail to understand what kind of data they have and how they do the validation:

- **Massive multitask language understanding (MMLU):** A benchmark of over 14,000 multiple-choice questions across 57 subjects, including math, history, law, medicine, and more. It tests a model's broad general knowledge and reasoning ability, especially at the high school and college level. Example.

Question: What was the primary purpose of the Monroe Doctrine (1823)?

Choices: A) To end slavery in the United States; B) To promote trade with Europe; C) To prevent European colonization in the Americas; D) To establish the League of Nations

Expected output: C

- **MATH:** Contains over 12,500 math problems from high school-level competitions. It tests multi-step mathematical reasoning, symbolic manipulation, and precise numeric or algebraic outputs. Example.

Question: If $3x + 5 = 20$, what is the value of x ?

Expected Output: 5

- **HumanEval:** A set of 164 coding problems designed to test *code generation* and completion abilities. Each task includes a function signature and description, and the model is expected to generate correct, working code based on it.
- **SimpleQA:** A fact-based question-answering dataset that contains over 6,000 simple questions, typically about entities and relationships. It focuses on *common sense* and *factual recall*, requiring short and direct answers.

Question: Who is the founder of Microsoft?

Expected Output: *Bill Gates and Paul Allen.*

From the dataset descriptions, we can see that mathematical problems are commonly used to evaluate LLM performance. While you may not need to solve math problems in everyday applications, they serve as an effective way to test a model's *reasoning* capabilities. Math tasks are well-defined and require logical, step-by-step thinking, similar to how children learn math in school.

Other datasets evaluate LLMs on factual knowledge and context understanding. These benchmarks focus on a model's ability to answer questions about historical events, scientific facts, or general knowledge, often pulled from encyclopedic sources like Wikipedia.

Datasets like MMLU, GPQA, and SimpleQA test how well the model can retrieve and reason over factual information. This helps measure a model's *breadth of knowledge*, as well as its ability to follow the nuances of a question and select the most accurate answer.

So far, we've covered some of the most widely used benchmarks. However, many others exist, such as TruthfulQA, ARC, BoolQ, BIG-Bench, and HellaSwag, each designed to evaluate different aspects of language understanding and reasoning. In fact, the list is much longer. Evidently, AI has compiled a database of over 100 benchmarks and evaluation datasets, spanning areas like reasoning, language comprehension, mathematics, question answering, and coding [10].

While benchmarks are helpful for comparing models and understanding their general capabilities, real-world applications often require custom evaluation methods tailored to specific tasks and goals. We'll explore how to design and run such evaluations in *Chapter 10*.

Small language models

In addition to the term LLM, you may also encounter the term **small language model (SLM)**. The distinction between the two is generally based on model size, measured by the number of parameters.

LLMs typically contain billions or even trillions of parameters, enabling them to capture a vast amount of knowledge and handle highly complex tasks. By contrast, SLMs are deliberately designed to be smaller and lightweight, often ranging from tens of millions to a few billion parameters. These smaller models are easier to run on resource-constrained devices (such as laptops, edge devices, or even mobile phones), are faster, and cheaper to serve in production environments.

However, the line between "small" and "large" is constantly shifting as model sizes grow over time. What was considered an LLM just a few years ago (e.g., a model with 1–2 billion parameters) might now be categorized as small compared to today's massive 70B–1T parameter models. There is no universally agreed-upon cutoff between SLMs and LLMs; the terminology is relative and context-dependent.

For the purposes of this book, we will use LLM as a general term to refer to language models of all sizes: large or small. Whenever model size becomes important to the discussion, we will specify it explicitly.

Multimodal models

As we learned, large language models are designed to work primarily with text. But in recent years, a new class of models has emerged, *multimodal models*, that can handle not just text but also other types of inputs such as images, audio, video, and even structured data. These models are built to understand and reason across multiple *modalities* (types of data), allowing for more natural and versatile interactions. For example, a multimodal model can answer questions about an image, describe a video, or generate captions for a photo.

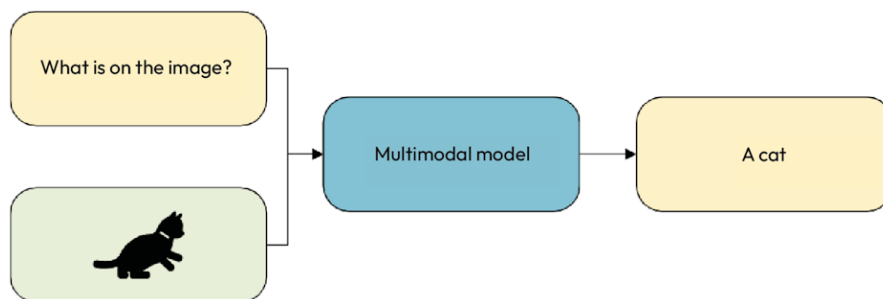


Figure 2.5: A multimodal model takes an image and text as input and returns text as output (example: Gpt-5.4)

At the core of many of these advancements is the concept of *foundation models*, i.e., large models trained on broad data at scale that can be adapted to a wide range of tasks.

Foundation models aren't limited to text; they can span multiple data types. For instance, OpenAI's GPT-4o and Google's Gemini are multimodal foundation models that were trained with both text and visual inputs from the start, enabling them to interpret images, analyze diagrams, and respond with natural language. These models blur the boundaries between language and perception, creating more intuitive and powerful AI experiences.

A key innovation driving multimodal generation, especially for images, speech, and video, is the use of *diffusion models*. Unlike LLMs, which predict the next token in a sequence, diffusion models gradually transform random noise into coherent outputs (like a photo or video) through a learned *denoising process*.

Popular examples include OpenAI's DALL-E for text-to-image generation and Google's Imagen. These models have become state-of-the-art for generating high-quality visuals from text prompts, and many are now being combined with LLMs to create rich, interactive applications that understand both language and vision.

As multimodal models continue to evolve, they are opening up entirely new possibilities in AI: virtual assistants that understand voice, text, and visuals; educational tools that can read and explain charts; or creative platforms that generate both written and visual content. These models represent the next step in general-purpose AI, combining the reasoning abilities of language models with the perceptual power of vision and audio systems, all under one unified architecture.

For the purpose of this book, we won't dive deeply into fusion techniques or the broader architecture of multimodal models, as our primary focus is on large language models and their use with text. This is because working with text remains the foundation of most real-world LLM applications today, from chatbots and summarization tools to AI agents.

While multimodal models are an exciting and rapidly evolving field, understanding how LLMs process, generate, and reason over language is essential before extending those concepts to other modalities.

Using LLMs

At this point, we've built a solid foundation for understanding how LLMs are designed, trained, run, and benchmarked. Now it's time to explore the practical aspects of using modern LLMs, focusing on the key parameters and characteristics that affect how they behave in real-world applications.

We'll begin with the idea of a context window, which defines how much information the model can "see" at once. We'll also cover the role of input and output tokens, and how they influence performance and cost. From there, we'll explore generation controls like temperature, top-k, and top-p, which affect randomness and diversity in outputs. Finally, we'll discuss advanced outputs like log probabilities (logprobs) and the important challenge of hallucinations, where a model confidently generates false or misleading information.

Context window

LLMs accept text as input and return text as output, or more precisely, they work with sequences of tokens. The length of these input and output sequences is limited by what's known as the **context window**, which is the maximum number of tokens the model can process at once, including both the prompt (user question/query) and the generated response. If the total token count exceeds this limit, some tokens may be truncated, ignored, or the request may fail entirely.

Early versions of LLMs supported a relatively small context window, typically around 4K to 8K tokens, which corresponds to roughly 3–6K words, or about 5–10 pages of a typical book. Modern models, such as GPT-5 or Claude Sonnet 4.5, have dramatically expanded this limit. Some now support 128K, 200K, or even up to 1M tokens in a single context window, an enormous amount of information. For perspective, 128K tokens is roughly 100K words, equivalent to 200–300 pages of an average book, meaning an entire medium-sized book can fit into the model's context at once.

This became possible due to a combination of architectural improvements and engineering optimizations. Models like GPT-4 Turbo and Claude 2.1 started using techniques such as *efficient attention mechanisms*, *sparse computation*, and *sliding window attention* to handle longer input sequences while keeping memory and compute requirements manageable. Advances in hardware (such as NVIDIA's H100 GPUs) and software frameworks have also made it feasible to support much larger context windows.

Here is a list of some models and their context window limits at the moment of writing this book:

Model	Max input tokens	Max output tokens
GPT-5.4	1M	128K
Claude Opus 4.6	1M	128K
Gemini 1.5 Pro	2M	8K
LLaMA 3.1 (70B / 405B)	128K	2K
Mistral 7B (latest variants)	8K	8K

Table 2.5: LLMs and their context window limitations

As we can see, the latest models now support at least 128K input tokens and even more. However, the output token length remains relatively limited, typically capped at around 4-8K tokens and 128K for more advanced models. This is largely due to the autoregressive nature of text generation, where tokens are produced one at a time in sequence, making longer outputs more computationally expensive and slower to generate.

While today's leading LLMs can technically accept very long prompts, they often use only a few thousand tokens effectively in practice. Many benchmarks and research papers show that once the context length goes beyond around 8K or 16K tokens, model performance starts to drop [11, 12].

This happens because with large inputs, models can start missing important information that isn't recent, struggle to process content in the middle of the prompt, or fall back on incorrect or generic answers when overloaded with too much text [13].

OpenAI conducted a comprehensive study on long-context performance called "needle-in-a-haystack" and published the results on the internet [14]. In their experiment, they inserted a small piece of information, referred to as the *needle*, into a much larger body of text referred to as the *haystack*. The goal was to determine whether large language models could accurately retrieve the *needle* when asked targeted questions about it.

They found that with an 8K-token context window, most GPT models performed well, maintaining high accuracy. However, once the context length increased beyond 16K tokens, performance dropped significantly, from over 80% accuracy down to around 20–40%. This highlights a key limitation of LLMs: while they can technically handle long inputs, their ability to reason over and retain specific information degrades as the context grows.

So, in most cases, it's still important to manage relatively small and focused context windows (typically up to 8–16K or even less) to maintain strong model performance.

Note that output tokens are not just limited in number compared to input tokens; they are also typically more expensive, often costing several times more per token. For example, as of April 2025, OpenAI's GPT-4o charges \$5 per million input tokens and \$15 per million output tokens, making output tokens three times as costly. This pricing reflects the higher computational and memory demands of the autoregressive generation phase, where tokens are produced one at a time, requiring repeated model evaluations.

Most LLMs provide configuration options to control the context window size or limit the number of input tokens. This allows developers to manage cost and latency more effectively, especially when working with long inputs or generating large outputs. Many APIs also allow specifying a maximum number of output tokens, helping ensure the response fits within budget or performance constraints.

Sampling and randomness

As we learned earlier, LLMs generate text in a non-deterministic way. This means that asking the same question multiple times can result in slightly different responses. Guided by probabilities, the model predicts the next token based on what is most likely to follow.

One straightforward approach is to always pick the token with the highest probability. This is called **greedy decoding**, a term borrowed from greedy algorithms in computer science. While simple and efficient, this method often leads to repetitive or overly predictable text, especially in tasks where variety, creativity, or natural flow are desired.

To improve this, modern LLMs often use a special token selection strategy called **sampling**. Instead of always choosing the top one token, the model picks the next token randomly based on its probability distribution.

Let's illustrate this with a simple example. Suppose the model is trying to complete the sentence: "The cat sat on the ...". Let's have the following probability distribution for the next token:

Token	Probability
Mat	55%
Sofa	28%
Floor	15%
Moon	5%
telescope	1%
....	

Table 2.6: Probability distribution

If the model always chose "mat", the text would sound predictable and robotic. Instead, sampling allows the model to occasionally pick "sofa", "floor", or even something unexpected like "moon", which makes the output more diverse and natural.

In practice, sampling works by building a list of all possible next tokens along with their probabilities (similarly to the table above). Those probabilities serve as weights for the selection algorithm. The model then randomly selects one token based on those weights. So, tokens with higher probabilities are selected more frequently. The goal is to strike a balance between predictability and creativity, avoiding both boring repetition and chaotic randomness.

Three key settings influence how sampling behaves: temperature, top-k, and top-p.

Temperature controls how confident or creative the model is when choosing the next token. A lower temperature makes the model more deterministic, increasing the gap between likely and unlikely tokens. A higher temperature "flattens" the probabilities, giving more weight to less likely tokens and making the output more diverse.

Typically, the temperature ranges from 0 to 2:

- **0 (deterministic)**: no randomness, greedy encoder
- **0.2—0.3 (low randomness)**: predictable, focused output
- **0.4—0.6 (moderate randomness)**: balanced, i.e., some creativity, still coherent

- **0.7—0.9 (high randomness):** more diverse and surprising responses
- **1.0+ (very high randomness):** unpredictable, possibly chaotic or nonsensical

When the temperature is set to 0, the model behaves deterministically, i.e., there is no randomness in the output. As we mentioned above, this mode is called greedy decoding because the model always selects the most likely next token at each step. When we talked about Seq2Seq models, we implemented the decoder using exactly the greedy decoding strategy. Note that if temperature = 0, all other randomness settings like top-p and top-k don't work anymore.

To illustrate how temperature works, let the model assign the following probabilities to the next tokens:

```
[40%, 20%, 15%, 10%, 5%, ... ] for tokens [A, B, C, D, E, ...]
```

With low temperature (e.g., 0.2-0.3), the distribution becomes sharper: token A might jump from 40% to 80% or more, making it much more likely to be chosen (80% of chances to be selected vs. 40% as before). In other words, the model becomes more confident and deterministic, favoring the most probable token almost every time.

With high temperature (e.g., 0.9 or 1.2), the distribution becomes flatter: token A might drop to 25-30%, while less likely tokens such as C or D rise in probability. This encourages the model to explore more diverse options, producing more varied and creative outputs.

Another common parameter used in sampling is top-k. This setting controls the number of highest-probability tokens that the model is allowed to consider when generating the next token. Instead of sampling from all tokens in the vocabulary (which could include tens of thousands of possible tokens), top-k sampling limits the selection to only the top k tokens with the highest probabilities. This helps the model focus on the most likely options while still allowing some variation and creativity.

For example, if our vocabulary contains 10,000 tokens, the model normally computes a probability for each token at every generation step. If we set top-k = 5, the model considers only the five most likely tokens and samples from them based on their relative probabilities, ignoring the other 9,995 options.

This approach reduces the chance of selecting extremely unlikely or nonsensical tokens, which could happen if sampling from the full distribution. Lower values of k make the output more focused and deterministic, while higher values introduce more diversity. At k = 1, top-k sampling becomes identical to greedy decoding, always picking the single most probable token.

As an alternative to top-k, another parameter used in LLM to control randomness is top-p sampling, or **nucleus sampling**. Unlike top-k, which selects a fixed number of top tokens, top-p selects a dynamic number of tokens whose cumulative probability adds up to a threshold p (e.g., 0.9 or 90%). The model then samples from this "nucleus" of high-probability tokens.

For example, let's have the following probabilities for the next token:

[52%, 21%, 17%, 5%, ..., 0.01%]

If top-p = 0.9, the model will look at the tokens until their combined probability reaches 90%. In our case, it will be only tokens with probabilities [52%, 21%, 17%], all remaining tokens will be excluded. This makes top-p more flexible compared to top-k. Lower values of p make the output more focused, while higher values allow more diversity and creativity.

If top-p is set very low, for example, 0.1, the model becomes much more deterministic, behaving similarly to greedy decoding. This is because only the very highest probability tokens (those whose cumulative probability adds up to 10%) are kept for sampling, leaving little or no room for randomness.

Note

By using parameters like temperature, top-k, and top-p, we can control randomness in generation and, in ideal cases, push the model toward more deterministic output (for example, by setting temperature = 0 or very low top-k/top-p).

However, in many cloud-hosted LLMs, achieving perfect determinism is not possible in practice. This is because deployed LLMs often use complex infrastructure: a **mixture-of-experts (MoE)** architecture [15], approximations (e.g., *quantization*), batching, parallelism, and hardware-level nondeterminism [16]. These factors can induce subtle variability even when randomness parameters are "turned off." Additionally, some LLM providers may intentionally restrict full determinism as part of system design or intellectual property management to protect from "reverse-engineering."

Hallucinations

As we learned, AI models have a probabilistic nature, which means they sometimes produce incorrect results. In fact, very few AI models, including LLMs today, achieve anything close to 100% accuracy on reliable and comprehensive benchmarks.

When a language model produces incorrect or misleading content, it's called a **hallucination**. This happens when the model generates something that is factually wrong, contradictory, or

even completely fabricated. This phenomenon is common across many types of generative AI models, including LLMs.

Note

The term **hallucination** in the context of LLMs has evolved over time. Initially, in the early 2000s, it was used in computer vision to describe the process of adding detail to images, such as generating high-resolution images from low-resolution ones. This task was called "face hallucination" [17].

By the late 2010s, the term began to denote instances where AI systems produced factually incorrect or misleading outputs in tasks like translation or object detection. For example, in 2017, Google researchers used "hallucination" to describe responses from neural machine translation models that were unrelated to the source text. Later this term drifted to LLMs as well [18].

The adoption of "hallucination" has sparked ethical debates, particularly concerning its overlap with the medical condition characterized by false sensory perceptions. Some critics argue that using this term in AI contexts may trivialize the experiences of individuals with mental health disorders [19]. It is suggested to use more precise terms like *fabrication*, *unfaithfulness*, or *confabulation*.

Hallucinations may occur due to the probabilistic nature of LLMs. There are several factors that contribute to the likelihood of this:

- Training data may contain errors or biases. Since most of such data is collected from the internet, it's not always accurate or consistent. While LLM developers try to clean and filter the data, doing that at scale is extremely challenging.
- The way text is generated can also lead to mistakes. As we discussed earlier, techniques like temperature, top-k, and top-p control the balance between creativity and accuracy. Sometimes, the model picks a more "interesting" token instead of the correct one.
- Unclear, ambiguous, and contradictory prompts can confuse the model. If the instructions have ambiguity or include conflicting information, the model may guess and produce a confusing or incorrect answer.
- Conflicts between the prompt and the model's internal knowledge or reasoning can also lead to hallucinations. For example, if a prompt asks about a fact the model learned differently during training, it may produce an answer that sounds plausible but is wrong.

There is currently no fully guaranteed way to eliminate hallucinations in LLMs. However, it is possible to reduce their likelihood by taking the following steps:

- Improve prompt quality by removing contradictions and clarifying ambiguities. Clear and focused prompts help the model generate more accurate responses.
- Adjust decoding parameters such as temperature, top-p, or top-k to strike a better balance between creativity and precision. Lower values tend to produce more consistent and grounded outputs.
- Use context enrichment techniques like **retrieval-augmented generation (RAG)** to ground responses in external, factually correct data. This approach, covered in *Chapter 5*, can reduce hallucination by providing the model with up-to-date and relevant information.

Later in the book, we'll explore additional techniques for improving and evaluating output quality, including methods to help reduce hallucinations.

LLM families

Today, we're witnessing an unprecedented surge in the development of LLMs. The success of OpenAI's GPT-3.5 and the widespread adoption of ChatGPT have sparked intense interest across the tech industry, prompting a wave of companies, research labs, and open-source communities to build their own LLMs tailored to various tasks, domains, and use cases.

After exploring the transformer architecture and understanding how LLMs are trained and used, it's time to examine the most prominent and widely used model families. In this section, we'll dive into influential models such as GPT, PaLM, Gemini, LLaMA, and Claude. We'll also touch on earlier landmark models like BERT and T5, which were highly impactful in their time. Our focus will be on understanding the architecture of these models, along with their strengths, limitations, and the roles they play in modern AI systems.

GPT models

In June 2018, OpenAI introduced the GPT-1 model. **GPT** stands for **generative pretrained transformer**. This model and the next versions (2, 3, 3.5, 4, etc.) were built on a *decoder-only* transformer architecture; see the right part of *Figure 2.3*.

Removing the encoder component simplified the overall architecture and improved efficiency, especially for open-ended tasks like question answering and storytelling. The encoder was initially considered more suitable for tasks requiring input-to-output mapping, such as machine translation and summarization. However, later versions of GPT (e.g., GPT-3 and GPT-4) demonstrated that the decoder-only architecture can also perform well in tasks like summarization and translation.

GPT-1 and GPT-2 were open-sourced and completely available from Hugging Face. The next versions became proprietary, and even had limited information about architecture parameters. Here are more details about GPT model versions.

Model	Released	License	Params	Layers	Embedding dimension	Training cost**
GPT-1	2018	Open source	117M	12	768	~1 petaFLOP /s-day
GPT-2	2019	Open source	Up to 1.5B	Up to 48	768–1600	Tens of petaFLOP /s-days
GPT-3	2020	Proprietary	175B	96	12,288	~3.1e23 FLOPs
GPT-3.5	2022	Proprietary	~175B	96	12,288	Slightly more than GPT-3
GPT-4	2023	Proprietary	1.8T*	120 *		
GPT-4o	2024	Proprietary				

Table 2.7: The list of GPT family models and their characteristics

* Not official data [20]

Note

** Note that petaFLOP/s-day is a unit used to express the total amount of compute consumed during training. To put this into perspective: it would take a single NVIDIA A100 GPU approximately 3.2 days of continuous training at peak performance to deliver 1 petaFLOP/s-day of compute. If you used 32 A100s in parallel, the same amount of compute could be achieved in about 2.4 hours.

There's not much official information about GPT-4 (and later versions) architecture, but some resources mention (like this one [20]) that GPT-4 is based on eight models, each with 220 billion parameters, which are linked in the **mixture of experts (MoE)** architecture.

The MoE model is a learning technique that combines multiple specialized models, known as "experts". This architecture excels in handling large and complex datasets by effectively partitioning the problem into simpler, more manageable subspaces, allowing for efficient, and specialized learning.

In September 2024, OpenAI introduced GPT-o1, with the full release following in December. This model brought a new approach: it spends additional computation time "thinking" before answering, generating a longer **chain-of-thought (CoT)** (see *Chapter 4*) internally. This class of models came to be known as *reasoning* models.

According to OpenAI, unlike GPT-4 or GPT-4o, GPT-o1 is not designed to replace those models, but rather to complement them. It was trained using a new optimization algorithm, specialized datasets, and reinforcement learning techniques. The result is a model that excels at reasoning but typically requires more compute per response, leading to higher latency and cost.

OpenAI benchmarked GPT-o1 at near PhD-level performance on a range of STEM tasks [21]. For example, it solved 83% of problems on the American Invitational Mathematics Examination, far outperforming GPT-4o, which scored just 13%. It also ranked in the 89th percentile in competitive coding benchmarks like Codeforces.

The o-series reflects a broader trend in LLM development: building models that don't just generate fluent language but can reason through problems. OpenAI has already introduced GPT-o3, the next step in this line, and skipped the name "o2" due to trademark conflicts. As the field continues to evolve, these reasoning-first models are likely to play an increasingly important role, especially in domains that demand advanced reasoning.

OpenAI was not the first to explore transformers for NLP tasks. However, OpenAI was the first to build and release a large-scale, general-purpose generative language model that achieved significant commercial success, starting with GPT-2 and especially GPT-3. Google researchers introduced the transformer architecture earlier [7] and applied it to machine translation tasks. However, OpenAI was the first to turn LLM into a widely adopted consumer product. The launch of ChatGPT, initially powered by the GPT-3.5 model, marked a major turning point. Within just two months, ChatGPT reached 100 million users, making it the fastest-growing consumer application in history at the time [22].

Gemini

Google was among the early pioneers in large language model research, developing influential models such as **text-to-text transfer transformer (T5)** and the **pathways language model (PaLM)** family. While these models introduced important innovations in scaling and transfer learning, they were primarily designed as text-focused systems and had architectural and integration limitations. These constraints motivated the development of a more unified and multimodal framework, ultimately leading to Gemini, Google's next-generation model family and successor to PaLM.

Building on this foundation, Gemini has become Google DeepMind's flagship multimodal large language model, positioned as a direct competitor to OpenAI's GPT-4. Officially released in December 2023, it marked a major leap in Google's LLM strategy by consolidating the strengths of its earlier models into a single, cohesive framework.

What sets Gemini apart is its native multimodal design, meaning it was trained from the ground up to handle text, images, audio, video, and code. This design allows Gemini to reason across different types of input in a more integrated way, which is especially useful in applications like science, education, and interactive assistants.

Gemini is a proprietary model, just like the latest GPT models from OpenAI. Google integrated Gemini into its search engine, Google Search. It also made the model available through Google AI Studio and Vertex AI. This makes Gemini easy to access for developers.

LLaMA

LLaMA or **Large language model Meta AI** is a family of *open-weighted** language models released by Meta AI (formerly Facebook AI). It was first introduced in February 2023. The goal behind LLaMA was to provide language models that are efficient and accessible to the research community, as an alternative to proprietary models like GPT-4.

Note

* An open-weight model is one where both the source code and the pretrained weights are publicly available from platforms like Hugging Face or GitHub. This allows users to download and run the complete model without needing to train it from scratch.

LLaMA was designed to be a smaller model, compared to other LLMs, with the goal of achieving competitive performance. The first version, LLaMA 1, ranged from 7B to 65B parameters and was trained on publicly available datasets.

LLaMA gained high attention because Meta released it under a non-commercial research license, allowing researchers to explore and build on top of it. This decision sparked a wave of

derivative models, community fine-tunings, and optimizations, including instruction-tuned versions and optimized models that could run on consumer GPUs.

In July 2023, Meta released LLaMA 2, a new model version, trained with RLHF and optimized for dialog and instruction-following tasks. LLaMA 2 was released with models at 7B, 13B, and 70B parameters, and was made freely available for both research and commercial use. It powered various open-sourced chatbot projects and served as the backbone for products like Meta's own AI assistant.

Claude

Claude is a large language model developed by Anthropic, a company founded by former OpenAI researchers. The model is named after Claude Shannon, widely regarded as the father of information theory. Like other LLMs such as GPT and LLaMA, Claude is based on the transformer architecture, but Anthropic has placed an emphasis on safety, helpfulness, and transparency throughout its design and training process.

A distinctive feature of Claude is Anthropic's training methodology called **constitutional AI**. Instead of relying on human feedback to guide the model's alignment, this approach uses a set of predefined principles, a "constitution," to help the model self-critique and revise its responses. The idea is to make Claude's behavior more predictable, ethically consistent, and aligned with human values, while reducing the need for manual human review at every step.

In practice, this approach results in a model that often produces responses that are carefully worded and generally avoids unsafe or harmful outputs. However, like all LLMs, Claude is not perfect; it can still produce incorrect or biased information, and its adherence to safety principles sometimes leads to overly cautious or evasive answers in situations where a direct response would be more useful [25].

Claude is available through Anthropic's API and popular integrations, making it a competitor to OpenAI's GPT models in the LLM ecosystem. Its design philosophy and alignment techniques have contributed to ongoing industry discussions about how to make LLMs not just powerful, but also trustworthy and responsible.

Model selection

We introduced some of the most widely used LLM families. However, the full list of models available is much broader. We didn't cover several notable examples, such as Mistral, DeepSeek, Phi, Grok, Command R, or Falcon, among others. Most likely, we will see many new LLM versions and families; the competition has just started. Each of these models contributes to the constantly evolving LLM ecosystem, with unique design goals, ranging from efficient inference and open-source accessibility to advanced reasoning capabilities.

This brings us to a critical question: Which model family, and ultimately, which specific model, should you choose for your application? Unfortunately, there's no universal answer. Model selection depends heavily on your specific use case, constraints, and requirements.

As we have seen before, benchmark scores can help guide model selection, but they shouldn't be treated as absolute recommendations. For instance, if one model scores 85% on a benchmark and another scores 88%, does that automatically make the second model a better fit? Not necessarily. Those extra 3 percentage points might not translate to better performance in your particular task. In fact, a model with lower benchmark scores could outperform others in your specific domain or under your specific workload.

Below are some key factors to consider when selecting a large language model for your application:

- **Model performance:** Evaluate the model's performance on benchmarks relevant to your domain. If your problem is similar to a well-known public benchmark, strong results there are a good sign that the model may perform well for your tasks too. However, in many cases, task-specific evaluation is essential; see *Chapter 8* for practical guidance on how to design and run such evaluations.
- **Open-source vs. proprietary:** Open-source models offer transparency, flexibility, and the ability to customize or fine-tune them to your needs (see *Chapter 6*). They also benefit from active community support. However, deploying and maintaining them often requires deeper technical expertise and infrastructure readiness. Proprietary models, on the other hand, typically offer more out-of-the-box APIs and support but come with licensing limitations, less control, and transparency.
- **Cost considerations:** High-performance proprietary models often involve high costs. Alternatively, running large models on your own infrastructure, either on-premises or on the cloud, can also be expensive due to compute (GPU) resources and maintenance costs. It's important to assess the total cost of ownership and usage.
- **Data privacy and security:** Make sure the model and its deployment approach align with your organization's data governance policies. If your application involves sensitive or regulated data, you may be limited to using models that can run within your private network. This can rule out cloud-based APIs like GPT or Claude unless a secure, isolated deployment is available.
- **Language availability:** Most large language models are primarily trained on English data and perform best in English. While many models support multiple languages, their performance can degrade for languages that are less represented on the internet or in training datasets (see *Chapter 4*). Be sure to validate model capabilities if your application targets non-English users.

- **Region availability:** Some models and APIs are only available in specific geographic regions. This can impact latency, data residency, and compliance with regulations such as EU data protection and residency laws. Make sure the model can be hosted or accessed in a region that meets your legal and operational requirements.
- **Features availability:** Depending on your use case, you might need specific capabilities such as function calling, code interpretation, or partial fine-tuning. Not all models support the same feature set, especially across open-source options.
- **Responsible AI:** Ensure the model aligns with your organization's ethical guidelines and risk policies. Consider how it handles bias, harmful content, misinformation, and fairness. Proprietary models may offer built-in safety mechanisms, while open-source deployments might require you to implement safeguards yourself. See more details about responsible AI in *Chapter 8*.

We've outlined some of the key factors to consider when selecting an LLM for your workload. This list isn't exhaustive; your specific needs may introduce additional requirements. Still, these are the most common and impactful areas to evaluate. By assessing them in the context of your goals and constraints, you'll be better equipped to choose a model that performs well and fits both your application and organizational standards.

Summary

This chapter laid out the technical foundation for the rest of the book. The material may feel very complex. The goal was to build a clear mental model of how large language models work under the hood. With this understanding, you are better equipped to build reliable AI systems, design effective prompts, and make informed decisions about model selection, deployment, and evaluation.

We explored the roots of modern LLMs in Seq2Seq architectures, the role of attention mechanisms, and how transformers enable models to capture large text and scale effectively. These concepts provide a framework for understanding and intuition about why models behave the way they do.

We also discussed the importance of data in AI systems. You learned how LLMs are trained, evaluated, and benchmarked, as well as the role of context windows and generation parameters such as temperature, top-k, and top-p. Finally, we reviewed major LLM families and their historical predecessors, giving you a clearer picture of today's model landscape.

In the next chapters, we will build on this foundation by focusing on how to interact with LLMs in practice. We will start with prompt engineering, moving toward RAG, LLM fine-tuning, designing, and evaluating AI agents.

Subscribe for a free ebook

New frameworks, evolving architectures, research drops, production breakdowns – *AI_Distilled* filters the noise into a weekly briefing for engineers and researchers working hands-on with LLMs and generative AI systems. Subscribe now and receive a free eBook, along with weekly insights that help you stay focused and informed.

Subscribe at <https://packt.link/80z6Y> or scan the QR code below.



3

Prompt Engineering

Traditional AI development typically involved building models to solve a single, specific task or, at best, a small set of related tasks. This task-by-task approach worked to some extent, but it was time-consuming, resource-intensive, and often required deep expertise in both machine learning and the specific problem domain.

Large language models (LLMs) have introduced a new paradigm for solving **artificial intelligence (AI)** problems. A single model became capable of solving a wide range of tasks without needing to be retrained for each one. This shift was made possible by the introduction of *prompt engineering*.

Prompt engineering allows developers to interact with pre-trained LLMs using natural language instructions, called **prompts**, to steer the model's behavior toward a desired outcome.

Prompt engineering has dramatically lowered the barrier to entry for building intelligent systems, opening up AI development to a much broader range of software engineers, product teams, and domain experts, many of whom don't have a background in classical machine learning. At the same time, data scientists have gained a powerful new tool to solve AI and NLP problems more quickly and efficiently.

With that in mind, this chapter will explore the core prompt engineering techniques that form the foundation for effective use of large language models. We'll begin with a basic understanding of prompt structure and then dive into popular approaches like chain-of-thought prompting and related methods. From there, we'll examine the role of examples in guiding model behavior and finish with more advanced techniques such as multi-prompt strategies.

The chapter will cover the following key topics:

- Understanding prompt structure
- Exploring core prompt techniques

Technical requirements

The code for this chapter is available on the book's accompanying GitHub repo: <https://github.com/PacktPublishing/Building-Agent-Powered-Applications/tree/main/Chapter3>

Understanding prompt structure

A **prompt** is a text-based instruction or context provided to a language model to guide its response. From a technical point of view, a prompt is simply an input string passed to the LLM for processing. In fact, any text can serve as a prompt; there are no strict rules about what it must contain. In practice, prompt design is often considered more of a craft or an art than a rigid engineering process.

You can also think of a prompt as a kind of *code* that gets interpreted and executed by a language model. However, unlike traditional code in software engineering, which follows strict syntax and structure, prompts are more like free-form instructions. They lack formal rules, can be written in natural language, and often rely on nuance, phrasing, and context to influence model behavior. This flexibility is powerful, but it also means that crafting effective prompts requires experimentation, intuition, and an understanding of how the model interprets language.

Despite this free-form nature, research and real-world experience have led to a set of foundational principles and techniques that consistently yield better, more reliable results. One of the first such techniques is structuring prompts into system and user prompts or messages. This separation is supported now by many modern LLM APIs, including GPT and Claude model families.

System and user prompts

System prompt defines the role, behavior, or constraints for the LLM. It acts as a directive or a set of rules that the model follows throughout the interaction. System prompt is sometimes also called **system message** or **system instruction**.

User prompt represents the queries, requests, or commands given by the end user. It provides the specific task or question to be addressed by the LLM. User prompt, sometimes also called **user message** or **user instruction**.

Let's look at system and user prompt examples for a customer support assistant:

- **System prompt:** "You are a friendly customer support assistant for a software company. Always provide clear and polite answers."
- **User Prompt:** "How can I reset my password?"
- **LLM Response:** "To reset your password, please go to the login page and click on 'Forgot Password.' Follow the instructions sent to your registered email."

When developing such an assistant, the system prompt is typically reused for every customer query, while individual questions are passed through user prompts. In this setup, the distinction between system and user prompts is largely logical rather than physical. This split helps organize interactions conceptually, but in most cases, the final input received by the LLM is a single, merged sequence of text, combining both the system prompt and the user prompt into one unified input.

However, LLM APIs may still handle these prompts differently behind the scenes. For example, in how they apply policies, message prioritization, caching, or logging. System prompts may be subject to stricter policies, while user prompts often require stronger privacy safeguards, as they may contain sensitive end-user data.

It's important to note that different LLMs may use varying terminology for system and user prompts. For instance, older versions of Anthropic's Claude models utilize a system parameter to set the assistant's role or behavior, technically functioning as a system prompt. Similarly, Google's Gemini models employ a *preamble* to establish the initial context and expectations for the model's responses, serving a role akin to system prompts.

The following code demonstrates how to use OpenAI's Chat Completions API in Python, including both a system prompt and a user prompt:

```
import openai
openai.api_key = "your-api-key"
response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=[
        { "role": "system", "content": "You are a friendly customer support ..." },
        { "role": "user", "content": "How can I reset my password?" }
    ]
)
print(response["choices"][0]["message"]["content"])
```

Next, let's look at how to send a similar request to Anthropic's Claude models, also including system and user prompts:

```
import anthropic
client = anthropic.Anthropic(api_key="your-anthropic-api-key")
response = client.messages.create(
    model="claude-opus-4-6",
    max_tokens=1000,
    messages=[
        { "role": "system", "content": "You are a friendly customer support ..." },
        { "role": "user", "content": "How can I reset my password?" }
    ]
)
print(response.content[0].text)
```

Note that some LLM APIs don't support separate system and user messages; they accept a single combined prompt. In those cases, the separation between system instructions and user input is logical only and must be handled at the application level. The example below shows how to call a LLaMA model with a single prompt parameter:

```
from transformers import AutoTokenizer, AutoModelForCausalLM
# Load model and tokenizer (ensure you have access)
model_name = "meta-llama/llama-2-7b-chat-hf" # Or another LLaMA-based model
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name)
# Construct a single prompt combining system and user roles
prompt = (
    "You are a friendly customer support assistant. ... \n" # system prompt
    "User: How can I reset my password?\n" # user prompt
)
# Tokenize input
inputs = tokenizer(prompt, return_tensors="pt")
# Generate response
outputs = model.generate(
    **inputs, max_new_tokens=150, do_sample=True,
    temperature=0.7, top_p=0.9)
# Decode and print the response
response = tokenizer.decode(outputs[0], skip_special_tokens=True)
print(response)
```

Note that the preceding code above performs inference locally on the running machine using the LLaMA 7B model via the Hugging Face transformers library. Running it requires a capable GPU and a properly configured CUDA environment.

System prompt structure

As mentioned earlier, there are no strict requirements for crafting prompts, but achieving good results often depends on defining a clear prompt structure. A prompt can be thought of as a combination of *instructions*, *rules*, and *hints*. Instructions and rules tell the LLM what it should or shouldn't do, while *hints* help set the context and guide the shape, style, or format of the desired output, such as tone, length, or specific tokens.

Let's look at an example of a system prompt for a financial assistant and examine its structure:

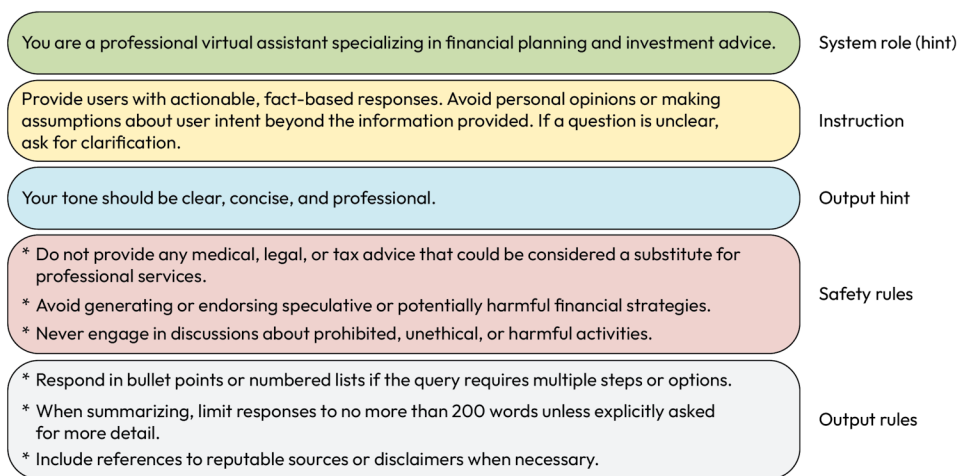


Figure 3.1: A system prompt example with instructions, rules, and hints

It has the following items:

- **System role (hint):** Provides context and defines the persona or domain expertise the model should adopt
- **Instruction:** Guides the task-specific behavior of the model
- **Output hint:** Specifies the tone, style, or overall structure of the response
- **Safety rules:** The list of constraints or guidelines to prevent harmful, inappropriate, or unethical content generation
- **Output rules:** Formatting or structural guidelines for how the output should be presented

In the following example, there are two *hints* that guide the LLM to respond in a specific way, such as adjusting tone or style, or providing additional context, like defining the role. Keep in mind that system prompts can vary significantly in structure. In other cases, they may include a list of detailed instructions, more complex output formatting, or even request responses in structured formats like JSON or YAML.

System prompts should be clear, without conflicting instructions, uncertainties, or ambiguity. As we discussed in *Chapter 2*, such cases can lead to reduced prompt performance. To illustrate how conflicting prompts can occur, consider the following example of a system prompt:

Prompt

```
You are a friendly and casual virtual assistant specializing in providing accurate
and professional financial advice.
Provide clear, detailed, and professional financial advice.
Keep your responses casual and light-hearted to engage users.
Respond in concise bullet points, but ensure your response is detailed and covers
all possible scenarios comprehensively.
Do not provide specific financial advice without disclaimers.
Always provide recommendations tailored to the user's situation.
Limit your responses to no more than 100 words. Include examples and detailed
explanations where necessary.
```

This system prompt has the following conflicts and ambiguities:

- **Tone conflict:** The role requires a friendly and casual tone, but the instruction demands a professional tone, creating inconsistency in how the response should sound.
- **Detail vs. brevity conflict:** The output hint asks for concise bullet points, while also requiring comprehensive and detailed responses, which are contradictory.
- **Ambiguity in tailoring advice:** The system role said that the assistant needs to "provide accurate and professional financial advice," but safety rules require avoiding specific financial advice without disclaimers. That creates ambiguity in how specific the advice should be.

This kind of system prompt would likely result in inconsistent and not optimal outputs because the LLM would struggle to prioritize which conflicting instructions to follow, leading to mixed or incoherent responses. Such prompts can be the source of LLM hallucinations (see *Chapter 2*).

User prompt structure

In contrast to the system prompt, the *user prompt* typically contains the specific task or question the model is expected to address. User prompts often originate directly from external sources, such as API requests or **user interface (UI)** inputs.

While user prompts can include instructions, it's generally recommended to keep them as simple and focused as possible, especially from a code clarity and maintainability perspective. The ideal approach is to pass in just the user's input along with minimal guidance. For example, building on the previous system prompt, a user prompt might be:

Prompt

The user question: What are some low-risk investment options for a beginner?

Here, The user question serves as a brief instruction in the system prompt, while What are some low-risk investment options for a beginner? is the actual user query, typically coming from an API request or user interface input.

Since user prompts often come from dynamic, external sources and aren't hardcoded in code, they can be harder to control or audit. This variability, especially when prompts are unstructured, inconsistently formatted, or vary significantly across use cases, may lead to reduced model accuracy or unpredictable behavior (hallucinations).

To improve the quality of user input, traditional *sanitization*, and cleaning techniques are often applied. These include removing potentially harmful code, trimming or discarding overly long inputs, cleaning up formatting, and styles, and extracting plain text from structured formats like HTML or XML. Python libraries such as *bleach*, *html-sanitizer*, and *validators*, among others, can be used to sanitize and validate user prompts before passing them to the model.

Examples and n-shot learning

One of the most powerful and widely used techniques for improving output quality is to provide examples to the LLM. This approach is known as a **few-shot** (or **n-shot**) **prompting**, also referred to as **in-context learning**. It involves including specific input-output pairs within the prompt to guide the model's behavior. By showing the model how a task should be performed, these examples help it to infer the desired format, tone, structure, and output.

When a prompt doesn't include any examples, it's referred to as **zero-shot prompting** (or **zero-shot learning**). In this case, the model relies entirely on the user's instructions to determine how to respond. We'll explore zero-shot prompting in more detail later in this section.

Note

The terms *zero-shot* and *n-shot* (*few-shot*) learning originated in machine learning well before the rise of LLMs, with early applications in fields like computer vision and traditional NLP. But in modern NLP, these ideas became central with the rise of prompt engineering and LLMs [26].

Zero-shot learning was adopted in computer vision around 2009. Researchers like Lampert et al. introduced methods enabling models to recognize unseen classes by leveraging semantic attributes. For instance, a model trained to identify horses and tigers could infer the concept of a zebra by understanding it as a horse-like animal with black and white stripes, despite never having seen a zebra during training.

Few-shot learning emerged from the field of *meta-learning*, which focuses on training models to quickly adapt to new tasks using only a few examples. A notable development was the introduction of **model-agnostic meta-learning (MAML)** [27] in 2017 by Chelsea Finn and colleagues. MAML trains models to be fine-tuned rapidly on new tasks with minimal data, laying the groundwork for many subsequent few-shot learning approaches.

Few-shot prompting is particularly useful for tasks requiring nuanced responses or where the desired output format is non-obvious. By leveraging examples effectively, users can reduce ambiguity, improve precision, and tailor the LLM's responses to better meet their specific needs. For instance, let's consider the following prompt example:

Prompt

```
You are an AI assistant trained to classify text into predefined categories. Your task is to analyze a given text and assign it to one of the following categories: positive, negative, neutral. Follow the provided examples to understand the classification criteria and format your responses accordingly:
Example 1: Text: "I absolutely love this product! It works perfectly and exceeded my expectations."
Classification: Positive sentiment
Example 2: Text: "The service was terrible. I waited over an hour and still didn't get my order."
Classification: Negative sentiment?
```

This prompt instructs the LLM to perform text sentiment analysis and includes two examples of inputs and classification outputs.

Examples can include only the input, or only the expected output, or both the input and the desired output. Each example provides the LLM with the following key information:

- **Expected input:** Helps the LLM understand the type and structure of inputs it should process
- **Output format:** Guides the model in structuring the response correctly
- **Relationship between input and output:** Helps LLM to understand the mapping between input and output

The final point is the most critical: it only becomes possible when examples include both inputs and their corresponding outputs. The LLM analyzes these examples, identifies those with inputs semantically similar to the user's query, understands the output format, and infers the mapping pattern between input and output to generate a response.

Most LLMs do not offer a dedicated API for providing examples. Instead, examples can be included either in the system prompt or the user prompt. If the examples are global and remain consistent across multiple user prompts, they should be placed in the system prompt to provide a uniform context. On the other hand, if the examples are specific to a particular task or user query, they can be included directly within the user prompt to guide the model's response for that specific interaction.

As we learned from *Chapter 2*, language models possess two primary capabilities: factual knowledge and reasoning. In few-shot prompting scenarios, LLMs can leverage both. They may retrieve semantically similar examples from their internal knowledge to generate outputs or analyze provided examples to infer underlying patterns and apply them to new inputs.

That's why few-shot (or n-shot) prompt engineering is sometimes referred to as **in-context learning**, meaning the model learns to perform a task based on the context provided within the prompt, without altering its underlying parameters (weights). This perspective highlights the model's ability to generalize and adapt its behavior dynamically by leveraging examples or instructions included in the input.

In-context learning viewpoint also gives us a powerful framework to interact with LLMs as *adaptable systems* that "learn on the fly" from the prompt, bypassing the need for computationally intensive fine-tuning. This concept redefines how AI systems can be used for task-specific purposes, making them highly flexible and accessible for a wide range of applications without modifying their base architecture or parameters.

So, few-shot learning can also serve as a foundation for **adaptive learning** in AI agents, where software (agent) tracks examples (cases) and uses them to guide future actions and improve decision-making over time. We'll explore this concept in more detail in *Chapter 7* when we talk about AI agents.

Example selection

The number and distribution of examples depend on the specific task. For simpler, well-understood tasks, such as sentiment analysis or basic fact retrieval, just a few examples may be enough, since the model can lean on its pre-trained knowledge. For more complex, nuanced, or domain-specific tasks, a larger set of examples (e.g., 10-shot, 25-shot) may be needed to guide the model toward the desired output style and reasoning process.

Equally important is the quality and *diversity* of your examples. If you're building a system to analyze user reviews, make sure your examples reflect the real variety of inputs: short and long reviews, clear positives and negatives, and more nuanced or mixed cases. If your examples are too narrow or repetitive, the model may underperform when faced with real-world variation.

Selecting the right number, quality, and diversity of examples is similar to curating a training dataset in machine learning. Just as in supervised learning, where dataset design impacts model performance, well-chosen examples in few-shot prompting act as implicit training signals for the LLM, all within a single prompt. High-quality, representative examples help the model generalize better, while poor examples can mislead it and produce inconsistent or suboptimal outputs.

To illustrate what poorly chosen examples look like, let's examine the following prompt example:

Prompt

```
Classify the sentiment of the following reviews as positive, negative, or neutral.  
Review: "I love this app! It works perfectly."  
Sentiment: Positive  
Review: "I hate this app. It's terrible."  
Sentiment: Positive  
Review: "It's okay I guess."  
Sentiment: Neutral  
Review: "Works fine."  
Sentiment: Neutral  
Review: "Great job."  
Sentiment: Positive  
Now classify this review:  
"This is the worst experience I've had with any service."
```

Here we can see several issues:

- **Low number of examples:** Only 5 examples are provided, which is often not enough to capture meaningful variation in tone or context.

- **Poor diversity:** The examples lack variety in sentence structure, length, tone, and expressions. They don't represent different user types or ambiguous/mixed reviews.
- **Label conflicts:** "I hate this app..." is labeled Positive, which is clearly incorrect, and "Works fine." (which sounds slightly positive) is labeled Neutral.
- **Unclear labeling pattern:** There's no consistent logic the model can learn from, which will confuse it during inference.

So, when adding examples to prompts, it's important to ensure they cover all major use cases, avoid contradictions, and clearly illustrate the relationship between inputs and expected outputs for the LLM.

Zero-shot prompting

Examples can improve prompt performance by guiding the LLM to respond within the context of the provided examples. However, they can also have a negative impact if the input task differs significantly from the examples, as the LLM may overly rely on the patterns or context of the examples, leading to poorer performance in such cases.

In **zero-shot prompting** (or **zero-shot learning**), we don't provide any examples in the prompt. This approach can be more effective when few-shot examples are too narrow or unrepresentative of the broader task. In such cases, the model may *overfit* to the examples, resulting in biased or less accurate responses when the actual input differs significantly from the provided examples.

For instance, consider the following prompt:

Prompt

Classify the following customer support questions as one of: ["Billing", "Technical", "Cancellation", "Sales"].

Example 1: "I need to update my credit card information."

Category: Billing

Example 2: "My app keeps crashing when I open it."

Category: Technical

Example 3: "I'd like to cancel my subscription."

Category: Cancellation

Question: "Do you offer student discounts?"

Category:

LLM might classify the user question, "Do you offer student discounts?" under the Billing category simply because it includes the word "discount", which is semantically related to billing. However, the intent of the question is more aligned with sales or pre-purchase inquiries, not actual billing issues. In such cases, a prompt without predefined examples may

yield a more accurate response, as it allows the model to interpret the input more broadly without being biased by narrow examples.

So, the following prompt can often produce better results than one that relies on few-shot examples:

Prompt

Classify the following customer support message into an appropriate category based on its content.

Message: "Do you offer student discounts?"

Category:

Note also that the previous prompt constrained the LLM's output to a fixed set of categories. In contrast, the version without examples is more open-ended, which can sometimes lead to unexpected or incorrect category suggestions.

Zero-shot prompting relies entirely on the model's pre-trained knowledge and the instructions provided in the prompt without including any examples. This approach offers greater flexibility and can be especially advantageous in scenarios such as:

- The task is well-defined and within the model's capabilities.
- Providing examples might introduce bias or limit the model's generalization
- The problem is too broad and open-ended and cannot be covered well by examples

Zero-shot prompting also slightly reduces token usage, making it a more cost-effective choice.

Static and dynamic few-shot prompting

If the number of examples is relatively small and describes the problem quite well, they can be included in every single prompt. This approach is called **static few-shot prompting** because the examples are the same for all user requests. This approach is quite straightforward and ensures consistency across different LLM requests.

However, care must be taken to make sure that the examples remain relevant, concise, and do not introduce unintended biases. Additionally, adding all examples directly into each prompt can increase token usage and latency, which may impact overall efficiency. Therefore, static few-shot learning is suitable when:

- The number of examples is relatively small (typically up to 10-20), ensuring they do not cause excessive token usage or introduce significant latency overhead
- The provided examples are universally relevant across all user prompts, maintaining consistency while avoiding biases, ambiguities, or unintended constraints that could negatively affect model responses

Dynamic few-shot prompting is an approach where the examples provided to the model are adaptively selected based on the specific user query or context. Unlike static few-shot prompting, which uses fixed examples regardless of input, dynamic few-shot learning retrieves or generates relevant examples "on-the-fly" to better align with the task at hand.

This method is more advanced but can address the more complex problems where a fixed number of examples can't cover all user inputs efficiently. It also can improve token usage, request latency, and overall prompt performance. Let's look at a few strategies for selecting examples in dynamic few-shot prompting:

- **Random selection:** Some examples are randomly selected. For example, we have 100 examples and select 10 of them randomly. This approach is quite straightforward to implement, but at the same time, it doesn't select the most suitable examples. Random selection is often used as a baseline to compare with other dynamic few-shot techniques.
- **Embeddings and similarity:** Embeddings and similarity metrics (see Chapter 1) can be used to intelligently select the most relevant examples for a given user query.
- **Example selection classifier:** In some cases, a classifier can be used before selecting the most relevant examples. This approach works well when examples can be grouped based on specific criteria.

Embedding and similarity

This approach selects examples that are semantically closest to the user's input. Imagine we have a chatbot that answers user questions, and we want to provide the chatbot's prompt with the most relevant few-shot examples. Suppose we have 100 stored examples (pairs of questions and answers), but for each new user query, we only want to include the top 5 most relevant ones.

The process can be implemented in the following way:

- Pre-compute embeddings for all 100 stored questions (not answers). These embeddings only need to be generated once, such as at application startup.
- Embed the user's question to produce a vector representation, i.e., the user question embedding.
- Find the top k matches (5 in this case) by comparing the user embedding with the stored examples' embeddings using a similarity metric such as cosine similarity or Euclidean distance
- Select the corresponding pairs (the matched questions and their answers) and include only those in the LLM request (context).

This way, the model receives only the most semantically relevant examples instead of all 100, keeping prompts concise and focused. The same idea can be applied to other tasks, such as classification, summarization, or reasoning.

To see how this works in practice for a classification task, we'll use the `SemanticSimilarityExampleSelector` utility from the LangChain library in Python:

```
import openai
import os
openai.api_key = os.getenv("OPENAI_API_KEY")
def run_prompt(prompt):
    response = openai.ChatCompletion.create(
        model="gpt-4o",
        messages=[{"role": "user", "content": prompt}],
    )
    return response['choices'][0]['message']['content']

user_question = "Identify key factors a company must consider when expanding into
a new international market."
countries_to_analyze = ["Brazil", "Germany"]

# --- Step 1: Problem Breakdown Prompt ---
problem_breakdown_prompt = f"""
You are an AI assistant specializing in helping companies develop marketing and
business expansion strategies.
{user_question}. List the major categories, such as economic viability, legal
regulations,
market demand, competition, and infrastructure.
"""

# Run Step 1
breakdown_response = run_prompt(problem_breakdown_prompt)
print("\nProblem Breakdown Response:\n", breakdown_response)

# Parse factors (simple split)
factors = [line.strip() for line in breakdown_response.split("\n") if
line.strip()]

# --- Step 2: Factor Analysis Prompts ---
factor_analysis_results = []
```

```

for factor in factors:
    factor_prompt = f"""
    You are analyzing a specific factor related to {user_question}.
    Factor: {factor}

    Compare {countries_to_analyze} countries based on this factor.
    Identify key elements, assess potential risks and opportunities, and provide a
    concise comparison.
    """

    factor_result = run_prompt(factor_prompt)
    factor_analysis_results.append(factor_result)
    print(factor_result)

# --- Step 3: Final Synthesis Prompt ---

# Prepare input for final synthesis
synthesis_input = "\n".join(factor_analysis_results)

final_synthesis_prompt = f"""
You are an AI assistant specializing in helping companies with marketing and
business expansion strategies.
You are provided with the following analyses related to {user_question}:

{synthesis_input}

Based on these analyses, determine which country ({countries_to_analyze}) is
better suited for expansion.
Justify your recommendation by carefully weighing economic, legal, and market
demand factors.
"""

final_response = run_prompt(final_synthesis_prompt)
print("\nFinal Recommendation:\n", final_response)

```

In the preceding code example, we begin by defining five labeled examples for a support request classification task. Next, we initialize a `SemanticSimilarityExampleSelector`, which relies on MiniLM sentence embeddings and a FAISS vector store (which will be introduced in *Chapter 5*). We configure the selector to return the top 2 most similar examples from the set of 5. Then, we construct a few-shot prompt template using LangChain's `FewShotPromptTemplate` class, which automatically incorporates the selected examples into the final prompt.

When a user query is passed in, the selector identifies the most relevant examples and inserts them into the prompt, grounding the model's response in contextually similar cases. For instance, for the user question "Can I get a refund?", the code selects the 3rd example and the 1st example, ranking them in order of semantic similarity.

```
43 print("Generated Prompt:\n" + final_prompt)
✓ 1.6s

Generated Prompt:

Classify the category of the following support request.

User: How do I cancel my subscription?
Category: Cancellation

User: I need help with my billing
Category: Billing

User: Can I get a refund?
Category:
```

Figure 3.2: The final prompt with the two selected examples from the preceding code

So, we can see that the code identified the 3rd example (line 10) and the 1st example (line 8) as the most similar to the user's question "Can I get a refund?", selecting them in that order based on semantic similarity.

Note that one of the main challenges with this approach is deciding how many examples to include (k). In this case, $k = 2$ (line 27). While the first example is highly relevant, the second is less so, highlighting the trade-offs when working with small sets of examples.

In real-world scenarios, you may have hundreds of stored examples and choose the top 5 or 10. However, not all selected examples will necessarily align closely with the user query, which can lower the quality of the final prompt. To address this, you can apply a similarity threshold using the `vectorstore_kwargs` parameter, ensuring that only examples above a certain score are included.

It's also important to note that the embeddings-and-similarity approach introduces additional overhead. It adds complexity to the overall solution and requires extra compute and memory resources to pre-compute example embeddings, embed each user query, and perform similarity searches. As a result, this method can impact latency and performance.

On the other hand, as we already mentioned above, reducing the size of the LLM context and selecting more tailored examples can improve both performance and accuracy. The trade-off depends on the task: the final decision on whether to adopt this approach should be based on task-specific testing and evaluation.

Example selection classifier

In some cases, a classifier can be used to help select the most relevant examples. As discussed in *Chapter 1*, a classifier is a model or a solution designed to perform classification tasks, i.e., assigning inputs to predefined categories.

This approach works particularly well when examples can be meaningfully grouped based on specific criteria. By using a classifier to route the user prompt to the most appropriate example group, we can improve context selection and enhance the LLM's performance.

For example, consider the task of generating SQL queries from natural language questions, such as the NL2SQL problem (see *Chapter 4*). This type of task is common in question answering and business analytics, where users need to interact with tabular data such as Excel spreadsheets, SQL, or NoSQL databases by using human language.

Since SQL queries can vary greatly in complexity, it can be useful to categorize prompts into three groups based on their structure:

- Simple queries: Basic SELECT statements without joins or subqueries
- Nested queries: Queries that include subqueries
- Queries with joins: Queries involving multiple tables using JOIN operations

Before calling the LLM, a classifier can be used to categorize user prompts into one of the three groups. This helps ensure that the model is provided with the most appropriate examples for the given query.

For instance, consider the following user questions (prompts) and the corresponding SQL queries needed to generate the proper answers:

User question (prompt)	Classified group	SQL query
Get all employees from the HR database.	Simple queries	<pre>SELECT * FROM employees</pre>
Get all orders where the customer has spent more than \$500 in total.	Nested queries	<pre>SELECT * FROM orders WHERE customer_id IN (SELECT customer_id FROM orders GROUP BY customer_id HAVING SUM(total_amount) > 500)</pre>

User question (prompt)	Classified group	SQL query
Show all products along with their category names.	Queries with joins	<pre>SELECT products.product_name, categories.category_name FROM products JOIN categories ON products.category_id = categories.category_id</pre>

Table 3.1: Prompts and their corresponding SQL queries

Using a classifier to route user queries into the right group allows us to provide more targeted examples for the LLM. The downside, of course, is that building a classifier is not always straightforward. Depending on the task, data availability, and performance needs, different approaches can be used. The following table lists some common methods, along with their strengths, weaknesses, and best-fit scenarios:

Method	Pros	Cons	Best for
LLM-based classifiers	Highly flexible; requires no labeled data	Expensive, relatively slow, sometimes inconsistent, and difficult to debug.	Tasks needing adaptability without fine-tuning, though cost and latency must be considered
Fine-tuned language model (like BERT)	Can be efficient for domain-specific tasks	Requires labeled data, compute resources, and experience	Structured text classification where accuracy and efficiency are balanced
Classical ML models (e.g., logistic regression, Naive Bayes)	Simple, fast, interpretable	Struggles with complex language and nonlinear patterns; requires labeled data	Small datasets with clear, well-defined text structures

Method	Pros	Cons	Best for
Rule-Based (Heuristic)	Explainable and requires no training	Poor scalability, can struggle with unstructured input, not generalizable	Well-defined, structured classification tasks with clear, stable rules

Table 3.2: An analysis of classifier building methods

As you can see, there's no single best option. LLM-based classifiers are the most flexible and relatively easy to implement, but they can also be costly and slow. In practice, smaller LLMs like GPT-4-mini or LLaMA can be used for classification, which makes them faster and more efficient than running a full-scale model or relying on large static few-shot prompts. We'll return to text classification tasks in more detail in *Chapter 4*.

Fine-tuned models strike a strong balance between efficiency and accuracy when labeled data is available. However, they require large, high-quality datasets, GPU resources, and advanced ML expertise. Fine-tuning is a substantial topic in itself and is beyond the scope of this book, though we'll touch on key aspects of it in *Chapter 6*.

Classical ML models remain a good choice for smaller, simpler tasks. This approach typically requires medium-sized training datasets and specialized ML knowledge. We won't cover these methods in depth here, but if you're interested, we encourage you to explore further resources dedicated to traditional machine learning techniques.

Core prompt techniques

In the previous section, we explored few-shot prompting, which is one of the most powerful prompt engineering methods. Now it's time to examine other core techniques that are widely used in real-world applications.

This section highlights the most common and influential prompting strategies. Since the field of prompt engineering is evolving rapidly, it's neither possible nor necessary to cover every variation or niche method. Instead, our goal is to provide a clear overview of the most impactful approaches and the intuition behind why they work.

It's also important to build a practical understanding of when and how to apply these techniques. As the industry evolves, some strategies may become less relevant, while others will remain foundational. By strengthening your intuition here, you'll be better equipped to adapt, combine, and innovate as new prompting methods emerge.

Chain of thought

One of the most popular and well-studied prompting techniques is **chain-of-thought (CoT)**. This method encourages the LLM to *think out loud* by generating intermediate reasoning steps before arriving at a final answer. It helps the model break down complex tasks into smaller, more interpretable parts, often leading to improved accuracy and transparency.

CoT prompting typically involves adding an instruction or hint to the prompt, such as:

- "Let's think step by step", the most popular prompt hint
- "Explain your reasoning step by step"
- "Walk through your thought process before answering"
- "Break the problem into smaller steps and solve each one"

There are many variations of this type of instruction, but the core idea is to encourage the LLM to explain its reasoning step by step, rather than jumping straight to an answer. This helps the model simulate a more deliberate thought process.

For example, consider the following prompt:

Prompt

A customer's monthly subscription revenue dropped by 25% in the last two months.
What could be the potential reasons for this decline? Think step by step.

Without the "think step by step" instruction, the LLM might jump to an answer right away. But with CoT prompting, the model is more likely to reason through possible causes systematically, diving deeper step by step.

CoT reasoning has become one of the most widely researched techniques. It works because LLMs "can think better loudly," or more accurately, generate more text that allows the model to derive deeper insights and conclusions. This technique can be useful for tasks that require complex reasoning rather than simply retrieving factual information.

By generating more text and reasoning through problems step by step, LLMs can leverage their autoregressive nature (see *Chapter 2*) to produce more accurate and detailed responses. More tokens lead to a richer intermediate context, which can result in better internal embeddings and allow the model to build a stronger semantic representation of the problem before producing a final answer. That's why CoT works.

However, newer models incorporate CoT-like reasoning natively. Advanced models, such as GPT-4o, GPT-o1, DeepSeek R1, Claude 3, and Gemini, have been trained with inherent step-by-step reasoning capabilities. Therefore, explicit CoT prompting is becoming less necessary because these models already reason well internally.

So, adding "Let's think step by step" can't always yield visible improvements as it did for older models (e.g., GPT-3.5). However, CoT is not entirely obsolete; while newer models handle reasoning better, explicit CoT prompting can still improve performance for complex tasks, such as logical puzzles, advanced analytics, and multi-hop reasoning. Note that performance can vary by model and task. So, CoT can still be used.

Using CoT has overheads as well. It forces LLM to be more verbose and thus to consume more output tokens. It also adds more latency to prompt execution due to the sequential LLM auto-regressive behavior.

Tree-of-Thought prompting

Tree-of-Thought (ToT) is a prompting technique that forces LLM to consider different paths (branches) and select the best one. ToT is an extension of CoT, but instead of one path, it forces the model to evaluate different branches within one prompt. Here are the most popular prompt hints for ToT:

- *"Explore multiple possible ways to solve this problem before deciding on the best approach"*
- *"Generate multiple approaches to solve the problem, compare those approaches and select the best one based on evaluation criteria"*
- *"Generate at least three different approaches to solving this and compare their effectiveness"*
- *"Break the problem into subproblems and explore different solutions for each"*
- *"Think about this problem from different perspectives: logical, intuitive, and probabilistic"*

ToT, similar to CoT, provides more content and can get better results for problems that require advanced reasoning, planning, and solving problems with multiple possible solutions.

One of the keys for ToT to start working is the *criteria* for the selection of the best outputs. The effectiveness of ToT depends on the ability to evaluate multiple reasoning paths and determine which one is most optimal. If such criteria are not provided, the LLM will use its own (default) reasoning capabilities to select the best answer. In some cases, these criteria can differ dramatically from what the problem actually needs.

For instance, consider this prompt:

Prompt

You are a business strategy consultant helping a SaaS company decide how to expand into a new market.

Step 1: Generate at least three different market expansion strategies based on industry's best practices.

Consider different approaches such as local partnerships, aggressive pricing, and feature differentiation.

Step 2: Evaluate each strategy using the following criteria:

```
Revenue Growth Potential (High, Medium, Low) - How much additional revenue can
this strategy generate?
Customer Acquisition Speed (Fast, Medium, Slow) - How quickly can this strategy
attract new customers?
Implementation Cost (High, Medium, Low) - How expensive is this strategy to
execute?
Risk Level (High, Medium, Low) - How risky is this approach in terms of
competition and market response?
Step 3: Select the best strategy based on the following rules:
Prioritize strategies with HIGH revenue growth and FAST customer acquisition.
Avoid strategies with HIGH risk unless they also have LOW implementation costs.
If two strategies are equally strong, prefer the one with a faster break-even
period.
Step 4: Justify why the selected strategy is the
best. Provide a clear explanation based on the criteria above.
```

This prompt defines 3 steps: generate multiple (three) strategies, evaluate, and select the best one. The prompt has well-defined criteria for how to evaluate and find the best strategy. That gives the LLM the reasoning path to select the best answer.

Currently, ToT prompting is not natively integrated into most LLMs. Implementing ToT typically requires external orchestration to manage multiple reasoning paths or branches.

Self-consistency prompting

Self-consistency prompting is a reasoning technique that forces an LLM to consider a few paths of generating a response. It is done by running the same prompt multiple times with moderate or high randomness (see *Chapter 2*) and selecting the most common output. Additionally, a *self-consistency hint* is used in prompts to force language models to try to discover different solutions.

Let's consider this prompt example:

Prompt

```
You are a senior business analyst at a SaaS company that has seen revenue growth
fluctuating over the past year. Based on the following business context, predict
the best sales strategy for the next quarter and justify your reasoning:
```

```
...
```

```
Given this data, provide a strategic sales recommendation by considering multiple
potential approaches. Ensure your response includes a detailed reasoning process.
```

```
Final Answer Format:
```

Justification: [Step-by-step reasoning]

Recommended Sales Strategy: [Summary of strategy].

In this prompt, we can notice two hints: "*considering multiple potential approaches*" (self-consistency hint), which forces the model to consider different paths, and "includes a detailed reasoning process." This instruction is actually modified to the CoT hint that pushes the LLM to discover more reasoning details. By running this prompt multiple times (for instance, 5 times) and with moderate temperature (0.4-0.6) or high (1+), we can get responses such as the following:

- "*Upsell AI-driven automation to enterprise clients*"
- "*Launch a targeted discount program for SMBs while keeping enterprise pricing stable*"
- "*Increase AI feature promotions for enterprise customers, as they are more willing to pay*"
- "*Reduce marketing spend further and focus on organic channels like webinars*"
- "*Enhance customer retention by offering AI automation consulting services*"

Now the question becomes: which answer should we select as the best one? There are several strategies for making this choice, often referred to as *voting mechanisms*:

- **Majority voting (most frequent answer):** Choose the response that appears most often. This works best when answers fall into well-defined categories, such as in classification tasks.
- **Clustering by similarity:** Group responses based on semantic similarity and select the largest group (cluster). For example, in the outputs above, answer #3 could be chosen as the final output since it aligns most closely with answers #1 and #5, forming the largest group.
- **Probability-based selection:** Use token-level logprobs (see *Chapter 2*) and choose the response with the highest overall probability.

Self-consistency prompting can give better performance for problems that require more advanced reasoning capabilities. But at the same time, the price for such additional reasoning is the extended latency, more token usage, and solution complexity, especially for advanced selection strategies.

Note that all of the prompt engineering techniques covered so far have been *single-prompt* techniques, where a single prompt is used. Self-consistency is also formally considered a single-prompt technique; however, it requires running the same prompt multiple times. Therefore, we can think of self-consistency as a *multi-prompt* technique to some extent, since it involves generating and comparing multiple outputs from repeated runs of the same prompt.

System role or persona-based prompting

System role technique, also known as the **persona-based prompting**, is a prompt engineering method where the LLM is assigned a role, specific persona, or expertise to guide its behavior, tone, and response style.

With this prompting technique, we define a role in the system prompt (or user prompt) to make responses more contextually accurate. The LLM will then *imitate* the behavior, knowledge, and style expected from that role or persona.

For example, consider the following prompt:

Prompt

```
You are an experienced data scientist with expertise in machine learning and statistical analysis.  
Explain the concept of PCA (Principal Component Analysis) in simple terms for a beginner.
```

In this example, the first sentence is a system role hint that prepares the LLM to answer as an "*experienced data scientist*". Roughly, this hint "*turns on*" the LLM capability to impersonate and respond as a data scientist.

System role technique allows:

- **Improves response relevance:** The LLM adapts its knowledge, tone, and style based on the role
- **Enhances accuracy:** The model filters responses to align with domain expertise based on the data it was trained on

Note that early proprietary LLMs like GPT allowed for *human impersonation*, enabling them to mimic individuals convincingly. For example, prompts like "You are Michael Jackson ..." would lead the model to respond as if it were the real Michael Jackson. However, in later versions, this capability was intentionally restricted due to privacy, ethical, and security concerns, preventing potential misuse such as identity fraud, misinformation, and deceptive interactions.

Other techniques

The field of prompt engineering is evolving rapidly, with new techniques and variations emerging all the time. While we focus mainly on the core techniques in this book, we encourage you, the reader, to explore the additional methods outlined below on your own. Gaining hands-on experience with these approaches will help you better understand their strengths, limitations, and potential applications. Here are a few additional techniques you can consider in more detail:

- **Meta prompting:** Gives LLM instructions about how to think or how to structure its response rather than to answer a specific question
- **Self-ask prompting:** Encourages the model to generate and answer intermediate questions that lead to the final answer
- **Reflection:** Involves the model reflecting on its previous outputs to improve future responses, fostering a form of self-improvement
- **Contrastive prompting:** Improves LLM reasoning by showing both correct and incorrect examples within the prompt
- **Template-based prompting:** Involves creating a fixed prompt structure where only specific variables (like user input) are inserted dynamically

It's important to recognize that the effectiveness of prompt engineering techniques is closely tied to the capabilities of the underlying LLM. A technique that yields excellent results with one model may perform suboptimally with another, especially when considering variations in model architecture, size, or version.

For instance, CoT prompting has been shown to significantly enhance reasoning tasks in large-scale models, particularly those with over 100 billion parameters. However, smaller models often struggle to produce coherent reasoning chains, leading to diminished performance compared to standard prompting methods. Additionally, as mentioned earlier, CoT prompting has become less impactful for many modern models, especially those optimized for reasoning, because step-by-step thinking patterns are now often already incorporated into the model's internal behavior.

As LLMs continue to evolve, some prompting approaches may become less effective or even obsolete, while others may gain prominence. Therefore, it's crucial to tailor prompt engineering strategies to the specific capabilities and limitations of the chosen model.

Multi-prompt engineering techniques

While many problems can be effectively solved with a single well-crafted prompt, more complex tasks often push this approach to its limits. For challenging tasks, a prompt may grow significantly in size, incorporating multiple instructions, branching flows, examples, and hints. As a result, the efficiency and maintainability of such a prompt can decrease considerably.

In such cases, the classical "divide and conquer" strategy, with splitting one prompt into multiple prompts, can reduce prompt complexity and improve overall task performance. Here are the most common cases when building a multi-prompt flow is beneficial:

- When the size of the prompt (system + user prompt) starts to exceed the LLM context window
- When the prompt fits within the context window but grows relatively large, performance may start to degrade. As discussed in *Chapter 2*, many modern LLMs begin to lose context once prompts exceed roughly 8–16K tokens, which can reduce the overall effectiveness of the response.
- When the prompt contains different types of tasks. For example, one prompt can have both a classification and a content generation task. So, splitting such a prompt into two with classification and generation can improve performance and make evaluation easier. As you will see in *Chapter 8*, the classification problems are more straightforward to evaluate compared to generation ones.

To demonstrate the last point above, let's consider the following prompt example:

Prompt

```
You are an AI assistant responsible for managing customer support emails.  
Your task is to respond only if the customer inquiries about their support ticket  
status or provides additional details related to an ongoing ticket.  
- If the customer requests a ticket status update, generate a professional  
response including the ticket details.  
- If the customer provides additional information about their issue, analyze it  
and attempt to determine a solution before responding.  
- If the email does not contain a ticket inquiry or relevant details, do not  
generate a response.  
Ensure that replies are clear, concise, and professional.
```

As we can notice, this prompt actually has two tasks: first, classification and next, email reply generation. So, this prompt can be split into the following prompts:

1. **Classification prompt:**

Prompt

You are an AI assistant managing customer support emails.
Your task is to classify each email into one of the following categories:

- Ticket Status Inquiry → The customer is asking for an update on their support ticket.
- Ticket Issue Update → The customer provides new details related to their existing support case.
- No Action Needed → The email does not contain a ticket inquiry or relevant issue details (e.g., general feedback, unrelated questions).

...

2. **The ticket status generation prompt:**

Prompt

You are an AI assistant managing customer support emails.
The customer has requested a status update for their support ticket. Using the ticket details provided generate a professional and informative response.

3. **Ticket solution generation prompt:**

Prompt

You are an AI assistant managing customer support emails.
The customer has provided additional details about an existing support ticket. Analyze the information and generate a professional response that includes a possible solution.

So, we split one prompt into three: one for problem classification and two for getting ticket status updates and solution generation.

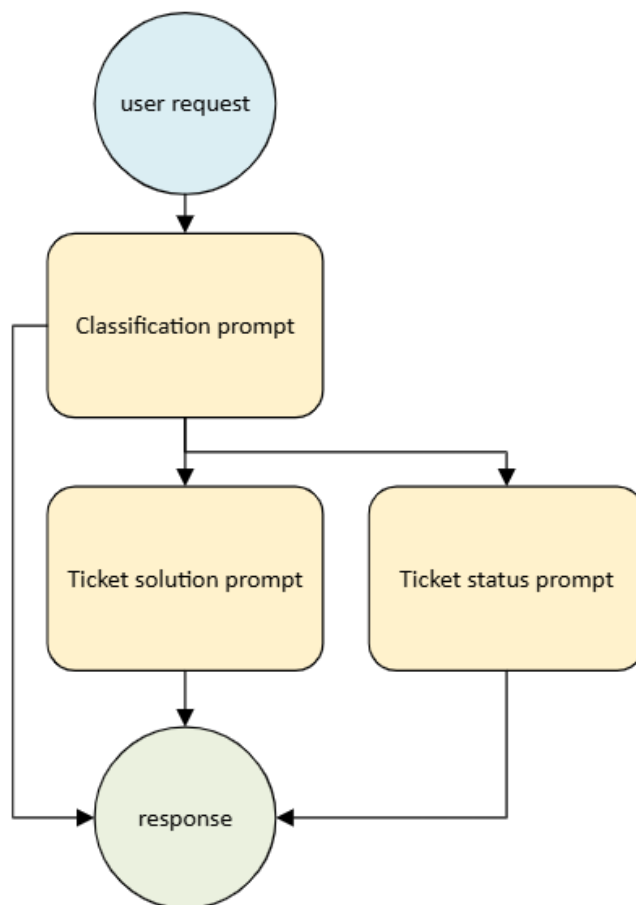


Figure 3.3: Three prompt flows to handle customer requests

By structuring the task this way, we simplify the overall flow and can now separately define details and examples for each individual prompt. This approach also allows us to test and refine each prompt in isolation, making it easier to manage complexity. Smaller, more focused prompts often lead to better accuracy and more targeted outputs.

However, it's important to note that the multi-prompt approach is inherently more complex than a single-prompt flow and introduces additional latency, since multiple prompt calls must be made separately. It's also worth mentioning that splitting one large prompt into several smaller ones can increase token usage, as some context needs to be passed between prompts. As a result, multi-prompt flows typically come with additional costs in both time and token consumption.

Static multi-prompt flows

The example above illustrates *static or deterministic* multi-prompt execution, where the structure, and flow are strictly defined in code and remain unchanged from one user request to another. This type of flow works well when each prompt has a clearly defined role. In the example provided, one prompt served as a classifier, while the other two were responsible for generating content based on the classification outcome.

Deterministic flows are also more stable and predictable due to their static nature. Since the sequence of prompts and logic paths is predefined, the behavior of the system remains consistent across different inputs. When a specific scenario can be reliably handled by one of a few well-defined flows, it is generally better to use deterministic execution.

This approach not only improves stability but also makes testing, debugging, and optimization much easier, since the possible execution paths are limited and well-controlled. Additionally, both individual prompts, and the overall flow can be tested in isolation using traditional testing techniques, similar to practices commonly used in regular software engineering.

However, deterministic multi-prompt flows can be rigid because of their fixed structure, which limits their ability to adapt to dynamic situations. In more complex cases, the large number of possible variations and edge cases can make it difficult, or even impractical, to design a fully deterministic flow that handles every situation. As a result, these rigid flows may struggle to deal with nuanced or unexpected inputs effectively.

For example, imagine we are developing a financial chatbot designed to assist users with personal finance questions. This chatbot needs to handle a wide range of ad-hoc financial calculations based on user queries. Since users can ask for anything from simple expense summaries to complex investment projections, it becomes very difficult to define a fully deterministic multi-prompt flow that covers every possible scenario. The high variability and unpredictability of user inputs would require a more flexible, dynamic approach rather than a rigid, predefined flow.

Next, we will explore several dynamic multi-prompt techniques.

Dynamic multi-prompt flows

In contrast to deterministic or static multi-prompt flows, *dynamic flows* do not follow a fixed prompt execution schema that applies to all user requests. Instead, the execution plan is determined on the fly, adapting to the specific input, context, or intermediate results.

For example, imagine we have three prompts, each responsible for a distinct task. In one request, only a single prompt may be triggered and executed once. In another, all three prompts might be used, but in a different sequence depending on the output of the first

prompt. This flexibility allows the system to tailor the reasoning or problem-solving process to the situation.

Dynamic multi-prompt schemas are particularly powerful when used in *multi-agent* communication. Each agent may handle a specialized task, such as retrieval, reasoning, or evaluation, and the orchestration between them is decided dynamically based on the evolving conversation. In practice, this means that the same high-level goal (e.g., answering a user's question) can be reached through different pathways, depending on what intermediate steps prove necessary.

While we will dive deeper into agents and their orchestration in *Chapter 7*, it is important here to highlight the key features of dynamic multi-prompt execution:

- **Adaptivity:** The LLM selects which prompts to run, and in what order, based on the input and intermediate results
- **Non-linear execution:** Prompts can be skipped, repeated, or reordered dynamically, rather than executed in a fixed sequence
- **Context sensitivity:** The choice of which prompt to invoke may depend on external context (e.g., retrieved knowledge, user feedback, or monitoring signals)

In short, dynamic multi-prompt flows represent a more flexible paradigm than static ones. They sacrifice predictability for adaptability, which is often important in open-ended tasks where the path to the answer cannot be fully known in advance.

However, this flexibility does not come for free; dynamic multi-prompt flows introduce several important limitations and trade-offs:

- **Implementation complexity:** Building a dynamic flow is significantly more challenging than a static one. The orchestration logic must account for branching, looping, and conditional execution. In most cases, specialized frameworks such as AutoGen (see *Chapter 7*) are required to manage these interactions effectively.
- **Stability and predictability:** Dynamic flows can be less stable and harder to predict. Without careful safeguards, they may trigger unnecessary LLM calls, waste resources, or even fall into endless loops. Designing robust stopping conditions, guardrails, and monitoring mechanisms becomes essential.
- **Increased token consumption and latency:** Because multiple prompts may be triggered on demand, dynamic schemas often consume more tokens and introduce higher latency compared to static flows. This makes them more expensive to run and

potentially slower to deliver results, especially for complex tasks that require many iterations.

- **Debugging and evaluation challenges:** Unlike static flows, where the execution path is known in advance, dynamic flows are harder to trace and evaluate. This makes debugging errors, reproducing results, and building reliable evaluation datasets more complicated.

In short, while dynamic schemas bring adaptability and power, they require careful design, monitoring, and cost management. To implement them successfully, one must balance flexibility with control, ensuring that the system remains efficient, reliable, and aligned with user expectations.

Multi-prompt chain of thought

Multi-prompt chain-of-thought (CoT) is an extension of the traditional single-prompt CoT approach. Instead of using one long prompt to guide the model through an entire reasoning process, multi-prompt CoT breaks down a complex problem into a series of smaller, specialized prompts, where each prompt handles a different stage of the reasoning process. Each step builds upon the previous output, gradually refining the solution.

This approach is especially useful for handling highly complex reasoning tasks or multi-step logic problems that might overwhelm a single prompt. By dividing the problem into smaller, focused steps, the model can reason more systematically and accurately.

Here's an example of one possible scheme for applying multi-prompt CoT:

- **Prompt 1: Problem breakdown:** The first prompt analyzes and decomposes the problem into several subproblems
- **Prompt 2: Step-by-step subproblem solving:** Each subproblem is solved individually using CoT reasoning
- **Prompt 3: Synthesis and final answer:** The AI integrates all partial solutions into a coherent final response

For example, imagine we are developing a chatbot designed to assist companies with building marketing and business expansion strategies. This chatbot needs to handle complex, open-ended questions, such as when a company is considering expanding into a new international market.

In such a case, the AI would need to evaluate multiple factors, including economic conditions, legal regulations, and competitive landscapes, before forming a well-rounded recommendation. Handling this level of complexity effectively would benefit from a multi-prompt CoT approach, where the AI first breaks down the problem, analyzes each factor individually, and then synthesizes the findings into a final strategic recommendation.

For this example, when the user asks, "Identify the key factors to consider when expanding into a new international market," the first step could be to use a prompt that breaks down the overall problem into smaller, more manageable subproblems:

Prompt

You are an AI assistant designed to help companies with building marketing and business expansion strategies.

Task:

identify the key factors to consider when expanding into a new international market.

Output

Format: List the most important categories (e.g., economic viability, legal regulations, market demand, competition, infrastructure).

For this case, the LLM might generate a list of key problems or factors like the following:

- **Economic viability:** GDP growth, consumer purchasing power, inflation.
- **Legal regulations:** Business laws, tax policies, compliance risks.
- **Market demand:** Industry trends, local consumer behavior, demand forecasts.
- **Competition analysis:** Local vs. international competitors, market saturation.
- **Infrastructure and logistics:** Supply chain readiness, local business networks.

At the second step, we can now analyze each factor individually. To do this, we need a specialized prompt that guides the LLM to focus on one specific task at a time. This prompt has a placeholder ([Insert Factor Here]) where each factor obtained from step #1 should be inserted:

Prompt

You are an AI assistant designed to help companies build marketing and business expansion strategies.

Analyze the following factor related to international market expansion: [Insert Factor Here].

For this factor, please:

- Identify key elements and considerations.
- Assess potential risks and opportunities.
- Provide a concise summary of your findings.

And finally, after getting all reasoning for every factor, we need to synthesize and form an answer with the prompt like this one:

Prompt

You are an AI assistant designed to help companies with building marketing and business expansion strategies.

You are provided with the following analyses related to international market expansion:

[Insert Factor Analysis 1 Here].

[Insert Factor Analysis 2 Here].

...

Based on these analyses, determine which country is better suited for expansion. Justify your recommendation by carefully weighing the economic, legal, and market demand factors.

This final prompt is responsible for generating the complete answer to the user's question.

Note that the three prompts shown here were crafted specifically for the example question: *"Identify the key factors to consider when expanding into a new international market"*. This intentional design choice makes the illustration more intuitive and easier to follow. In real-world use, however, more general prompts, intended to handle a wider variety of questions, would need slightly different structures to accommodate diverse problem types.

Summarizing the steps above, we now have the following prompt flow schema for the multi-prompt CoT approach:

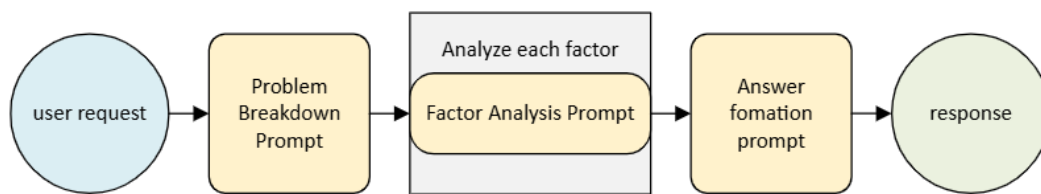


Figure 3.4: Multi-prompt CoT flow

It's also important to note that the second stage in the schema should be executed separately for each factor. Above, we have the process with three prompts that can result in more than three LLM calls, since the second prompt must be applied to each factor generated by the first prompt.

As we can see, single-prompt CoT and multi-prompt CoT have several fundamental differences, but they also share some important similarities. Let's compare them feature by feature:

CoT feature	Single-prompt CoT	Multi-prompt CoT
Handles complexity	Moderate	Advanced due splitting prompts
Step-by-step analysis	Yes	Yes (more granular)
Independent subproblem solving	No	Yes
Implementation complexity	Low	Higher
Token usage	Moderate	Higher
Latency	Moderate	Higher

Table 3.3: Comparison of single-prompt and multi-prompt CoT

Multi-step CoT can be viewed as a partly static (or deterministic) flow, since it follows predefined stages like breaking a complex problem into sub-problems and analyzing them separately. However, because the number and structure of those sub-problems are determined dynamically, it also functions as a dynamic flow.

Iterative self-refinement

Iterative self-refinement prompting is a technique where an LLM gradually improves its responses by first generating an initial output and then refining it through multiple rounds of self-correction, feedback loops, or guided modifications. This approach is also often referred to as **self-reflection**.

With iterative self-refinement, we break down the task into multiple sequential steps. For example, we can structure the process into tasks like the following:

- **Initial generation:** The model provides a first attempt
- **Self-reflection:** The response is reviewed for accuracy, clarity, or style
- **Refinement steps:** The model improves the response based on evaluation criteria
- **Final output:** After multiple iterations, the most polished version is presented

The steps above can differ depending on the specific task. The first stage can be generated by an LLM or obtained from humans as input. The next steps can refine the output sequentially for specific aspects.

For example, consider the following refinement steps:

1. Generate an initial response

Prompt:

```
"Write a detailed summary of the inflation effects on the global economy."
```

2. Request self-reflection and improvements

Prompt:

```
"Refine this summary to include specific examples and a more structured format."
```

3. Apply final refinement for style and depth

Prompt:

```
"Make this summary more engaging by using concise, reader-friendly language."
```

Iterative refinement is particularly useful for some complex tasks like long form writing, code debugging, and decision-making, where a single response might be suboptimal. This approach can work better because the LLM has more context and more attempts to improve the initial response with a specific focus.

Iterative refinement is very similar to how humans work with large documents. People mostly start with a draft and then improve it, focusing on some aspects within each iteration. For instance, first we add some raw content, then we organize it and finally correct grammar, styles, and phrasing. Similarly, for LLM, it can be easier to focus on one specific aspect rather than on all of them altogether.

Note that the list of refinement steps can either be fixed and defined deterministically or dynamically generated by the LLM itself. This allows for two general approaches:

- Generate self-assessment and identify areas for improvement
- Propose the next logical refinement step based on detected weaknesses
- Continue iteratively refining until the output meets the desired quality standards

In this case, the LLM adaptively improves the quality of the document through multiple passes. So, the self-refinement technique can be both a static or dynamic multi-prompt flow.

Prompt chaining

Prompt chaining is a technique where multiple prompts are sequentially linked together, with each prompt building upon the response of the previous one. This is another way to reduce the overall prompt complexity by using the "divide and conquer" principle. There are several prompt chaining variations:

- **Sequential chaining (step-by-step breakdown):** Each step builds on the previous until reaching a final decision. For example, it could be the following prompt sequence: ***Generate market research data → Analyze trends → Generate investment recommendations***
- **Branching chaining (parallel analysis):** Different prompts explore multiple perspectives or alternatives. For example, we can have the following branch:
 - Branch 1: AI writes the pros of an expansion
 - Branch 2: AI writes the cons of an expansion
 - Branch 3: AI weighs the pros and cons and makes a recommendation
- **Recursive chaining (self-Refinement):** An LLM reviews and refines its own response in multiple cycles. For instance, we can have the following chain: ***Initial draft → AI critiques it → AI refines it → Final version***
- **Multi-agent chaining:** Different prompts simulate different roles or experts in a discussion. For instance:
 - **Prompt 1 (CEO View):** "Analyze AI adoption from a CEO's perspective"
 - **Prompt 2 (CTO View):** "Analyze AI adoption from a technology leader's perspective"
 - **Prompt 3 (CFO View):** "Analyze AI adoption from a financial risk perspective"
 - **Final Prompt:** "Combine all perspectives into a final report"

As we can see, prompt chaining shares some similarities with iterative self-refinement prompting (self-reflection) and multi-prompt flows like the multi-prompt CoT approach. However, the key difference is that prompt chaining focuses specifically on linking multiple prompts into a sequence (called a **chain**), creating a structured flow where the output of one prompt becomes the input for the next.

The LangChain framework has made prompt chaining a central concept in its design. Let's look at a simple example of chaining two prompts:

```
from langchain.prompts import PromptTemplate
from langchain.llms import OpenAI
from langchain.chains import LLMChain, SimpleSequentialChain
```

```
# Initialize the language model
llm = OpenAI(temperature=0.5)

# Step 1: Create a prompt template to generate a business idea
idea_prompt = PromptTemplate.from_template(
    "Suggest an innovative business idea in the {industry} industry."
)
idea_chain = LLMChain(llm=llm, prompt=idea_prompt)

# Step 2: Create a prompt template to write a marketing pitch for the business
idea
pitch_prompt = PromptTemplate.from_template(
    "Write a short marketing pitch for this business idea: {business_idea}"
)
pitch_chain = LLMChain(llm=llm, prompt=pitch_prompt)

# Step 3: Combine the chains into a sequential flow
chain = SimpleSequentialChain(chains=[idea_chain, pitch_chain])

# Run the chained prompts
result = chain.run("technology")
print(result)
```

The code above first creates a chain for the initial prompt (lines 9–12), then defines a second chain for the next prompt (lines 15–18). Finally, it links the two chains together sequentially (line 21) and runs them as a single flow. The output of the first chain is automatically passed as the input to the second chain.

LangChain also supports more complex and advanced chaining flows, where multiple prompts, models, or tools are combined in dynamic sequences. These flows can involve conditional branching, iterative loops, or even agent-like decision-making, allowing developers to build sophisticated multi-step reasoning pipelines. This flexibility makes LangChain a powerful framework for handling more advanced tasks beyond simple sequential prompt chaining.

Summary

So, we've just completed a deep dive into the core techniques of prompt engineering. After building a foundation in AI basics and exploring LLMs and transformer architectures, this chapter shifted focus toward the practical aspects of creating modern AI-powered applications.

We started by defining the basic prompt structure, introducing user and system prompts, and outlining typical patterns used when building LLM-based applications. From there, we moved into one of the most powerful techniques, few-shot prompting, exploring both zero-shot and

few-shot approaches. We discussed where these techniques work well and where they can be less effective. We also covered static and dynamic few-shot prompting as practical alternatives to full-scale fine-tuning.

Next, we explored a range of core prompt engineering techniques, including CoT, ToT, self-consistency, persona-based prompting, and several others. Beyond single-prompt techniques, we also examined multi-prompt strategies such as multi-prompt CoT, iterative refinement, and prompt chaining. These techniques enable more structured, multi-step reasoning and interaction with external tools.

It's important to note that prompt engineering remains a rapidly evolving field, with new methods and variations appearing almost daily. Every technique can perform differently depending on the LLM being used; what works well for one model might be less effective for another. This is why experimentation, practice, and iterative evaluation are crucial for solving real-world problems effectively.

In the next chapters, we will revisit some of these techniques as we explore AI agents in more detail. There, we'll see how many of these reasoning and action strategies play a critical role in driving agent decision-making and task execution.

Get this book's PDF version and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.

Part 2

Building LLM Applications in Practice

Part 2 moves from foundations to application design and implementation. It explores the major language tasks that modern LLMs can solve, including summarization, generation, information extraction, classification, question answering, and reasoning. It then expands into two of the most important techniques for building real-world AI systems: **retrieval-augmented generation (RAG)** and fine-tuning. Together, these chapters show how to extend model capabilities, ground outputs in external knowledge, adapt models to domain-specific tasks, and design more effective AI applications for practical use.

This part contains the following chapters:

- *Chapter 4, Understanding Language Tasks*
- *Chapter 5, Generation, Question Answering, and Reasoning*
- *Chapter 6, Retrieval-Augmented Generation*
- *Chapter 7, LLM Fine-Tuning*

4

Understanding Language Tasks

In the previous chapters, we introduced the foundations of **artificial intelligence (AI)** and **natural language processing (NLP)**, explored what happens under the hood of **large language models (LLMs)**, and learned core prompt engineering techniques. Now it's time to move from theory and tooling to the practical problems these models can solve.

Modern LLMs are capable of performing a wide range of language-related tasks, from summarizing documents to answering questions, translating between languages, and extracting structured information from unstructured text. These capabilities form the foundation for many modern AI applications and agents.

An effective agent often needs to summarize large documents, classify incoming messages, extract key facts, or answer questions based on available information. Usually, such agents combine several of these tasks together in complex flows to accomplish more sophisticated goals.

In this chapter, we will explore the most important language tasks that modern LLMs can perform. We begin with summarization, one of the most widely used applications of language models. Then we will move through other key areas, including text classification, information extraction, and multilingual processing. Understanding these tasks will provide a practical foundation for building more advanced AI systems later in the book.

We will cover the following key topics:

- Summarization
- Text classification
- Information extraction
- Multi-language support

Technical requirements

The code for this chapter is available on the book's accompanying GitHub repo: <https://github.com/PacktPublishing/Building-Agent-Powered-Applications/tree/main/Chapter%204>

Text summarization

Today, the amount of information is growing exponentially, with an enormous number of documents, financial reports, posts, news articles, blogs, and papers being created every single day. This rapid growth makes the need for efficient tools to process and analyze the ever-increasing volume of data essential, making summarization an essential skill to extract key insights quickly.

Summarization is the process of *compressing* a longer text into a shorter, more concise version while preserving its key ideas and essential details. It is sometimes referred to as **semantic compression**, emphasizing that the goal is not just to shorten the text, but to retain the meaning and critical information.

Note

Historical note

Summarization is one of the oldest challenges in NLP, with research dating back to the 1950s and 1960s. Before the broad development of neural networks, summarization was primarily addressed using rule-based and statistical methods (1950s-2000s). Early approaches focused on identifying key sentences or phrases using metrics like term frequency, sentence position, or cue words. Another strategy involved developing knowledge-based systems that relied on handcrafted rules for domain-specific summarization. While these methods were effective for some basic tasks, they often lacked flexibility and coherence.

Summarization is a form of **lossy compression**, where certain details from the original text may be omitted to focus on the most important information. So, nuances or less critical details can be lost.

Summarization can vary significantly depending on the domain and purpose, with each scenario requiring specific approaches to effectively capture and condense the information. Let's look at a few common types of summarization tasks:

- **News summarization:** Focuses on compressing news articles into concise summaries that highlight the most critical facts and events. Services like Google News or similar utilize summarization to show short news descriptions.
- **Dialog summarization:** Focuses on summarizing conversations, such as meetings, emails, or chat conversations. Such summarization is presented in products like Microsoft Teams (meeting summary), Microsoft Office Outlook (email summary), and similar products.
- **Financial summarization:** Focuses on condensing financial documents such as earnings reports, financial statements, analyst reports, or market updates into concise summaries that highlight key metrics, trends, and risks.

So, we can see that summarization tasks are customizable, requiring a tailored approach to focus on the specific details and priorities of each domain. Now, let's dive deeper into different types of summarizations.

Extractive summarization

Extractive summarization involves selecting and extracting the most important sentences, phrases, or paragraphs directly from the source text (original document) to form a summary. The selection of important text parts can be done using deterministic algorithms or machine learning models.

For example, let's take a short paragraph with five sentences about AI and see which ones might be chosen for the summary:

Text

AI is transforming industries across the globe. In healthcare, AI algorithms assist doctors in diagnosing diseases more accurately and predicting patient outcomes. In the financial sector, AI is used to detect fraudulent transactions and automate trading. Education is another area where AI-powered tools personalize learning experiences for students. In summary, despite its many benefits, AI also raises ethical concerns, such as bias in decision-making and job displacement.

An extractive summary of this text, limited to two sentences, can look like:

- "AI is transforming industries across the globe"
- "In summary, despite its many benefits, AI also raises ethical concerns, such as bias in decision-making and job displacement"

These sentences are considered important based on several factors: their positions in the text (with the first and last sentences often carrying greater significance), the presence of the word "AI," which is the most frequent word, and the fact that the last sentence includes the phrase "in summary," a common *cue* indicating key information.

Extractive summarization typically works as follows:

- The text is split into sentences using a sentence tokenizer (see *Chapter 1*)
- Each sentence is ranked by importance based on specific criteria
- The top-ranked sentences are selected and combined to form the summary

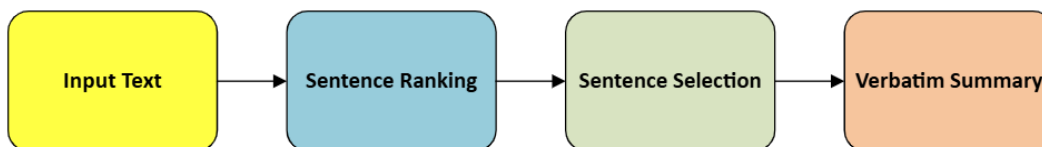


Figure 4.1: Extraction summarization design flow

Extractive summarization can be thought of as a sentence-ranking task: the goal is to identify which sentences carry the most important information. There are two main ways to determine sentence importance, using algorithmic approaches or machine learning models:

Algorithmic approaches rely on calculating text characteristics, such as:

- **Term frequency (TF):** How often keywords appear in a sentence (see *Chapter 6*)
- **Inverse document frequency (IDF):** Words that appear rarely in the document but frequently within a particular sentence are given more weight (see *Chapter 6*)
- **Sentence position:** Sentences at the beginning or end of a paragraph or document are often considered more important
- **Cue words:** Phrases like "important," "conclusion," or "in summary" often signal key content

Machine learning approaches take this further by using AI models to assess the importance of sentences. They can assign importance scores to sentences based on context, semantics, and relationships across the document. For example, the BERTSUM model (a fine-tuned version of BERT) can classify sentences as "summary-worthy" or not, producing more accurate and context-aware summaries.

Note that extractive summarization can be applied not only to sentences but also to larger text units such as paragraphs, pages, or entire sections. This approach can be useful for summarizing large documents, such as books or conversations.

Extractive summarization has several key disadvantages and limitations:

- It often produces disjointed summaries by extracting sentences verbatim
- It lacks the flexibility to rephrase or restructure content for better coherence
- It frequently includes redundant or very similar information and may struggle to capture deeper semantic context or nuances

Despite these limitations, extractive summarization still has niche applications and remains computationally efficient. Extractive methods remain relevant in situations where high factual accuracy is critical or where preserving the exact wording of the original text is necessary.

Abstractive summarization

Abstractive summarization is a technique that generates a concise summary by paraphrasing the main ideas of a text, rather than directly copying sentences from the original. The summaries it produces may include new phrases and sentence structures that do not appear in the source material.

For example, the abstractive summarization for the text in the section above about AI can look like:

Text

AI is revolutionizing industries by enhancing healthcare diagnoses, detecting financial fraud, and personalizing education, but it also raises ethical concerns like bias and job displacement.

We can see that this summary describes the original text much better than extractive summarization. Such a summary is also more *coherent* and *fluent* rather than just extracting a few sentences.

Abstractive summarization involves understanding the context and meaning of the original text and then generating a shorter, rephrased version. This complexity demands deep semantic understanding and the ability to create new text, which is why it was only addressed with transformer-based architectures and LLMs.

Abstractive summarization typically follows these steps:

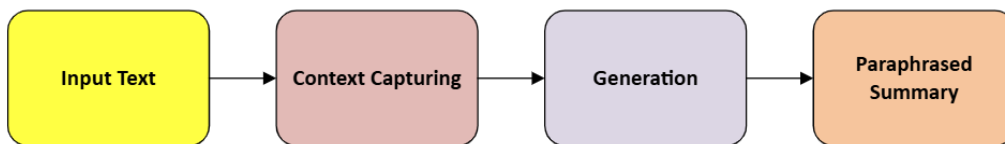


Figure 4.2: Abstractive summarization design flow

As we noted earlier, summarization is a form of lossy compression. The key challenge lies in how LLMs decide which information is essential to keep and which details can be safely omitted. Here are a few factors that LLMs typically consider:

- Context understanding learned during the initial self-supervised pretraining phase
- Fine-tuning on labeled summarization datasets to adapt to specific tasks and identify key patterns of importance
- Prompt instructions, which guide the model to focus on specific aspects or styles of summarization
- Few-shot examples with the relationship between input text and expected output

As we discussed in *Chapter 2*, conflicts between these factors can lead to summary fabrication, where the model introduces information that was not present in the original text. The model may "believe" that facts learned from its pretraining data are more important or more reliable than those in the original input. Additionally, the probabilistic nature of LLMs can introduce another source of errors, leading to inaccuracies even when the model is trying to stay faithful to the original content.

Thus, while abstractive summarization is highly powerful, it also comes with several challenges:

- Models may generate incorrect or fabricated information, compromising factual accuracy
- Ensuring coherence, relevance, and faithful representation of nuanced or domain-specific content can also be difficult

Hybrid summarization can help to address some of the challenges mentioned above. It combines both extractive and abstractive approaches to leverage the strengths of each method. In this technique, the most relevant pages, paragraphs, sentences, or phrases are first extracted from the source text, ensuring that key information is preserved. Then, an abstractive process is applied to rephrase, condense, and enhance the extracted content, improving coherence, readability, and overall fluency.

Summarization parameters

Today, with LLMs, we can summarize text by just a simple prompt like "summarize this ..." or "extract bullet points from ..." and so on. But in this case, we completely rely on the LLM to determine what the summary should look like. The LLM makes decisions about what is important and what is not, and how concise the summary should be. But what if such a summary isn't good enough for us? So, let's consider how we can set up and configure the summarization process to achieve a summary closer to our needs.

Summary length

The first most often used parameter is the **summary length** that defines our semantic compression ratio. LLMs are not like traditional APIs that can generate exactly the length we ask for. Usually, the final text length is *approximately* what is asked in the prompt. That is especially true if asked to make a summary of up to some words, tokens, or character numbers. But LLMs work relatively well for the number of sentences or paragraphs in a summary.

Maximum tokens

The LLM output length can be limited with the *maximum tokens* parameter as well (see *Chapter 2*). This setting establishes a physical limit for the summary length. However, if the maximum token limit is set too low, the quality of the summary may drop significantly. Typically, short summaries (such as executive briefings) might require 50–100 tokens, while medium-sized summaries may range from 200–300 tokens.

Roughly speaking, the summary length defined in the prompt acts as a "soft" or logical limit, while the maximum tokens parameter acts as a "hard" limit. In practice, the best approach is to define a desired summary length through the prompt that is slightly lower than the hard maximum token limit, giving the model enough flexibility to complete its output naturally without being abruptly cut off.

Randomness settings

To control the level of randomness, we can use the *temperature* parameter (see *Chapter 2*). Lower temperature values (e.g., 0.2–0.3) make the model more deterministic and focused, producing more straightforward summaries. Higher values (e.g., 0.8+) introduce more creativity and variability, which might result in diverse but less consistent summaries. With low temperature, the model "drifts" more toward extractive summarization, and with higher to more abstractive.

Refocus

LLMs summarize text by focusing on what they determine to be the most important topics, a judgment that largely comes from their training data. However, to produce a more targeted

summary, we may need to *refocus* the LLM by explicitly guiding it to prioritize the information that is most relevant to our specific goals.

Re-focusing can also be done by providing important keywords in the system prompt to focus on, such as topics, schemas, parts of text, and so on.

Model selection

In general, larger language models tend to perform better at summarization because they have more attention heads and parameters, enabling them to consider multiple aspects of the input simultaneously.

However, model size is not the only factor that determines effectiveness. In domain-specific tasks that rely heavily on specialized terminology or factual precision, even large general-purpose models may fall short. Without domain-specific knowledge, an LLM may attempt to generalize or hallucinate based on its broad training data, resulting in summaries that diverge from the source material or produce less relevant summaries. In these cases, smaller models that have been fine-tuned on relevant, domain-specific datasets can outperform larger models in terms of accuracy, contextual relevance, and terminology usage. We will consider domain-specific fine-tuning in *Chapter 7*.

Larger models may also be suboptimal for summarization tasks where natural language quality is not a priority, for example, in scenarios where summaries are used as intermediate representations. In such cases, large models can incur unnecessary computational costs.

Another important factor in model selection for summarization tasks is the context window, especially the effective context window (see *Chapter 2*). A larger context window allows the model to process more text in a single pass, which is essential when summarizing long documents or performing multi-document summarization.

So, overall, summarization tasks generally benefit from at least medium-sized, and ideally LLMs, especially when output quality is a top priority. However, exceptions exist. In specialized domains or use cases where computational cost outweighs the need for fluency, smaller fine-tuned models may be the better choice. Ultimately, model selection should be guided by the trade-offs between performance, domain alignment, input size, and cost.

Multimodal summarization

Today, the majority of documents contain not only text but also different kinds of other information (*modalities*) like charts, graphs, pictures, or even audio and video files. For instance, business reports can have text summaries, charts, and tables; research papers can contain figures, equations, and explanatory text; and presentation slides can integrate images, diagrams, and videos.

Multimodal summarization is the process of generating summaries for documents that combine multiple types of content, such as text, tables, images, videos, or audio (in advanced use cases). The challenge for such summarization is extracting and interpreting information across different modalities, combining key information from different formats into a unified summary.

There are a few approaches to summarize multi-modal documents. Let's consider some of them.

Text-first multimodal summarization

One approach to summarize multi-modal documents is to extract text descriptions from non-text content (e.g., images, videos, audio) and merge them with existing textual information into a unified text document. This combined document is then summarized using text-based summarization techniques.

To capture text from images, videos, and audio, specialized tools can be used. **Optical character recognition (OCR)** tools like Tesseract and Google Vision API can extract text from images, while models such as **BLIP (Bootstrapping Language-Image Pretraining)** and **CLIP (Contrastive Language-Image Pretraining)** provide captions. For videos, combining OCR and speech-to-text models like Whisper captures both visual and audio content. Audio transcription can also be handled by Whisper or Azure Speech Services.

The limitation of this approach is that the final quality of the summary is highly dependent on the accuracy of these intermediate tools. For example, errors in OCR may lead to missing or misrecognized text, while speech-to-text systems can struggle with background noise, accents, or domain-specific terminology. Captioning models may also oversimplify or misinterpret complex visual scenes. These issues propagate into the summarization pipeline, reducing the overall reliability of the final summary.

It's also important to emphasize that while human language is flexible and universal, it cannot always fully capture the richness of other modalities. An image, for instance, may convey visual details, emotions, or spatial relationships that are difficult to describe in words. Similarly, video may communicate motion, timing, or atmosphere that is lost in textual description. As a result, text-based summarization of multimodal content is always, to some extent, an abstraction—useful and powerful, but not a perfect substitute for the original modality.

Multimodal models

Another approach is to use **multimodal models** like GPT-4o or Gemini that can handle multiple modalities (text, images, audio, video) simultaneously. These models encode different data types into a shared representation and generate summaries that reflect all the content holistically. Such models are often called **foundation models** (see *Chapter 2*).

One of the main challenges in multimodal summarization is the quality of non-text modalities for specific tasks. As we discussed in *Chapter 2*, multimodal models are typically trained on large datasets in a self-supervised manner. However, in many cases, certain modalities within foundation models may not provide sufficient quality for specialized tasks.

Summarizing tabular data

One additional challenge could be with summarizing tabular data from tables presented in the documents. Tabular data is organized in a different way than regular text. In tables, the data is attributed by its location, so LLMs need to understand such relationships well.

Most LLMs are trained on internet text, which includes a variety of table-like formats such as Markdown tables, HTML tables, and CSV snippets. However, they do not inherently understand the native structure of tables the way a spreadsheet program does. Instead, they interpret rows and columns simply as sequences of tokens, without a built-in awareness of true tabular relationships or structure.

If the table is relatively small and has a straightforward structure, in most cases, LLM can understand it quite well. Such tables can be passed to the LLM prompt directly. But if tables are large and have more advanced structures (like pivot tables), in many cases, they require some pre-processing before summarizing.

One of the ways to pre-process tables can be flattening or row-by-row presentation, where we read each row (record) and convert it into a short text snippet. Each snippet includes the column label and the cell value. For example, we have this table:

Quarter	Revenue	Expenses
Q1	10000	8000
Q2	15000	10000

Table 4.1: A table example to summarize

A flattened text representation might look like this:

```
Row 1: Quarter = Q1, Revenue = 10000, Expenses = 8000  
Row 2: Quarter = Q2, Revenue = 15000, Expenses = 10000
```

The main advantages of this approach are that it is simple to implement, easy to interpret, and LLMs can naturally process column–value pairs as sequential tokens. The drawback, however, is that for large tables the resulting text can become very long and inefficient.

Another approach is to use a templated style to describe each row or the overall table. For instance, the table above can be parsed in the following text:

```
"In Q1, the revenue was 10,000 USD, while expenses were 8,000 USD. In Q2, the revenue was 15,000 USD..."
```

This approach gives the most LLM-understandable text, but also is the most verbose and will require more token usage.

Another approach is to use different tabular formats like Markdown, CSV (or TSV), or JSON formatting. LLMs are trained in these formats and can understand short and medium-sized tables relatively well. Note that the more verbose the format is, the better it is for LLM to understand, but such a format consumes more tokens. So, JSON and XML formats will be more verbose but clear for LLM. CSV is the least verbose for tabular data, but less clear for LLM and thus might be less efficient for large tables.

Large document summarization

As we discussed in *Chapter 2*, modern LLMs have dramatically expanded their context window sizes, reaching up to 128K tokens and, in some cases, even 1M tokens. However, long context windows come with their own challenges. In practice, the effective context window is often closer to 8K–16K tokens. Beyond that, issues such as context dilution and degraded accuracy tend to appear.

Therefore, when working with large documents, we usually need to split the text into manageable chunks and summarize them step by step. There are several well-established strategies for chunk-based summarization, which we will now explore:

MapReduce

The core idea is the following:

- Chunk the original document into smaller, manageable sub-documents
- Create summaries for each chunk – chunk-level summaries (map)
- Merge chunk-level summaries into the final summary – second level (reduce)

While this approach is quite straightforward, it raises a few challenging questions:

- How to chunk the document, what size of chunks to select?
- What summary length to select for chunks?
- What if merged chunk summaries don't fit into the LLM's effective context window?
- How to get a concise and meaningful final summary if chunk summaries are fragmented and lose key points?

The first question can be addressed with the following approaches:

- **Natural breaks:** In this approach, we select chunk size based on natural document breaks like chapters, sections, paragraphs, and so on. In this way, we can get natural first-level summaries. The challenges here could be if the size of the sections differs significantly. For instance, some document sections are small (one paragraph), and others are 10-20 pages. In this case, small sections can be grouped, and bigger ones can be split.
- **Split by tokens:** In this approach, we chunk documents by the number of maximum tokens or words that can be filled into document sections like pages, paragraphs, or sentences. For instance, we can split text by paragraphs that can fit into a maximum of 10K tokens. This method is quite straightforward but can lead to disconnected text sections.

Now, let's answer the question about what length of chunk summaries (or first-level summaries) to select. One approach is to make the first-level summaries the same length as the final-level summary. For example, setting one paragraph summary for each chunk, and exactly one paragraph summary for the final summary. This approach can work well if the length of the final summary is not small. It also can simplify your workflow because you apply the same summarization parameters (e.g., prompt instructions, maximum token lengths, or style guidelines) at both levels.

However, this approach has risks of losing details: if the length of the final summary is small (one sentence), each chunk can omit important details. In this case, it's recommended to have a chunk summary longer than the final one.

Another challenge is the effective context window constraint. This directly relates to our third question:

```
"What if merged chunk summaries don't fit into the LLM's effective context window?"
```

Merging large chunk summaries can quickly consume all effective context windows. In this case, it's recommended to reduce chunk summaries or use multi-level summarization.

Multi-level summarization is when we first chunk a document and make chunk summaries: first-level summaries. Next, we merge these chunk summaries into one document and now apply chunking and summarization to it. Or another approach is to group chunk summaries into documents and then apply summarization for each of such groups, retrieving second-level summaries.

The more levels we get, the more disadvantages we can encounter, such as increased chances of losing context, hallucinations, and fabrications. This directly relates to our fourth question:

"How can we produce a concise and meaningful final summary if chunk summaries are fragmented and lose key points?"

In practice, it's not recommended to have more than 2-3 summary levels. With 2-3 levels, we can summarize quite large (1K+ pages) documents effectively with modern LLMs.

For instance, a 500-page book (~200K tokens) can be chunked into sections of 15-20 pages (~6,000-7,000 tokens each) to fit within an LLM's 8,000-token effective context window. First-level summaries condense each chunk into ~500 tokens, which are then grouped into second-level summaries covering 120-150 pages. Finally, these mid-level summaries are distilled into a concise final summary.

A multi-level map-reduce approach is sometimes called **hierarchical summarization**.

The following diagram illustrates a two-level summarization process:

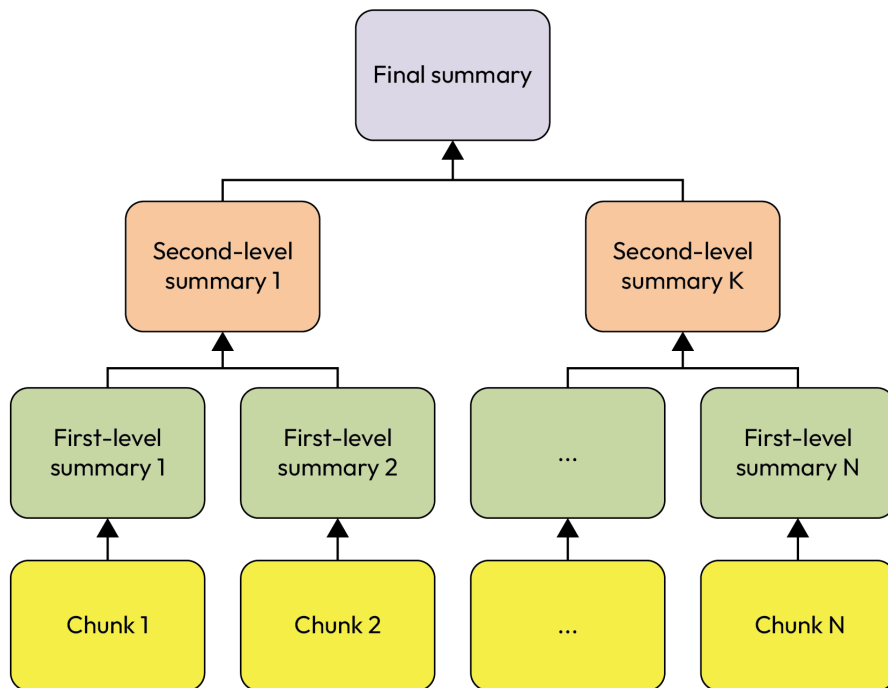


Figure 4.3: Multi-level map-reduce summarization

First, the document is divided into N chunks, and a first-level summary is generated for each chunk. Next, these summaries are grouped into larger segments and summarized again, producing K second-level summaries. Finally, the second-level summaries are combined and distilled into a single coherent final summary.

Incremental summary aggregation

In some cases, summaries can be built *iteratively* by starting with an initial summary and gradually enhancing it. In this approach, the summary is constructed incrementally by reviewing document chunks one by one and updating the summary as new information is added. This method works particularly well when a base summary already exists.

For example, when summarizing a scientific paper, the abstract can serve as the starting point. An LLM can then iteratively expand and refine this base summary by incorporating key details from each section of the paper, ultimately producing a more comprehensive and polished final summary.

This approach can also work well when we need to extract key points by filtering content and identifying only new, unique information at each iteration. For example, when summarizing customer complaints from a set of product reviews, we could start with the first review by extracting complaints, then move to the second review, capturing only complaints that have not yet been mentioned. This process would continue iteratively, building a summary that highlights truly unique feedback across all reviews.

In this section, we explored summarization as one of the most fundamental language tasks that modern LLMs can perform efficiently, including key techniques such as chunking, hierarchical summarization, and managing context window constraints.

Building on this foundation, we will now explore other core NLP tasks that power modern AI systems and agents.

Text classification

As discussed in *Chapter 1*, classification is one of the most fundamental and widely applied tasks in AI. It has long been a central focus of traditional machine learning, where models are trained to assign predefined labels to structured inputs.

Text classification is a specific type of classification task in machine learning, where the input data is text, often in natural language. The objective is to assign predefined categories or labels (classes) to these text inputs based on their content.

Text classification falls under the broader category of *language understanding* tasks, where the language model must understand the meaning, intent, or topic of a given text. In other words, the model must identify what the text is about, its topic, sentiment, intent, or other property,

and then map that insight to a well-defined category such as "spam," "support request," "politics," "positive review," etc. Achieving this mapping often requires the system to recognize subtle linguistic cues, resolve ambiguity, account for context, and sometimes infer unstated implications.

Modern language models acquire their ability to understand text through exposure to massive amounts of training data. For example, the model might encounter thousands of documents where the word "war" appears in negative or emotionally charged contexts. Over time, by processing vast amounts of text, the model learns that the term "war" is frequently associated with negative sentiment even though this connection was never explicitly provided.

The model also implicitly learns relationships between related concepts and terms. For instance, words like "battle," "fight," and "invasion" often co-occur in similar contexts. Through techniques like embeddings, the model adjusts their vector representations to be closer together in the embedding space, allowing it to recognize that they are semantically similar.

In the post-training or fine-tuning phase, the model is refined on specific labeled datasets to perform concrete tasks like classification or summarization. For example, fine-tuning on the IMDB Reviews dataset trains a model to associate text patterns with positive or negative sentiment, while a dataset like AG News enables it to recognize topics and classify news articles accordingly.

Now let's explore the types of text classification problems commonly encountered when building real-world LLM-based applications, including AI agents. As we mentioned in *Chapter 1*, three types of classification tasks are applied to text classifications as well:

- **Binary classification:** The model predicts one of two possible categories (e.g., spam vs. not spam, positive vs. negative).
- **Multi-class classification:** The model predicts one category out of several (e.g., classifying news into politics, sports, technology, or health).
- **Multi-labeled classification:** The model can assign multiple categories to the same input (e.g., tagging a movie as comedy, romance, and drama at the same time).

Text classification comes in many forms, with a wide range of variations. Some tasks, like spam detection and sentiment analysis, have become classics, extensively studied, and widely adopted in real-world applications.

Below, we'll explore several common types of text classification tasks and how they are typically categorized:

Class	Problem	Classification type	Description
Content filtering and moderation	Spam and phishing detection	Binary	Classifying emails or messages as spam or phishing.
	Toxicity detection	Binary, multi-class, or multi-label	Identifying offensive, hateful, or abusive language.
	Misinformation detection	Binary or multi-class	Identifying false or misleading content – fake news, incorrect statements, etc.
Emotion analysis	Sentiment analysis	Binary or multi-class	Classifying text as positive, negative, or neutral.
	Emotion detection	multi-class or multi-label	Identifying emotions such as anger, joy, fear, sadness, etc.
Dialog and assistant tasks	Intent detection	multi-class or multi-label	Recognizing user intent in a message (e.g., "ask for refund," "track shipment")
	Question type classification	multi-class or multi-label	Classify the question type – factual vs opinion-based, yes/no, simple vs complex, brief vs detailed answer, etc.

Class	Problem	Classification type	Description
Document management	Document classification	multi-class or multi-label	Classifying documents (e.g., legal vs. medical vs. financial).
	Document tagging	multi-class or multi-label	Assigning one or multiple tags or labels to content
	Compliance classification	Binary or multi-class	Identifying whether the document complies with legal or policy standards.

Table 4.2: Categorization of common types of text classification

As we can see, each text classification task comes with its own specific challenges, goals, and nuances. While they all follow the general pattern of mapping text to predefined labels, the nature of the labels, the structure of the input, and the reasoning required can vary significantly.

In the following sections, we'll take a closer look at several key text classification tasks such as sentiment analysis and intent recognition. This exploration will help build intuition for how different types of text classification challenges can be approached in practice.

Sentiment analysis

Sentiment analysis is a type of text classification task that assigns a sentiment label, such as positive or negative, to a given piece of text. It is most commonly formulated as a binary classification problem, where the text is labeled as either positive or negative. However, it can also be extended to a multi-class classification setup, where additional labels like neutral or mixed are included to capture more nuanced sentiment categories.

Sentiment analysis was one of the earliest and most widely adopted tasks in modern NLP, especially with the rise of user-generated content in the early 2000s, such as product reviews, social media posts, and online forums. It quickly gained popularity due to its high business value, enabling companies to automatically analyze customer opinions, monitor brand reputation, and extract insights from platforms like Amazon, Yelp, TripAdvisor, and Twitter.

Early implementations of sentiment analysis were often based on simple rule-based or *lexicon-based* approaches. These systems relied on predefined dictionaries of positive and negative words, where the presence (and sometimes frequency) of such words in a text was used to determine its overall sentiment. To improve accuracy, basic NLP preprocessing was applied, such as tokenization, stop word removal, and sometimes stemming or lemmatization (see *Chapter 1*), to normalize the text.

Here's a simple lexicon-based example of how sentiment analysis can be implemented using the spaCy library:

```
import spacy
nlp = spacy.load("en_core_web_sm")
# Define simple sentiment lexicons
positive_words = {'good', 'great', 'excellent', 'amazing', 'love', 'wonderful',
                  'happy', 'fantastic', 'liked'}
negative_words = {'bad', 'terrible', 'awful', 'hate', 'horrible', 'worst',
                  'disappointed', 'sad', 'boring'}

def analyze_sentiment(text):
    doc = nlp(text.lower())
    # Filter tokens: keep lemmatized words, ignore stop words and punctuation
    tokens = [token.lemma_ for token in doc if token.is_alpha and not
              token.is_stop]

    # Count sentiment words
    pos_count = sum(1 for word in tokens if word in positive_words)
    neg_count = sum(1 for word in tokens if word in negative_words)
    # Determine sentiment
    if pos_count > neg_count:
        return "Positive"
    elif neg_count > pos_count:
        return "Negative"
    else:
        return "Neutral"

# Example usage
text = "The movie was amazing! I loved every minute of it."
sentiment = analyze_sentiment(text)
print(f"Sentiment: {sentiment}")
```

The code loads the small English language model `en_core_web_sm`, defines basic vocabularies for positive and negative words, and then processes the input text through tokenization and lemmatization. It filters punctuation and stop words, counts the occurrences of positive and negative terms, and returns the overall sentiment of the sentence: positive, negative, or neutral.

We can see that the core of this algorithm lies in its positive and negative vocabulary words. These vocabularies are static and manually crafted based on general language intuition about which words convey positive or negative sentiment. Any words in the text that are not included in these lists are treated as neutral and are assumed to have no impact on the sentiment analysis outcome.

While this method is relatively easy to implement and performs reasonably well on simple datasets, it lacks contextual understanding and often struggles with sarcasm, negation, and domain-specific vocabulary. These limitations are why more advanced, machine-learning-based approaches have become the standard in modern sentiment analysis.

The next major step in the evolution of sentiment analysis involved moving from rule-based systems to classical machine learning models, such as Naive Bayes, **Support Vector Machines (SVM)**, and Random Forests. These models brought a more statistical and data-driven approach to solving sentiment classification, allowing systems to learn from labeled data instead of relying on handcrafted word lists.

While these methods remain a surprisingly efficient model even today, they also come with several notable limitations. They treat words as *independent features*, ignoring word order, syntax, and deeper linguistic context. This makes them ill-equipped to handle complex language structures, such as sarcasm, negation, or subtle shifts in tone.

More advanced models, particularly large language models, can overcome many of the limitations faced by traditional approaches like Naive Bayes. Today, LLMs can handle sentiment analysis with impressive accuracy and flexibility. In fact, even a simple prompt such as "Analyze this text and determine whether the sentiment is positive or negative" can yield strong results without any additional fine-tuning.

This is possible because LLMs don't just rely on the presence of positive or negative words; they are trained on massive datasets and can understand context, tone, syntax, and subtle semantic cues. They can account for negation (e.g., "not bad" vs. "bad"), detect sarcasm more effectively, and interpret the sentiment of longer, more nuanced passages.

As a result, LLMs are generally very effective at sentiment analysis across many domains. Yet, because sentiment analysis is often a relatively straightforward task, it may not require the power of very large models. Smaller models such as DistilBERT, MiniLM, or T5-small can achieve strong performance, especially when fine-tuned or guided with well-structured prompts.

If you're developing a sentiment analysis solution, it's usually best to start with a lightweight model and evaluate whether it meets your needs before moving to larger, more resource-intensive alternatives.

Intent detection

Intent detection is a fundamental task in conversational AI systems, particularly in chatbots, virtual assistants, and voice interfaces, often referred to as **conversational agents** (see *Chapter 8*). The goal is to determine what the user wants to do, their intent, based on the message they send. This typically involves mapping free-form natural language input into a predefined action or workflow.

For example, in a flight booking assistant, when the user says:

```
"I want to book a flight to New York next weekend."
```

The system should classify the intent as *Book Flight* action, extract parameters (destination: New York and time: next weekend), and execute that action.

Intent detection is the entry point for the chatbot (assistants) *orchestration mechanism*. It tells the system what action to trigger next. In modern conversational architecture, once the intent is identified, it is used to run some code, an API, or a tool to perform some action like getting more information or updating database records.

Intent detection is often formulated as a multi-class classification problem in its simplest form, where the system selects one intent or action that best matches the user's input. In this setup, each user message is classified into a single predefined category, corresponding to a specific task or workflow. For example, consider a financial chatbot where the user asks the following question:

Single action selection Multi-class classification

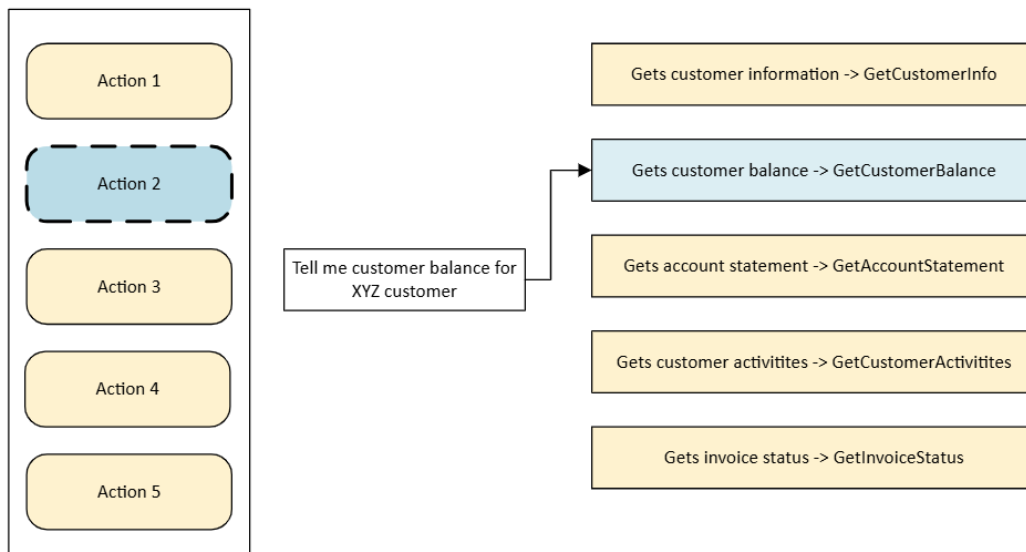


Figure 4.4: Intent detection with single action selection

In this example, when the user says, "Tell me the customer balance for XYZ customer," the chatbot should recognize the intent as `GetCustomerBalance` and execute the corresponding action to retrieve and return the appropriate information. This is a single-intent scenario.

However, in real-world applications, user requests often involve more complex flows. A single input may require the system to perform multiple independent actions. For example, to respond to the user request, "For XYZ customer, give me the current balance and contact info," the system must identify and execute multiple actions. In this case, both `GetCustomerBalance` and `GetCustomerInfo`. This transforms the task into a multi-label classification problem, where the model may return none, one, or multiple intents for a single input.

The example above of selecting multiple actions assumes that all selected actions are independent. That isn't always the case. In many real-world scenarios, actions must be performed in sequence, where the output of one step informs the next. This requires not just intent detection, but a form of dynamic orchestration: selecting a set of initial actions, executing them, analyzing the results, and then determining which additional steps are needed to fully resolve the user's request.

As conversational interfaces evolved, rule-based methods were gradually replaced or augmented by machine learning-based models, particularly multi-class classifiers. These models could learn patterns from examples and generalize to variations in phrasing.

With the rise of pretrained transformer models like BERT, intent detection saw a significant leap in accuracy and robustness. These models capture contextual relationships in text and are capable of understanding user queries with greater nuance. Fine-tuning these models on domain-specific intent datasets has become a common practice in modern chatbot development.

More recently, LLMs have enabled zero-shot or few-shot intent detection using natural language prompts, removing the need for traditional training in many cases. For instance, a prompt, such as the following, can work quite well, even without fine-tuning, making LLMs a powerful tool for rapid prototyping or low-volume scenarios:

Prompt

```
Classify the user's intent as one of: CheckBalance, TransferMoney, GetHelp, or
BookAppointment.
Message: Can you move $200 to my savings account?
```

That said, in production systems where latency, cost, and reliability are important considerations, lightweight fine-tuned models or hybrid approaches (combining classification with prompting) are frequently considered. The choice of approach depends on several factors, including the complexity of the intent task, the volume of user interactions, and the overall system architecture.

Information extraction

Information extraction is the task of pulling information from unstructured text. It transforms raw language into a well-defined structure that machines can understand and use. While human readers naturally recognize facts, entities, and relationships in text, machines need these elements to be made explicit in order to reason over them.

At its core, the goal of information extraction is to identify and organize key pieces of information, such as names, dates, relationships, or events, from raw text. This structured output becomes the foundation for building knowledge bases, automating workflows, and powering downstream applications like question answering and conversational agents (see *Chapter 8*).

Information extraction differs fundamentally from classification. In classification, we assign the entire input text to one or more predefined categories. In contrast, information extraction involves identifying specific pieces of information within the text and labeling each one according to its type, such as a person, organization, date, or event.

Information extraction spans a variety of sub-tasks, each producing a different kind of output:

- **Named entity recognition (NER):** Identifies entities like people, organizations, locations, dates, products, and more. For example, from the sentence "Barack Obama was born in Hawaii," NER can extract the following entities: "Barack Obama" (person), "Hawaii" (location).
- **Relation extraction:** Detects semantic relationships between entities. For example, from the sentence "Barack Obama was born in Hawaii," relation extraction can detect the relationship "born in".
- **Event extraction:** Captures actions or occurrences along with associated participants, time, and location. For example, from the sentence "John filed a lawsuit on Monday," event extraction will extract the event of *filing* where the actor is John, the object is the lawsuit, and the time is Monday. This event can be presented in a more structured JSON object.

These problems act as foundational blocks for more advanced AI tasks. For instance, in conversational AI, after selecting the correct action, the next important task is to extract specific pieces of information and pass them into the chosen action. That's exactly where information extraction comes in.

Named entity recognition

Named entity recognition (NER) is an NLP technique used to identify and classify key pieces of information, called **entities**, in text. These entities often include names of people, organizations, dates, and more.

For example, in the sentence, "*Microsoft released GitHub Copilot in June 2021*", NER would tag "Microsoft" and "GitHub Copilot" as organizations, and "June 2021" as a date.

NER typically extracts entities with the following categories:

- **Person:** Individual names (e.g., "Angela Merkel")
- **Organization:** Companies, institutions (e.g., "Microsoft", "UNESCO")
- Location and **geo-political entities (GPE)** which include countries, cities, regions (e.g., "Paris", "Mount Everest")
- **Date:** Any time expression (e.g., "April 2023", "Today", "12:30")
- **Money:** Prices, currencies (e.g., "\$50 million")
- **Value:** numbers, percents

In specialized domains, custom entity types can be defined as well. For example:

- In biomedical texts: gene, disease, drug
- In legal documents: case number, judge, court
- In e-commerce: product, brand, discount

While NER might appear straightforward, it encompasses several nuanced challenges:

- **Ambiguity:** Words like "Apple" at the beginning of sentence can refer to a fruit or a technology company, depending on context. Disambiguating such terms requires models to effectively leverage surrounding textual cues.
- **Nested entities:** Phrases like "University of Washington" contain entities within entities, "University of Washington", an organization, and "Washington", a location.
- **Domain shift:** In one domain, a term can be an entity, but not in another. Examples include Jaguar: a car or an animal, Python: a programming language or a snake.
- **Noisy and informal texts:** Social media and user-generated content often contain slang, abbreviations, and misspellings. NER must be robust to such noise.

There are several approaches to handling NER. We will consider some of the most popular ones:

- Classical NLP pipelines (e.g., spaCy, NLTK)
- Transformer-based fine-tuned models (e.g., BERT, RoBERTa, DistilBERT)
- Large language model

Classical NLP pipelines

Modern NLP libraries like spaCy or NLTK make it easy to perform NER using pre-trained models that work out of the box. These models are trained on large, annotated datasets, and learn to recognize entity types (such as PERSON, ORG, or DATE) by analyzing patterns and context in word sequences. This data-driven approach enables modern NER systems to achieve high accuracy and adapt well to a wide range of real-world applications.

It might seem like NER models can only recognize entities they've seen during training, or only well-known names from their training data. But that's not exactly true. Modern NER models don't rely on memorizing specific entities; instead, they learn to recognize patterns and linguistic context that indicate entity types. This means they can often generalize to new, previously unseen entities as long as the surrounding context is similar to what the model has learned during training.

Let's look at the code example that shows how NER can be applied:

```
import spacy
from spacy import displacy
# Load English model with NER support
nlp = spacy.load("en_core_web_sm")
Sample text
text = "Apple was founded by Steve Jobs and Steve Wozniak in California in 1976."
# Process the text
doc = nlp(text)
# Print named entities
print("Named Entities:")
for ent in doc.ents:
    print(f"- {ent.text} ({ent.label_})")
```

This preceding code produces the following named entities:

```
Apple (ORG)
Steve Jobs (PERSON)
Steve Wozniak (PERSON)
California (GPE)
1976 (DATE)
```

The example above shows how we can use spaCy and a built-in language model to perform named entity recognition. spaCy comes with pre-trained language models that can handle NER out of the box. In this example, we used one of the smallest models for demonstration purposes. However, larger models like `en_core_web_md` or `en_core_web_lg` may yield more accurate results.

Libraries like spaCy or NLTK are easy to use, lightweight in terms of resource requirements, and often provide surprisingly good accuracy on general text. But as mentioned earlier, they tend to struggle with more complex or domain-specific cases.

Transformer-based fine-tuned models

To overcome the limitations of models used in libraries like spaCy, transformer-based models can be used for NER tasks as well. These models are built on architectures like BERT and typically require fine-tuning on annotated NER datasets such as CoNLL-2003, OntoNotes, or custom domain-specific corpora.

For specialized domains such as medical, legal, or financial text, domain-specific fine-tuning can improve accuracy. In some cases, you can use already pre-trained, fine-tuned, and openly available models published on Hugging Face. Here are a few well-known examples:

Model	Based on	Fine-tuning dataset	Description
dslim/bert-base-NER	bert-base-cased	CoNLL-2003	NER fine-tuned BERT model
Davlan/bert-base-multilingual-cased-ner-hrl	bert-base-multilingual-cased	WikiANN	Multilingual NER BERT fine-tuned model
Jean-Baptiste/roberta-large-ner-english	roBERTa-large	conll2003	NER Fine-tuned RoBERTa-large

Table 4.3: Transformer-based NER models

As we discussed earlier, fine-tuning and hosting transformer-based models come with non-trivial requirements. It demands specialized knowledge in machine learning, access to appropriate infrastructure (such as GPUs or cloud compute), and introduces ongoing maintenance and monitoring overhead. This makes it important to weigh the trade-offs between model performance and operational complexity, especially for production use.

Large language model for NER

The third approach to solving NER is to use LLMs. In many cases, they can outperform traditional methods due to their exposure to large and diverse training data, strong language understanding, and reasoning capabilities. However, their performance may be limited in highly specialized domains or scenarios that were not well represented in the training data.

Even a simple prompt like the following can be quite effective:

Prompt

Extract all persons, organizations, and locations mentioned in the following text.

One of the biggest strengths of LLM-based NER is flexibility. You can define custom entity types on the fly (e.g., products, error codes, symptoms, investments) and change the extraction format as needed: JSON, key-value pairs, tables, etc. This makes LLMs especially useful in exploratory data analysis, prototyping, and domain-specific extraction when labeled training data is limited or unavailable.

Similar to many text classification tasks, NER can often be handled by smaller models, such as the mini series. These models strike a good balance between performance and efficiency. Likewise, information extraction tasks typically don't benefit from high output variability. Since the goal is to extract consistent and accurate entities from text, these tasks are best performed with lower temperature, which reduces randomness and makes the model more deterministic in its predictions (see *Chapter 2*).

However, LLM-based NER also comes with limitations. Results may vary depending on prompt phrasing, and consistency across large datasets can be an issue. LLMs may also hallucinate or miss entities if the prompt isn't clear enough. Additionally, calling an LLM (e.g., via OpenAI's API) introduces latency, cost, and data privacy considerations.

Relation and event extraction

Once entities are identified in text, the next step in extracting structured meaning is to understand how those entities are connected. This is the purpose of **relation extraction**, a task that identifies and classifies semantic relationships between entities appearing in the same sentence or document.

For example, take the sentence, "**Satya Nadella is the CEO of Microsoft.**"

A relation extraction system could map two entities, "Satya Nadella" and "Microsoft", with the "being CEO" relationship. Such systems are critical in building **knowledge graphs (KG)**, powering *question answering* systems, and enabling semantic search. Common relation types include `works_for`, `located_in`, `founded_by`, and `acquired_by`.

Note

KGs are outside of the scope of this book, but in short, a KG is a structured representation of information where entities (like people, places, or organizations) are connected by relationships. These connections form a graph of nodes (**entities**) and edges (**relations**), allowing systems to reason over data more like a human would through contextual links. Relation extraction tasks can handle KG data population.

There are generally two approaches to relation extraction:

- **Supervised relation extraction** relies on labeled data where relationships between known entity pairs are annotated.
- **Open relation extraction** does not rely on predefined relation types. Instead, it attempts to extract relation phrases directly from text, offering flexibility but typically requiring post-processing or clustering to group similar relations.

Modern approaches often build relation extraction on top of NER: entities are first identified, and then potential relationships between entity pairs are classified.

Large language models can also perform relation extraction using prompt-based techniques. Relation extraction still poses challenges, particularly when relationships are implied, ambiguous, or span multiple sentences. But as a core component of information extraction pipelines, Relation extraction plays a foundational role in transforming unstructured text into usable, interconnected knowledge.

Another important task in IE is *event extraction*. This task focuses on identifying what happened in a text, who was involved, and when and where the event occurred. While named entity recognition pulls out key elements like names or dates, event extraction connects these elements into a meaningful action or occurrence. For example, from the sentence, "**Microsoft acquired GitHub in 2018 for \$7.5 billion.**"

An event extraction system can identify the following events:

```
Event type: acquisition
Participants: Microsoft (acquirer), GitHub (acquired)
Date: 2018
Amount: $7.5 billion
```

Unlike relation extraction, which links two entities with a static relation, event extraction captures more complex interactions often involving multiple roles, time expressions, and attributes. Events are typically built around triggers, verbs, or action phrases like acquired, resigned, filed for bankruptcy, or launched, and their surrounding context.

In more advanced systems, events can also be nested or span across sentences. For instance, "**The CEO resigned after the company was hit with a major lawsuit.**"

Here, one event (resignation) is caused by another (lawsuit), and a full extraction pipeline would model the causal relationship as well.

Event extraction is widely used in areas like news monitoring, financial analytics, legal automation, and clinical reporting. For example, tracking real-time events in financial news

(e.g., mergers, earnings, leadership changes) enables structured analysis for trading systems or risk management.

Traditionally, event extraction was handled by rule-based systems or supervised learning on datasets like ACE 2005 (<https://catalog.ldc.upenn.edu/LDC2006T06>) or Rich ERE (<https://catalog.ldc.upenn.edu/LDC2023T04>). Today, it's often implemented using fine-tuned transformer models or LLMs with prompt-based approaches. For example, consider this prompt:

Prompt

What event is being described in this sentence, and who are the participants?
Return the JSON object that represents the event.

It will extract the event from the input text and return it as a JSON document.

Despite advances, challenges remain, especially around ambiguous language, implicit events, or the need for domain-specific triggers. However, as LLMs continue to improve and integrate with retrieval or symbolic reasoning components, event extraction is becoming more scalable and adaptable across industries.

In this section, we explored information extraction as a fundamental NLP capability, focusing on how structured data can be derived from unstructured text. This capability is essential for many real-world applications. In subsequent chapters, we will see how this extracted information is used in AI agent orchestration and evaluation.

Multilanguage support

Early NLP systems were typically built for one language at a time, often requiring separate models, rules, or datasets per language. In many cases, a model trained for a specific task, like summarization or classification, would only support a single language, usually English. When such models needed to handle input in other languages, developers often relied on machine translation: translating the non-English text into English, applying the NLP model, then translating the output back.

Translating text into another language and then back to the original, commonly known as *back-translation* or *round-trip translation*, can often lead to loss of context, distortion of meaning, and inconsistent results. This approach, while sometimes used for data augmentation or quick evaluations, has notable limitations, especially when applied to complex NLP tasks.

For example, let's consider the following text:

"After months of radio silence, the startup finally broke cover with a game-changing AI prototype."

This sentence contains:

- **Idioms:** "radio silence" (no communication), "broke cover" (sudden reveal after hiding), "game-changing" (major or disruptive impact)
- **Contextual nuance:** implies secrecy, strategic reveal, and major innovation

If we translate this sentence in Google Translator (May 2025) to Thai language, we will have the following:

- "After months of radio silence" -> หลังจากไม่มีวิทยุมาหลายเดือน
- "The startup finally broke cover" -> สตาร์ทอัพในที่สุดก็เปิดเผยที่กำบัง
- "With a game-changing AI prototype" -> ด้วยต้นแบบ AI ที่จะเปลี่ยนแปลงเกม

And if we translate this back to English, we will get, **"After months without radio, the startup finally reveals bunker with game-changing AI prototype."**

This sentence ends up noticeably different from the original, even with back-translation to a relatively common language like Thai. When idioms are used, back-translation often fails to preserve meaning, and it can also mistranslate proper named entities, brand names, or domain-specific terminology, leading to confusion or misrepresentation.

To address these challenges, modern NLP approaches started using multilingual models trained on diverse datasets spanning dozens or even hundreds of languages. Some of the first widely adopted multilingual models were mBERT, XLM-R, and mT5; all of these were designed to understand and process multiple languages natively, without relying on machine translation as an intermediate step.

By learning from large-scale multilingual datasets, these models develop shared semantic representations, allowing knowledge to transfer between languages. For instance, a sentiment classification model trained in English can generalize to related tasks in French or Hindi with little additional training. This capability is especially valuable in low-resource languages, where labeled data is scarce but multilingual models can still perform reasonably well.

Multilingual models enable cross-lingual tasks such as retrieving documents in one language using a query in another or performing question answering across language boundaries, making them well-suited for global applications like multilingual chatbots and international knowledge bases without requiring separate models for each language.

Modern large language models are increasingly trained on multilingual datasets to support these capabilities. For example, OpenAI's GPT-3 was trained primarily on English text but also included non-English data, allowing it to perform tasks in multiple languages. Newer models such as GPT-4o significantly improve multilingual performance, supporting over 50 languages and introducing better tokenization for non-Latin scripts, which increases efficiency and accuracy for diverse language inputs.

Nevertheless, English still dominates training data because it remains the most prevalent language on the internet; as of 2024, about 49.7% of all websites are in English, far exceeding other major languages such as Spanish (5.9%) and German (5.4%) [32].

This imbalance is shaped by several factors:

- **Historical development:** The internet was primarily developed in English-speaking countries, leading to an initial abundance of English content
- **Global reach:** English serves as a lingua franca in international business, science, and technology, encouraging content creation in English to reach a broader audience
- **Educational prevalence:** English is widely taught as a second language worldwide, increasing its usage online

As a result, even the most multilingual LLMs are often inherently biased toward English and perform better in English, simply because that's where most of the training data comes from.

Language detection

One of the popular tasks is to detect the language from arbitrary text. In its simplest form, **language detection** aims to assign a language label (e.g., English, Spanish, Swahili) to a given text snippet. This is an important step in many NLP problems, especially when dealing with user-generated content from global sources, where the language is not always known in advance. For example, language detection is used to route customer support queries to the correct regional agent or to select the appropriate language-specific model for translation or summarization.

Language detection, while often treated as a solved problem in popular languages, still faces several challenges, especially in real-world scenarios involving short or informal text. Some of the most common difficulties include:

- **Short text:** Detecting the language of very short inputs, such as a single word, search query, tweet, or headline, can be difficult.
- **Lexical overlap and similarity:** Many languages share identical or very similar words and phrases and use the same or similar scripts. It often happens among linguistically related families (e.g., German and Dutch, or Serbian and Croatian).

- **Code-mixed and multilingual text:** In multilingual settings, users often switch between languages within a single sentence or paragraph (known as **code-switching** or **code-mixing**). This is common in informal communication, like social media or messaging apps.
- **Noisy or informal text:** Misspellings, abbreviations, emojis, transliterations, and inconsistent punctuation are common in casual communication, social media, and messages.
- **Named entities and borrowed words:** Named entities like brand names (Netflix, Samsung) or global terms (pizza, email, OK) are shared across many languages and provide little signal for language identification.

Early approaches to solving language detection were primarily rule-based systems, which relied on manually crafted heuristics, such as character frequency tables, specific letter combinations, stop word patterns, or script identifiers. These systems often used simple rules like "if the text contains 'the', 'and', or 'is', it's likely English," or mapped unique Unicode character ranges to specific languages (e.g., Cyrillic to Russian or Bulgarian).

While effective for a limited number of popular languages and longer text inputs, rule-based methods struggled with short inputs, ambiguous words shared across languages, and multilingual phrases. As the number of supported languages grew and multilingual content became more common, these rule-based techniques were gradually replaced or supplemented by statistical and machine learning models that could learn patterns across large datasets and generalize better to noisy, user-generated text.

Modern large language models can detect language with remarkable accuracy, especially those trained on massive multilingual datasets. Even a simple prompt like "detect the language of this sentence" can yield highly accurate results across a wide range of languages, including low-resource or code-mixed ones, due to the deep contextual understanding and robust pretraining these models possess.

However, despite their strong performance, LLMs are not always the most practical solution for this task. Language detection is a relatively straightforward classification problem, and relying on large models introduces significant overhead in terms of latency, compute, cost, and infrastructure complexity. Running an LLM just to identify a language, especially for high-throughput or real-time systems, can be overkill and economically inefficient, particularly when simpler tools can achieve comparable results for many common use cases.

fastText and *langdetect* are among the most popular lightweight alternatives to transformer-based models and LLMs for language detection. *fastText*, developed by **Facebook AI Research (FAIR)**, uses a specialized language identification model built on character n-gram embeddings and a linear classifier. It supports 176 languages and is highly fast and efficient,

making it well-suited for real-time applications. Trained primarily on Wikipedia data, fastText delivers solid accuracy for longer and well-formed texts, though it may struggle with short or noisy inputs.

Machine translation

One of the key NLP tasks is translating text from one language to another, a process known as **machine translation (MT)**. The goal is to convert content from a source language into a target language while preserving its meaning, fluency, and context. MT is a foundational component in multilingual applications, powering tools like Google Translate, real-time transcription services, multilingual chatbots, and cross-lingual information retrieval.

Historically, early machine translation systems were rule-based, relying on hand-coded grammar rules, dictionaries, and phrase mappings. These were rigid, fragile, and difficult to scale across languages.

In the mid-2010s, **neural machine translation (NMT)** transformed the field. Sequence-to-sequence models (see *Chapter 2*) with attention mechanisms (like Google's Transformer architecture) allowed systems to generate more fluent and context-aware translations. Today, NMT dominates the field, with state-of-the-art systems leveraging transformers, multilingual pretraining, and large-scale fine-tuning.

Despite these advances, modern MT still faces several challenges:

- Handling low-resource and rare languages with little training data
- Preserving named entities, idioms, and cultural context
- Ensuring consistency in style, tone, and terminology, especially in domain-specific or professional content

Today, we can select a few approaches to perform machine translation: specialized models and LLMs.

Specialized machine translation models are purpose-built systems optimized solely for translating between language pairs. These models are trained on large-scale parallel datasets, where sentences in one language are aligned with their translations in another. Popular examples include Google Translate, DeepL, Facebook's M2M-100, and many others.

These systems are typically based on neural machine translation architectures, especially the transformer, which has become the backbone of modern MT. Earlier approaches performed translation in steps (word-by-word or phrase-by-phrase). In contrast, NMT models generate fluent, end-to-end output, using attention mechanisms to align source and target content dynamically. Specialized MT systems are highly optimized for:

- Speed and scalability (millions of daily queries)
- Fluency and grammar in the target language
- Coverage across curated language pairs
- Customization and terminology control (e.g., glossaries or formality settings)

Modern translation systems like Google Translate exemplify a hybrid approach to machine translation. When dealing with single words or short phrases, these systems often rely on specialized dictionaries to offer multiple translation options and usage examples. This ensures more accurate and contextually appropriate results, especially when users are looking for word meanings or alternatives.

For longer inputs such as sentences or paragraphs, these systems switch to neural machine translation, typically based on transformer architectures. NMT models analyze the entire context of the input to produce more fluent, grammatically correct, and semantically coherent output. The combination of dictionary-based and neural techniques offers a balance of precision and context awareness, improving accuracy across a wide range of translation tasks.

In contrast to these task-specific MT systems, LLMs are general-purpose models capable of performing translation as one of many tasks. Rather than being trained exclusively on language translation datasets, LLMs learn from broad multilingual datasets, allowing them to build shared semantic representations across languages. This enables translation even of a simple prompt like "Translate the following sentence to French."

LLMs offer several advantages in translation scenarios:

- **Flexibility:** They can translate, summarize, extract information, or classify across languages using a single model
- **Broad language coverage:** Especially useful for unseen or low-resource language pairs
- **Contextual understanding:** Capable of handling subtle meanings, idiomatic expressions, and ambiguous phrasing more naturally

However, LLMs also come with important limitations:

- They are generally slower and more resource-intensive, making them less suitable for real-time or high-volume applications
- Translation consistency may vary across language pairs, especially for morphologically rich or low-resource languages
- They are prone to hallucinations, omissions, or over-literal translations, particularly when the source context is sparse or ambiguous

As a result, LLMs are most effective when translation is one component of a larger workload, such as translating and summarizing a report, extracting entities from foreign-language documents, or generating multilingual conversational responses. For high-throughput, real-time translation needs, like powering multilingual websites or chat services, specialized MT engines remain the more efficient and reliable option.

Summary

Congratulations! You've just completed a deep exploration of several core practical language tasks. We started with summarization and continued with text classification, information extraction, and multilingual tasks. Each of these capabilities illustrates a different way in which language models interpret, organize, and process human language.

These tasks form the backbone of many modern AI applications and intelligent systems. Summarization helps condense large amounts of information, classification enables systems to categorize and interpret text, and information extraction converts unstructured language into structured data that machines can analyze and use.

Multilingual capabilities further extend these applications across languages, enabling global systems that can understand and process information from diverse sources. Together, these practical tasks provide an essential foundation for building more advanced AI systems that interact with language in increasingly sophisticated ways.

The next chapter explores advanced AI capabilities, including generating fluent human-like text, answering questions, and performing complex reasoning through multi-step problem-solving.

Subscribe for a free ebook

New frameworks, evolving architectures, research drops, production breakdowns – *AI_Distilled* filters the noise into a weekly briefing for engineers and researchers working hands-on with LLMs and generative AI systems. Subscribe now and receive a free eBook, along with weekly insights that help you stay focused and informed.

Subscribe at <https://packt.link/80z6Y> or scan the QR code below.



5

Generation, Question Answering, and Reasoning

In the previous chapter, we explored several core language tasks, such as summarization, text classification, information extraction, and multilingual processing. These tasks form the backbone of modern **natural language processing (NLP)** and have been studied for decades. Even before the emergence of transformer architectures and **large language models (LLMs)**, researchers developed methods to solve many of these problems with varying degrees of success.

However, the arrival of LLMs has significantly expanded what machines can do with language. Tasks that once required specialized models, complex pipelines, or carefully engineered rules can now often be solved by a single model with the right prompt and context. As a result, LLMs have moved beyond traditional NLP capabilities and opened the door to more advanced forms of language understanding and generation.

In this chapter, we will explore several of these more advanced capabilities. We begin with text generation, examining how modern models can produce coherent and high-quality text that often approaches human-level fluency. We then look at **question answering (QA)**, where models understand a question and generate relevant answers.

Finally, we turn to reasoning, one of the most exciting and rapidly evolving areas in modern AI, where language models attempt to perform multi-step thinking, combine pieces of information, and solve problems that require more than simple pattern matching.

We will cover the following key topics:

- Text generation
- Question answering
- Reasoning

Together, these capabilities represent an important step toward more intelligent AI systems. They also play a critical role in the design of modern AI applications and agents, which often rely on generation, question answering, and reasoning to perform complex tasks and interact naturally with users.

Text generation

Text generation is a fundamental task in NLP and a core capability of LLMs. It involves producing *coherent* and *contextually relevant* text based on a given input, such as a prompt or description.

Note

While basic string manipulation code can also generate text, in the context of NLP and LLMs, the generated output must be natural human language.

Many LLM tasks, such as summarization and translation, also involve generating text. However, the term *text generation* typically refers to creating new, open-ended content from scratch or with minimal guidance.

We can think of text generation as, in some sense, the opposite of summarization. Instead of compressing information into a shorter form, the model expands a prompt by adding details and producing richer content. Effectively, LLM does "*decompressing*" of the input.

To generate text, language models leverage their learned capability to understand language, factual knowledge obtained from training data, and human preferences acquired through **reinforcement learning from human feedback (RLHF)**. In addition, models can draw on their reasoning capabilities to generate text.

For example, we have the following prompt:

Prompt

Write a short story about a boy who lives in a forest with his father, mother, and seven sisters. The story should be a few paragraphs long, inspiring, and end on a positive note.

Such a prompt can generate the following text:

In a quiet forest untouched by the noise of the outside world, there lived a boy named Luca. He shared a cozy wooden cabin with his father, a skilled carpenter; his mother, a healer who knew every herb in the woods; and his seven lively sisters, each with a heart full of dreams.

....

One autumn, when the river overflowed and threatened their cabin, it was Luca who led his family to higher ground, finding a clearing where they could build a new home together. It was not the cabin or the place that made them strong – it was their bond. As they worked side by side under the open sky, laughter echoing between the trees, Luca realized the truth his parents had always lived by: the richest life isn't built with stone or gold, but with kindness, courage, and the people you love.

Their new home stood taller and stronger, just like the boy who helped build it – a boy who had learned that even the smallest voice can lead, and that from the deepest woods, the brightest dreams can grow.

You may ask the following question: how did the LLM come up with all these details about the boy and his story? We didn't provide any of those details in the prompt.

This happens because language models are trained to fill in gaps creatively and plausibly based on patterns they've learned from vast amounts of text. When the prompt is open-ended, the model uses its knowledge of storytelling conventions, common narrative structures, and typical character traits to generate a coherent story, even when only minimal information is given.

To understand this better, let's take a closer look at why the model selected the name 'Luca' in this particular case. Names like Luca are common in European languages and also familiar to English-speaking audiences. They frequently appear in children's stories and fairy-tale-like narratives. Since we asked for a story with a positive and inspiring tone, the model naturally chose a warm, simple, and easy-to-pronounce name. If we run the same prompt again, the model might select a different name, but it would likely be similar in style: friendly, familiar, and fitting the overall mood of the story.

Even though the generated story is influenced by countless examples the model encountered during training, the produced text is most likely unique. If we attempt to search for even a single paragraph from this story using exact-match search techniques (bracing with quotation marks) in search engines like Google Search or Microsoft Bing, we will likely find no identical or even closely matching text on the internet.

Language models don't copy and paste from their training data. Instead, they generate text step by step by predicting the next word based on learned patterns, associations, and probabilities. This makes each output a novel combination of linguistic structures rather than a reproduction of existing content.

A key reason for this variability lies in the model's sampling strategies (see *Chapter 2*). The model introduces controlled randomness through parameters like temperature scaling, top-k sampling, and nucleus (top-p) sampling. These techniques allow it to explore alternative word choices and sentence structures, so even with the same input, the results can differ.

Because of this, the number of possible variations is technically unlimited. Combined with the autoregressive nature of generation, where each word depends on all the ones before it, even a small change early in the sequence can cascade into a completely different outcome. This creates a vast space of potential token sequences, making every story generation unique.

The following figure illustrates how easily a story can branch into completely different paths, with each word influencing the direction and outcome of the narrative:

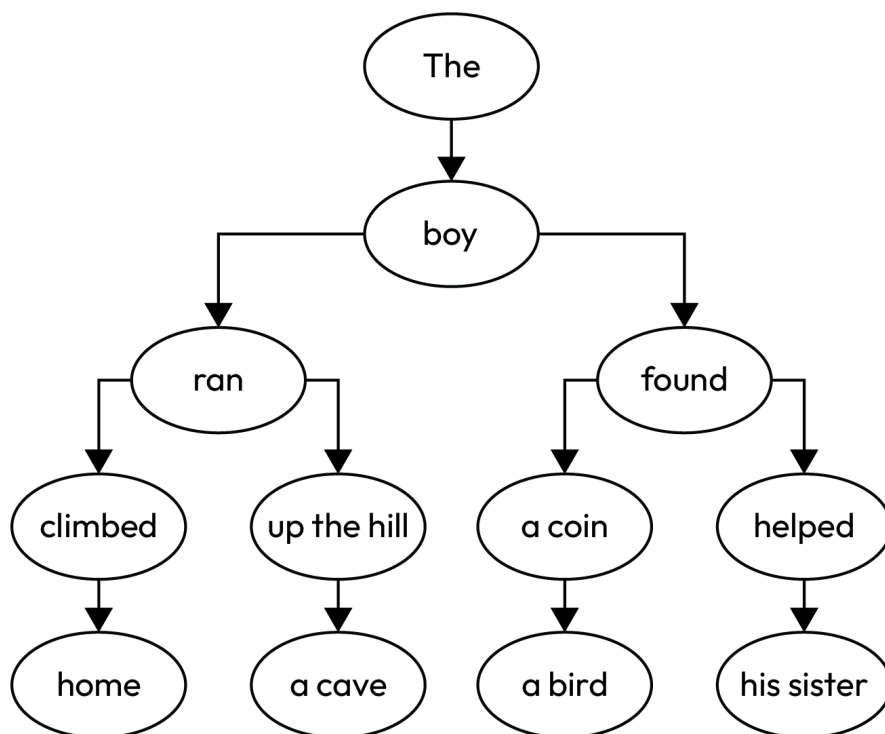


Figure 5.1: Branching diagram showing alternative word choices during generation

In reality, a language model considers thousands of possible next-word candidates at each step during sampling, far beyond the few branches shown here. This gives a sense of the enormous scale and combinatorial richness involved in generating even a single story.

Today, text generation is widely applied in real-world tools. For example, email clients like Microsoft Outlook can generate email drafts from short user prompts. Microsoft Word and PowerPoint now offer features to generate entire documents or slide decks based on high-level instructions.

Customer service platforms use text generation to assist in drafting responses to support tickets. In marketing, AI tools can create social media posts, product descriptions, and ad copy tailored to a brand's tone. Even in education, generative tools are used to produce practice questions, lesson plans, or personalized study materials.

Text generation parameters

For text generation tasks, several key parameters influence the quality and style of the generated output. These include the instruction or prompt used to guide the model, the desired output length, the use of few-shot examples to provide context, and the level of randomness applied during generation. In this section, we'll explore how each of these parameters affects the final text.

Instruction and prompt

For text generation, one of the most important settings is the instruction, and any additional hints provided in the prompt. The more detailed and specific the instructions, the more closely the generated text will align with the intended content, style, and structure. Conversely, the less specific the instruction, the more creative freedom is given to the language model.

And this leads to one of the key challenges in LLM-based text generation: the level of specificity and familiarity of the requested content. If we ask the model to generate a type of document that is widely available and well-represented in its training data, the LLM will likely produce a solid draft.

However, if the request is highly specialized, unusual, or outside of common patterns, the model may struggle. It might fill in gaps with plausible-sounding but inaccurate information or produce text that feels generic rather than domain-specific. This is why the level of detail in the prompt becomes even more critical when generating less common or highly specialized content.

Note that, by default, language models tend to produce responses in a neutral-to-positive tone and a moderately formal style. This behavior is typically shaped during post-training, for example, through techniques such as RLHF.

In addition to relying on the model's default behavior, we can explicitly control the tone and style of the generated text by including specific hints in the prompt. For example, the following prompt asks the model to use a friendly and casual tone:

Prompt

Write a short email to a colleague reminding them about tomorrow's meeting. Use a friendly and casual tone.

As we can see, effective text generation depends on balancing instruction specificity with the model's inherent capabilities. Clear, detailed prompts guide the model toward accurate, structured, and context-appropriate outputs, while less detailed prompts allow for more creative and open-ended generation.

Few-shot prompting

In cases where the language model needs additional guidance to produce the desired output, few-shot examples can be helpful (see *Chapter 3*). For text generation tasks, these examples typically include sample outputs that illustrate the expected format, tone, or structure. The model uses these examples to infer the relationship between the instruction and the desired outcome, allowing it to better align its response.

Here's an example of a few-shot prompt for a text generation task, where the model is expected to generate a short motivational quote based on a topic:

Prompt

Instruction: Generate a short, motivational quote based on the given topic.

Topic: Courage

Quote: "Courage doesn't mean you're not afraid—it means you choose to act despite the fear."

Topic: Failure

Quote: "Failure is not the end, but the beginning of a wiser journey."

Topic: Growth

Quote: "Growth happens when you step outside your comfort zone and keep going."

Each example shows a clear pattern: topic -> quote. The model learns not only the expected format but also the tone and length of the response.

However, for creative and open-ended tasks, a zero-shot setup (prompts without examples) can often be more effective. This is because providing examples may unintentionally constrain the model's creativity by anchoring it to specific patterns or styles.

Output length

One important aspect of text generation is controlling the length of the output. Similar to summarization tasks, text generation needs length guidance.

By default, the length of text generated by a language model depends on the specificity of the instructions provided in the prompt and the model's internal understanding of the task. For instance, if you ask an LLM to generate a dialog between two people without specifying the number of exchanges, the model will typically produce a short conversation with a few utterances per participant.

Similar to summarization, there are two primary ways to influence the length of generated text: through prompt design and through model settings. Prompts can include explicit instructions such as "*Write a one-paragraph story...*" or "*Limit the response to three bullet points.*" In addition to prompt-based guidance, most LLM APIs expose a `max_tokens` parameter, which sets a hard limit on the number of tokens the model can generate.

Generating long-form text introduces new challenges, too. Because LLMs generate text token by token in a left-to-right sequence (autoregressive generation), early decisions heavily influence later ones. Additionally, longer generations are more susceptible to drift, repetition, or loss of focus, especially when the prompt lacks structure or specific objectives.

To address these challenges, intermediate checkpoints or multi-step prompting (e.g., generating an outline first, then expanding each section) can be used. In *Chapter 3*, we explored the method of interactive text improvement (self-refinement), which allows the model to revise and enhance its responses step by step.

Randomness

Similar to summarization, increasing randomness in text generation leads to more creative and diverse outputs. However, in text generation, randomness plays an even more critical role because the goal is often to produce open-ended, imaginative, or stylistically varied content. Whether generating stories, dialogues, or marketing copy, we rely on randomness to help the model explore a broader space of possibilities rather than repeating predictable or templated language.

That's why medium (0.4–0.7) to high (0.8–1.0+) temperature settings are more commonly used in text generation tasks where creativity and diversity are desired. In contrast, low temperature settings (0.1–0.3) are typically applied when the output needs to be more deterministic, consistent, and closer to template-based generation.

In addition to temperature, top-p, and top-k are key parameters that shape how random, creative, or focused the model's output is. For text generation tasks, it is generally recommended to align top-p (or top-k) with the temperature setting to achieve the desired balance of creativity and control.

For example, a lower top-p value (e.g., 0.5) combined with a low or medium temperature can produce more focused and conservative output. On the other hand, using a higher top-p (e.g., 0.95–1.0) with a higher temperature encourages more diverse, imaginative, and less deterministic text, which is ideal for creative generation tasks.

Model selection

Similar to summarization, model selection plays a critical role in text generation tasks. In general, larger language models tend to perform better at generation tasks as well. This happens because larger models have a greater capacity to internalize linguistic patterns, world knowledge, and stylistic nuances during training.

However, larger models are not always the best choice. In domain-specific contexts where the required knowledge is narrow or highly specialized, even large general-purpose LLMs may underperform due to limited exposure to the relevant terminology or structure. In such cases, smaller models that have been fine-tuned on high-quality, domain-specific datasets can deliver better accuracy, relevance, and task alignment, often at lower computational cost.

Code generation

One important subtask of text generation is **code generation**, where the output is software source code. Code generation is a complex area that involves unique challenges and deserves more detailed exploration. For the purpose of this book, we will provide a high-level overview of the task and examine how it is addressed using modern large language models.

Software code can be considered a specialized subset of natural language, most commonly based on English, but governed by strict syntax, semantics, and formatting rules. Programming languages are designed to be both human-readable and machine-executable.

While many keywords and conventions are derived from English, code follows a much more rigid structure than natural language. This combination of natural language elements (like variable names, comments, and function names) with formal grammar rules makes code generation a *hybrid task* that requires understanding both natural language intent and formal language precision.

All code generation tasks can be classified as follows [29]:

Type	Mapping	Task	Description
Understanding	Code→Label	Code classification	Classify code based on functionality, purpose, or attributes
		Bug detection	Detect bugs or vulnerabilities in code
		Clone detection	Identify duplicated code
	Code→Code	Code-to-code Retrieval	Find relevant or similar code based on other code
	NL->Code	Code search	Find code based on natural language queries
Generation	Code→Code	Code completion	Predict the next code by part of the other code (starting code)
		Code translation	Translate code from one language to another
		Code repair	Identify and fix bugs in the code
		Test generation	Generate tests for code
	Code->NL	Code summarization	Generate a summary of code: a natural language description of the code
	NL->Code	Code generation	Generate code from a natural language query

Table 5.1: Code generation tasks

As we've seen, code generation encompasses a wide variety of tasks. Moreover, some of the tasks listed in *Table 5.1* can be further broken down into sub-tasks, each with its own specific challenges and nuances. For example, the **natural language to code (NL2Code)** problem includes sub-problems tailored to individual programming or query languages, such as NL2Python, NL2SQL, and others.

Among these, **natural language to SQL (NL2SQL)** stands out as particularly distinct and valuable, with broad applications in QA over structured data. Its ability to translate natural language queries into SQL makes it highly relevant for business intelligence, data analytics, and enterprise reporting scenarios.

One significant challenge for language models in code generation is the vast and ever-expanding vocabulary inherent in modern software development. Programming languages encompass not only built-in syntax and keywords but also a multitude of external libraries, frameworks, and APIs. Each of these introduces its own set of identifiers, such as class names, function names, and variables.

For instance, consider how JSON serialization and HTTP request executions are handled across different Java libraries:

Problem	Library/ Framework	Code example
JSON serialization	Jackson	<code>objectMapper.writeValueAsString(myObject)</code>
	Gson	<code>gson.toJson(myObject)</code>
	org.json	<code>new JSONObject(myObject).toString()</code>
HTTP request execution	Apache HttpClient	<code>httpClient.execute(httpGet)</code>
	OkHttp	<code>client.newCall(request).execute()</code>
	Spring WebClient	<code>webClient.get().retrieve().bodyToMono(...)</code>

Table 5.2: Handling of JSON serialization and HTTP request executions in different libraries

As we can see, the same functionality can be expressed in many different ways. Now, if we multiply this variability across dozens of programming languages (actually thousands) and thousands of frameworks and libraries, it becomes clear how much knowledge a language model must remember and generalize from to generate code effectively.

This growing and ever-changing vocabulary makes it difficult for language models to generalize effectively, especially when encountering unfamiliar libraries, APIs, or naming conventions. Unlike natural language, where the vocabulary is relatively stable and governed by shared linguistic norms, programming introduces an almost unlimited set of identifiers. As a result, models must learn not only common syntax and semantics but also develop strategies to handle rare, unseen, or ambiguous tokens.

That's why, in practice, when generating code using tools like ChatGPT or Gemini, it's common to observe that for less-known libraries or niche frameworks, the language model may fabricate function names, classes, or other identifiers. Essentially, the model generates "hallucinated" code by attempting to reason about what those elements might look like.

This challenge highlights a core limitation of models attempting to support such broad coverage of programming languages and tools. To mitigate this, language models must be trained or fine-tuned on extremely large and diverse code corpora. Exposure to a wide variety of real-world projects, libraries, and usage patterns improves the model's ability to generate accurate, syntactically correct, and semantically relevant code.

All of this highlights the need for targeted fine-tuning to enable most language models to generate code effectively. Alternatively, the model must be large enough to have been trained on extensive code datasets, at least for the most widely used programming languages like Python, JavaScript, C/C++, and so on.

The first widely known code generation models, such as OpenAI's Codex (used in early versions of GitHub Copilot) and Google's PaLM-Coder, were fine-tuned versions of existing general-purpose language models, GPT and PaLM, respectively.

To assess the performance of code generation models, benchmarks like HumanEval have been created. HumanEval was introduced by OpenAI in 2021 alongside the Codex model. It comprises 164 hand-crafted Python programming challenges, each accompanied by unit tests to verify functional correctness.

Over time, models have significantly improved their performance on the HumanEval benchmark. For instance, OpenAI's GPT-4o has achieved a *pass@1* score of 90.2%, indicating that the model's first attempt at solving each problem passes all associated tests 90.2% of the time.

Synthetic data generation

A practical application of text generation is the creation of synthetic data. In general, *synthetic data* refers to any data that is artificially generated by machines rather than collected from real-world human activity. In the context of NLP, this includes any text produced by language models, rule-based generators, or other automated systems.

Synthetic text is widely used in both research and industry. In machine learning, it can help address data scarcity issues by generating labeled examples for underrepresented classes or edge cases. Researchers also use synthetic data to train or evaluate models.

Synthetic data is also highly present on the internet today. Some studies estimate that at least 30% of text on active web pages originates from AI-generated sources, with the actual proportion likely approaching 40% [30]. Gartner predicted that by 2024, 60% of data used in artificial intelligence and analytics projects would be synthetically generated [31].

Despite its usefulness, synthetic data differs from human-generated text in important ways. While LLMs are capable of producing grammatically correct and coherent outputs, the semantic depth, diversity, and authenticity of human-generated content can still be difficult to replicate. Synthetic texts may over-represent high-frequency patterns seen during pretraining or exhibit stylistic uniformity. Additionally, synthetic data can unintentionally carry model biases or inaccuracies.

Synthetic text can be generated using zero-shot or few-shot prompting techniques. In zero-shot prompting, a language model is given an instruction without any examples. Few-shot prompting, on the other hand, provides the model with a handful of examples to guide its output.

For more structured and contextually rich synthetic data, multi-stage prompting approaches can be used. This involves a sequential process where the model first generates a high-level data plan or outline, which serves as a blueprint. Subsequently, the model produces individual data samples based on this plan, ensuring coherence and alignment with the desired objectives.

Advanced techniques also include **meta-prompting**, where a language model orchestrates multiple specialized "expert" agents to collaboratively generate diverse and high-quality synthetic data.

So, we've explored how language models can summarize content, generate open-form text, classify inputs, extract structured information, and operate across multiple languages. Now it's time to dive into one of the most exciting and powerful capabilities of LLMs, which is *question answering*, or simply QA.

Question answering

QA is where everything comes together. QA pushes models to interpret questions, retrieve or infer relevant knowledge, and deliver precise, context-aware answers. In many ways, this is where LLMs stop just processing language and start using it intelligently.

Note

Note that QA is the standard abbreviation for *question answering* in research papers, academic benchmarks, and most technical contexts. In contrast, **questions and answers (Q&A)** is a more informal, user-facing term typically used in product interfaces, UI labels, or conversational messages, where it refers broadly to "questions and answers" rather than the specific NLP task.

At its core, QA is about providing the right information in response to a specific question. It is a foundational skill for large language models because it combines multiple competencies:

- Understanding context and intent
- Locating or recalling relevant knowledge
- Formulating a coherent and relevant response

These abilities are central to many real-world applications: chatbots, customer service agents, and tutoring systems all depend on reliable QA performance.

QA is more than a single task. It's a framework that encompasses many sub-problems, from simple fact retrieval to complex, multi-hop reasoning. This makes it a powerful lens through which to evaluate and build intelligent systems.

We can also view each problem we've explored so far as a specific form of question answering. In a way, every NLP task poses its own kind of question to the model. For instance, summarization asks, "*What is most important in this text?*"; text generation asks, "*How should this continue or look?*"; and text classification asks, "*What does this text mean or express?*"

From this perspective, QA becomes a unifying framework rather than just another task. Almost every interaction with a language model can be reframed as the model answering a question, whether explicit or implicit. This is why QA sits at the heart of natural language understanding and generation.

Broadly speaking, we can group and distinguish QA systems into several key categories:

- Closed-domain vs. open-domain QA
- Multi-hop and conversational QA

Let's explore these types of QA and the way LLMs solve them.

Closed-domain and open-domain QA

Closed-domain QA operates within a specific, pre-defined context (scope) such as a single document, a knowledge base, or a product manual. The model is expected to find the answer within this limited scope. For example, we have a bank database and ask a question like, "What is the interest rate on a savings account?"

Another illustrative example comes from the SQuAD (<https://rajpurkar.github.io/SQuAD-explorer/>) dataset, which is one of the most widely used benchmarks in question answering tasks.

id	title	context	question	answers
string · lengths	string · classes	string · lengths	string · lengths	sequence
24	100%	507~863	1~2.57k	100%
5733be284776f41908661182	University... e_Dame	Architecturally, the school has a Catholic character. Atop the Main Building's gold dome is a golden statue of the Virgin Mary. Immediately in front of the Main Building and facing it, is a copper statue of Christ with arms upraised with the legend "Venite Ad Me Omnes". Next to the Main Building is the Basilica of the Sacred Heart. Immediately behind the basilica is the Grotto, a Marian place of prayer and reflection. It is a replica of the grotto at Lourdes, France where the Virgin Mary reputedly appeared to Saint Bernadette Soubirous in 1858. At the end of the main drive (and in a direct line that connects through 3 statues and the Gold Dome), is a simple, modern stone statue of Mary.	To whom did the Virgin Mary allegedly appear in 1858 in Lourdes France?	{ "text": ["Saint Bernadette Soubirous"], "answer_start": [815] }

Figure 5.2: An example record from the SQuAD dataset on Hugging Face

In this dataset, each record consists of three key components: a **context** (a passage from a Wikipedia article), a **question** based on that context, and a correct **answer** (ground truth). All questions in SQuAD are tightly *grounded* in the provided context, meaning the answer must be found within the accompanying *passage* (provided context).

Closed-domain QA was one of the earliest question answering problems tackled in NLP. Early systems approached this task using rule-based methods, where answers were extracted using handcrafted patterns or keyword matching. These systems were extremely limited in flexibility and struggled with the variability of natural language.

Today, LLMs have become the primary solution for closed-domain QA. The shift to LLMs has been driven by the inherent complexity of the task, which often requires deep language understanding, reasoning, and generalization, capabilities that traditional rule-based systems simply couldn't provide.

Here's a simple example of a prompt you could use to solve a closed-domain QA task:

```
Prompt
Answer the question based only on the context provided below.
```

Context:

"Albert Einstein was born in Ulm, Germany, in 1879. He developed the theory of relativity, which revolutionized theoretical physics. He won the Nobel Prize in Physics in 1921. He also developed an explanation of the photoelectric effect."

Question:

"When did Albert Einstein win the Nobel Prize?"

This prompt is straightforward and shows how language models can extract precise facts from a given context. However, even such simple examples can be challenging for LLMs when multiple plausible details appear in the same context.

For instance, we could also ask, "Why did Einstein win the Nobel Prize?" The context provides several key facts like his birthplace, his theory of relativity, and his explanation of the photoelectric effect. But the context never explicitly connects the award to a specific reason. A model might easily pick the wrong one, for example, associating the prize with "the theory of relativity" because it seems more prominent or familiar.

This subtle distinction highlights a core challenge in closed-domain QA: even when the correct information exists within the provided context, models can still produce answers that sound reasonable but actually are incorrect. It emphasizes the importance of careful context interpretation and demonstrates why plausible yet incorrect answers remain a common pitfall.

A second major challenge in closed-domain QA is **grounding**. Large language models are pre-trained on massive and diverse datasets, which means they carry strong priors about the world. Even when explicitly instructed to "answer only based on the provided context," a model may still mix that background knowledge with the passage or, worse, generate details that sound plausible but are not actually present in the text.

This happens because an LLM doesn't simply read the input text. As we learned from *Chapter 2*, an LLM transforms text into embeddings and enriches them with knowledge encoded in its parameters, learned from large-scale internet data. As a result, the model's internal representations of meaning are always influenced by both the prompt and its prior knowledge (model weights). When these two sources of information conflict, the model may "fill in" missing details or adjust facts to match what it expects to be true (override input embeddings), rather than what the context actually says.

The quality and clarity of the context, therefore, play a critical role. If the passage itself is ambiguous, contradictory, or phrased in a way that differs from typical real-world knowledge, the model may default to its pre-trained assumptions instead of the supplied information, effectively letting its parameterized knowledge overwrite the input embeddings. Even small inconsistencies between the context and the model's internal knowledge can lead to confident yet incorrect answers.

To mitigate this, modern QA systems employ grounding and verification techniques, citation alignment, or answer attribution. These methods help ensure that model responses stay anchored to the actual evidence rather than drifting toward prior knowledge. We'll explore these strategies in greater detail and how **retrieval-augmented generation (RAG)** formalizes them in *Chapter 5*.

In contrast, *open-domain QA* assumes that the answer may come from anywhere, not just a predefined passage or document. Open-domain QA systems must handle questions without knowing in advance where the relevant information resides. To address this, they typically include a retrieval component that searches across large corpora such as Wikipedia, domain-specific knowledge bases, or even the open web. Once the most relevant documents or passages are retrieved, a language model processes them to generate the final answer.

This setup is far more flexible than closed-domain QA but also more complex. The quality of the final answer depends not only on the model's reasoning ability but also on the effectiveness of retrieval. If the system fails to surface the right context, even the strongest model may produce an incorrect or hallucinated response. Conversely, when retrieval works well, open-domain QA systems can handle a much broader range of questions, including those requiring information scattered across multiple sources or absent from any single document.

We'll explore both closed-domain and open-domain QA in more detail in the next chapter, which focuses on RAG.

Multi-hop and conversational QA

Now let's look at a more advanced type of QA: **multi-hop QA**. This form of QA requires a model to combine multiple pieces of information, often drawn from different parts of a document or even across multiple documents, to answer a single question. Multi-hop questions involve reasoning through intermediate steps to connect related pieces of information.

For example, consider the question, "Which organization owns the operating system developed by Linus Torvalds?"

To answer this, the system must first identify that Linus Torvalds developed Linux and then determine that Linux is managed by the Linux Foundation. This is a *two-hop* question, meaning it requires two distinct reasoning steps to reach the correct answer: identifying the relevant entity and then tracing its organizational affiliation.

Multi-hop QA challenges models to build reasoning chains, maintain contextual awareness across steps, and resist distractions from irrelevant but related information.

Consider another more advanced question: "What currency is used in the country where Toyota's stock is traded?"

This question already requires three hops:

- Identify the stock exchange that lists Toyota → Tokyo Stock Exchange
- Determine the country of Tokyo Stock Exchange → Japan
- Get the currency used in Japan → Japanese Yen (JPY)

As we can see from these examples, multi-hop QA is inherently more complex than single-hop or factoid-style questions. It requires models not only to retrieve relevant information, but also to connect multiple intermediate facts in a logical sequence. This demands reasoning capabilities, such as tracking entities across sentences, resolving references, and inferring relationships between concepts that are not explicitly stated in the same place.

The emergence of LLMs, such as GPT-3 and beyond, introduced capabilities to handle such advanced questions. Techniques like **chain-of-thought (CoT)** prompting have further improved performance in multi-hop QA tasks.

Now let's consider one of the most complex types of question answering: *conversational QA*. While traditional QA systems typically handle single, isolated questions, conversational QA involves *multi-turn* dialogues, where each question and answer is part of an ongoing conversation. This introduces new challenges. Now the system must not only understand each question independently but also interpret it in the context of previous questions (turns). It requires the ability to maintain dialog memory, resolve references (like pronouns or implicit subjects), and track the evolving state of the conversation over multiple exchanges.

For example, consider the following interaction:

- **User:** Who is the CEO of Microsoft?
- **System:** Satya Nadella is the CEO of Microsoft
- **User:** Where did he complete his MBA?
- **System:** He earned his MBA from the University of Chicago Booth School of Business

In this example, the second question contains the pronoun "he", which refers to Satya Nadella, mentioned in the previous turn. This is a case of *coreference*, where a pronoun or phrase in one sentence refers back to an entity introduced earlier. A conversational QA system must correctly resolve such references, linking them to the appropriate entity, in order to maintain continuity and coherence. This requires the system to carry over context from earlier turns and use it to interpret the current question accurately.

Modern LLMs are well-suited for conversational QA because of their ability to handle longer context windows and maintain coherence across multiple dialog turns. Still, practical systems must often manage context size constraints, prevent error accumulation, and decide how much past dialog to retain or prioritize. We'll explore these challenges in more detail in *Chapter 8*, where we dive into the design of conversational agents such as chatbots and virtual assistants.

QA for structured data

Up to this point, we've focused primarily on question answering over text data (unstructured data). However, a large portion of the world's information is stored in structured formats, including relational databases (SQL databases), spreadsheets, data warehouses, and structured documents like JSON or XML. Answering questions over this kind of structured data is a common and highly valuable capability, particularly in domains like finance, operations, customer analytics, and enterprise reporting.

Structured data can be grouped by the format and the level of schema rigidity it offers. At the core, we have relational data, such as SQL databases and spreadsheets, which follow a strict tabular schema with clearly defined columns and data types. These are ideal for analytical queries and are widely used in business intelligence, finance, and reporting systems.

Next, we have semi-structured data, such as JSON, XML, or YAML, which still contain structure, often in the form of key-value pairs or nested objects, but allow more flexibility in schema. These formats are commonly used in APIs, logs, and document stores like MongoDB or DynamoDB, where each record may have slight variations in structure.

If the data is relatively small, we can often provide the data directly in the prompt to the LLM. In many cases, this works surprisingly well: LLMs can understand the schema, interpret the question, and generate an accurate response without requiring any additional tools or context management.

However, challenges quickly arise as the size and complexity of structured data increase. As we discussed earlier, the effective context window of LLMs remains relatively small. Complex data schemas place additional strain on the model.

To address these challenges, one of the most widely adopted solutions is to use a language model to translate a natural language question into a formal query or code, such as SQL or Python. The generated query or code is then executed directly on the underlying data source, which could be a relational or a NoSQL database, an Excel spreadsheet, or a structured API (like Python Pandas data frames). This approach leverages the strengths of both components: the LLM handles the user's intent and question understanding, while the database engine efficiently retrieves and processes the data.

For example, in the case of tabular data stored in a relational database, a typical QA pipeline involves parsing the user's question, generating the appropriate SQL query, executing it on the database, and then returning the result as the answer; see the following figure:

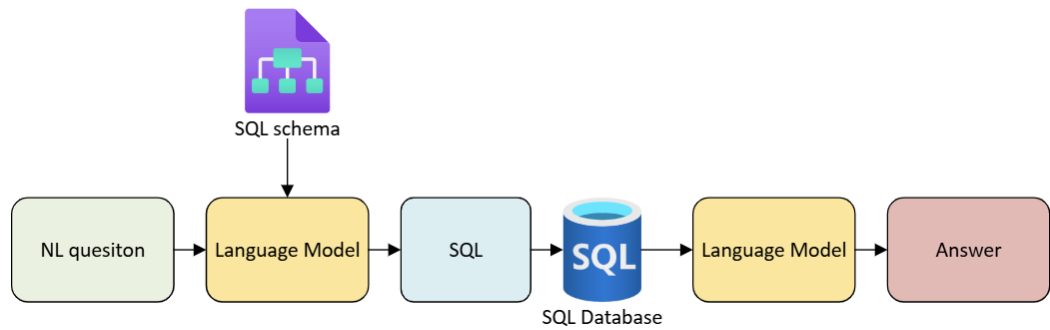


Figure 5.3: QA over structured data via SQL generation

To illustrate how this works, let's consider a simple example using a table of customer transactions:

transaction_id	customer_id	Date	category	amount	currency
1001	C001	2023-01-15	travel	1200.00	USD
1002	C002	2023-01-20	groceries	85.50	USD
1003	C001	2023-02-05	travel	1500.00	USD
1004	C001	2023-03-01	utilities	200.00	USD

Table 5.3: A customer transaction table

For this data, we can ask questions like the following: "What was the total amount spent by customer C001 on the travel category in 2023?"

To solve this type of problem, we can use a language model to generate a SQL query. This can be done by passing the table schema along with the question in the prompt, as in the following example:

Prompt

You are an assistant that converts natural language questions into SQL queries based on a given table schema.

Table name: transactions

Columns:

- transaction_id (string)
- customer_id (string)
- date (date)
- category (string)
- amount (float)
- currency (string)

Question:

What is the total amount spent by customer C001 on travel in 2023?

This prompt will generate the following SQL query:

```
SELECT SUM(amount)
FROM transactions
WHERE customer_id = 'C001' AND
      category = 'travel' AND
      date BETWEEN '2023-01-01' AND '2023-12-31';
```

This result is fairly accurate, and the generated SQL would correctly return the data needed to form the final answer. However, in other cases, SQL generation can be more challenging.

To see how this happens, let's look at a slightly different phrasing of the original question: "What was the total spent by c001 on Travel in 2023?"

This rephrased question preserves the same meaning but adds ambiguity that may mislead the LLM:

- **Implicit column references:** The word "amount" is missing, so the model must infer which numeric column to aggregate
- **Ambiguous filters:** The phrase "by c001" could apply to several columns, especially since the model doesn't have access to actual data values, only the column names
- **Casing issues:** The data values in the table might be case-sensitive (e.g., lowercase "travel" in the category column, uppercase "C001" in customer_id)

These subtle differences illustrate a broader challenge: even small variations in phrasing can introduce ambiguity that reduces the reliability of SQL generation and overall QA accuracy. While LLMs are strong at interpreting natural language, they operate with limited context and no direct visibility into the actual data values, making them prone to inference errors when information is implicit or inconsistent.

There are several strategies to mitigate these challenges, even when working with very large tables that can't be fully embedded into the LLM context:

- **Include a few sample rows in the prompt:** Providing several representative examples helps the model better understand the structure and semantics of the data.
- **Include known categorical values:** Some columns, like category in our previous example, may have a limited, well-defined set of values, even if the full table spans millions of rows. By explicitly passing this list into the prompt (e.g., "category values: travel, food, office, insurance, ..."), we give the LLM context to disambiguate references in the question.

With these enhancements, the revised prompt might look like this:

Prompt

You are an assistant who converts natural language questions into SQL queries based on a given table schema.

Table name: transactions

Columns:

- transaction_id (string)
- customer_id (string)
- date (date)
- category (string)
- amount (float)
- currency (string)

Sample rows:

transaction_id	customer_id	date	category	amount	currency
1001	C001	2023-01-15	travel	1200.00	USD
1002	C002	2023-01-20	groceries	85.50	USD

Known values for `category` column: travel, food, office, utilities, insurance, entertainment

Question:

What is the total amount spent by customer C001 on travel in 2023?

With this additional context, the LLM can more confidently generate the correct SQL. When the question mentions "total spent", the model can infer from the sample rows that this likely

refers to the amount column. Similarly, "by C001" clearly maps to the `customer_id` field, and "travel" is correctly understood as a value in the category column thanks to the explicit list of possible categories provided in the prompt.

Another challenge in QA over structured data is that, in real-world applications, information is rarely stored in a single table or one document. Instead, it is typically distributed across multiple relational tables, documents, collections, or other structured formats, depending on whether the system uses SQL or NoSQL technologies. This introduces an additional layer of complexity for question answering.

To perform accurate QA over such databases, a system must not only understand the user's query but also have a comprehensive understanding of the database schema. This includes knowing which entity (table, collection) contains the relevant information, how those entities are related through references and relationships, and what joins or filters are needed to retrieve the correct data.

For example, a question like "What was the total revenue generated by product category X last quarter?" may require joining a products table with sales, and possibly filtering using a dates or transactions table. Without awareness of the relationships, it's nearly impossible for an LLM to select the correct sources or build accurate queries, especially in enterprise-scale environments with complex and normalized schemas. Adding data schema definitions (like T-SQL) will improve SQL generation accuracy.

Although a major challenge in real-world systems is schema size, in many enterprise databases, schemas can include thousands of tables with intricate relationships. Passing the full schema into the LLM's context window is often infeasible or inefficient.

To handle such scenarios, a two-step prompting strategy can be used:

- **Table selection phase:** Use the LLM to select the relevant tables based on the user's question. For this step, only the table names and short descriptions can be provided to LLM.
- **Query generation phase:** Once the relevant tables are identified, provide their full schemas and selected tables to the LLM to generate the final SQL query.

This modular strategy helps QA systems scale to large and complex database environments. In this setup, we effectively use a multi-prompt flow: the first prompt performs a multi-label classification task to identify the relevant tables, and the second prompt generates the corresponding SQL query.

Another frequent issue in structured QA is naming ambiguity in database schemas. In the earlier example, table and column names like `customers`, `order_date`, and `total_amount` are self-explanatory for both humans and LLMs. However, in many systems, especially legacy

ones, the naming conventions are cryptic and difficult to interpret without additional metadata.

Here's an example of one table definition from an SAP ERP database:

```
CREATE TABLE VBAK ( -- Sales Document: Header Data
    MANDT CHAR(3),           -- Client
    VBELN CHAR(10),          -- Sales Document Number
    AUART CHAR(4),           -- Sales Document Type
    ERDAT DATE,              -- Date on which the record was created
    ERNAM CHAR(12),          -- Name of Person who Created the Object
    VKORG CHAR(4),           -- Sales Organization
    VTWEG CHAR(2),           -- Distribution Channel
    SPART CHAR(2),           -- Division
    KUNNR CHAR(10),          -- Sold-To Party (Customer Number)
    NETWR DECIMAL(15,2),     -- Net Value of the Order
    WAERK CHAR(5),           -- Currency
    PRIMARY KEY (MANDT, VBELN)
);
```

In this schema:

- The table name VBAK gives no immediate hint about its contents
- Column names like VBELN, AUART, and KUNNR are non-intuitive to users unfamiliar with SAP's naming conventions

If LLM was not trained explicitly for such a schema, it might not be able to generate the correct queries. To enable accurate QA in such cases, it is essential to provide metadata or natural language descriptions of each object (table, column). These descriptions serve as a translation layer, allowing the LLM to understand what kind of data is stored and how to map natural language questions to the underlying schema.

Reasoning

We've come a long way. From mastering summarization to generating content, classifying text, extracting structured information, handling multilingual inputs, and answering complex questions, we've explored the full breadth of what large language models can do.

But now, we arrive at something even more profound: *reasoning*. **Reasoning** is the beating heart of intelligence. It's what allows us to draw conclusions, connect dots across facts, and make decisions. And for a long time, this was the line that traditional NLP systems couldn't cross. They could summarize, match, classify, and parse, but they couldn't think.

Today, that has changed. Because of massive advances in pretraining, model architecture, and prompt engineering, LLMs are showing impressive capabilities in performing tasks that look and feel like human reasoning. We're talking about solving logic puzzles, navigating cause-and-effect chains, reasoning over timelines, and even making analogies, i.e., tasks that require not just language fluency, but true *cognitive inference*.

This is more than just another capability. Reasoning is the culmination of everything we've covered so far. Summarization taught the model to distill. Generation unlocked fluency and creativity. Classification and extraction brought structure. And now, all of these skills come together in a new, more powerful way: the ability to reason.

At first glance, reasoning and QA may seem like the same thing. After all, both involve taking a question and producing an answer. But look a little closer, and a subtle difference emerges.

QA is about providing answers to user queries, often by retrieving facts from known knowledge or context. Reasoning, on the other hand, is about how to conclude when the answer isn't explicitly stated. It requires the model to infer, synthesize, and sometimes simulate a step-by-step thought process to reach the final answer. In other words, reasoning is a method for answering questions.

In this section, we'll dive deep into what reasoning looks like for LLMs, how to prompt for it effectively, the limitations still in place, and how reasoning unlocks the next frontier: intelligent retrieval and autonomous AI agents.

LLM reasoning

Since LLMs are statistical sequence models, their reasoning emerges from patterns in data rather than strict logical rules. Let's walk through the major forms of reasoning LLMs perform today, progressing from the simplest to the most complex.

Commonsense reasoning

Commonsense reasoning refers to the ability to apply broad, everyday knowledge about the physical, social, and human world to interpret or generate language in a way that feels natural and intuitive to humans. This includes assumptions we rarely state explicitly, like how objects behave, what people are likely to do, or how events unfold in daily life.

Commonsense doesn't follow strict rules; it's fuzzy, experiential, and often context-dependent. For humans, it's second nature. For LLMs, it's one of the most elusive but essential types of reasoning.

Examples:

- *"Can a person fit inside a shoebox?"* → **No**
- *"If you drop a glass on a hard floor, what is likely to happen?"* → **It will break**
- *"Why would someone put a wet umbrella in a stand near the door?"* → **To avoid dripping water indoors**

These are not facts we memorize; they are generalizations we've internalized through lived experience. For an LLM, answering correctly requires having seen enough real-world textual data to generalize similar patterns.

Today, modern LLMs like GPT-4, Claude, or Gemini perform impressively on many commonsense tasks from simple physical-world questions (e.g., "Can an elephant jump?") to understanding social norms (e.g., "Is it polite to interrupt someone while they're talking?"). Much of this is due to their exposure to vast, diverse web text that reflects the statistical patterns of everyday reasoning.

Despite these advances, LLMs still struggle in several areas. They may fail when context is ambiguous or requires unstated assumptions, relying on guesses about what is "normal" (e.g., interpreting "Jamie put the milk in the cupboard" and assuming it refers to a fridge). They can also confuse real-world knowledge with patterns in text.

Additionally, LLMs often struggle with multi-step reasoning that combines common sense with logical operations across several steps. These limitations highlight the gap between learned language patterns and true understanding. While techniques like better prompting, retrieval, and fine-tuning can mitigate some of these issues, achieving robust commonsense reasoning remains an open challenge and motivates the use of grounding, tools, and agent-based approaches explored in later chapters.

Arithmetic, logical, and symbolic reasoning

Arithmetic, logical, and symbolic reasoning refers to the ability to perform structured, rule-based reasoning, including mathematical operations, logical inference, and manipulation of abstract symbols like equations or code. These forms of reasoning differ from common sense in that they rely on explicit rules, formal systems, and precise structure.

Symbolic reasoning serves as an umbrella category, encompassing arithmetic (e.g., numerical operations), logic (e.g., deduction, truth tables), and other structured formats like algebra, formal proofs, rule-based programming, and decision trees. These skills are foundational in domains such as finance, engineering, law, compliance, analytics, and code generation.

Examples:

- "What is $47 + 138$?" → **Arithmetic**.
- "If A is true and B is false, what is A AND B?" → **Boolean logic**.
- "All students passed the test. Alex is a student. Did Alex pass the test?" → **Deductive logic**.
- "If a train leaves at 2 PM and arrives at 5:30 PM, how long is the trip?" → **Mixed temporal + Arithmetic reasoning**.
- "Solve for x: $2x + 3 = 11$ " → **Symbolic reasoning**.

Prior to LLMs, arithmetic and logical reasoning were handled through specialized rule-based systems, symbolic solvers, or programmatic tools. These could solve structured problems well, but only within narrow domains (e.g., WolframAlpha for math, Prolog for logic).

Today, LLMs can do the following well:

- Solve basic arithmetic, short logical expressions, and simple word problems
- Follow clear logical structures when prompted, techniques like chain-of-thought or so
- Perform well on tasks involving *deduction*, *inference trees*, or *truth tables*, especially with few-shot prompting

But still, there are several areas where LLMs struggle with this type of reasoning:

- **Multi-step computation without tools:** LLMs can lose track of state (intermediate steps), especially in long problems.
- **Precision vs. plausibility:** LLMs are language models; they optimize for likely continuations, not always correctness. This can cause them to confidently output incorrect math or logic.
- **Hallucinated logic:** Sometimes, models invent steps or skip required operations unless prompted to reason step-by-step.
- **Arithmetic consistency:** A model might answer " $47 + 138 = 175$ " and " $138 + 47 = 174$ " in the same session. Unlike a calculator, LLMs don't naturally preserve mathematical consistency unless fine-tuned or externally validated.

While LLMs have shown impressive progress in arithmetic and logical reasoning, they still can't compete with specialized tools when it comes to precision, reliability, performance, and efficiency. If your goal is simply to compute something like $a + b$, it's far better to use a calculator or math engine. It will be faster, more accurate, and cheaper than calling an LLM.

That said, LLMs can still be highly effective for basic to medium-complexity arithmetic and logical tasks, especially when integrated into broader workflows such as answering natural language questions, generating explanations, or parsing messy user input.

Temporal reasoning

Temporal reasoning refers to the ability to understand, interpret, and reason about time-related information, including event order, durations, intervals, scheduling, and temporal dependencies. It involves understanding both explicit time expressions (like dates or durations) and implicit ones (like before/after relationships or contextual timing).

For humans, this kind of reasoning is intuitive: we effortlessly understand that "Monday comes before Tuesday," or that "if the meeting started at 2 PM and ended at 3:30 PM, it lasted 90 minutes." For LLMs, however, temporal reasoning presents unique challenges due to its reliance on context, implicit cues, and arithmetic operations.

Examples:

- "*John arrived before Mary. Mary arrived after Tom. Who arrived first?*" requires reasoning across multiple relative time statements.
- "*If your flight is at 3:45 PM and you need to arrive 2 hours early, when should you get to the airport?*" involves time subtraction and context application.
- "*The meeting is on the second Monday of next month.*" requires calendar understanding and date inference.

But it's important to remember that LLMs are fundamentally text-generation models, not exact execution engines. As a result, they can struggle with:

- **Date and calendar logic:** Computing dates like "the third Tuesday of next month" or interpreting recurring events can break down without tool support
- **Grounding to the current time:** LLMs without real-time access may misinterpret statements like "tomorrow" or "next Monday" unless prompted with context

To improve LLM performance on temporal reasoning tasks, we can apply techniques such as chain-of-thought prompting, explicitly providing the current date or time (so the model can accurately interpret expressions like "today" or "next Monday"), or using NL2Code strategies to offload date computations and scheduling logic to code.

Causal reasoning

Causal reasoning is the ability to understand and infer *cause-and-effect* relationships. This reasoning includes determining what causes an event, what consequences might follow, or how actions lead to specific outcomes. For humans, this reasoning is foundational: we understand that pushing a glass off a table causes it to fall and break. For LLMs, causal reasoning is more complex because it requires understanding not just what happened, but why it happened or what will happen next.

While common sense helps LLMs interpret typical situations, causal reasoning enables them to predict outcomes, explain events, and reason about change, making it especially important for tasks like planning, decision-making, and autonomous agent behavior.

Causal reasoning is central to decision-making, planning, and explanation, and while LLMs are improving rapidly in this area, they still face challenges with ambiguity, counterfactuals, and multi-step cause-and-effect chains. For best results, causal tasks should use structured prompting, and when accuracy matters, external grounding through retrieval or logic systems can enhance performance.

Analogical reasoning

Analogical reasoning is the ability to recognize relational similarities between different domains. It includes drawing inferences in a new context based on structural or functional patterns observed in a known one. It plays a central role in human creativity, abstract thinking, metaphor, and problem-solving.

Humans often use analogies to explain unfamiliar concepts (e.g., "The heart is like a pump") or to solve problems by mapping knowledge from one domain onto another. This process is grounded in a deep understanding of how different systems work, along with real-world, sensory, and experiential knowledge.

Large language models also demonstrate the ability to recognize and generate analogies. Modern LLMs can complete standard analogy structures, interpret, and generate metaphors, and understand figurative comparisons quite effectively, especially when they are phrased in familiar formats or appear frequently in training data.

However, LLMs often struggle with more complex or abstract analogies. Analogies that span unfamiliar domains or rely on deep structural mapping rather than surface similarity are especially challenging.

This is because LLMs don't perform analogical reasoning the same way humans do. While humans rely on intentional abstraction, conceptual understanding, and real-world grounding, LLMs generate analogies through statistical pattern matching, identifying relationships based on how words and ideas co-occur in text. As a result, LLMs are quite effective at reproducing familiar analogies, but they can be unreliable or inconsistent when tasked with constructing novel, cross-domain, or structurally complex analogies.

Multi-hop reasoning

Multi-hop reasoning refers to the ability to answer a question or solve a task by chaining together multiple pieces of information. Each individual fact may be insufficient on its own, but when logically linked to others, it leads to a coherent conclusion. This type of reasoning mimics how humans piece together knowledge to handle more complex, layered queries.

Examples:

- "If you finish the project early, then you can travel. But only if you get approval." requires evaluating multiple conditions and dependencies.
- "Which department owns the customer account managed by the director who reports to the VP of Sales?" requires linking organizational relationships and roles. It is common in enterprise analytics or CRM tools.

Modern LLMs perform quite well on simple and middle-level multi-hop QA, particularly when the reasoning steps are clearly stated and close together in the context. However, they can lose coherence over longer chains, especially when relationships are implicit or dispersed. A small mistake early in the chain can lead to incorrect final conclusions.

Reasoning models

In *Chapter 2*, we briefly introduced a new class of large language models built specifically to excel at reasoning, such as models like GPT o1 or o3, Claude Sonnet, DeepSeek R1, and "thinking-style" Gemini variants. In this section, we'll take a closer look at how these advanced models approach reasoning tasks and can solve them more efficiently.

Reasoning models are trained using **reinforcement learning (RL)** techniques that encourage them to "think" before producing an answer. The model internally performs a sequence of intermediate steps, breaking down the problem, exploring possible solutions, and refining its response. This process can be viewed as a form of *internal* dialog, although it is not always explicitly exposed to the user.

For example, consider a request to analyze a company's performance. Internally, the model can start producing the text like: *"The user is asking for a performance analysis; first, evaluate financial indicators, then consider growth trends..."* After completing this internal reasoning, the model produces a final, structured answer based on those intermediate steps. So, in this sense, reasoning models can be viewed as having CoT capabilities built in by design.

If we take a look at evaluation benchmarks published by OpenAI, we can see that reasoning models perform significantly better on datasets like MATH, GPQA, and SimpleQA. These benchmarks require more advanced reasoning capabilities. The strong performance of reasoning models on these datasets highlights their internal ability to "think before answering."

Of course, the improved reasoning capabilities don't come for free. They come at a cost in both price and latency. Reasoning-optimized models are significantly more expensive, often consume more tokens due to their longer internal deliberation, and tend to respond noticeably slower than general-purpose models.

For example, as of May 2025, GPT-o3 costs nearly 5× more per token than GPT-4.1 and may add 3 to 5 seconds of latency per API request, depending on the prompt complexity and token length. This overhead is a direct result of the model's internal "thinking process" to generate longer chains of reasoning before producing a final output.

This trade-off means that while reasoning models are powerful, they should be strategically applied to problems where depth, accuracy, or decision-critical reasoning truly matter. For simpler or high-volume tasks, more lightweight models may still be the better option in terms of cost and speed.

Moreover, prompt engineering techniques such as CoT, Tree-of-Thoughts, self-consistency, and multi-prompt flows (discussed in *Chapter 3*) can often achieve comparable or even better performance than reasoning models, especially in terms of accuracy, latency, and cost.

Modern reasoning models can sometimes expose parts of their intermediate reasoning ("internal dialog"). Analyzing these explanations can help determine whether a reasoning model is truly necessary for a given task. If the model consistently follows the same or very similar reasoning patterns, it may indicate that the task does not require deep reasoning. In such cases, a well-designed prompt that guides the model through a structured process can often achieve comparable results without relying on a specialized reasoning model.

Reasoning models, in contrast, require less prompt design effort since advanced reasoning mechanisms are already built into their architecture. Ultimately, the choice between reasoning models and prompt-based techniques should be driven by empirical evaluation, something we'll explore in detail in *Chapter 10*.

Note that tasks that don't heavily rely on reasoning can often be handled more effectively by general-purpose models like GPT-5.2. For example, often we don't need reasoning models for summarization, classification, content generation, information extraction, and factual question answering tasks.

When deciding whether to use a reasoning-optimized model, two key factors should be evaluated:

- *Does the task actually require reasoning?* If so, what type of reasoning: arithmetic, causal, analogical, etc.? This helps determine whether a reasoning model would significantly improve performance.
- *Are cost and latency acceptable trade-offs?* As we mentioned, reasoning models are more expensive and slower, so it's critical to assess whether the accuracy gains justify the additional overhead.
- *Can prompt engineering be more efficient?* Can using advanced prompt engineering be more efficient than using reasoning models? Does the reasoning model follow the same reasoning patterns for all user questions?

For example, consider a real-world business scenario such as reallocating marketing spending based on last quarter's trends and compliance requirements. This task may involve multi-hop reasoning (linking different budget components), causal reasoning (predicting outcomes of reallocations), and arithmetic operations. In this case, a reasoning model like GPT-4o or Claude Opus could produce more accurate and justifiable recommendations than a standard model.

Modern language models, such as GPT-5.x or Claude Opus 4.6, can be viewed as hybrid systems that combine both generation and reasoning capabilities. A single model dynamically adapts its behavior, producing fast, direct responses for simple tasks or engaging in deeper, multi-step reasoning when needed.

Reasoning behavior can often be influenced by prompting. For example, CoT instructions such as "think step by step" encourage the model to produce more deliberate, structured reasoning. In addition, many platforms allow you to control the level of reasoning through inference settings, enabling a trade-off between speed, cost, and accuracy.

Unknown reasoning

Despite years of research and growing empirical evidence, how large language models actually "reason" remains only partially understood. Above, we have described all known facts about LLM reasoning capabilities. We can describe what they do, what types of reasoning they can solve (causal, commonsense, and others), but not how this process is represented internally.

As we saw in *Chapter 1*, we can visualize how neural networks store information within their weights: even a tiny 4×4 matrix with 16 model parameters can encode associations such as Michael Jackson was a singer, or Michael Jordan played basketball. However, the leap from storing patterns to performing reasoning, chaining facts, evaluating alternatives, or inferring unseen relationships, is far less transparent.

Yet we still lack a unified theory explaining how models generalize reasoning beyond pattern completion. Even the most advanced LLMs like GPT-5.4, Claude Opus 4.6, Gemini Thinking, or DeepSeek demonstrate reasoning-like behavior that can be measured but not fully interpreted.

Interestingly, this opacity mirrors our understanding of the human brain itself. Despite decades of neuroscience, we still do not fully know why certain neurons fire and others remain dormant, how a specific thought or intuition arises, or what biological representations correspond to reasoning and creativity. Both human and machine intelligence reveal patterns we can observe but not completely explain.

In that sense, the mystery of reasoning is shared between the systems we build and the minds that build them. Understanding this phenomenon, whether it emerges from neurons or neural networks, remains one of the most profound scientific and philosophical challenges ahead.

Summary

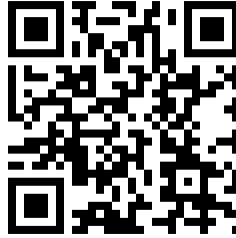
In this chapter, we explored more advanced language capabilities that modern LLMs enable. We began with text generation, examining how models can produce coherent, high-quality content, and then moved to question answering, where models interpret and respond to queries based on context or learned knowledge. Finally, we discussed reasoning, where models go beyond pattern matching to perform multi-step thinking, combine information, and solve more complex problems.

Together, these capabilities significantly expand what AI systems can achieve today. They make it possible to build more sophisticated applications and intelligent agents that can generate content, answer questions, and make informed decisions in dynamic environments. Modern AI agents often combine generation, question answering, and reasoning into coordinated workflows to accomplish complex tasks.

At the same time, these capabilities are still evolving. While today's models demonstrate impressive performance, they can struggle with deep reasoning, ambiguous context, and long, complex inputs. As research continues, we can expect further improvements in reasoning reliability, better integration with external tools and data, and more robust agent-based systems. These developments will play a key role in shaping the next generation of AI applications explored in the following chapters.

Get this book's PDF version and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.

6

Retrieval-Augmented Generation

At this point in our journey, we have explored large language models in depth, both in terms of how they work and how they can be applied to a wide range of practical tasks. Up to now, LLMs have served as the core engine behind every solution we have built.

But in the real world, solving advanced problems often requires more than just a powerful model. We need to bring in additional tools, external knowledge, and domain-specific data to make our AI systems truly useful and grounded in context. This is where **retrieval-augmented generation (RAG)** comes into play.

As we learned in *Chapter 2*, LLMs are trained on large, public, general-purpose datasets available up to a specific cutoff date. This cutoff often lags real time by a year or more. In addition, these datasets typically do not include private, proprietary, or newly published information.

This creates a critical limitation: LLMs alone cannot access up-to-date, rare, or restricted knowledge. Their factual accuracy is bound by their training data. RAG addresses this limitation directly by retrieving relevant external information and feeding that content into the LLM as part of its input prompt. The model can then generate answers that are both context-aware and grounded in retrieved facts.

In this chapter, we'll discover how RAG works, when to use it, how it differs from traditional search engines, and how it helps bridge the gap between static model knowledge and dynamic, real-world information. Specifically, we will cover the following key topics:

- RAG components
- RAG implementations

Technical requirements

The code for this chapter is available on the book's accompanying GitHub repo: <https://github.com/PacktPublishing/Building-Agent-Powered-Applications/tree/main/Chapter%206>

RAG components

A typical RAG system contains two primary components:

- **Retriever (R letter in RAG):** Responsible for information retrieval
- **Generation (G letter in RAG):** Responsible for generating the response

The following diagram illustrates the high-level architecture of a typical RAG system, showing the interaction between generator, retriever, and data source components:

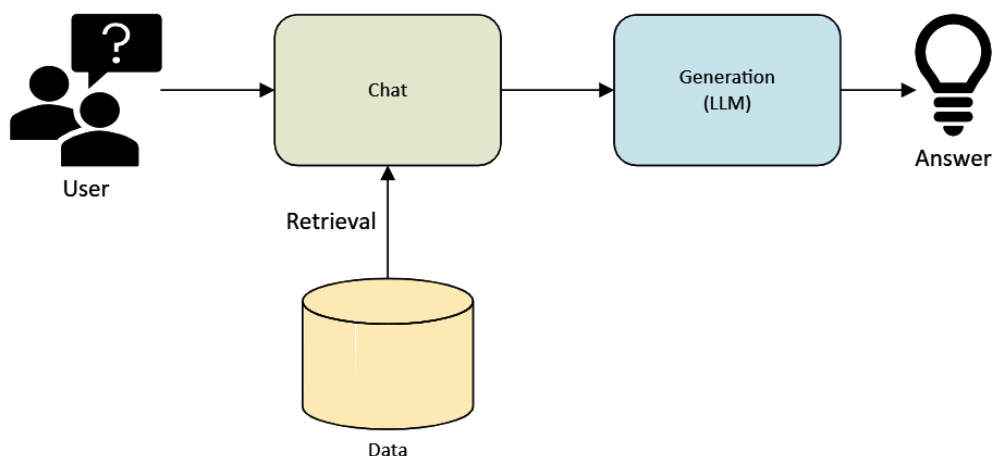


Figure 6.1: Retrieval and generation in RAG

In RAG, the retrieval component is responsible for searching and delivering relevant information for questions asked by users, while the generation component takes the found information and generates a coherent, context-aware final response.

A helpful analogy is the process of writing an essay before the era of the internet. Imagine a student who got an assignment for a specific topic. First, the student visits the library to search for and gather relevant books and resources. This mirrors the retrieval step. Then, after reviewing and synthesizing the material, the student composes an essay in their own words, drawing on what they've learned. That's the generation step.

Just like a student relies on trusted sources to write an informed paper, a RAG system combines the strength of retrieval (finding useful information) with the creativity and fluency of a language model (generating the final output). This division of labor makes RAG both more accurate and more adaptable to real-world use cases where up-to-date or proprietary knowledge is essential.

In other words, RAG provides *grounded* data to LLM to make an inference. So, we extend or *augment* (the letter "A" in RAG) the existing LLM knowledge with new information to apply reasoning on it. As a result, RAG allows LLMs to tackle more targeted, real-world problems that require up-to-date or domain-specific knowledge.

The retriever component can vary in design depending on the complexity and nature of the data. There are several types of retrieval today:

- Static retrieval: Predefined content is retrieved via APIs or fixed logic
- Lexical retrieval: Based on keyword matching, similar to traditional search engines
- Semantic retrieval: Uses embeddings to match meaning, not just words
- Hybrid retrieval: A combination of the above techniques

In the next section, we will look more closely at how these retrieval approaches are designed and implemented in practice.

Static retrieval and parser

Static retriever is the simplest form of RAG retrieval. It fetches predefined data and injects it into the LLM's prompt (context). Such data might come from files, databases, external APIs, system metadata, or other sources. Often, it's hardcoded for specific needs, such as returning the current date, local weather, or stock prices for a given company.

For example, imagine a personal assistant that needs to generate a daily summary for a user. To do this, the assistant must know the current date and time. LLMs don't inherently know this information. In this case, the retrieval step can be as simple as calling a function that returns the current date and time, which is then injected into the prompt for the LLM. Similarly, any other info can be injected into the LLM context.

So, while static retrievers are easy and straightforward to implement, they come with practical limitations:

- **Context window constraints:** Retrieving large documents can exceed the LLM's effective context window, leading to degraded performance or failure to process the input
- **Scaling complexity:** When the application requires multiple data types, like product details, discounts, and customer reviews, scaling a static retriever becomes cumbersome

In short, while static retrieval is simple and effective for small, targeted use cases, it doesn't scale well as the volume, variety, or complexity of required data grow.

Note that the basic static retrieval may or may not require additional parameters to be called. For example, retrieving the current date or system time doesn't require any parameters. The retriever simply returns the value. In contrast, fetching the weather forecast does require parameters like location and date to return the relevant information.

In general, RAG is not directly responsible for extracting retrieval parameters from user questions. These should be obtained by other components that come earlier in the RAG pipeline. However, for the sake of completeness in this section, we'll introduce an additional component called the **parser**, which is responsible for extracting the necessary parameters for retrieval.

For example, consider a weather forecast chatbot where users ask about the weather for specific dates and locations. The chatbot retrieves the relevant forecast data and responds with accurate, up-to-date information. Take the following questions:

1. "What will be the temperature in Boston next Friday?"
2. "Will it rain in Paris on 26 Feb?"

In these cases, the parser needs to extract the following structured information:

1. Region = Boston, USA, date = 01/20/2025 (assuming today is Wednesday, 01/17/2025)
2. Region = Paris, France, date = 02/26/2025 (assuming the current year is 2025)

As you can see, such a parser acts as an information extraction component we explored in *Chapter 4*. Often, the parameters are named entities such as dates, locations, numbers, or times. In other cases, they might include custom entities like product IDs, client names, or domain-specific tags. The extracted parameters are then passed to the retriever to obtain the relevant content.

Generator

The *generator* in RAG is responsible for the *generation stage*. It takes the user's question along with the retrieved contextual information (from retrieval) and uses a language model to produce the final response. In its simplest form, the generator is a prompt that embeds the relevant facts and instructs the model to generate an answer grounded in that data.

For example, imagine a weather chatbot. The system prompt for it might look like this:

Prompt

```
You are a weather chatbot that retrieves weather forecasts.  
Don't generate predictions; present data found between <weather>...</weather> tags.
```

Now consider the following user interaction:

```
User question: What will the weather be like in Toronto on May 22?  
Context: <weather>{"region":"Toronto","date":"2025-05-22","temp":"23","unit":"C"}  
</weather>  
Response: The forecast for Toronto on May 22 will be 23 °C.
```

In this example, the chatbot first extracts key details from the user's query (location and date), queries a weather service to get the requested weather information, and then injects the retrieved result into the prompt. The generator then uses this combined input to produce a relevant, accurate, and grounded response.

The example above demonstrates that the generator can be implemented with relatively straightforward instructions. But, in other scenarios, the prompt design can be more complex. In such cases, advanced prompting techniques, such as few-shot examples, CoT, ToT and others, may need to be applied.

Let's consider the second example: a company wants to build an internal AI assistant that answers employee questions about company policies, such as parental leave, travel reimbursement, or remote work rules. The policy documents are long, vary in format, and occasionally can contain outdated or conflicting information.

To ensure accurate and reliable answers, the system must go beyond simply retrieving relevant sections. It needs to summarize complex content, resolve any inconsistencies, and clearly cite sources. This is where prompt engineering plays an important role: it helps structure the

model's behavior and constrain the output. A well-crafted system prompt for this task might look like the following:

Prompt

```
You are an internal company assistant that answers employee questions based ONLY
on retrieved policy documents provided between <doc>...</doc> tags.
- Summarize clearly if the documents are long.
- If multiple documents provide conflicting information, mention the discrepancy.
- Cite the policy title and section in your answer using [doc:PolicyName,
SectionName].
- If no relevant information is found, say "I couldn't find that in the current
policy documents."
- Do not make up or guess information.
```

With a prompt like this, a user might ask a question such as "What's the travel reimbursement process for international conferences?" and together with context obtained from the retriever, we can have the following user prompt:

Context:

```
<doc>
Policy: Travel Policy 2022
Section: International Travel
Text: Employees must obtain written manager approval before booking.
Reimbursements require submission of original receipts within 10 business days.
Policy: Travel Policy 2021
Section: Travel Guidelines
Text: For international trips, reimbursement forms must be submitted within 5
business days with a scanned copy of receipts.
Policy: Expense FAQ
Section: Conferences
Text: Reimbursement is handled through the SAP system. Use the "Conference
Travel" category when submitting.
</doc>

User question: What's the travel reimbursement process for international
conferences?
```

This example illustrates why prompt engineering is important in RAG setups. A well-crafted prompt helps guide the model in the following key areas:

- **Grounding and hallucination control:** The prompt explicitly instructs the model not to invent answers and to stay within the retrieved text
- **Summarization and disambiguation:** The generator must condense multiple documents and highlight contradictions
- **Fallback strategy:** The system knows what to say when information is missing

Looking at the previous two examples, we can also observe that the generator effectively serves as a **question answering (QA)** component for *closed-domain* problems (see *Chapter 5*). So, the retrieved content from the retriever defines the boundaries of the domain, and the generator's role is to formulate an answer based solely on that context.

As we discussed in *Chapter 4*, exploring QA for closed-domain tasks, one of the key challenges for LLMs is generating *grounded* responses. This means the answers should be strictly based on the provided context, without hallucinating or introducing facts from the model's internal knowledge. Even when relevant information is retrieved and included in the prompt, the model may still incorporate plausible but incorrect answers. This makes prompt design and context control essential when using LLMs in closed-domain RAG setups.

Lexical retrieval

Static retrievals are well-suited for straightforward, predefined scenarios. However, in many real-world applications, we need more flexible and intelligent ways to retrieve information. To illustrate when a more advanced retrieval approach is necessary, let's walk through the following example:

Email chatbot: Imagine we need to develop a chatbot that will need to answer questions based on previous email conversations. For example, a user wants to know when some topic was discussed and what the resolution was. That chatbot needs to operate on a large number of emails (thousands or even millions) and find the relevant information to answer the user's questions. So, such a chatbot can be implemented as follows:

- Extract key words from user questions – **parser**
- Use keywords to find the relevant email(s) – **retriever**
- Generate the answer using relevant email(s) retrieved by retriever – **generator**

The following diagram illustrates this email chatbot workflow, showing how a user query is processed through keyword extraction, retrieval of relevant emails, and final answer generation:

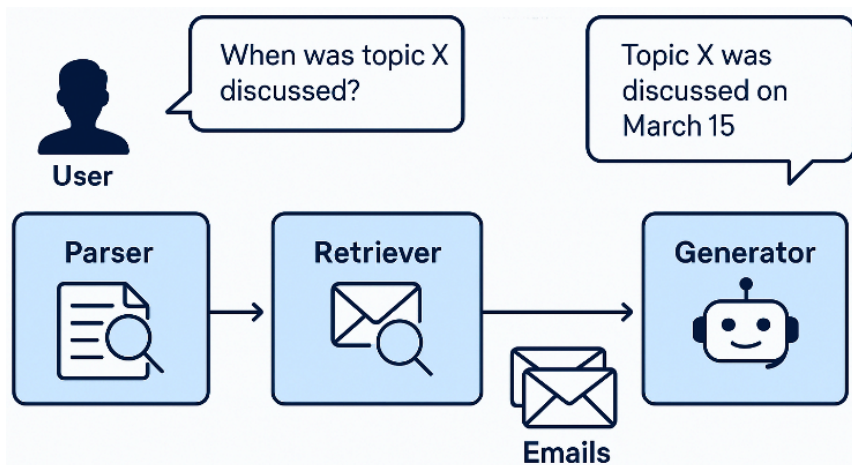


Figure 6.2: Email chatbot with lexical retrieval

In this case, the retriever needs to be able to search for relevant emails by keywords. Such retrievers are called *lexical* retrievers because they make *lexical* (or *keyword-based*) searches. The retriever matches user queries with stored text using exact words (tokens) or phrases. This approach relies on traditional, well-developed keyword-based search.

Lexical retrieval can be implemented in the following way:

- **User query processing:** The retriever tokenizes the user's query into individual words and performs stemming or lemmatization (see *Chapter 1*).
- **Keyword matching:** Retriever searches for document(s) that contain the exact words from the query.
- **Ranking by frequency and relevance:** Documents are ranked based on the frequency of keywords presented in the documents. Ranking means we assign some number (score) to a document and sort by the scores.
- **Context injection:** The highest-ranked document(s) are passed as the context to the LLM.
- **Response generation:** LLM generates an answer using the retrieved context.

The first two steps identify candidate documents based on keyword matching. The 4th and 5th steps form context for the generator. Since multiple documents may match, the 3rd step becomes essential. Let's examine it in more detail.

There are several well-established approaches to rank relevant documents that rely on how frequently keywords (terms) appear in documents and how relevant they are. We will consider several of the most commonly used techniques:

- **Term frequency and inverse document frequency or TF-IDF**
- **BM25**

Note

Throughout this chapter, we will use the word "term" as a synonym for "keyword". In the field of information retrieval, a term is the more commonly used and formally accepted term to describe a unit of search. It is typically a word or phrase that appears in documents or user queries. While keyword is a widely understood term, it is less preferred in technical literature and retrieval algorithms.

Term frequency and inverse document frequency

Term frequency – inverse document frequency (TF-IDF) is commonly used in RAG and across the broader field of *information retrieval*. TF-IDF measures how important a word is within a specific document. It also considers how common that word is across the entire document collection (corpus). This helps highlight keywords that are more meaningful and relevant.

TF-IDF consists of two main components: term frequency and inverse document frequency.

Term frequency (TF) measures word importance in a document; TF calculates how often a word appears in a document compared to the total words in that document. TF is calculated by the formula:

$$TF(t, d) = \frac{f_t}{N_d}$$

where:

- f_t - Number of times term t (word) appears in document d
- N_d - Total number of words in document d

For example, in a document with 100 words, if the word "invoice" appears 5 times, then

$$TF("invoice") = \frac{5}{100} = 0.05$$

TF ranges from 0 to 1. Higher TF means the word is more frequent in the document and thus considered more important in that document. When $TF = 0$, then the word is not present in the document, and 1 when the entire document has only that word. Typically, TF doesn't count stop words.

Inverse document frequency (IDF) measures word uniqueness in all documents (corpus). IDF calculates how unique or rare a word is across all documents in the dataset by the following formula:

$$IDF(t) = \log\left(\frac{N}{df_t}\right)$$

where:

- N - Total number of documents in the corpus
- df_t - Number of documents containing term t

Example: If the corpus has 10,000 documents and the word "invoice" appears in 1,000 documents, then

$$IDF("invoice") = \log\left(\frac{10000}{1000}\right) = \log(10) = 1.0$$

If the word "customer" appears in all 10,000 documents, its IDF is:

$$IDF("customer") = \log\left(\frac{10000}{10000}\right) = \log(1) = 0$$

As we can conclude from the formula above, common words (e.g., "customer") have low IDF, and rare words have high IDF. That is exactly what IDF calculates: how rare or common the word is.

So, now combine TF and IDF into one final TF-IDF formula:

$$TF_IDF(t, d) = TF(t, d) \times IDF(t)$$

To illustrate TF-IDF in practice, let's consider a corpus with 10,000 documents where each document has 100 words. Let's also have the following:

- The word **"customer"** appears in all 10,000 documents and 10 times in one selected document, then $IDF = 0$ and $TF = 0.1$ ($10/100$)
- The word **"invoice"** appears in approximately 3,700 documents and 5 times in the selected document, then $IDF = 1.0$ and $TF = 0.05$
- The word **"payment"** is found in 820 documents and 2 times in the document, then IDF is 2.5 and $TF = 0.02$

So, for this example, we will have the following values:

Term (word)	TF	IDF	TF-IDF score
Invoice	0.05	1.0	0.05
Customer	0.1	0	0
Payment	0.02	2.5	0.05

Table 6.1: TF-IDF scores for a sample document corpus

This example demonstrates how TF-IDF can equalize the influence of terms that are either frequent but common or rare but informative, depending on their combined relevance.

Here is an example of how to implement TF-IDF by using the **Scikit-learn (sklearn)** library in Python:

```
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd

# Step 1: Collect Documents (Corpus)
corpus = [
    "Invoice payment is pending",
    "Customer made a payment",
    "Invoice was processed for customer",
    "Payment is completed",
    "Customer has pending invoices"
]

# Step 2-5: Preprocessing + TF-IDF Vectorization
vectorizer = TfidfVectorizer(lowercase=True, stop_words="english")
```

```
tfidf_matrix = vectorizer.fit_transform(corpus)

# Get feature names (words)
feature_names = vectorizer.get_feature_names_out()

# Convert TF-IDF matrix to DataFrame for readability
df_tfidf = pd.DataFrame(
    tfidf_matrix.toarray(), columns=feature_names,
    index=[f"Doc {i+1}" for i in range(len(corpus))]
)

# Display the TF-IDF table
print(df_tfidf)
```

The code produces the following output:

	completed	customer	invoice	invoices	payment	pending	processed
Doc 1	0.000000	0.000000	0.609818	0.000000	0.506204	0.609818	0.000000
Doc 2	0.000000	0.707107	0.000000	0.000000	0.707107	0.000000	0.000000
Doc 3	0.000000	0.462208	0.556816	0.000000	0.000000	0.000000	0.690159
Doc 4	0.830881	0.000000	0.000000	0.000000	0.556451	0.000000	0.000000
Doc 5	0.000000	0.462208	0.000000	0.690159	0.000000	0.556816	0.000000

This is the TF-IDF matrix for a small set of example documents (a corpus of 5 sentences). It uses Scikit-learn's `TfidfVectorizer` to automatically preprocess the text (lowercasing and removing English stop-words), convert each document into a vector of a TF-IDF matrix, and then stores the result in a readable pandas DataFrame. Each row represents a document, and each column corresponds to a word from the vocabulary. It shows how important each word is in that specific document relative to the entire corpus.

Now we can summarize the key strengths and limitations of using TF-IDF in practical applications. We will look at the pros first:

- **Efficient for small to medium-sized datasets:** It's relatively fast and computationally inexpensive compared to deep learning-based approaches (see below)
- **No training required:** TF-IDF is a deterministic algorithm, meaning it does not need labeled data or model training

Next, we will consider the cons:

- **Cannot recognize synonyms:** Words like "car" and "automobile" are treated as completely different terms.
- **Lacks semantic understanding:** It does not distinguish context, so it cannot differentiate between meanings in phrases like "river bank" vs. "finance bank".
- **Struggles with long queries:** It may retrieve too many relevant documents, making it harder to rank them effectively.
- **Scaling issues for large corpus:** With a growing number of documents and vocabulary size, TF-IDF can become not efficient for both CPU and memory.

While TF-IDF remains a simple and effective baseline, its limitations motivate the need for more advanced retrieval approaches, which we will explore further.

BM25

While TF-IDF offers a solid foundation for ranking documents based on keyword relevance, it also comes with certain limitations, as we've mentioned above. To overcome some of these challenges, more advanced ranking algorithms have been developed.

One of the most widely adopted alternatives is **best matching 25 (BM25)**. It belongs to a broader family of **best matching (BM)** algorithms, with earlier versions like BM11 and BM15. The number 25 simply denotes a specific version in this series. Among them, BM25 has become the de facto standard for lexical retrieval due to its balance of simplicity, tunability, and effectiveness. It is commonly used in search engines, chatbots, and RAG.

BM25 shares conceptual roots with TF-IDF but introduces a few key improvements:

- **Term frequency saturation:** It avoids overemphasizing words that appear too frequently in a document. TF-IDF assumes that more occurrences = higher relevance, which is not always true. BM25 dampens the effect of high word frequency, preventing long documents from dominating.
- **Document length normalization:** It adjusts for document length bias, preventing longer documents from scoring artificially higher. Longer documents naturally have more words, leading to inflated scores in TF-IDF. BM25 normalizes for length to ensure fair ranking.

For the purposes of this book, we won't delve into the implementation details of BM25. It is similar to TF-IDF but with some more advanced optimizations and additions, as we mentioned above. Instead, we encourage you to explore BM25 further if you're interested. Here, we'll demonstrate how BM25 can be used in practice, showing a simple implementation in Python:

```
from rank_bm25 import BM25Okapi

documents = [
    "Invoice payment is pending",
    "Customer made a payment",
    "Invoice was processed for customer",
    "Payment is completed successfully",
    "Customer has pending invoices"
]

# Tokenize documents
tokenized_docs = [doc.lower().split() for doc in documents]

# Initialize BM25 model
bm25 = BM25Okapi(tokenized_docs)

# Query example
query = "pending invoice"
tokenized_query = query.lower().split()

# Get BM25 scores
scores = bm25.get_scores(tokenized_query)

# Rank documents by score
ranked_docs = sorted(zip(scores, documents), reverse=True)

# Display results
for score, doc in ranked_docs:
    print(f"Score: {score:.4f} - Document: {doc}")
```

This code demonstrates how to use BM25 to retrieve relevant documents. It starts by defining a small corpus of documents and tokenizing them into individual words. Then, it initializes the BM25 model with these tokenized documents. When a query is provided (e.g., "pending invoice"), it calculates BM25 relevance scores (BM25 matrix) for each document and ranks them from most to least relevant. Finally, it prints the ranked documents along with their scores.

Inverted indexes

As we discovered above, both the most popular lexical retrievals, TF-IDF and BM25, require computational operations to perform search. Such search operations can be significantly optimized by using inverted indexes.

An **inverted index** is a data structure used to quickly find documents containing a given word. Instead of scanning all documents, an inverted index directly maps words to the documents they appear in, making retrieval extremely fast. For example, we have the following documents:

- 1. **D1:** "The customer paid the invoice"
- 2. **D2:** "Payment is due for the customer's invoice"
- 3. **D3:** "The invoice is overdue"

The inverted index will take all unique words from the documents and map them to document addresses where they are present:

Word	Documents (posting list)
Customer	D1, D2
Pay	D1
Invoice	D1, D2, D3
Payment	D2
Overdue	D3

Table 6.2: An inverted index for a sample document corpus

Typically, inverted indexes are implemented as hash tables (maps, dictionaries) and therefore getting the documents where the word is present takes constant time ($O(1)$).

Practical implementations of TF-IDF and BM25 often rely on inverted indexes to achieve efficient performance. As a result, inverted index structures are widely used in many systems and databases, including Elasticsearch, Solr, PostgreSQL Full-Text Search, and even large-scale engines like Google Search.

These techniques form the foundation of lexical retrievals, but as we saw earlier, they rely heavily on exact word matching. In the next section, we will explore semantic retrieval approaches that go beyond keywords and capture meaning.

Semantic retrieval

As we noticed from the section above, lexical retrievals (BM25, TF-IDF) are quite efficient and can process a large number of documents, especially with optimizations like inverted indexes. But at the same time, they perform keyword matches that don't consider *semantic meanings*. That means they may miss some important documents to filter or find documents with high keyword frequency, but not so relevant to the search.

For example, imagine a user searching for: "*How to reduce expenses in a small business?*" A lexical retriever will look for documents containing exact terms like "*reduce*", "*expenses*", and "*small business*". It might miss a document titled "*Cost-cutting strategies for startups*", which uses synonyms or close terms like "*cost-cutting*" instead of "*reduce expenses*" and "*startups*" instead of "*small business*".

In contrast, semantic retrieval uses embeddings (see *Chapter 1*) to identify the most relevant documents based on their meaning rather than exact wording. These embeddings capture both semantic relationships and contextual nuances, allowing the retriever to match conceptually similar content.

Basically, semantic retrievals work in this way:

- Compute embedding vector for every document in the corpus, i.e., document reprocessing
- Compute embedding vector for user query (question), which is the query embedding
- Compute the similarity metric between the query embedding and each document embedding
- Find the top-k document embeddings that are closest to the query embedding

The following diagram visualizes the semantic retrieval workflow, where both documents, and the user query are transformed into embeddings and matched based on similarity:

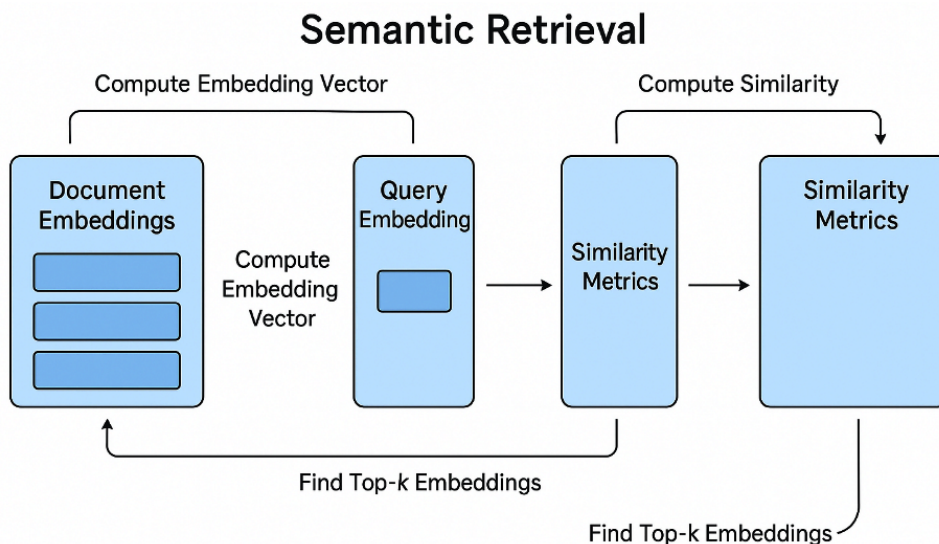


Figure 6.3: Semantic retrieval

As we can see, there are several tasks we need to solve in order to implement semantic retrieval:

- Preprocess documents, extracting clean text, removing formatting, markup, and non-text elements
- Decide how to segment the text for embedding, e.g., chunking long documents, merging short ones, or truncating where appropriate
- Select a model or algorithm to compute embeddings
- Choose a similarity metric to compare query and document embeddings
- Apply a ranking strategy to identify and return the most relevant documents

Let's look at these tasks in more detail.

Document preprocessing

Just like in lexical retrieval, semantic retrieval also requires pre-processing the documents first. Often, the raw content comes in structured formats such as HTML, XML, or JSON, from which we need to extract plain text. Additionally, documents may include non-text elements like tables, charts, or images. In such cases, we must convert these multi-modal documents into a text-only format, much like we did for multi-modal inputs in summarization tasks (see *Chapter 4*).

Select document length

Since the documents can have different structures, we may need to develop a strategy to chunk, merge, or truncate documents. That is because:

- Sentence-level embedding models (see *Chapter 1*), similar to LLMs, can process text up to some maximum limits
- Long documents can dilute relevance
- Small documents like short chat conversations can also affect embedding quality and performance

Embedding models work best with relatively short inputs, typically a few sentences or paragraphs. To understand why, consider a text that has 10,000 words (tokens). If we encode this entire text into a fixed-size embedding vector with, let's say, 1,000 dimensions, we're roughly compressing 10 words into a single number. This extreme compression makes the embedding non-reversible and likely to lose important contextual details, especially for long documents. The longer the input, the greater the risk of information loss.

At the same time, very short texts, such as single words or brief utterances, can also be challenging for semantic retrieval. With minimal context, the model has little semantic information to work with, which can lead to ambiguous or poorly differentiated embeddings. For example, the word "bank" on its own could refer to a financial institution or a riverbank, but without surrounding words, the embedding model can't disambiguate the meaning.

So, to achieve optimal embedding quality, it's important to adapt the input length (hyperparameter for RAG) through strategies like *chunking*, *merging*, or *truncation*, depending on the nature of the documents and the retrieval task. These strategies should be task-specific and reflect the expected query types.

For example, if users tend to ask high-level questions (e.g., "*What is the main idea of this document?*"), it may be appropriate to truncate the content or summarize it before embedding. On the other hand, if queries are likely to target specific facts or details (e.g., "*What date was the invoice sent?*"), a better approach is to chunk the document into smaller, semantically meaningful sections, such as paragraphs or sections, then embed each chunk separately.

So, as a result, the strategy to select the document length for embeddings directly depends on the type of QA (see *Chapter 4*) problem you're solving, and more broadly, the retrieval task and user intent. Different QA scenarios require different document preprocessing and embedding strategies to optimize both retrieval accuracy and efficiency.

Here we will briefly highlight a few types of QA problems and the proper document length strategy:

QA type	Query example	Recommended strategy
Open-ended/high-level	"What is this document about?"	Truncate or summarize the key sections; embed the whole chunk.
Fact-based/extractive	"When was the invoice sent?"	Chunk the document into paragraphs or semantic blocks, embed each one separately and retrieve the most relevant chunk.
Short-form queries	"Due date?" or "Client name?"	Merge short queries with additional context (e.g., surrounding metadata or headings) to improve embedding quality.
Multi-turn/conversational QA	"What did they agree on in the previous meeting?"	Chunk and track dialog turns or message sequences with context windows for embedding (similarly to what we did for summarization; see <i>Chapter 4</i>).

Table 6.3: Recommended document length strategies for different types of QA tasks

The challenge arises when we need to develop a general-purpose RAG system that must handle a wide variety of question types. In such cases, it's difficult to apply a one-size-fits-all strategy for document chunking, merging, or truncating. Some questions may require a deep understanding of long-form content, while others may target specific facts buried in small sections.

This variability makes it crucial to design adaptive preprocessing and retrieval strategies, potentially using metadata, query classification, or even dynamic chunking based on the question's intent. Building such flexible systems often involves trade-offs between accuracy and coverage across different question types, and may require additional logic such as reranking, hybrid search, or other post-retrieval techniques to maintain consistent performance.

Embedding models

The next important question is which embedding model to use for computing embeddings. Sentence-level (or text-level) embeddings are the most frequent choice for semantic RAGs.

Here is the list of the most popular sentence-level embeddings for RAG used today:

Model	Best for	Parameters	Vector size	License	Model type
Sentence-BERT (SBERT)	General-purpose retrieval	~110M to ~350M	384–1024	Apache 2.0	Open-source, Transformer-based
OpenAI embeddings (text-embedding-ada-002)	General-purpose retrieval	Undisclosed	1536	Proprietary	Cloud-based, Transformer-based
E5 (Microsoft)	Large-scale retrieval	~200M–800M	768	MIT	Open-source, Transformer-based

Table 6.4: Popular sentence-level embedding models for semantic retrieval in RAG systems

From the preceding table, we can see that SBERT and E5 are open-source and can be hosted locally or in a cloud-managed environment. OpenAI embeddings are available from API access and are proprietary and don't require a host or management. When using cloud API-based models, we also must understand that data will be sent into the model owners' infrastructure. That can cause privacy and data boundary issues.

Choose a similarity metric

The next step is to choose a similarity metric to measure how close the query embedding is to each document embedding. As discussed in *Chapter 1*, cosine similarity is one of the most widely used options. It's particularly effective when working with normalized embeddings, as it measures the angle between vectors rather than their magnitude. To use cosine similarity effectively, you should either normalize the embeddings manually or use models that produce pre-normalized vectors, such as OpenAI's text-embedding models.

Find the most relevant documents

Once we have computed the similarity between the query embedding and all document embeddings, the next step is to identify the most relevant documents. This typically involves ranking the documents based on their similarity scores and selecting either the top-k most similar ones or documents that exceed a certain similarity threshold.

In small datasets, ranking can be performed using a simple linear scan. For example, suppose we have 5 documents and calculate the cosine similarity between the user query and each document, resulting in the following scores: [0.3, 0.01, 0.65, 0.85, 0.07].. Sorting these scores in descending order gives us [0.85, 0.65, 0.3, 0.07, 0.01]. If we want to retrieve the top 2 most relevant documents, we will select those with similarity scores of 0.85 and 0.65, which correspond to the 4th and 3rd documents in the original list.

However, as the number of documents grows into thousands or millions, this approach becomes computationally expensive. Sorting scores will require $O(N * \log N)$ time complexity. For example, if we have one million documents, we will need to perform around 20 million operations ($\log_2(1,000,000) \approx 20$)..

The algorithm to find the top-k closest items we described above is called **nearest neighbor search (NNS)**. It has two primary versions:

- Exact NNS
- **Approximate NNS (ANNS)**

Exact NNS guarantees finding the true top-k nearest neighbors based on a chosen similarity or distance metric. As shown earlier, this can be implemented by computing similarity scores for all documents and then sorting the results, which takes $O(N * \log(N))$ time. However, more optimized versions of exact NNS avoid full sorting by using either:

- **Quickselect**: An efficient selection algorithm that finds top k most similar items in average $O(N)$ time. It's faster than sorting but has a worst-case time complexity of $O(N^2)$.
- **Min heap**: Builds a heap of size k, maintaining the top k results. This approach runs in $O(N * \log k)$ time and is commonly used in practice due to its predictable performance and simplicity.

So, as we can see, exact k-NN is simple to implement and gives the most accurate list of closest embeddings. But for very large datasets and high-dimensional embeddings, it can still be relatively slow.

One important point to emphasize is that the exact NNS requires computing the distance or similarity metric (e.g., cosine similarity) between the user's query embedding and every

document embedding. That is required for every new search request and can be a very intense computing operation.

For instance, with 1 million normalized document embeddings, each of 1,000 dimensions, you'd need to perform 1 million dot products for every query. Each dot product involves roughly 1,000 multiplications and additions, totaling about 1 billion operations, and that's before you even sort or pick the top-k.

As a result, the exact NNS method becomes impractical at scale, especially when you need to handle frequent queries. To address this scalability challenge, many systems use more advanced NNS optimizations, such as *ANNS*.

Approximate nearest neighbor (ANN)

As we said above, for large-scale datasets, **approximate nearest neighbor (ANN)** algorithms are often used instead of exact NNS to dramatically improve efficiency at the cost of a small trade-off in accuracy. Unlike exact NNS, which guarantees the top-k nearest neighbors, ANN algorithms aim to return near-optimal results much faster.

One of the key benefits of ANN is that it avoids computing the similarity metric between the user query and every single document embedding. Instead, it leverages specialized indexing structures to narrow down the search space and quickly identify the most likely candidates. This not only cuts down computational costs but also speeds up the final top-k selection process.

That said, because ANN does not exhaustively compare the query to all documents, it can return sub-optimal results. This means:

- It may miss some true nearest neighbors (false negatives)
- Or, it may include less relevant neighbors (false positives)

Let's illustrate this with a simple example. Suppose we have the following seven distances between the user query and seven documents: [1, 4, 2, 7, 5, 3, 6]. We want to find the top-3 nearest neighbors. An exact search would return [1, 2, 3] — the smallest distances. An ANN method, however, might return [1, 2, 4], missing the actual third-nearest neighbor [3] and mistakenly including a farther one [4].

ANN is much faster than exact NNS due to using various indexing techniques to accelerate search. These methods vary depending on the approach:

- **Tree-based**, such as *Annoy*
- **Graph-based**, such as **hierarchical navigable small world (HNSW)**

- **Quantization-based**, such as FAISS using **inverted file index (IVF)** and **product quantization (PQ)**
- **Hashing-based**, such as **locality-sensitive hashing (LSH)**

Let's dive into these algorithms in more detail to understand how they work and when to use each one.

Tree-based methods

Tree-based methods are one of the simplest approaches for efficient NNS. These methods organize embeddings in a hierarchical tree structure, allowing for fast lookup compared to exact NNS search. They often can give up to $O(\log N)$ time complexity and thus are significantly faster than $O(N)$ with quick select.

Tree-based methods split embeddings by dimensions, building a tree structure. In this way, they achieve $O(\log N)$ performance. But tree-based algorithms depend on the embedding space size. In practice, they start struggling with more than 50 dimensions.

Here are a few of the most popular tree-based algorithm implementations:

Method	Best for	Speed	Accuracy	Scalability
K-d tree	Small datasets (<100K), low dimensions (<50D)	Fast	Exact	Limited
Ball tree	Medium datasets (~1M), moderate dimensions (50D–100D)	Fast	Exact	Limited
Annoy	Large datasets (~1M), approximate k-NN	Faster than k-tree	Approximate	Scales well

Table 6.5: Popular tree-based algorithms

Today, *Annoy* is considered the most popular tree-based method, but it becomes less efficient with growing embedding dimension space. Typically, 100-300 dimensions are considered the maximum practically feasible for such algorithms. Since for most modern RAGs we typically need large-dimensional embeddings (768+, 1024+), tree-based methods lose their importance.

Graph-based methods

Graph-based methods organize high-dimensional vectors into a navigable graph structure, enabling efficient nearest neighbor search. These methods are widely used in large-scale

retrieval systems, including RAG. Graph-based methods have a similar computational complexity to tree-based methods $O(\log N)$ but handle high-dimensional vectors much more efficiently. Graph methods can also handle very large datasets, but often require more memory.

There are several methods for graph-based algorithms:

Method	Description	Best for
Hierarchical navigable small world (HNSW)	Connects points in a multi-layer graph for fast lookup	Large datasets (~10M vectors)
Navigable small world (NSW)	Simple proximity-based graph search	Medium datasets (~1M vectors)
K-NN graphs (KGraph)	Precomputes k-NN graph for efficient lookup	Small-to-medium datasets (~100K vectors)
Disk-based ANN (DiskANN)	Graph-based ANN optimized for disk storage	Billion-scale retrieval

Table 6.6: Graph-based algorithms

Here is a detailed explanation of each:

- **HNSW** is the most popular graph-based ANN algorithm because of its speed and high accuracy (~99%), meaning it produces a small number of false positive results. It builds a multi-layer graph with $O(N * \log N)$ indexing speed and needs $O(\log N)$ compute time to search.
- **NSW** is a simpler version of HNSW. It builds a single-layer graph with no hierarchy and links points based on proximity, forming a small-world network. Because of that, it is slower than HNSW but consumes less memory. It needs $O(N * \log N)$ for indexing and $O(\sqrt{N})$ search time. It is good for medium-sized datasets (up to about 1M vectors).
- **K-nearest neighbor graph (KGraph)** precomputes an NNS graph where each node is connected to its k nearest neighbors. It searches via graph traversal instead of brute-force comparison. KGraph is useful when queries are repetitive (e.g., caching frequently searched embeddings). It is quite slow to build the graph as it requires $O(N^2)$ compute time, but relatively fast for searches, $O(\log N)$.
- **DiskANN** is optimized for large-scale databases and stores data on disk instead of RAM, reducing memory usage. It is used in Microsoft's Bing Search engine and has built complexity $O(n * \log N)$ and search time $O(\log N)$.

Overall, graph-based methods offer an excellent balance between speed, accuracy, and scalability, making them a preferred choice for large-scale semantic retrieval systems.

Quantization-based methods

Quantization-based approach is used in large-scale nearest neighbor search to reduce memory usage and improve retrieval speed. These methods compress high-dimensional vectors into lower-dimensional representations while maintaining a high level of accuracy. There are a few methods that use a quantization-based approach:

- **Inverted file index (IVF)**
- **Product quantization (PQ)**

IVF is used to reduce the number of distance computations by clustering vectors and only searching within the closest cluster(s) instead of scanning the entire dataset. It partitions vectors into clusters by using k-means clustering, and each vector is associated with its nearest cluster center. So, in the searching phase, first we calculate in which cluster the query vector is, and next, search among the vector clusters. This algorithm has $O(N * \log N)$ time complexity for indexing and $O(\sqrt{N})$ time for search.

PQ divides each vector into parts and then associates every part with some number. For instance, we have 128D vectors and split them into 16 sub-vectors of 8D each. Next, we associate every 8D sub-vector with one number. Search happens using PQ codes, which is faster. This algorithm also has $O(N * \log N)$ time complexity to build and $O(\log N)$ time complexity to search.

So far, we've explored the most widely used ANN algorithms and developed an understanding of how they work. While we didn't dive deep into the low-level implementation details, the goal was to give you the foundational intuition needed to evaluate and choose the right approach. Now, let's shift focus and see how these algorithms are applied in real-world applications, using libraries and services that bring ANN-based retrieval into production.

RAG implementations

Till now, we've explored the foundational building blocks of RAG, including the retriever, generator, and parser components. We've also taken a deeper dive into both lexical and semantic retrieval and examined the key challenges they present. Now it's time to bring it all together and look at how to implement RAG systems end to end.

The good news is that, in many practical scenarios, you don't need to build everything from scratch. There's a growing ecosystem of robust tools, libraries, frameworks, and databases that can help you get started quickly. However, because there's no one-size-fits-all solution for every use case, it's important to understand the trade-offs and inner implementation details of

these components. That's exactly why we took the time to dive deeper into the underlying algorithms to better understand their strengths, limitations, and how to make correct choices when designing real-world RAG pipelines.

Depending on our goals, task domain, and technical constraints, we can implement the RAG system using one of the following three approaches:

- **Fully custom pipeline:** Build the entire stack by yourself, implement your own generator, parser, and retrievals
- **Modular hybrid approach:** Customize certain components like the generator and parser, while leveraging existing vector databases or retrieval frameworks for semantic or lexical retrieval
- **RAG as a service:** Use an end-to-end managed solution that abstracts away all the implementation complexity

Let's consider all these approaches in detail.

Custom pipeline

A fully custom RAG pipeline offers the highest level of flexibility and control. It gives you the ability to design and fine-tune each component to match the exact requirements of your use case.

However, this flexibility comes at a cost. You are fully responsible for designing, integrating, and maintaining the entire pipeline. That includes selecting and configuring embedding models, implementing retrieval logic, crafting prompt templates, hosting services, managing infrastructure, and ensuring scalability and reliability.

Earlier in this chapter, we explored how the generator and parser components can be implemented and highlighted common challenges like grounding, disambiguation, and context management. In the following sub-sections, we'll shift our focus to the retrieval component, which is often the most complex and performance-critical part of the RAG pipeline. We'll look at how to implement different types of retrievals, how to structure and index your data, and how to choose the right tools and strategies for your specific use case.

Lexical retriever

We begin our retrieval implementation with lexical retrieval, which includes two of the most widely used algorithms: TF-IDF and BM25. These approaches rank documents based on the presence and relative importance of terms (keywords) using straightforward statistical formulas. While conceptually simple and highly interpretable, lexical retrieval methods remain effective.

Moreover, lexical retrievals typically offer better scalability and efficiency compared to semantic ones: they run faster, require fewer computational resources, and can be deployed using lightweight infrastructure. This makes them particularly suitable for applications where scale, latency, and high output are priorities.

As we demonstrated before, TF-IDF and BM25 can be easily implemented using libraries like `sklearn` in Python. While these examples work well for prototyping or very small-scale experiments, such code typically cannot be used in production "as is." Transitioning to a real-world application requires addressing several important challenges:

- **Corpus size and memory usage:** In the basic code example, we loaded a small corpus (just a few short sentences) entirely into memory. However, in real-world production scenarios, the document corpus may contain hundreds of thousands or even millions of large documents. Loading all of them into memory at once is typically infeasible. Instead, you'll need to implement a streaming or batch processing approach to handle documents in chunks and store intermediate results in persistent storage. To improve throughput and scalability, this pipeline should also be parallelized where possible.
- **Persistent index storage:** The vectorized representation (TF-IDF matrix) and vocabulary can grow significantly as the corpus scales. For example, in a production-grade RAG system with 1 million documents and a vocabulary of 100,000 unique terms (words), the TF-IDF matrix would have dimensions of $1,000,000 \times 100,000$ — totaling 100 billion entries. Even though the matrix is usually sparse (with many zero values), storing it fully in memory is still impractical. To address this, the matrix should be serialized using efficient sparse formats (e.g., CSR or COO) and stored in persistent storage like local disk, cloud storage, or vector databases. This enables fast access, consistent query responses, and reusability without reprocessing on every startup.
- **Efficient similarity search:** Performing similarity calculations (like cosine similarity) across large sparse matrices can be compute-intensive. Efficient indexing and fast retrieval techniques (such as sparse matrix multiplication or optimized top-k selection) must be applied.
- **Preprocessing consistency:** Any text cleaning, tokenization, stop-word removal, and stemming or lemmatization applied during vectorization must also be applied to incoming user queries.
- **Updating indexes:** When new documents arrive, the entire TF-IDF matrix may need to be recreated or partially updated, which introduces complexity in dynamic and real-time systems.

As we've seen, implementing lexical retrieval in real-world RAG systems can become quite complex. While a full exploration of the technical and architectural details required for

production-grade solutions is beyond the scope of this book, we encourage readers who are interested in building such systems from scratch to dive deeper into those topics. Our goal here is to surface the most critical challenges you're likely to encounter when transitioning from a basic prototype to a scalable and maintainable retrieval pipeline.

Semantic retriever

As we discussed earlier, implementing semantic retrieval involves several key steps: preprocessing documents, selecting an appropriate embedding model, designing an effective chunking strategy (document length strategy), choosing a similarity metric, and applying a nearest neighbor search algorithm to retrieve the most relevant documents.

When building a custom semantic retrieval pipeline, you have full control over each of these components. For example, based on the structure and content of your documents, you can design a chunking strategy that preserves context and improves retrieval quality. You can also choose an embedding model that best captures the semantic meaning relevant to your domain. Finally, you have the flexibility to tune how documents are ranked and selected based on your specific performance and accuracy goals.

The challenge arises when you need to design a general-purpose solution that works across a wide variety of documents and user queries. In such cases, a one-size-fits-all chunking or embedding approach may not deliver optimal performance. You may need to implement dynamic strategies that adapt based on query intent, document type, or user context. This often introduces additional complexity, such as incorporating query classification, metadata-aware indexing, or hybrid retrieval methods that combine lexical and semantic methods.

Once we've computed embedding vectors for all document chunks, the next step is to store them in a way that allows fast and efficient retrieval. In small-scale setups, it's feasible to hold these vectors in memory using simple data structures like arrays or matrices. However, as the number of documents grows into the thousands or millions, this approach quickly becomes impractical due to memory and performance constraints. For large-scale retrieval tasks, we need specialized tools that support both storage and fast similarity search.

This is where ANN libraries come into play. While they are often referred to as ANN libraries because of their core search functionality, these tools typically offer much more. In addition to implementing fast nearest neighbor search algorithms like HNSW or IVF, they also handle essential infrastructure concerns such as efficient indexing, memory optimization, persistence, and support for batch operations. These features make them well-suited for building production-grade semantic retrieval systems.

There are several highly optimized ANN libraries that can be used in code, each with different strengths in terms of ANN method used, speed, accuracy, scalability, and memory efficiency. These libraries can be used in code to build semantic retrievers.

Below is a breakdown of the most widely used ANN libraries and the best methods they implement:

Library	Best for	ANN method	Scalability	GPU support	Language support
FAISS	Large-scale, high-speed retrieval	IVF, PQ, HNSW	Large (billion-scale)	Yes	Python, C++
HNSWlib	High recall, real-time search	HNSW (Graph-Based)	Medium (10M–100M)	No	Python, C++
Annoy	Small-scale, lightweight ANN	Random Projection Trees	Small (1M vectors)	No	Python, C++
ScaNN	AI-driven ANN for deep learning	Quantization + Graph Search	Medium (10M–100M)	Yes (Google AI)	Python, C++

Table 6.7: ANN libraries and their methods

Let's look at these libraries in more detail:

- **Facebook AI similarity search (FAISS)** is one of the most widely used ANN libraries for large-scale search. It is developed by Meta (previously Facebook). Today, it is often used for custom search engines, recommendation systems, and RAG. It supports IVF, PQ, and HNSW. FAISS is not optimized for real-time streaming updates. So, if your RAG needs to add documents constantly, and they should appear in RAG retrievals in real-time, FAISS could not be the best solution.
- **HNSWlib** is a high-performance library for ANN search based on the HNSW graph algorithm. It is designed for high accuracy (99%), real-time search, making it one of the fastest ANN libraries available. Unlike FAISS, HNSWlib is optimized for batch indexing and supports dynamic insertion and deletion. It also has some more memory optimizations for medium-sized datasets (10-100M vectors). It supports both L2 (Euclidean) and cosine similarity metrics. This library has limitations as well: it doesn't support GPU, has higher memory usage compared to others, and is not ideal for billion-scale datasets; FAISS is better for larger scales.

- **Annoy** or, **approximate nearest neighbor oh yeah**, is a lightweight, tree-based ANN library developed by Spotify. It is optimized for fast, memory-efficient ANN search, making it ideal for small-to-medium datasets (~1M vectors). It supports disk-based indexing. Index can be saved to disk and reloaded later. Spotify developed this library for music recommendations. Annoy also supports Cosine, Euclidean (L2), Manhattan (L1), and Dot similarity metrics. The downside of this library is slower performance than FAISS and HNSWlib for larger datasets (10M+ vectors), less accurate with 85-95% accuracy, no real-time updates, requiring reindexing for new data, and no GPU support.
- **Scalable nearest neighbor (ScaNN)** is an AI-optimized ANN search library developed by Google. It is designed for high-speed, high-accuracy ANN retrieval, specifically optimized for deep learning embeddings and large-scale search applications. It uses a hybrid ANN approach; it uses quantization + graph search for high recall & fast lookup, integrates well with TensorFlow, and supports GPU acceleration. The limitations of ScaNN are that it is less flexible than FAISS unable to support online updates, it has limited documentation and community support compared to FAISS and HNSWlib and is not as easy to use outside of Google's AI ecosystem.

To demonstrate how these libraries can be used in practice, let's walk through a code example that shows how to implement semantic retrieval using FAISS in Python:

```
from sentence_transformers import SentenceTransformer
import faiss

# Step 1: Sample documents
documents = [
    "The invoice has been paid.",
    "Customer requested a refund.",
    "Payment is still pending.",
    "The shipment was delivered yesterday.",
    "Refund has been processed."
]

# Step 2: Generate embeddings using a transformer model
model = SentenceTransformer('all-MiniLM-L6-v2')
doc_embeddings = model.encode(documents, convert_to_numpy=True)

# Step 3: Build FAISS index
dimension = doc_embeddings.shape[1] # e.g., 384 for MiniLM
index = faiss.IndexFlatL2(dimension) # L2 similarity (can also use cosine)
```

```
index.add(doc_embeddings)

# Step 4: Encode query
query = "Has the customer paid the invoice?"
query_embedding = model.encode([query], convert_to_numpy=True)

# Step 5: Perform similarity search
k = 2 # top-k results
distances, indices = index.search(query_embedding, k)

# Step 6: Show results
print("Query:", query)
print("\nTop matching documents:")
for rank, idx in enumerate(indices[0]):
    print(f"{rank + 1}. {documents[idx]} (distance: {distances[0][rank]:.4f})")
```

This example defines a small corpus of five short sentences, then generates sentence embeddings for each document using a lightweight transformer model. It proceeds to build a semantic FAISS index, computes the embedding for a user query, and finally performs a top-2 similarity search to retrieve the closest matching documents.

While this code illustrates the core logic of semantic retrieval using FAISS, it's only a minimal prototype. To implement a production-level solution, you'll need to address several important concerns:

- **Corpus scalability:** The current approach loads all documents into memory. In real systems, you must handle large corpora efficiently, using batch processing and persistent storage.
- **Index persistence:** The FAISS index should be saved and loaded from disk or integrated into a vector database due to memory limitations and to support consistent querying across sessions.
- **Embedding management:** Embeddings need to be cached, versioned, and periodically updated as documents change.
- **Query pre-processing and normalization:** Queries may need to be cleaned, tokenized, or adjusted for domain-specific language.
- **Performance tuning:** For very large datasets, you'll likely want to switch to more advanced ANN and consider using GPU acceleration for faster lookups.

These aspects introduce significant complexity but are essential for building robust, scalable, and reliable semantic retrieval in production environments. While a full deep dive into these

production-level implementation details is beyond the scope of this book, we hope this section has given you a clearer intuition and foundation for exploring the topic further.

Hybrid approach, vector databases

Previously, we explored a fully custom RAG pipeline, where all components, like generator, parser, and retrieval, are built and maintained individually. We emphasized that while this approach offers maximum flexibility, it also introduces substantial complexity, especially in implementing the retrievals at scale.

In this section, we explore a more balanced alternative: the hybrid approach. Here, the generator, and parser remain custom, allowing you to customize them to your specific needs, but the retrieval is delegated to an existing tool or service. This significantly reduces the overall development effort while still offering room for customization.

Today, many modern databases, and search engines support built-in lexical and semantic retrieval capabilities, providing efficient, scalable alternatives to custom implementations. Let's now explore some of the most widely used tools and databases that offer out-of-the-box support for lexical retrieval.

Database	Type	Lexical retrieval method	Technical characteristics
Elasticsearch	Search engine	BM25 (default), BM25F, TF-IDF (optional)	Scales to petabytes of data, supports distributed indexing, and real-time search.
PostgreSQL	SQL database with full-text search	TF-IDF, BM25-like ranking	Supports millions of documents, indexing in GIN/GiST, slower than dedicated search engines.
MySQL FULLTEXT Search	SQL database with full-text search	BM25 (since MySQL 5.7)	Works best for datasets <10M rows, lacks advanced ranking, no real-time indexing.
SQLite FTS5	In-process SQL database with full-text search	BM25	Lightweight, ideal for small applications, stores data in a single-file DB.

Database	Type	Lexical retrieval method	Technical characteristics
Microsoft SQL Server (Full-Text Search)	SQL database with full-text search	TF-IDF-based scoring	It can handle large enterprise-scale data, optimized for structured text search.
Amazon OpenSearch (Elasticsearch fork)	Cloud service	BM25, TF-IDF	Scales horizontally, integrates with AWS analytics, supports multi-region indexing.
Azure Cognitive Search	Cloud service	BM25	Highly scalable, supports AI-powered re-ranking, and integrates with Azure AI.
Google Cloud Search	Cloud service	BM25 + Proprietary Ranking	Google-powered, massively scalable, optimized for real-time enterprise search.

Table 6.8: Examples of modern databases that support lexical retrieval

So, we can see there are many on-premises and cloud-based databases that support lexical retrieval out of the box, offering efficient and scalable performance even on large datasets. We also can notice that among lexical algorithms, BM25 has become the de facto standard due to its superior ranking effectiveness across a wide range of use cases. In contrast, TF-IDF is often available as an optional method and may still be suitable in specific scenarios, particularly when documents have similar lengths and well-balanced term distributions. In such cases, TF-IDF can offer faster performance with simpler implementation.

Vector databases

Now let's look at how we can implement semantic retrieval using existing tools. Earlier, we explored how ANN libraries can be used to build custom semantic retrieval pipelines by computing document embeddings and performing ANN search. While this approach offers full control, it also comes with significant responsibilities: managing data pipelines, building and maintaining indexes, handling storage, and ensuring scalability. These tasks can become especially challenging when operating on a large scale.

In practice, however, it's often more efficient to use purpose-built special *vector databases* for semantic retrieval. **Vector databases** are specialized databases designed to store, index, and retrieve high-dimensional vector embeddings. These databases are optimized for semantic search, recommendation engines, and RAG.

Vector databases not only integrate high-performance ANN algorithms like HNSW, IVF, and PQ (which we covered earlier), but also offer a range of production-ready capabilities, including:

- Scalable vector storage and indexing
- Fast similarity search across millions or even billions of vectors
- Metadata filtering and hybrid search (combining semantic and keyword retrieval)
- Cloud-native deployment, high availability, and API access

Using a vector database significantly reduces engineering complexity while still giving you the flexibility to implement custom query generation, parsing logic, and post-processing, making it an ideal solution for hybrid RAG implementations.

Here are some of the most popular vector databases available today:

Vector database	Characteristics	ANN methods	Scalability	Deployments
Pinecone	AI-powered search & RAG	HNSW, IVF	High (~billion-scale)	Cloud-native
Weaviate	Open-source AI search	HNSW, IVF-PQ	Large (~100M+ vectors)	On-prem and Cloud
Milvus	Large-scale AI search	HNSW, IVF, PQ	High (~billion-scale)	On-prem and Cloud
Qdrant	Fast, real-time ANN search	HNSW	Medium (10M–100M vectors)	On-prem and Cloud

Vector database	Characteristics	ANN methods	Scalability	Deployments
Vespa	AI-powered search and ranking	HNSW, Query Ranking	Medium (~100M vectors)	On-prem and Cloud

Table 6.9: Popular vector databases with supported ANN algorithms

Many general-purpose databases have also recently added native support for storing vector embeddings and performing ANN search. This allows teams to integrate semantic retrieval directly into existing data infrastructure without adopting a specialized vector database. The following table lists some of the most widely used general-purpose databases that now include vector search capabilities:

Database	Type	DB extension and supported ANN methods	Best for
PostgreSQL	Relational SQL	pgvector (HNSW, IVFFlat)	AI search in structured data
MySQL	Relational SQL	HeatWave ML (Vector indexing)	Enterprise AI & RAG
Elasticsearch	NoSQL Search Engine	Dense Vector Fields (HNSW)	Hybrid search (Text + ANN)
MongoDB	NoSQL Document Store	Atlas Vector Search (HNSW)	AI-powered document retrieval
Redis	Key-Value Store	RedisVector (HNSW, Flat)	Real-time search and caching

Table 6.10: Popular general-purpose databases

So, as we can see, we have today a large selection of specialized and general-purpose databases to store embeddings and perform efficient semantic search.

Using these vector databases can significantly simplify the development of semantic retrievals in your RAG system. They abstract away many complex engineering tasks such as indexing,

vector storage, query optimization, and scaling, allowing you to focus more on your application logic, prompt design, and user experience.

However, this convenience comes with trade-offs. The downside is that certain aspects of the retrieval process may be more difficult to customize. For example, you may have limited control over the exact ANN algorithm parameters, vector pre-processing steps, or how chunking and filtering are applied at query time. Some solutions may also restrict embedding model choices or tie you to specific interfaces and data schemas.

Therefore, while vector databases are powerful tools, they may not be ideal when you need deep algorithmic control, fine-grained tuning, or full transparency of the indexing and retrieval logic. Always weigh the flexibility you need against the complexity you're willing to manage.

Now, to get a better understanding of how such databases work, let's take a closer look at the Pinecone vector database and how it can be used to implement semantic retrievals.

Pinecone is a fully managed, cloud-native vector database optimized for fast, scalable approximate nearest neighbor search. It supports both semantic and lexical retrieval, offers real-time updates, and can scale to handle billions of vectors efficiently. However, Pinecone is currently available only as a cloud service (primarily on Google Cloud), with no on-premises deployment option. This may limit control over indexing strategies and introduce data residency or compliance concerns for some use cases.

Let's take a look at how we can perform semantic search by using this vector database in Python:

```
import pinecone, openai

# Initialize Pinecone
pinecone.init(api_key="<your-pinecone-api-key>", environment="<your-pinecone-environment>")

# Step 1: Create Pinecone index
index_name = "semantic-search-demo"
pinecone.create_index(index_name, dimension=1536, metric="cosine")
index = pinecone.Index(index_name)

# Step 2: Embed some documents (using OpenAI for simplicity)
openai.api_key = "<your-openai-api-key>"
documents = [
    "Pinecone is a vector database used in RAG systems."
    "Retrieval-augmented generation improves LLM accuracy.",
    "Semantic search uses embeddings to find similar text."
]
```

```

]

def get_embedding(text):
    response = openai.Embedding.create(input=text, engine="text-embedding-
ada-002")
    return response['data'][0]['embedding']

# Step 3: Upsert documents into Pinecone
doc_ids = [f"doc-{i}" for i in range(len(documents))]
vectors = [
    (doc_ids[i], get_embedding(documents[i]))
    for i in range(len(documents))
]
index.upsert(vectors)

# Step 4: Semantic search with query
query = "What database is used in retrieval augmented generation?"
query_vector = get_embedding(query)

# Retrieve top 2 most similar documents
results = index.query(vector=query_vector, top_k=2, include_metadata=False)

# Step 5: Display results
print("Top results:")
for match in results['matches']:
    print(f"- {match['id']} (score: {match['score']:.4f})")

```

In this demo code, we first initialized Pinecone. Next, we created a vector index with a 1536-dimensional space, matching the size of OpenAI's embeddings model. Then we defined our demo corpus with a few short documents. We used OpenAI's embedding API to convert each document into a vector, and then inserted (upserted) the document embeddings into the Pinecone index. After that, we created a semantic search query, embedded it using the same model, and queried the index for the top-2 closest matches. Finally, we printed the IDs and similarity scores of the top results.

As we can see, storing embeddings and performing similarity search are offloaded to Pinecone. However, document preprocessing and embedding generation are still on us. Additionally, notice that in this example, we only store the document ID and its corresponding embedding in Pinecone. The actual document content remains under our control and must be stored and managed separately. When performing a search, Pinecone returns only the vector IDs and

similarity scores, meaning we must map these IDs back to the original documents. This division of responsibility is exactly what defines the hybrid approach.

As we can see, the hybrid approach reduces development overhead by offloading low-level components to existing tools or services, but this also comes at the cost of reduced flexibility and customization.

RAG as a service

So, now it's time to look at RAG as a service. Instead of building and managing individual components like retrievers, parsers, and generators, RAG as a service allows you to use fully managed solutions that deliver end-to-end RAG functionality out of the box. These services handle everything from indexing and semantic search to grounding and generation, enabling faster prototyping and deployment with minimal engineering effort.

Below is the list of some popular RAG services as of today:

Service	Retrievals	Description
Microsoft Azure AI Search	Hybrid (lexical and semantic)	Cloud-based hybrid search engine that supports both traditional keyword (lexical) search and vector-based (semantic) retrieval.
Vertex AI RAG Engine	Semantic (via Vertex AI Search or vector DB)	Fully managed RAG orchestration on Google Cloud, handles ingestion, indexing, retrieval, and generation. Integrates with Vertex AI Search or third-party vector databases.
Coveo AI search	Semantic (vector + metadata)	Enterprise-grade search platform with built-in vector retrieval optimized for RAG-style applications.

Table 6.11: Popular RAG services

Note that this area is evolving rapidly, and we can expect more RAG-as-a-Service platforms to emerge in the near future, both as part of modern cloud ecosystems (like Amazon Web Services, Microsoft Azure, or Google Cloud) and as standalone offerings. Several new solutions are already being announced for 2025, including enterprise-ready platforms like Personal AI, Vectara, and Nuclia, highlighting the growing investment and innovation in RAG solutions.

Using these services can significantly simplify the development of real-world RAG applications. In most cases, all you need to do is upload your documents and send user queries to the service's API. The platform handles everything else, including document storage, embedding generation, index creation and updates, semantic or lexical search, and final answer generation.

However, this convenience comes at a cost. These services typically offer limited flexibility to customize the retrieval pipeline, such as tuning chunking strategies, swapping embedding models, or adjusting ranking logic. Additionally, they can be expensive at scale and may introduce latency, especially for time-sensitive applications. Since most of these offerings are cloud-based, organizations must also consider data residency, security, and compliance concerns when integrating such services into their infrastructure.

Summary

Congratulations! We've just finished an exciting exploration of retrieval-augmented generation.

RAG has become one of the most powerful and widely used technologies in modern AI systems, especially in the era of LLMs and AI agents. It empowers AI agents to access up-to-date information and stay grounded in factual knowledge.

We started by exploring the core components of RAG, including the retriever, generator, and parser, and how they work together to ground language models in external knowledge. We then examined retrieval techniques, comparing lexical approaches such as TF-IDF and BM25 with semantic retrieval based on embeddings, similarity metrics, and nearest neighbor search, along with the trade-offs and scalability challenges involved.

Finally, we moved from theory to practice by exploring different implementation approaches, from custom pipelines to managed services, and highlighted key engineering considerations such as chunking, indexing, and performance. Overall, the chapter emphasized that effective RAG systems require not just models, but thoughtful system design that balances accuracy, efficiency, and scalability.

This chapter is a great launchpad for anyone looking to build robust RAG pipelines, whether you're prototyping, scaling a production system, or exploring new hybrid strategies. The landscape is evolving fast, and your understanding of the core mechanics will help you adapt, innovate, and stay ahead in this exciting space.

In the next chapter, we will compare RAG with fine-tuning as approaches to extending an LLM's knowledge base.

Subscribe for a free eBook

New frameworks, evolving architectures, research drops, production breakdowns – *AI_Distilled* filters the noise into a weekly briefing for engineers and researchers working hands-on with LLMs and generative AI systems. Subscribe now and receive a free eBook, along with weekly insights that help you stay focused and informed.

Subscribe at <https://packt.link/80z6Y> or scan the QR code below.



7

LLM Fine-Tuning

If you've made it to this chapter, congratulations! You already have a solid grasp of how **large language models (LLMs)** work, how to tackle a wide range of **artificial intelligence (AI)** and **natural language processing (NLP)** tasks such as summarization, classification, and reasoning, and how to extend model capabilities using RAG to access private or up-to-date information.

Up to this point, everything we've done has relied on prompt engineering and external tools. We've shaped model behavior through instructions, context injections, and retrieval pipelines, all without changing the model itself. This approach is incredibly powerful and has defined the new paradigm of adaptation without retraining. That underpins modern AI development today.

However, there's another path, one that changes the model itself. That path is *fine-tuning*. Fine-tuning allows us to teach an existing model new behaviors, adjust the way it responds, improve domain expertise, or even specialize in narrow tasks.

That said, fine-tuning is a deep and complex topic. In this chapter, our focus will be *conceptual* rather than procedural. We'll explore what fine-tuning really means and how it differs from prompt engineering and **retrieval-augmented generation (RAG)**, along with when fine-tuning is justified versus when it isn't worth the effort. We'll also examine the trade-offs, costs, and challenges involved in modifying model weights compared to adapting model behavior externally, and how organizations combine these approaches to strike a balance between flexibility, cost, and performance.

We will cover the following key topics:

- The anatomy of fine-tuning
- When and how to fine-tune

By the end of this chapter, you'll have the intuition to decide whether your next AI system should rely purely on prompting and retrieval or whether it's time to go deeper and truly fine-tune your model.

The anatomy of fine-tuning

In *Chapter 1*, we introduced fine-tuning as the process of **retraining** an existing AI model on new data. During this process, the model's weights are adjusted based on the new examples it learns from. In *Chapter 2*, we also saw fine-tuning as a key stage in the development of modern LLMs, where a pre-trained model is refined to become more human-like, aligned, and practically useful in real-world applications. Now, let's dive a bit deeper into what fine-tuning actually means and how it works.

At a high level, every AI model can be described by two core components:

- **Architecture:** The design or blueprint of the model. It defines the model size and how many layers it has, what type and structure of layers (attention, convolution, normalization, feed-forward, etc.), how they are connected, and what activation functions they use.
- **Parameters (weights):** Millions or billions of numerical values that define the model.

You can think of the parameters as an enormous matrix of decimal numbers, sometimes billions or even trillions of them. Each number encodes a small piece of "knowledge" or "association" that the model has learned from data.

When we fine-tune a model, we usually don't alter its underlying architecture. So, the number of layers, connections, and parameters remains the same. What changes are the weights: the numerical values that define how the model processes information. As these weights are updated based on new training data, the model's behavior adapts, emphasizing new patterns, new cases, or decisions that better align with the fine-tuning dataset.

In essence, fine-tuning teaches an existing model to "think" a bit differently, usually without rebuilding it from scratch.

To make this idea more tangible, let's revisit the house price prediction model we discussed in *Chapter 1*. Recall that the model takes features like the number of rooms, square footage, bathrooms, and location, and predicts the house price.

In that simple model, the architecture might consist of just a few input neurons (one per feature), a small hidden layer, and one output neuron for the predicted price. The model's parameters (weights) could be represented as a small 5×5 matrix, a table of decimal numbers that determine how each input feature influences the final prediction.

For example, imagine our model initially learned these weights (not real weights, just for demo purposes only):

Feature	Rooms	Bathrooms	Area	Lot size	Zip code
Hidden 1	0.2	0.4	0.8	0.1	0.3
Hidden 2	0.1	0.6	0.5	0.2	0.4
Hidden 3	0.3	0.7	0.6	0.2	0.5
Hidden 4	0.2	0.2	0.3	0.1	0.2
Hidden 5	0.1	0.5	0.7	0.2	0.3

Table 7.1: Demo weights for a housing price prediction model

Each row labeled "Hidden 1" through "Hidden 5" represents a neuron in the hidden layer of our small neural network. Each neuron learns to combine the input features (like number of rooms or lot size) using different weights to capture subtle relationships. For example, one neuron might focus more on "size-related" features, while another might learn to emphasize "location-related" ones.

When we fine-tune the model on this new dataset, we don't rebuild its architecture; we keep the same five weights and the same layers. But through training, the model can adjust its weights slightly based on the new patterns it detects. After fine-tuning, some numbers may shift like this:

Feature	Rooms	Bathrooms	Area	Lot size	Zip code
Hidden 1	0.2	0.4	0.9	0.1	0.35
Hidden 2	0.1	0.6	0.6	0.2	0.4
Hidden 3	0.3	0.7	0.6	0.2	0.55
Hidden 4	0.2	0.2	0.3	0.1	0.2
Hidden 5	0.1	0.5	0.7	0.2	0.3

Table 7.2: Updated weights for a housing price prediction model

Notice that only a few numbers changed, but those small adjustments mean the model now places slightly more importance on "Area" and "Zip Code" when predicting price. As a result, the model's predictions better reflect the new housing trends.

This simple example captures the essence of fine-tuning. In large-scale models like GPT or Claude, the same principle applies. But these LLMs already have billions of parameters. Each fine-tuning step subtly shifts how the language model interprets relationships between words, meanings, and reasoning patterns, leading to a different overall "personality" or capability.

AI model fine-tuning can be viewed from several perspectives. Among them, two are the most fundamental:

- **Scope:** Whether we update all model parameters (full fine-tuning) or only a subset using parameter-efficient methods such as **low-rank adaptation (LoRA)** or adapters (partial fine-tuning)
- **Intent and purpose:** Whether the goal is to teach new skills or knowledge (generic fine-tuning) or to align model behavior with human preferences and values (alignment fine-tuning, as in reinforcement learning from human feedback or direct preference optimization)

To better understand these perspectives, it's important first to see how fine-tuning actually changes a model internally and how adjusting parameters reshapes its knowledge, reasoning, and behavior.

How fine-tuning affects the model

During fine-tuning, the pretrained weights are adjusted to better fit a new domain, style, or task. Technically, changing these parameters means changing the model itself. The model's internal knowledge, learned associations, and overall behavior change as well.

However, typically, much of the model's prior knowledge is retained. Even though all parameters are eligible for updates, often most of them change only slightly, while a smaller portion shifts more noticeably to capture new patterns.

This happens because fine-tuning adjusts parameters selectively, rather than replacing them completely. During training, the model compares its predictions with the desired outputs and adjusts the weights to reduce the difference. The goal of training is to find weight values that make the model's predictions align as closely as possible with the expected results.

If most of the new examples are similar to what the model already saw (was trained on), the weight adjustments are minimal or even zero, since the existing parameters already produce correct results. Only the parameters that strongly influence new or unfamiliar aspects of the data receive larger updates.

And here, the curious reader might ask a fundamental and surprisingly deep question:

When we fine-tune a model on new data, does it add new knowledge or replace what was already there?

The short answer is: both, but not in the same way as a database or file storage system does. In a traditional database, every piece of information has its own dedicated place. If the database has limited capacity and we insert new records, older ones may need to be deleted or overwritten to make space. It's a clear, discrete replacement. You remove one row and add another.

AI models, however, work very differently. As we discussed in *Chapter 1*, knowledge in AI models is not stored as isolated facts or records. It is distributed and encoded across many parameters (weights). Each weight captures not a single fact, but a tiny fragment of many overlapping relationships, patterns, and associations.

Because of this distributed representation, there isn't one specific "slot" where a fact like "Paris is the capital of France" or "Vitali Klitschko is a former professional boxer" resides; it's spread across thousands or millions of connections.

When we fine-tune, we're not adding or deleting individual records. Instead, we're shifting the distribution of parameters, strengthening some patterns while weakening others. If the new data aligns with existing knowledge, those "associations become stronger" (parameters remain the same). The model refines what it already knows. If the new data contradicts previous information, the corresponding patterns are redirected or overwritten, effectively replacing earlier associations (parameters are overridden significantly).

So, in practice:

- Fine-tuning on data that aligns with the model's existing knowledge typically refines parameters (small adjustments)
- Fine-tuning on data that conflicts with the model's existing knowledge can overwrite parameters (larger shifts)

Roughly speaking, if we fine-tune a model using the same or very similar data it was originally trained on, its parameters will remain almost unchanged. The model already fits that data well, so there is little new to learn or adjust. In contrast, fine-tuning on completely new or very different data forces many parameters to shift significantly as the model learns new relationships.

For example, consider the housing price prediction model we discussed in *Chapter 1*. Suppose it was initially trained on residential housing data in Idaho, USA. If we fine-tune it again using updated Idaho housing data, the model will make only minor adjustments. Perhaps the change will reflect current price trends, but most of its internal knowledge will remain stable.

However, if we fine-tune the same model on commercial property data from Tokyo, Japan, the relationships between features will be completely different. For instance, now the correlation between area, location, and price will be very different. The model will need to modify a large portion of its parameters to adapt, effectively reshaping its internal understanding of what "price" means in this new context.

After such retraining, the new fine-tuned model will likely perform poorly on the original Idaho data. The patterns it learned earlier, such as how the number of bedrooms or lot size affect housing prices, no longer match the structure of the Tokyo dataset. The model "forgets" what it learned in the past. By adapting to the new domain, the model has overwritten its previous associations. This phenomenon is known as **catastrophic forgetting**.

In essence, fine-tuning is a "dialog" between data and parameters. The data defines where and how much the model changes. If the new data reinforces what the model already understands, the adjustments are small, and the knowledge base remains. If the data introduces new or conflicting patterns, the model must reorganize its internal structure (parameters), sometimes at the cost of what it knew before.

This interplay between data and weights is what determines how deeply fine-tuning reshapes a model's behavior.

Full fine-tuning

Now that we understand how model parameters are changed during fine-tuning, let's look at the scale of those changes, whether we adjust all parameters or only a subset of them. This distinction defines two main approaches: full fine-tuning and partial fine-tuning.

Full fine-tuning is the process of retraining a pretrained model by updating all of its parameters (weights) on a new dataset. It is conceptually similar to training a model from scratch, with the key difference that the model does not start from initial (often random) parameters. It begins with pretrained weights that already capture some patterns and knowledge. Because of this head start, full fine-tuning typically requires less data and compute resources than training a model entirely from scratch.

Although full fine-tuning benefits from starting with pretrained weights, it still typically requires substantial computational resources, and these demands become especially significant for large models like LLMs that have billions of parameters.

The larger the model, the greater the cost, because every parameter must be recalculated and updated. This makes full fine-tuning extremely demanding in terms of GPU, memory, training time, and energy consumption.

In addition to compute, full fine-tuning also needs a large and diverse dataset. When all parameters are being updated, the model must see enough examples to correctly adjust its

internal representations. If we were to fine-tune on just a few samples, the model would simply memorize those examples rather than learn meaningful patterns, degrading its general performance.

To illustrate this effect, let's consider building a simple linear regression model with just two parameters, similar to what we did for the housing price model in *Chapter 2*. Such a model represents a straight line in a 2D plane. Technically, we need only two data points to define that line, something we all learned back in school.

However, that's true only in theory. In practice, real-world data always contains noise, outliers, and inaccuracies. If we rely on just two samples, the resulting line will fit those points perfectly, but may be completely wrong for the rest of the data distribution. To capture the true underlying trend, the model needs many diverse examples so it can "average out" random fluctuations and learn a more general relationship.

Because of this, full fine-tuning of LLM often requires tens of thousands to millions of high-quality examples. The number of examples depends on the complexity of the task and model size. To be more specific, let's provide some examples from real experiments:

- In the paper, "*How Much Data is Enough Data? Fine-Tuning Large Models*" [38], researchers fine-tuned the LLaMA 3 model (8 billion parameters) for translation tasks, adapting it to specific domains and language pairs. They experimented with dataset sizes ranging from 1K to 207K sentence pairs, observing that model quality improved steadily with more data, plateauing around 100K samples.
- In "*Sample Size Considerations for Fine-Tuning Large Language Models*" [39], researchers fine-tuned BioBERT, a domain-adapted variant of BERT, for biomedical **named-entity recognition (NER)** tasks using datasets between 5K and 50K labeled sentences. They found that performance improved rapidly up to around 30K samples, after which additional data yielded diminishing returns, showing that even mid-scale datasets can saturate full fine-tuning performance.

The amount of data required for effective fine-tuning is not fixed. It depends heavily on the size and capacity of the model, the task, and the domain. Larger models, with billions or even hundreds of billions of parameters, have a much greater "capacity" and can model far more complex relationships. However, that also means they need more fine-tuning data to adjust these parameters meaningfully and avoid overfitting or underutilization.

Just to remind, as we discussed in *Chapter 1*, *overfitting* occurs when a model learns the training data too precisely, capturing noise or random details instead of general patterns. As a result, it performs well on the data it was trained on but fails to generalize to new, unseen examples. In the context of fine-tuning, overfitting happens when the dataset is too small or narrow

compared to the model's size. The model adapts too strongly to specific samples and loses its broader understanding.

So, if we fine-tune a small model (for example, 125M or 355M parameters) on a few thousand examples, it can adapt relatively quickly, because its internal capacity is limited and each example influences a significant portion of its weights.

But a massive model (let's say 100B parameters) has far more internal degrees of freedom. A small dataset will barely nudge its weights, and the changes will be too weak or unstable to create a consistent new behavior. As a result, large models typically require proportionally larger and more diverse fine-tuning datasets to achieve stable and meaningful adaptation.

We also have to mention that the relationship between the model size and the number of examples to fine-tune on is not perfectly linear. For instance, doubling model size doesn't necessarily require doubling data. But in general, larger models need more data to make fine-tuning effective and to prevent overfitting or catastrophic forgetting. For very large models, high-quality and domain-representative data becomes even more important than sheer quantity, since every sample contributes only a tiny influence across billions of parameters.

To illustrate that, consider the following examples:

- Fine-tuning a BERT-base (110M) model might work well with tens of thousands of examples
- Adapting a LLaMA 2 70B or GPT-3-class (175B) model typically requires hundreds of thousands to millions of examples for full fine-tuning to meaningfully alter its behavior

On the other hand, using too much data for a small model can also be counterproductive. Smaller models, like **small language models (SLMs)**, have a limited capacity. They can't fully capture the complexity and variety present in massive datasets. When trained or fine-tuned on very large datasets, these models may reach a point where additional data no longer improves performance.

In some cases, too much diverse data can even confuse a small model, reducing accuracy as it struggles to balance competing patterns. For such models, a smaller but well-curated and domain-focused dataset usually yields better results than a massive, unfocused one [40].

Partial fine-tuning

As we can see, full fine-tuning requires substantial computational power and large, high-quality datasets. These requirements grow rapidly with model size. The larger the model, the greater the cost of updating all its parameters. In practice, full fine-tuning is therefore typically applied to smaller models (like SLMs), where it remains feasible, or carried out by major AI

organizations such as OpenAI, Anthropic, Google, or Microsoft when developing or adapting their large models.

We can also notice that during full fine-tuning of large language models, often only a fraction of parameters change significantly, while most remain nearly the same. This happens because the pretrained representations are already general and stable and don't require much updating.

As we also discussed in *Chapter 1*, the earlier layers of a neural network typically handle more basic and fundamental representations, while the later layers focus on higher-level, task-specific abstractions. For example, as illustrated in *Figure 1.7* (image classification), the first layers analyze low-level visual patterns such as lines, curves, and edges, whereas the deeper layers combine those into more complex concepts like objects, people, or animals.

A similar hierarchy exists in language models. The early transformer layers capture fundamental linguistic patterns such as word structure, syntax, and grammatical dependencies, while the later layers model more semantic and contextual relationships, including meaning, reasoning, facts, and top-level concepts.

Because of this, fine-tuning often affects the higher layers more strongly, as they are responsible for task-specific reasoning and output behavior, while the lower layers tend to remain more stable and preserve the model's general language understanding.

We can also recall from *Chapter 2*, where we explored Seq2Seq models and decoder architectures, that the final feed-forward layer is responsible for producing the probability distribution of the next token. If we were to modify only this last layer, we would directly influence the model's final decision-making process. Essentially, we are changing the way we determine which tokens are most likely to appear next. This provides useful intuition: fine-tuning affects these higher layers, adjusting how the model interprets and generates its final outputs while leaving the deeper, more general linguistic knowledge largely intact.

That's why **partial fine-tuning** was introduced. It is an approach that keeps the earlier layers of the model unchanged and updates only the later layers, which have the most direct impact on the model's task-specific reasoning and output generation. For LLMs, this strategy allows the model to retain its general language understanding from pretraining while adapting efficiently to new domains or objectives.

The main advantage of partial fine-tuning lies in its efficiency. Since only a small fraction of parameters are updated, training costs drop dramatically both in terms of compute time and GPU requirements. This makes it feasible to adapt large models on modest hardware setups, such as a single high-end GPU, rather than large, distributed GPU clusters.

Another benefit is that partial fine-tuning requires less data. Because the model's core knowledge and language understanding remain intact, the fine-tuning dataset can be smaller, more specific, and focused on the new domain or task rather than broad and diverse. This makes partial fine-tuning particularly practical for specialized applications; for example, adapting a general-purpose LLM to legal, medical, or financial language without retraining the entire model.

In practice, partial fine-tuning can be implemented in several ways. Let's look at a few examples that illustrate how much of a model typically needs to be updated to achieve meaningful adaptation:

- **BERT-base (110M parameters) fine-tuning:** When adapting BERT-base for a specialized domain such as legal document classification, researchers often freeze the first 10 of its 12 transformer layers, updating only the last two layers and the classification head. This means that roughly 10–15% of the parameters are fine-tuned, while the rest remain unchanged. That is often enough to capture the nuances of legal terminology without retraining the entire model.
- **LLaMA 2 (7B parameters) fine-tuning:** When adapting LLaMA 2 for domain-specific text generation (for instance, financial report writing), partial fine-tuning can focus only on the last 2–4 transformer blocks. This approach modifies less than 3–5% of the model's total weights yet achieves comparable results to full fine-tuning on targeted tasks while requiring only a fraction of the compute and data.

These examples show that even minimal adjustments to the upper layers of large models can yield significant improvements in domain or task performance.

Parameter-efficient fine-tuning: adapters and LoRA

As we saw, partial fine-tuning updates only a subset of model parameters to reduce cost and data requirements. But researchers went further, developing methods that make this process even more efficient and modular. It is collectively known as **parameter-efficient fine-tuning (PEFT)**. Adapters and LoRA are among the most widely adopted PEFT approaches.

The idea behind adapters is simple: instead of modifying the original model weights, we insert small trainable modules, called **adapter layers**, between existing layers of the network. During fine-tuning, only these adapter layers are updated, while the main model remains frozen (unchanged).

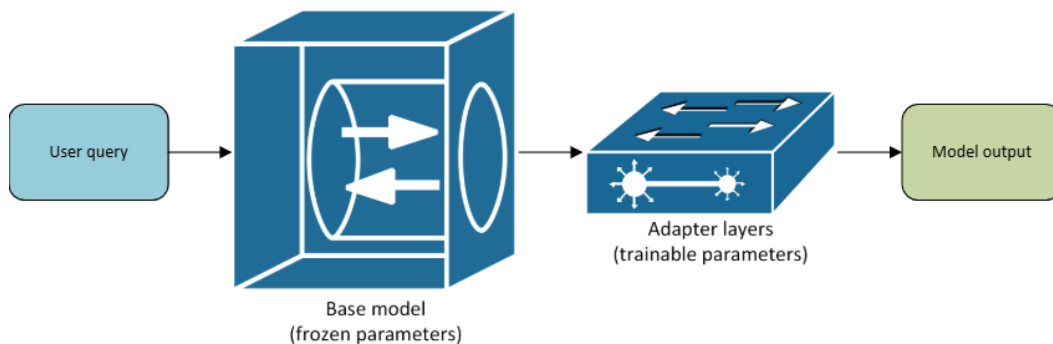


Figure 7.1: Adapter-based fine-tuning

This drastically reduces the number of trainable parameters and allows the base model to be reused across multiple tasks. For example, in a transformer with 300 million parameters, an adapter-based setup may introduce only 10–30 million new parameters per task.

Each adapter acts like a "plug-in" that learns domain-specific adjustments while the frozen (unchanged) layers preserve the model's general language understanding. This makes adapters ideal for scenarios where the same base model must be adapted to many tasks or clients. In this case, the system loads different adapter weights without retraining the whole model.

Another variant of PEFT is LoRA, which was introduced in 2022. Unlike adapters, which insert new layers between existing ones, LoRA modifies the model's existing weights by adding a pair of small, trainable low-rank weight matrices. During fine-tuning, the original model weights remain frozen, and only these compact matrices are updated.

This technique captures how the weights should shift toward the new task but represents that change in a much smaller parameter space. It is often less than 1% of the model's total parameters.

In practice, LoRA enables efficient fine-tuning of very large models even on a single high-end consumer GPU. It has become the de facto standard for adapting open-source large models and is supported by popular frameworks like Hugging Face PEFT, QLoRA, and OpenLLM.

The field of parameter-efficient fine-tuning is evolving rapidly, with new methods and optimizations emerging almost monthly. Techniques such as prefix tuning, bitfit, prompt tuning, and QLoRA extend the same core idea, achieving powerful adaptation with minimal parameter updates and modest hardware requirements.

So, in this section, we explored the key ideas and high-level principles behind fine-tuning. If you're interested in going deeper, we encourage you to study fine-tuning implementations in

detail, including modern open-source libraries such as Hugging Face PEFT and LoRA modules in PyTorch. What truly matters is understanding the core principles of parameter-efficient fine-tuning. Once you grasp those, working with the actual tools becomes straightforward.

Supervised fine-tuning

So far, we've looked at fine-tuning through the lens of scope. We discovered how much of the model we update, whether it's all parameters (full fine-tuning) or just a subset (partial or parameter-efficient fine-tuning). Another fundamental aspect of fine-tuning is its intent and purpose. It is about the question of what exactly we are trying to teach the model. What new knowledge, skills, or behaviors do we want to acquire from the new data?

Fine-tuning can serve very different purposes. In some cases, the goal is to extend the model's knowledge or skills. For example, adapting it to legal or medical terminology, or teaching it to perform better on specific NLP tasks like summarization or translation. In other cases, the goal is to shape how the model behaves, making it more helpful, safe, polite, or aligned with human preferences.

These two directions give rise to two major families of fine-tuning:

- **Supervised fine-tuning (SFT):** Where the model learns directly from human-labeled examples to produce desired outputs for specific inputs
- **Preference-based fine-tuning:** Where the model learns how to behave by optimizing for human feedback, typically through methods like **reinforcement learning from human feedback (RLHF)** or **direct preference optimization (DPO)**

Let's start with the first one, SFT, which is conceptually the most straightforward and builds directly on what we learned in *Chapter 1* about supervised learning.

Back then, we explored supervised learning using the housing-price prediction example. The housing prices model learned to map inputs (house features like area, location, and number of rooms) to a target output (price) by minimizing the difference between predicted and real values.

That same fundamental idea lies at the heart of SFT for large language models. In SFT, the model is trained on a dataset of input–output pairs, where the desired behavior is explicitly demonstrated. For example, fine-tuning on a summarization dataset such as CNN/DailyMail teaches the model how to summarize text, what information is important, what can be omitted, and how to express the summary concisely.

During training, the model adjusts its parameters (weights) to reproduce these desired outputs as closely as possible, just as our housing-price model in *Chapter 1* adjusted its weights to better match real prices.

To build better intuition for how SFT works, let's look at the fine-tuning process on the CNN/DailyMail dataset. The overall flow can be roughly illustrated as follows:

- The model receives as input: a news article from the CNN/DailyMail dataset
- It generates a candidate summary based on its current parameters
- The generated summary is compared with the reference (expected) summary from the dataset
- If the generated and expected summaries differ significantly, the model updates its weights to reduce this difference
- This process repeats over thousands of examples and multiple training cycles (called **epochs**) until the model's outputs closely align with the reference summaries

The main challenge of supervised fine-tuning lies in the data itself. This approach requires large, high-quality labeled datasets, where every input has a well-defined expected output. Such datasets are typically expensive and difficult to create, often requiring human labeling and annotations.

Another challenge comes from the types of tasks we are solving with fine-tuning. For structured or well-defined problems such as classification, information extraction, or reasoning over factual data, supervised datasets are natural to build. For every input, we can have well-defined outputs.

However, for open-ended generation tasks, there is rarely a single "correct" answer. For example, for tasks like story writing, email drafting, or other creative or conversational outputs, it isn't hard to define what "good" output should look like. The quality of such responses often depends on style, tone, context, or even subjective preference, making it difficult to define consistent labels or "gold-standard" outputs. This inherent ambiguity limits how well SFT can capture the nuances of generative tasks.

It's also important to note that even with large datasets, SFT does not always guarantee optimal performance. In some cases, after multiple training iterations, the model outputs may still diverge significantly from the expected results. Additionally, SFT is prone to overfitting, particularly when fine-tuning on small, narrow, or non-diverse datasets. The model can become too specialized in reproducing training examples and lose generalization ability, performing poorly on unseen or slightly different inputs.

Finally, SFT tends to bias the model toward the training distribution, which can reduce creativity or flexibility. This can be a limitation for generative and conversational tasks. For these reasons, modern fine-tuning pipelines often combine SFT with preference-based methods like RLHF or DPO.

Preference-based fine-tuning: RLHF and DPO

SFT assumes that every question has a correct expected answer. But as we mentioned already, many real-world tasks don't have a single "right" response. In such cases, what matters most is human preference and which response feels preferable to people.

That's where preference-based fine-tuning comes in. Instead of training the model to reproduce exact target outputs, we train it to prefer some responses over others. For example, those that align better with human expectations, communication norms, and values. This approach forms the foundation of *alignment fine-tuning*, where the goal is not to add new knowledge, but to shape model behavior in ways humans find acceptable and useful.

One of the first and most widely used methods for implementing preference-based fine-tuning is **reinforcement learning from human feedback (RLHF)**, which we briefly mentioned in *Chapter 2* as the final stage of training for many modern LLMs.

In *Chapter 1*, we also introduced **reinforcement learning (RL)** more broadly, comparing it to supervised and unsupervised learning. Traditional RL is a way of training models through interaction with an environment. The model takes actions, receives feedback in the form of rewards or penalties, and gradually learns better behavior based on that feedback.

We also defined the concept of a *policy*, a model component that learns a strategy for selecting actions that maximize long-term rewards (or minimize penalties). In essence, a policy represents the type of experience humans develop through trial and error, learning which actions lead to the most favorable outcomes.

In the context of RLHF, the policy is the language model itself. The policy defines the strategy for deciding which token to generate next. During RLHF training, the model operates within a controlled environment designed to guide it toward behaviors that align with human preferences.

To guide learning, a separate *reward model* is trained to estimate how much humans would prefer one response over another. It acts as a stand-in for human judgment, assigning a numeric reward ("good" or "bad") to each generated answer. During training, the language model (policy model) explores different ways of responding to prompts, and the reward model evaluates those responses. Human-aligned answers receive higher rewards, while unaligned ones receive lower scores.

The language model then updates its internal parameters to favor higher-reward responses in future interactions. Over time, it learns patterns that people consistently prefer, not because it was told the exact "correct" answer, but because it has internalized what feels right to humans.

So, roughly speaking, in RLHF, we introduce an additional model, the reward model, that acts as a judge or teacher, guiding the main language model to respond in ways that humans would

prefer. During training, the reward model provides feedback on how good or bad each response is. After fine-tuning is complete, the reward model is no longer needed. The updated language model (now aligned through RLHF) is the one used for inference.

The reward model itself also needs to be trained on human preference data. It acts somewhat like a classifier or regressor, assigning a score that reflects how likely humans are to prefer one response over another.

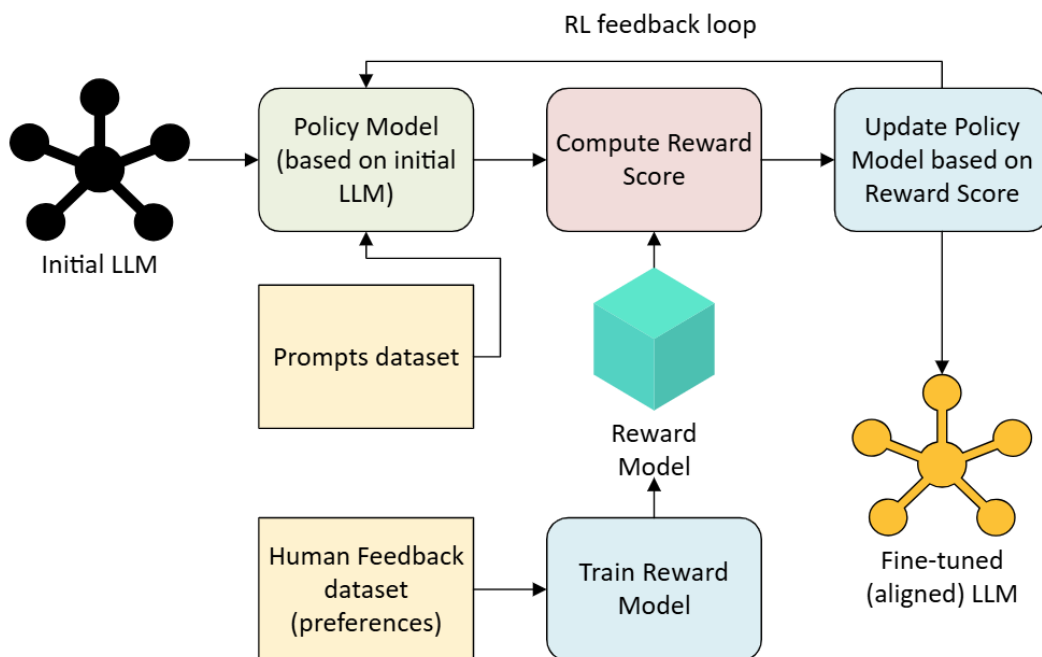


Figure 7.2: RLHF training pipeline

So, looking at the preceding figure, we can see that RLHF relies on two datasets: one containing human-labeled preferences, which is used to train the reward model, and another containing a set of prompts (user questions) used to train and update the policy model (our main LLM).

Once both datasets are prepared, the reinforcement learning feedback loop begins. A prompt is sampled from the prompts dataset and passed to the policy model (our LLM), which generates a response. That response is then evaluated by the reward model, producing a reward score that reflects how much humans would prefer it. This score is used to update the policy model (LLM), gradually teaching it to produce responses that are more aligned with human feedback and preferences.

Note

The RLHF technique was first applied by OpenAI in early 2022 with the release of InstructGPT (Ouyang et al., 2022). This marked a turning point in aligning large language models with human preferences by training them to prefer responses rated as more helpful and safe by human evaluators. The same approach later became a cornerstone of GPT-3.5, dramatically improving its usability, alignment, and conversational quality and enabling it to power the first version of ChatGPT.

We can notice that RLHF is quite complex. It requires multiple models, two separate datasets, human labeling, and a reinforcement learning loop with iterative updates. This makes it computationally expensive and difficult to scale compared to standard fine-tuning methods. For this reason, newer techniques such as **direct preference optimization (DPO)** have been developed to achieve similar alignment benefits without heavy reinforcement in the learning stage.

DPO simplifies preference-based fine-tuning significantly. It trains the language model directly on human preference data. It uses pairs of responses: one preferred and one rejected. It optimizes the model's parameters so that it assigns higher probability to the preferred responses. In essence, DPO bakes the preference signal directly into the model without needing a separate reward stage or policy optimization algorithm.

Mathematically, DPO reuses the same approach used in supervised fine-tuning but reformulates it to compare the probabilities of the preferred and rejected answers. This makes it far simpler and more stable than RLHF. There's no reward model, no reinforcement loop, and no risk of instability or reward hacking.

Despite its simplicity, DPO can achieve alignment quality comparable to RLHF on many benchmarks [44]. Because of these advantages, DPO has quickly become a popular method for alignment fine-tuning in both research and open-source communities. It enables organizations to refine large models using human preference data, turning model alignment into a more accessible, scalable, and transparent process.

Despite its advantages, DPO is not a universal replacement for RLHF. While it removes much of the complexity of reinforcement learning, it also inherits several limitations.

First, DPO relies entirely on pre-collected preference data, pairs of responses where humans (or a teacher model) have already decided which one is better. That means it cannot actively explore or discover new behaviors beyond what's represented in this dataset. In contrast, RLHF's reinforcement loop allows a model to generate new candidate responses during training, evaluate them via the reward model, and iteratively improve them, giving it a more dynamic learning process.

Second, once a model is trained with DPO, it's harder to adjust its behavior afterward. Because DPO learns directly from fixed examples of what people prefer, changing the model's tone or priorities usually requires collecting a new preference dataset and retraining from scratch. RLHF, by contrast, can adjust its behavior simply by retraining or refining the reward model.

Finally, because DPO skips the explicit reward modeling stage, it offers less interpretability. There's no separate "reward function" that developers can analyze or debug to understand what the model values. For this reason, many large AI labs continue to use RLHF (or hybrid methods) for high-stakes systems where fine control and monitoring are essential.

So, we've explored the fundamentals of fine-tuning, including full, and partial approaches, as well as supervised and reinforcement learning methods. With this foundation in place, we can now turn to the practical side: when fine-tuning is truly needed and how to apply it effectively in real-world scenarios.

When and how to fine-tune

At this point, we already have a solid understanding of what fine-tuning is, how it works, and the different forms it can take. Now it's time to move from theory to practice and answer several fundamental questions:

- How to perform fine-tuning?
- How do we choose the right model and strategy for a given task?
- When should we use prompting, RAG, or fine-tuning?
- How do we build effective datasets?
- And finally, how do you host fine-tuned models in production?

In the following sections, we'll explore these practical questions step by step, illustrating each with examples and guidelines that will help you decide how to apply fine-tuning effectively in real-world scenarios.

How to fine-tune

Now that we understand the theory and types of fine-tuning, let's look at how it is actually performed in practice.

Today, developers and researchers have several options for fine-tuning pretrained language models. Each approach offers a different balance of cost, control, scalability, and data privacy:

- Fine-tuning on local or on-prem hardware
- Fine-tuning in the cloud
- Fine-tuning as a managed feature

Fine-tuning on local or on-premises hardware

In this setup, you manage the entire fine-tuning process by yourself, from preparing data and configuring the environment to training and deploying the model. It's the most flexible approach, giving you complete control over the model, data, and infrastructure, but it also requires proper hardware and technical setup, including GPU acceleration (NVIDIA's Compute Unified Device Architecture, or CUDA) and using frameworks such as PyTorch or TensorFlow.

Relatively small models (such as with 1B to 7B parameters) can often be fine-tuned on a single modern GPU (e.g., NVIDIA RTX 4090 or A6000 with 24–48 GB VRAM) using parameter-efficient techniques like adapters or LoRA. Medium-sized models (13B–30B) may require multi-GPU configurations or distributed setups, while very large models (65B+ or more) are typically fine-tuned only by organizations with significant compute infrastructure, such as OpenAI, Anthropic, Google, Meta, or Microsoft.

Building and maintaining such infrastructure requires significant upfront investment, but it can pay off if fine-tuning needs to be done frequently or across multiple models. The main advantage of local or on-premise fine-tuning is maximum privacy and control. All data and model artifacts remain entirely within your environment.

Fine-tuning in the cloud

A popular alternative is to use cloud-based infrastructure to perform fine-tuning. Major cloud providers such as **Amazon Web Services (AWS)**, Microsoft Azure, and Google Cloud offer managed GPU and TPU clusters that can be rented on demand.

This eliminates the need to purchase or maintain expensive hardware, making it ideal for teams that fine-tune models occasionally or for specific projects. Cloud setups allow you to scale compute resources as needed, often with minimal configuration effort.

However, over the long term, if you fine-tune models frequently or at a large scale, renting cloud GPUs can become more expensive than maintaining dedicated hardware.

In terms of workflow, the process is nearly identical to local or on-premise fine-tuning. You use the same libraries and training scripts. The key difference is that only the compute environment is hosted in the cloud. Many organizations also integrate fine-tuning pipelines with their broader ML infrastructure, using tools for experiment tracking, checkpoint management, and automated deployment.

Fine-tuning as a managed feature

The newest and simplest option is fine-tuning as a managed feature, offered by major LLM providers such as OpenAI or Anthropic. In this case, the user simply uploads a dataset containing example prompts and desired responses. The provider then handles the entire process: data processing, training, evaluation, hosting, and deployment of a custom model that is accessible via API.

This approach is ideal when:

- You need fast results without building or maintaining infrastructure
- You are comfortable with a black-box process; you cannot access training internals, adjust model layers, or export weights
- You have no privacy restrictions preventing you from transferring or storing your data with a third-party provider

Often, fine-tuning as a service is implemented using parameter-efficient methods such as adapters or LoRA. In these setups, a small number of additional trainable layers or low-rank matrices are inserted into the base model (for example, GPT or Claude), while the vast majority of the original model parameters remain frozen and are used across different customers.

This approach allows the provider to train and deploy fine-tuned variants efficiently without re-training the entire model or duplicating massive weight files. It also means each customer can have their own lightweight, customized version of the same base model, while sharing most of the underlying parameters. Such architecture dramatically reduces the cost and storage footprint of fine-tuning, while enabling isolation between customers' adaptations. In practice, this is what makes commercial fine-tuning APIs scalable and economically viable.

The trade-off is reduced flexibility, transparency, and portability. When using managed fine-tuning, you have limited control over how the process is configured; you typically can't adjust training hyperparameters, choose specific layers to update, or inspect intermediate checkpoints. The model weights and adapter layers are trained entirely within the provider's cloud environment and remain there permanently.

You also cannot download or host the fine-tuned model independently; inference requests must be made through the provider's API. In practice, only a small portion of the model's parameters, the newly trained adapter or LoRA weights, belong to your fine-tuned version, while the core model remains proprietary and shared across all customers.

Choosing the right model and strategy

Now that we understand how fine-tuning works, the next step is to decide what to fine-tune. Specifically, which model family, model size, and fine-tuning strategy best fit your goals? This decision is rarely straightforward. There is almost never a single "best" model; rather, the right choice depends on your task type, domain scope, data quality, and operational constraints such as cost, latency, and infrastructure.

In *Chapter 2*, we discussed how to select an appropriate LLM for prompt engineering workloads. Many of those principles also apply here: understanding your task, aligning with model benchmarks, and assessing real-world trade-offs between capability and efficiency. Fine-tuning, however, introduces new dimensions such as data scale, compute requirements, and the depth of model adaptation. That makes the choice even more nuanced.

To navigate this decision, we'll break the process into three key steps:

- **Select the model family:** Identifying the right architecture and pretraining base for your task
- **Choosing the model size:** Balancing model capacity against efficiency, cost, and deployment limits
- **Selecting the fine-tuning strategy:** Deciding how deeply to adapt the model, from lightweight parameter-efficient methods to full fine-tuning

Let's dive right into it.

Select the model family

Choosing the right model family is the first and very important decision in fine-tuning. Model families differ not only in architecture but also in training objectives, data composition, license type, and intended use cases.

Historically, encoder-only architectures (such as BERT, RoBERTa, or DeBERTa) dominated *natural language understanding* tasks like classification and entity extraction [24]. However, the field has largely shifted toward decoder-only models, such as GPT, LLaMA, Mistral, Phi, or Gemma, which combine strong reasoning and generation capabilities with surprising competence in understanding tasks [84,85]. This shift means that, for most modern workloads, a decoder-only model is often the most practical starting point.

Although encoder-only models like BERT can still outperform decoder-only models in terms of efficiency and accuracy per parameter. Their bidirectional attention and compact structure make them well-suited for single-pass understanding tasks where no text generation is required. However, the final choice should always be based on empirical evaluation within your target task and dataset.

It is also worth noting that encoder-decoder architectures, such as those used in T5 or BART, get additional traction these days. This architecture was originally introduced in the transformer framework to unify understanding and generation [7]. However, encoder-decoder models have higher training and inference complexity. As a result, most modern fine-tuning efforts start from encoder-only or decoder-only families. The encoder-decoder architecture is often the second choice when a task requires both a deep understanding of the input and high-quality generation.

With that context in mind, let's look at the main dimensions to consider when evaluating model families for fine-tuning:

- **Pretraining data and domain:** Check what type of datasets the base model was trained on. Selecting a model whose pretraining data aligns with your target domain reduces the amount of new supervision needed.
 - **General-purpose models** (e.g., LLaMA, Mistral) are trained on broad internet text and perform well across varied tasks
 - **Domain-specific models** (e.g., BloombergGPT, FinGPT, MedPaLM, CodeLLaMA) are pretrained on specialized corpora such as financial filings or biomedical papers and require less data to adapt within that same domain

- **Benchmarks and evaluation scope:** Review how the model performs on standard benchmarks and assess how closely those benchmarks align with your intended task or domain. As emphasized throughout this book, treat these benchmarks as guides rather than absolute rankings. They provide a useful high-level signal of model capability, but the final decision should always rely on empirical evaluation using your own validation data (see *Chapter 8*). Benchmarks help narrow the options; real-world testing confirms the fit. For example, if your task involves reasoning or multi-turn interaction, prefer models tested on benchmarks such as MMLU, ARC, or BIG-Bench
 - For instruction-following or chat applications, check open benchmarks like AlpacaEval, Arena Hard, or MT-Bench
- **License, openness, and ecosystem:** Depending on whether you plan commercial use or internal deployment, licensing can be decisive
 - Open-weight models like LLaMA 3, Mistral, and Gemma allow full fine-tuning and local hosting
 - Closed models like GPT-4, Claude, or Gemini require API-based adaptation (via techniques like instruction tuning through managed services)
- **Architecture maturity, community, and tooling support:** Prefer model families with a strong community ecosystem, mature tooling, and active maintenance. Models integrated into frameworks like Hugging Face Transformers, PyTorch, or vLLM typically offer ready-made support for PEFT methods, optimized inference, and deployment pipelines. Such maturity greatly simplifies fine-tuning, serving, and ongoing monitoring.

With these factors in mind, you can select the most suitable model family for your use case. The next step is to choose the model size and configuration.

Choosing the model size

Once the model family is selected, the next question is: How large should your model be to perform well for a given task? Model size (the number of parameters) strongly affects both capability and cost. Many model families are released in multiple size variants, for example, 1B, 3B, 7B, 13B, or 70B parameters.

In general, larger models possess greater representational capacity: they can store more factual knowledge, capture subtler patterns in data, and perform complex reasoning more effectively. However, this comes at the expense of training time, GPU requirements, and inference time.

For example, small to mid-sized models (from hundreds of millions up to around 7–13B parameters) can typically be fine-tuned efficiently on modern single-GPU or small-cluster setups, making them ideal for enterprise or on-premise scenarios. By contrast, very large models (30B–70B+ parameters) demand distributed infrastructure and often require managed cloud environments, significantly increasing engineering and operational complexity.

It's also important to emphasize that data availability plays an important role when deciding on the appropriate model size. Larger models require proportionally more high-quality training data to reach their full potential. If sufficient data is not available, the additional parameters may lead to overfitting, inefficiency, or wasted compute rather than better performance.

This relationship is formalized in scaling laws, which show that model performance improves predictably only when model size, data volume, and compute budget are scaled together. In practice, if your fine-tuning dataset is small or narrow, using a very large model offers little benefit. A smaller, well-aligned model will often perform better and train more efficiently.

The choice of size also influences the fine-tuning strategy:

- For full fine-tuning, where all model weights are updated, compute cost and memory usage scale roughly linearly with model size
- For PEFT methods such as LoRA or adapters, the size of trainable parameters remains small even for large base models, making adaptation feasible without retraining the entire model

Recent research and industry experience show that smaller, well-aligned models can often match or even surpass larger ones for domain-specific tasks when fine-tuned with high-quality, curated data [45], [46]. For instance, models like Phi-3 Mini (3.8B) and Mistral 7B achieve impressive reasoning and instruction-following performance at a fraction of the size of older 30B+ models, highlighting how modern training techniques and data curation can compensate for reduced scale [47].

The decision about model size should balance the following:

- **Task complexity:** Reasoning vs. simple classification
- **Domain scope:** Broad, open-domain vs. narrow, specialized
- **Data availability** and quality: The better the data, the smaller the dataset size you may need
- **Operational constraints:** Cost, latency, and hardware availability

So, in summary, performance increases predictably with size only when compute and data scale in tandem. Beyond that, returns diminish. Therefore, use empirical evaluation to compare candidate sizes in context, rather than assuming that "bigger is always better."

Selecting the fine-tuning strategy

Once the model family and size are chosen, the next question is how to fine-tune and, more importantly, when each fine-tuning method makes sense. Different strategies are suited for different goals, depending on task type, data scale, and deployment constraints.

Full fine-tuning is best reserved for cases where you need to teach the model something fundamentally new:

- **New capability:** Tasks that require learning skills not present in the base model (e.g., legal reasoning, domain-specific coding languages, or specialized analysis of proprietary documents)
- **New knowledge:** When you have tens of thousands or more well-labeled examples covering the new domain

Use full fine-tuning only if you have strong infrastructure support and clear evidence that simpler methods won't achieve your goals. It provides maximum flexibility but comes with the highest cost.

Another important consideration is the risk of *catastrophic forgetting*. It is when fine-tuning causes the model to lose most of its general knowledge. This tends to happen when the model's capacity is insufficient to integrate new information while preserving prior knowledge. In such cases, the model can effectively become overspecialized, behaving almost like a newly trained model rather than an adapted one.

So, full fine-tuning has become a more specialized approach today, used primarily when deep adaptation is truly required. In most practical scenarios, parameter-efficient methods such as LoRA or adapter tuning are now the preferred choice, offering comparable performance with far lower computational cost and easier deployment. They are ideal when:

- You're adapting an existing model to a specific domain or customer context (e.g., summarizing financial reports, generating marketing copy, or analyzing medical text)
- You have limited labeled data or constrained compute resources
- You need to maintain multiple domain variants of a single base model without duplicating billions of parameters

PEFT preserves the base model's general knowledge while introducing lightweight, modular updates. It's the strategy of choice for most enterprise fine-tuning workflows today, especially when model refreshes or quick iterations are required.

Once a model performs a task correctly but lacks human-like alignment, preference-based fine-tuning is applied. These methods optimize the model against human feedback or preference data instead of explicit task labels.

Use RLHF or DPO when:

- The model already performs the task but needs to be more aligned with user intent or conversational quality
- You aim to enhance helpfulness, safety, politeness, or coherence in generated text
- You're building agentic or customer-facing systems that require nuanced social interaction

RLHF remains standard for high-stakes systems (e.g., commercial chatbots), while DPO is gaining traction for its simplicity and stability. It skips the reward model stage while achieving comparable alignment quality.

Prompting, RAG, or fine-tuning

Now that we understand how to fine-tune models, it's time to compare fine-tuning with two other key strategies to use and adapt LLMs: prompt engineering and **retrieval-augmented generation (RAG)**. The central question is:

Should we rely on powerful general-purpose models like GPT-4 or Claude through prompt engineering alone, or should we go further and modify model weights via fine-tuning?

Of course, this comparison is not perfectly one-to-one. Prompt engineering can and often should be used on top of fine-tuned models. However, once a model is fine-tuned for a specific task or domain, prompts typically become simpler and more deterministic, since much of the

task-specific knowledge is already encoded in the model's weights via training on task-specific datasets.

When comparing these approaches, it's also important to note that the model scale and deployment context often differ:

- Prompting and RAG typically use large, managed models (e.g., GPT-4, Claude 3, Gemini 1.5)
- Fine-tuning is often applied to smaller, open-weight models (e.g., LLaMA-3, Mistral, Phi-3, Gemma) for cost efficiency, control, or privacy

So, let's compare these two approaches using a common LLM with prompt engineering versus an open-source LLM with fine-tuning across several key dimensions:

Aspect	Prompt engineering / RAG	Fine-tuning
Cost	Typically relies on large proprietary LLMs (GPT-4, Claude Sonnet, Gemini), which are expensive per token but require no upfront cost. You pay as you use.	Fine-tuning smaller open models has upfront costs (data collection, training, hosting), but typically lower inference costs later. Over time, especially at scale, this can be notably cheaper.
Development complexity	Very low; requires no ML expertise. Ideal for rapid prototyping and experimentation using prompt iterations or RAG pipelines.	High – requires ML knowledge, data collection, infrastructure setup, and model maintenance. Although using a fine-tuning as a service feature can reduce such complexity.
LLM capabilities	Large general models provide broad factual knowledge and strong reasoning skills (a high number of attention heads). Excellent for open-ended question answering (QA) or complex multi-domain reasoning.	Smaller fine-tuned models have less world knowledge but can outperform larger ones in narrow domains where their training data is focused (e.g., legal, financial, or medical texts).

Aspect	Prompt engineering / RAG	Fine-tuning
Languages	Most large models perform best in English and major Western languages. Low-resource languages often see degraded performance.	Fine-tuning on domain- or language-specific data can improve results for low-resource languages (e.g., Arabic, Thai, Swahili).
Latency	Large managed LLMs are relatively slow and can be unsuitable for low-latency or large-scale real-time workloads.	Smaller fine-tuned models are typically faster, can be hosted locally, and are practical for real-time or edge deployments.
Privacy	Using proprietary LLM APIs requires sending data to a provider's cloud, which may conflict with privacy or compliance requirements.	Fine-tuned open models can be self-hosted in private or on-prem environments, giving full control over data, model weights, and telemetry.
Adaptability	RAG allows dynamic updating of knowledge by retrieving external documents without retraining. Prompts can evolve instantly.	Fine-tuning bakes in knowledge and behavior. It is ideal for stable, repetitive tasks but is less flexible for rapidly changing information.
Maintenance	Easy; prompt updates or retriever changes can be deployed instantly.	Harder; retraining or new fine-tunes are required for behavioral updates or knowledge refreshes.

Table 7.3: A comparison of prompting/RAG and fine-tuning

When we apply managed fine-tuning to commercial LLMs, we combine the best of both worlds: the extensive factual knowledge and reasoning power of large models with task-specific adaptation through fine-tuning. However, this approach still inherits the typical drawbacks of large, hosted models, such as high token costs, inference latency, and limited privacy control.

The interesting aspect is to compare RAG and fine-tuning as two different ways of extending a model's knowledge. Both approaches can introduce new information, but they do so in

fundamentally different ways. Fine-tuning integrates new data into the model's weights, permanently adapting its internal representations and behavior. In contrast, RAG keeps the base model unchanged and augments its responses at inference time by retrieving relevant information from external sources.

In other words, fine-tuning teaches the model new knowledge, while RAG "reminds" it of relevant facts when needed. Fine-tuning is best suited for stable, well-defined knowledge that rarely changes, whereas RAG excels when information must stay current or when data cannot be embedded into the model for privacy or cost reasons.

Roughly speaking, with fine-tuning, we inject new data directly into the model parameters, spending significant compute, and time for this "knowledge injection". Once the model is fine-tuned, however, it can recall and generalize over that information internally, without needing to search or retrieve it at runtime. That often makes inference faster and more self-contained.

In contrast, RAG does not modify the model's weights. Instead, it must retrieve relevant information on demand either through keyword-based (lexical retrieval) or semantic (embedding-based retrieval) search and then feed that context into the LLM for processing. This makes the RAG pipeline more flexible and immediately updatable (new data can be added via reindexing), but also slower and more resource-intensive at inference time due to the retrieval and context-injection steps.

In short, fine-tuning provides permanent knowledge integration but a high update cost, while RAG offers instant knowledge updates but a higher runtime overhead.

So, in summary:

- Prompt engineering and RAG excel when tasks are broad, reasoning-heavy, or require access to dynamic and up-to-date information. They enable rapid iteration, minimal infrastructure, and high adaptability.
- Fine-tuning shines when the domain is well-defined, stable, and data-rich, or when cost, latency, or privacy make API-based models impractical.
- Managed fine-tuning offers a middle ground—combining the power and knowledge of large proprietary LLMs with limited but effective task adaptation.

Ultimately, these methods are complementary, not exclusive: in modern AI systems, prompt design, RAG, and fine-tuning are often used together to solve complex real-world applications like AI agents.

Fine-tuning datasets

As we've emphasized throughout this book, data is the foundation of any AI system. Model architectures and fine-tuning techniques are simply mechanisms for extracting and encoding knowledge from data. If the data is poor in quality, unrepresentative, or mislabeled, no model can produce reliable results.

In fine-tuning, therefore, one of the most important factors is the quality and relevance of your dataset. Before optimizing models or experimenting with tuning methods, the first and most essential step is to curate and validate the right data for your task.

Today, there are numerous high-quality public datasets available for common NLP tasks such as summarization, sentiment analysis, and question answering. Many of these datasets have been mentioned throughout this book as examples when exploring how LLMs are trained and evaluated. They often serve as benchmarks defining canonical AI problems and providing a common ground for comparison and competition between model providers. Examples include datasets like CNN/DailyMail (summarization), SQuAD (question answering), and IMDB or SST-2 (sentiment analysis).

However, when addressing real-world business problems, these public datasets are rarely sufficient or directly applicable. In most enterprise or domain-specific scenarios, the available public datasets:

- Do not reflect private data
- Have already been used to fine-tune existing proprietary or open-source models, meaning similar solutions are often freely available on platforms like Hugging Face or GitHub
- Cannot be used for proprietary applications due to licensing or compliance limitations

As a result, organizations typically need to create or curate their own private fine-tuning datasets to achieve meaningful performance improvements and differentiation.

Building private fine-tuning datasets is a task-specific process and, to some extent, falls outside the full scope of this book. However, it's important to outline the core principles and high-level requirements that define a good dataset for fine-tuning.

Before discussing how to collect and prepare such data, let's clarify what it means to have a good dataset. A well-designed fine-tuning dataset typically exhibits several key characteristics:

- **Diversity:** It should include a wide range of examples that capture the variability of the task: different topics, phrasing styles, document types, and edge cases. Diversity helps the model generalize better and reduces overfitting to narrow patterns.
- **Representativeness:** The dataset should reflect the actual data distribution the model will encounter in real usage. For example, if you are building a financial summarizer, the dataset should include authentic financial reports, not generic news articles.
- **Quality, cleanliness, consistency:** Data should be accurate, consistent, and free from noise such as formatting errors, irrelevant text, or mislabeled samples. Poor data quality directly leads to poor model performance, even if the dataset is large.

In practice, for fine-tuning the quality of the dataset matters far more than its quantity. A smaller but diverse, representative, and well-labeled dataset will almost always outperform a larger one that is noisy or inconsistent.

We must also mention that different fine-tuning strategies require different types of datasets. In practice, most fine-tuning data falls into one of two broad categories: supervised fine-tuning datasets or preference-based datasets.

SFT uses explicitly labeled input–output pairs, where each input example has a desired response or target. These datasets teach the model what to do by directly mapping questions, instructions, or prompts to the expected answers or completions.

Typical examples include:

- **Task-specific datasets:** Examples include classification (text $\rightarrow$ label), summarization (document $\rightarrow$ summary), or translation (text_A $\rightarrow$ text_B)
- **Instruction-following datasets:** For example, "Write a short summary of the following article" $\rightarrow$ [target summary], often formatted in conversational or natural instruction style
- **Domain adaptation datasets:** Using domain-specific text (e.g., financial reports, medical notes) paired with structured outputs such as highlights, sentiment tags, or key entities

SFT datasets are relatively straightforward to construct and interpret, and they form the foundation for most instruction-tuned and chat-style models. Many well-known datasets such as Alpaca, Dolly, and FLAN are examples of this category.

For preference-based fine-tuning, the dataset contains pairs or rankings of model outputs showing which one humans (or an evaluator model) prefer. These datasets are typically used in methods like RLHF or DPO we have explored before. Such a preference dataset might look like this:

```
{
  "prompt": "Explain why the sky appears blue.",
  "chosen": "The sky appears blue because shorter blue wavelengths of sunlight are scattered more by air molecules.",
  "rejected": "The sky is blue because that's how humans perceive it."
}
```

It teaches the model how to respond better. It emphasizes alignment with human judgment, improving attributes like clarity, tone, politeness, factuality, and safety.

In real-world systems, these two dataset types often complement each other:

- SFT datasets establish core task competence
- Preference datasets refine the model's style, helpfulness, and human alignment

Many modern LLM pipelines use a two-stage fine-tuning process:

- SFT to build base task proficiency
- DPO/RLHF to optimize for preference alignment

Now that we understand the principles and approaches for fine-tuning and building effective datasets, we can move on to the practical aspects of hosting fine-tuned models.

Hosting fine-tuned models

So, finally, after completing fine-tuning, we obtain a model that consists of two main components:

- **Code:** Typically, Python scripts using ML frameworks like PyTorch or TensorFlow and configuration files. That code and configurations define the model architecture and inference logic.
- **Model weights:** One or several large binary files (often in formats like .bin, .safetensors, or .pt) containing the learned parameters.

Together, these are all that's required to load the model and run inference.

Conceptually, LLMs function like stateless web services, responding to requests without retaining session state between them. The key difference is that LLMs are computationally heavy, often requiring specialized hardware like GPUs to perform efficient inference.

Running LLMs efficiently depends heavily on the number of GPU vcores, GPU memory (VRAM), system RAM, network, and storage speed. The larger the model (in parameters), the more GPU memory and system resources are needed to load and run it. Here are several approximate setups required to run LLMs efficiently:

Model size	Example models	Typical GPU	GPU VRAM	System RAM	Notes
< 1 B (small)	DistilBERT, TinyLLaMA	can run on CPU		8 GB+	Suitable for lightweight tasks or edge devices.
2–7 B (medium)	Mistral 7B, LLaMA 3 8B, Phi-3 Mini	1 × NVIDIA A100 40GB / RTX 4090 24GB	20–30 GB	32–64 GB	Most common for fine-tuned enterprise models.
13–30 B (large)	LLaMA 2 13B, Gemma 27B	2–4 × A100 80GB or 2 × H100 80GB	60–100 GB	64–128 GB	Requires model sharding across GPUs.
70 B+ (very large)	LLaMA 3 70B, GPT-4-class	8 × A100 80GB (multi-node setup)	300–500 GB+	256 GB+	Demands distributed inference infrastructure.

Table 7.4: Hardware requirements for efficient LLM deployment

So, we can see that for small and medium-sized fine-tuned models (up to roughly 7–13 B parameters), a single high-end GPU machine can usually handle inference well. This setup is common for enterprise or research applications, as these models provide a good balance of quality, cost, and deployability.

However, for very large models (tens or hundreds of billions of parameters), hosting requires distributed GPU clusters with model parallelism or inference orchestration frameworks such as vLLM, DeepSpeed-Inference, or **Text Generation Inference (TGI)**.

Because LLMs are stateless services, they can be scaled horizontally: multiple replicas can serve inference requests behind a load balancer, similar to RESTful APIs. For small models, this scaling can be achieved using standard web infrastructure patterns (containerization, autoscaling, caching). For very large models, scaling typically involves advanced orchestration and specialized GPU schedulers.

Fine-tuned models can be hosted either:

- **On-premise:** For maximum privacy and control, though it requires expertise to maintain GPU hardware and high-availability clusters
- **In the cloud:** Using platforms like Microsoft Azure, AWS, or **Google Cloud Platform (GCP)**, which offer managed GPU instances, auto-scaling, and monitoring

For production workloads, cloud deployment is typically preferred, as it simplifies scaling, reliability, and monitoring. On-premises setups are more common in regulated industries (e.g., finance, healthcare) where data residency or compliance policies prohibit cloud use.

Summary

Great, we've completed the fine-tuning chapter. Now we understand what fine-tuning is, how it works, and when it should be applied. Fine-tuning allows us to take a pretrained model and adapt it to a specific task, style, or domain by retraining it on new data. Unlike prompting or retrieval-based methods, fine-tuning changes the model itself, embedding new knowledge and behavior directly into its parameters.

We learned that choosing the right model and fine-tuning strategy is a balance between capacity, cost, and purpose. The decision depends on task complexity, domain scope, available data, and infrastructure constraints. Larger models offer richer reasoning and factual knowledge but come with higher cost and latency. Smaller fine-tuned models, on the other hand, can achieve good performance when trained on high-quality, domain-specific datasets and deployed on modest hardware.

The central topic of this chapter is that data quality is everything. Architecture and training methods only determine how knowledge is extracted; the real substance comes from the data itself. A good fine-tuning dataset must be diverse, representative, consistent, and clean.

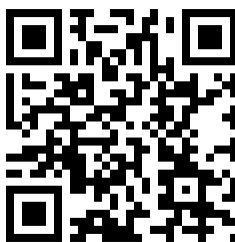
Finally, we explored how to host fine-tuned models in practice. Even modestly sized models often require GPUs to run efficiently, while very large ones need distributed infrastructure and careful scaling.

Today, fine-tuning together with RAG and prompting remains one of the most effective ways to turn general-purpose LLMs into specialized, high-performing systems.

In the next chapter, we introduce AI agents, focusing on their architecture, core components, and how they plan, reason, use memory, and interact with tools and multi-agent systems.

Get this book's PDF version and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.

Part 3

Designing, Building, and Evaluating AI Agents

Part 3 brings together the concepts introduced earlier and applies them to the design and development of agentic systems. It begins by exploring the architecture of AI agents, including core components such as memory, tools, planning, reasoning, and orchestration. It then moves on to practical approaches for building agents and agent-based workflows. The book concludes with an evaluation, showing how to assess LLM applications and agents in practice using the right metrics, datasets, and testing strategies. By the end of this part, you will have a clear picture of how modern AI agents are structured, how they can be built, and how to evaluate them responsibly and effectively.

This part contains the following chapter:

- *Chapter 8, Exploring the Architecture of AI Agents*
- *Chapter 9, Building AI Agents*
- *Chapter 10, Evaluating LLM Applications and Agents*

8

Exploring the Architecture of AI Agents

At this point, we have built a strong foundation in **artificial intelligence (AI)**, **natural language processing (NLP)**, and **large language models (LLMs)**. You've seen how LLMs work under the hood and how to apply them to a wide range of real-world tasks using prompt engineering, **retrieval-augmented generation (RAG)**, and fine-tuning. Now we move to one of the most important and rapidly evolving topics in modern AI: AI agents.

The era of AI agents is just beginning. We are entering a phase where software no longer simply responds to user input but can act on behalf of the user. These systems can plan, make decisions, execute multi-step workflows, interact with external tools, and coordinate with other agents. In many cases, they can handle tasks that previously required continuous human involvement.

In this chapter, we focus on the architecture of AI agents. You will learn what an agent is, how it differs from traditional AI applications, and what core components define modern agent systems. We will break down how agents reason, use memory, interact with tools, and operate within larger workflows, including multi-agent environments. We will cover the following key topics:

- Agent basics
- Agent memory
- Tools and orchestration
- Multi-agent flows

Technical requirements

The code for this chapter is available on the book's accompanying GitHub repo: <https://github.com/PacktPublishing/Building-Agent-Powered-Applications/tree/main/Chapter8>

Agent basics

The idea of creating autonomous entities capable of performing tasks on behalf of humans has been a long-standing dream throughout history. From ancient myths of mechanical beings to the early concepts of automatons, humanity has always envisioned artificial agents that could think, act, and work independently. Today, modern LLMs bring us closer to this vision, enabling machines to reason, make decisions, and interact dynamically.

Agent terminology

First, let's define the term agent. In AI and robotics, an **agent** typically refers to an *autonomous* system that operates within an environment, processes information from inputs, and takes actions to achieve specific goals. Many agents also possess *learning* and *adaptability*, allowing them to adjust their behavior over time.

Note

The concept of AI agents emerged in the 1950s–1960s with early work on **multi-agent systems (MAS)** [48]. By the 1970s–1980s, game theory introduced *rational agents* [49], while **reinforcement learning (RL)** formalized *agent-based learning*, and robotics advanced *embedded agents*. Modern AI agents now power self-driving cars, virtual assistants (Siri, Google Assistant), and game AI (AlphaGo, OpenAI Five). So, the agent concept is not new, has evolved over time, and now has a new shape and form.

Today, agents already exist in many forms. For example, physical robots operate in the real world, interacting with their surroundings through sensors and actuators. Similarly, virtual agents such as game bots operate within digital or simulated environments.

The agent environment includes everything the agent can perceive and interact with. An agent maintains its state, which includes the information it observes. It also follows a **policy** – a strategy that determines its next action based on that state or observation. Each agent action generates feedback (or reward), a signal indicating its success or effectiveness, which the agent uses to adapt and improve future behavior.

The following diagram presents a simplified view of an agent–environment loop, highlighting how the agent receives observations, updates its state, applies a policy, and takes actions that impact the environment:

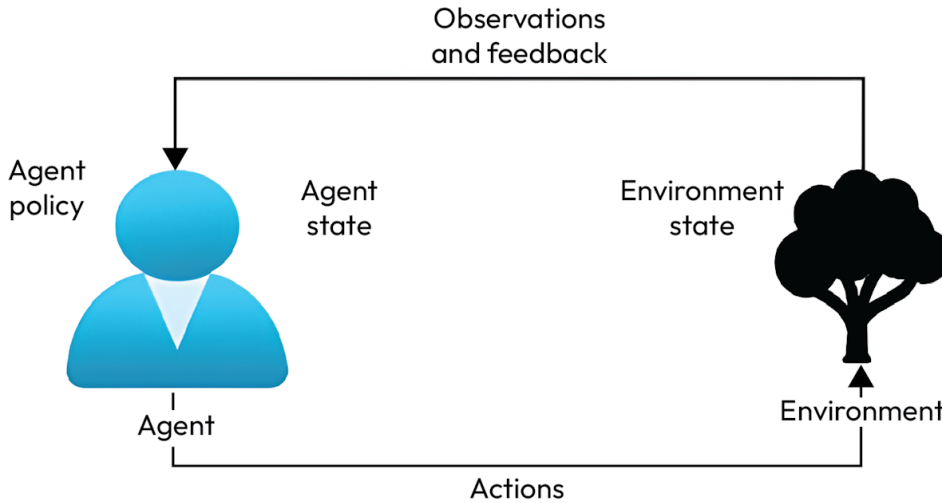


Figure 8.1: The agent and its environment

This abstraction applies to all known types of agents:

- Physical robots operate in the physical world, receiving sensory inputs such as camera images, LiDAR data, or touch sensors, and producing actions like movement, grasping, or speech.
- Game agents function within virtual environments, perceiving the game state such as object positions, scores, or player actions. The bot makes decisions to navigate, attack, or strategize based on predefined or learned policies.
- Software agents, including modern LLM-powered agents, act within digital ecosystems, observing inputs such as user queries, system data, or API responses, and taking actions like generating text, querying databases, or invoking external tools.

Despite these differences, all agents share the same foundational loop of perception → decision → action → feedback, continuously interacting with their environment to pursue their goals.

The next term often associated with agents is **assistant**, which is sometimes used interchangeably. Today, the difference between these terms can be blurred, but assistants are still typically more *reactive* rather than proactive. They mainly act more as interfaces, retrieving predefined information, responding to user requests, and performing tasks within limited boundaries.

Assistants are primarily triggered by humans. For instance, a user asks a question in a chat or requests to create an appointment with a doctor via phone. In contrast, agents can be triggered by external events, such as stock price changes, system alerts, datetime changes, or requests from other agents. However, agents can also be initiated by humans when needed, making them more flexible in their operation. That's why agents and assistants are often used interchangeably.

In this book, for simplicity, we will use the term *agents* as the primary term, also embracing assistants as a simplified version of agents. So, the assistant will be considered as agents but with some limited functionality, less autonomous entities.

Another term closely associated with modern AI assistants and agents is *Copilot*, popularized by Microsoft as a brand for AI-powered helpers integrated into its products. A **Copilot** can be considered a type of agent or assistant that works within a specific application or platform to augment user capabilities. Examples include Microsoft 365 Copilot, GitHub Copilot, Dynamics 365 Copilots, and others. These copilots can help you get insights from the last meeting, draft a document, summarize an email, or assist in coding.

Another dimension of the term agent is the level at which it operates:

- **Business-level agents:** Top-level entities that interact with humans or other business-level agents, delivering tangible business value by executing end-to-end workflows
- **Low-level (code-level) agents:** Lower-level entities that operate within software systems, performing specialized tasks that support business-level agents

Business-level agents are designed to deliver complete business functions. They can be purely software-based or embodied in hardware powered by software (like Boston Dynamics robots). Examples include:

- **Personal travel planning agent:** Assists users in planning and booking trips
- **Sales leads generation agent:** Identifies and qualifies potential customers (leads)
- **Autonomous inspection robot:** Navigates industrial facilities, gathers sensor data, and reports safety or maintenance issues

Low-level or code-level agents operate at the software component level, handling discrete, specialized tasks. They are typically invisible to end users but work as intelligent building blocks that enable business-level agents to function effectively.

One business-level agent can be implemented as several (multi-agent) low-level agents. Low-level agents can be shared among business-level ones. So, low-level or code-level agents can be considered as components of business-level agents.

For instance, a personal travel planning agent can be powered by multiple code-level agents, each specializing in a specific part of the workflow:

- **Travel planner agent:** Proposes a complete travel itinerary based on user preferences
- **Travel operations agent:** Executes bookings for flights, hotels, and activities
- **Travel supervisor agent:** Monitors and coordinates the work of the planner and operations agents, ensuring consistency and handling conflicts

Business-level and low-level agents differ in scope and function, but they share one fundamental trait. They interact dynamically rather than through fixed, code-defined logic or workflows. This flexibility allows them to adapt and collaborate in solving ambiguous or evolving problems.

A good analogy of business-level and low-level (code-level) agents is organizations/companies and their employees. A company is like a business-level agent that solves top-level problems like building houses, developing software, manufacturing equipment, and so on. Low-level or code-level agents are like employees who handle lower-level tasks within the company. Usually, employees communicate and interact with each other to solve big business problems, and they also operate at some level of autonomy. Employees develop the most optimal strategy, perform discussions, execute those strategies, and so on. Similarly, code-level agents dynamically communicate with each other, develop execution plans, and proceed with it.

While discussing agent terminology and diving deeper, it's also important to cover their capabilities—what an agent can actually do. These capabilities are often referred to as *skills*.

A **skill** is a distinct, modular ability that an agent can use to achieve a goal or perform a task within its environment. In the physical world, a robot might have skills such as navigating to a location, picking up an object, or moving it to a target area. In the digital world, a software agent might have skills such as searching a database, retrieving relevant documents, analyzing data, sending an email, or updating a record in a business system.

Skills can be *built-in* (preprogrammed functions), *learned* (acquired through training or adaptation), or *integrated* (connected via APIs or external tools). They can be quite straightforward and not intelligent, like calling an API and making a turn in a virtual game environment, or more advanced, like analyzing a financial report and monitoring financial transactions.

These are the key definitions of agents and related concepts. The field of AI is evolving rapidly, and the meaning and scope of some of these terms may continue to shift over time. With this foundation in place, let's take a closer look at LLM-based agents, which are the primary focus of this book.

LLM agent architecture

LLM agents or **LLM-powered agents** are a type of agent where a large language model serves as the "brain" handling reasoning, planning, and decision-making. LLM agents extend the capabilities of a standalone LLM by integrating with additional components and tools. That enables the agent to call external systems, retrieve up-to-date or private information, execute actions, run generated code, and leverage history or user feedback to improve future interactions.

These capabilities transform an LLM from a passive text generator into an active problem solver that can interact with the world and adapt to evolving contexts.

An LLM-based agent typically includes the following primary components:

- **Memory:** The agent's state: all information it retains to operate effectively
- **External tools:** Capabilities that extend the agent beyond the LLM itself
- **LLM:** The core and engine for reasoning and decision-making

The following diagram illustrates the high-level architecture of an LLM-based agent, showing how the LLM interacts with memory, external tools, and the user:

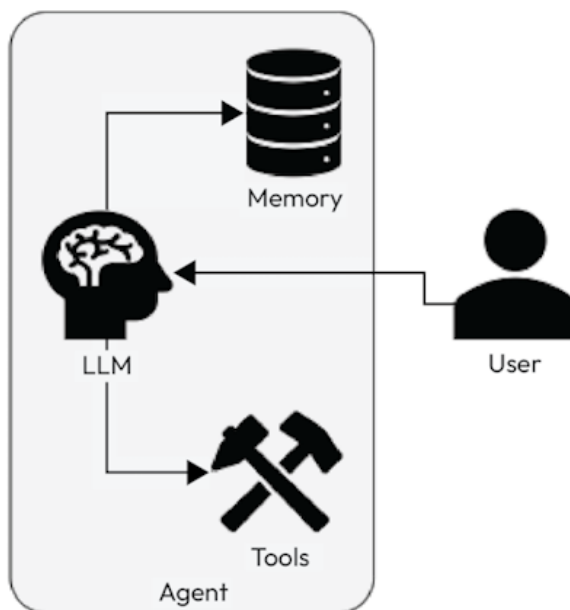


Figure 8.2: LLM-based agent components: LLM, memory and tools

LLM agents, like other types of agents, operate in a specific environment, maintain their own state, and perform actions. Just like a game bot, a warehouse robot, or an autonomous drone, an LLM agent perceives information (observations), decides what to do next based on its policy and state, executes actions, and processes the resulting feedback to improve future behavior.

In this chapter, when we use the term agent, we'll be referring specifically to LLM-based agents rather than other types of AI agents unless explicitly mentioned.

Conversational agents

Conversational agents interact with users via dialog-based interfaces like chats, emails, and voices. Chat GPT website (chatgpt.com), Google Gemini chat (gemini.google.com), and Microsoft 365 Copilot are examples of conversational agents (or assistants). Sometimes, conversation agents are also called *communication agents/assistants*.

One of the core functions of typical conversational agents is to maintain multi-turn dialogues, remembering previous conversations in order to provide context-aware interactions. A multi-turn conversation is one in which the exchange involves more than a single question-and-answer pair. For example, a user might first ask one question, then follow up with a second question that builds on information from the earlier answer.

From a technical perspective, conversational agents (or assistants) are often framed as the implementation of **question answering (QA)** problems (see *Chapter 5*), but with important extensions. Unlike traditional single-turn QA systems we explored before, they often must handle context retention, coreference resolution, clarification questions, and dynamic retrieval of information. In other words, they combine the core principles of QA with additional capabilities for memory, reasoning, and interactive information gathering.

Such capabilities of conversational agents make memory the critical component since LLMs don't "remember" what was sent in previous requests (see *Chapter 2*).

The memory can be used to solve the following tasks:

- **Follow-ups:** Answer follow-up questions when new ones are about some aspects of previous questions and answers
- **Ambiguity:** Resolve ambiguity when the question has some hidden context not mentioned in the question but is implied implicitly from a previous conversation
- **Personalization:** Adjust responses based on personal preferences obtained from previous conversations

Conversation agents can also use external tools to enhance reasoning capabilities, such as calling web search engines (Google Search, Microsoft Bing, etc.) or generating code (Python) and executing it in special environments, returning results to the user.

Background agents

Unlike conversational agents, **background agents** operate behind the scenes and do not interact directly with humans. They are triggered by specific events such as receiving an email, detecting a file change, or reaching a scheduled time. Software engineers have long been familiar with similar concepts in the form of background processes, cron jobs, and event-driven services. In this sense, background agents function much like traditional software automation, but with added intelligence and decision-making capabilities.

If the agent is going to be deployed in the cloud (AWS, Azure, Google Cloud), it can be triggered by various cloud-based events, such as blob storage changes, database updates, message queue events, API calls, or scheduled tasks.

We can call such events *basic infrastructure* events. They are triggered by low-level system-generated triggers. These events serve as the foundation for more complex, *business-level events*. By leveraging infrastructure events, we can construct agents that respond to higher-level business triggers such as stock price changes, weather updates, major customer transactions, inventory fluctuations, or social media trends.

For example, we can have a stock monitoring agent that will be triggered every minute (timer trigger) and check stock prices. If the price changes dramatically, the agent can start analyzing news and social media, making notifications about the proper situation. So, here we used an infrastructure-level event (timer trigger) to build a business-level event (stock prices change).

We can also notice that traditional automation is also triggered by external events and operates in the background. Given this, a key question arises:

What differentiates traditional automation from AI-driven agents?

While both rely on event triggers, AI agents introduce a new level of adaptability, reasoning, and contextual awareness. Unlike conventional automation, which follows predefined rules and workflows, AI agents can dynamically interpret inputs, learn from interactions, and make autonomous decisions based on complex reasoning. This distinction makes AI agents far more flexible, enabling them to handle complex tasks, engage in multi-turn decision-making, and adapt to changing environments in ways that traditional automation doesn't.

So, in short, background agents are "smart automation," more adaptive and more learnable.

We've now defined conversational and background agents. In practice, however, many agents are hybrid and capable of working in the background to complete tasks while also initiating or responding to user interactions when needed. A good analogy is an employee who can work independently but can also raise questions to their manager when necessary or respond promptly to any inquiries.

We can expect that many modern business-level agents will follow this hybrid model, combining an interactive interface with the ability to operate autonomously in the background, executing business tasks without constant human oversight.

LLM agent examples

LLM agents can tackle a wide variety of problems because they operate at the *language level*. A significant portion of human activity can be modeled using language. Moreover, the latest foundation models can process multiple modalities (see *Chapter 2*), including images, audio (speech), and video, which makes them even more versatile in real-world scenarios.

Because of this, LLM-powered agentic flows can be applied across many domains. Below, we explore just a few examples.

Customer support agent

The goal of this agent is to respond to and resolve customer inquiries. It can be triggered when a customer submits a chat question, sends an email to support, opens a ticket on a helpdesk platform such as Microsoft Dynamics or Zendesk, or makes a phone call to customer support.

This type of agent can use the LLM's factual knowledge base, extended by the company's own support data via RAG, to provide accurate, context-aware assistance. It can also integrate with company APIs to resolve certain issues automatically, either on its own or with human confirmation. Over time, the agent can learn and adapt, refining its behavior based on previous experience and aggregated insights for similar issues.

Lead generation agent

For many businesses, finding and qualifying potential customers is one of the most time-consuming and repetitive operations. A lead generation agent can streamline this process by conducting research, generating outreach emails, initiating contact with potential customers, and answering initial inquiries.

It can create appropriate records (such as leads or opportunities) in a CRM (customer relationship management) system and hand them over to sales representatives to finalize deals. In some cases, the agent can close deals by itself, involving human input only in critical situations. Over time, it can improve its strategies for identifying new customers and re-engaging existing ones by learning from previous interactions.

Personal email filtering agent

This agent reads and analyzes incoming emails to categorize and prioritize them automatically. If a message is urgent, it immediately escalates to the user; if it is unimportant, it is ignored; and if it is important but not urgent, the agent summarizes it and includes the summary in a daily activity digest. When emails contain open questions or require clarification, the agent can initiate a follow-up dialogue to gather the necessary information.

By categorizing, prioritizing, and summarizing messages, the agent improves the efficiency of personal email management, reduces cognitive load, ensures urgent matters are addressed promptly, and streamlines daily communication without requiring constant manual inbox monitoring.

Chat GPT, Gemini Chat, and Microsoft 365 Copilot

Although they may appear to be simple interfaces for making LLM calls, websites such as ChatGPT (chatgpt.com), Gemini (gemini.google.com), and Microsoft 365 Copilot (copilot.microsoft.com) operate as full-fledged conversational agents. They not only generate responses but also manage conversation history across turns, invoke external tools such as web search or code execution, and provide advanced features like deep search, file uploads, and integration with business data.

As we've seen, these are only a few examples of what LLM-based agents can do. The possibilities extend far beyond, with agents designed to serve both individual users and businesses in countless ways. Moreover, agents are not limited to working in isolation. They can communicate and collaborate with one another, combining their skills to solve complex problems and complete multi-step tasks that span different domains. This cooperative capability opens the door to highly adaptive, distributed systems capable of tackling challenges that would be difficult or impossible for a single agent to handle alone.

Now let's shift from examples to the core agent components, such as memory and tools.

Agent memory

As we discussed in *Chapter 2*, LLMs are stateless*. They don't retain information between requests. If you ask an LLM (not Chat GPT or Microsoft Copilot) for a recipe for pho soup and then, in a next request, ask it to "give me a shopping list for that recipe", the model won't remember what "that recipe" refers to. The information isn't stored inside the LLM itself. To overcome this limitation, we need to augment LLMs with memory by providing past interactions back into the model's context window.

Note

* - an exception to this statement is real-time **speech-to-speech (S2S)** models, which are stateful by design. They continuously process incoming speech and generate spoken responses in real time. One example is OpenAI's GPT-Realtime models.

Memory can be considered an additional *layer of context* we augment to the LLM, enabling it to operate not only on user input, the model's internal factual knowledge and its reasoning capabilities, but also on persistent information from prior interactions.

This idea is conceptually similar to RAG we explored in *Chapter 6*, where the LLM is provided with external documents or knowledge snippets retrieved from a database, vector store, or web search. In both cases, we are enriching the model's prompt with information it did not retain internally.

However, there are important differences between memory and RAG:

- **Source of augmentation:**
 - RAG retrieves external knowledge (e.g., articles, papers, structured databases)
 - Memory recalls interaction history or agent state (e.g., past questions, answers, decisions, preferences)
- **Purpose:**
 - RAG aims to ground responses in factual, external sources to improve accuracy and reduce hallucinations
 - Memory aims to maintain continuity, personalization, and statefulness across interactions
- **Update mechanism:**
 - RAG relies on indexing and retrieval pipelines that are usually external to the model and updated asynchronously.
 - Memory relies on runtime updates. New interactions are appended, summarized, or transformed into structured memory objects as the agent runs.

Agent memory can take different forms:

- **Short-term memory (STM):** Retains recent information within a single session
- **Long-term memory (LTM):** Stores persistent knowledge over time, including user preferences, domain-specific facts, structured information, or historical records

We can make an analogy here: STM is like RAM, which holds information temporarily for quick access while a program is running, whereas LTM is like disk storage, where information persists and can be recalled long after the immediate session has ended. Just as computers rely on both RAM and disk to function effectively, agents need both STM and LTM to combine responsiveness with lasting knowledge.

Short-term memory

Short-term memory (STM) refers to the ability to temporarily store and recall recent interactions within a session or task. A session, for conversational agents, typically refers to a single conversation or topic. For background agents, a session is often bounded by a single task or workflow execution.

STM often plays a critical role for conversational agents, where efficient multi-turn conversation is just impossible without such memory. It allows answer follow-ups, resolves coreferences (see *Chapter 5*), and ambiguity, incorporating hidden context and applying personalization within the current interaction.

STM is equally important for background agents. Here, this memory can store the agent's current task-related information and agent state. For example, a background scheduling agent must keep track of the meeting it is currently organizing, the deadlines it is monitoring, or the files it is processing.

The very basic STM implementation for a conversational agent can be just storing all user questions and LLM answers in a list and appending them to every new LLM request, such as the following:

```
class SimpleSTM:
    def __init__(self):
        # Stores conversation history as (user, bot) pairs
        self.memory = []
    def add_interaction(self, user_input, llm_response):
        """Stores user question and LLM response in memory."""
        self.memory.append((user_input, llm_response))
    def get_context(self):
        """Formats stored memory as conversation context for LLM."""
        return "\n".join([f"User: {q}\nLLM: {a}" for q, a in self.memory])

# Example Usage
stm = SimpleSTM()
stm.add_interaction("What is AI?", "AI stands for Artificial Intelligence.")
stm.add_interaction("How does machine learning work?", "Machine learning is a
subset of AI.")
context = stm.get_context()
print(context)
```

In this simple case, memory is represented as a collection of pairs containing user inputs (*questions*) and the corresponding LLM responses (*answers*). In other scenarios, however, memory can take the form of a collection of objects with more complex or varied structures.

A simple implementation like the one above can be used in practice. However, as we can predict, it doesn't address several important aspects that arise when deploying agents at scale or in more complex real-world scenarios.

- LLM context window and efficiency
- Context relevance

Let's now examine these aspects in more detail

LLM context window and efficiency

As we learned in *Chapter 2*, every LLM has a limited effective context window. Also, the larger the memory we carry forward, the heavier every single LLM interaction becomes. With the simple "append everything" strategy, the context grows dramatically with every new user interaction.

This has two important consequences:

- **Cost:** The total number of tokens sent to the LLM increases quickly
- **Latency:** Larger prompts take longer to process, making the agent slower to respond and operate

In other words, while naive appending ensures nothing is forgotten, it does so at the expense of efficiency and costs. Without some form of memory management strategy, agents risk becoming prohibitively slow, fragile, and costly to operate at scale.

There are a few strategies for solving such problems:

- Cutting memory
- Summarize memory
- Advanced hybrid approaches

Let's explore these in detail.

Cutting memory

Cutting memory can be done in two ways:

- Cutting by token size
- Cutting by natural breaks

Cutting strictly by token size is the most straightforward approach. The idea is simple: once the accumulated conversation exceeds a certain token threshold, the oldest parts are discarded.

This ensures the context window never overflows.

In practice, such thresholds are usually set based on experimentation with specific models, balancing context retention against cost and latency. As we learned in *Chapter 2*, today the thresholds of 8K–16K tokens are generally considered safe for many LLMs.

To apply this method, we first need a way to measure or estimate token usage. We can do this using the following:

- **Tokenizer libraries:** Each model has its own tokenizer (e.g., tiktoken for OpenAI models, sentencepiece for LLaMA-family models). These provide exact token counts.
- **Word-based approximation:** For rough estimates, a heuristic like $\text{tokens} \approx 1.3 * \text{words}$ works reasonably well for natural text. For structured formats such as JSON or code, the coefficient is typically higher (1.5–2.0×) because of the heavy use of braces, punctuation, and formatting symbols.

Cutting memory at an exact token boundary can be problematic. It can often split sentences or even words unnaturally. This can confuse the model. A more robust approach is to trim *at natural boundaries* such as sentence ends or paragraph breaks (see *Chapter 4*).

The main downside of cutting is that important context can be lost. Once truncated, details of earlier interactions are gone forever. Therefore, cutting works best when:

- Most conversations are naturally short and fit within the effective context window
- Occasional loss of older interactions is acceptable
- The priority is safeguarding requests against exceeding token limits rather than optimizing for information preservation

In such cases, cuts act as a protective measure, ensuring that the agent remains operational and does not fail or degrade in performance due to context overflow.

Summarizing memory

Another approach to overcome effective context window limitations is to summarize interactions. For example, if a typical memory object (a user question plus an agent answer) contains 1,000 tokens, and we can summarize it down to 300–350 tokens, then we can fit roughly three times as much history into the same context window.

As discussed in *Chapter 4*, summarization is inherently *lossy*. Summaries capture the essence of the original text while omitting details deemed less important. This leads to the first risk of summarization: *loss of critical context*.

If the agent operates in a specialized domain (e.g., medicine, law, engineering), what counts as "important" can often be predefined more reliably. For example, always retain dosage values in medical instructions or contract clauses in legal texts.

Another open question is the *target summary size*. Smaller summaries compress more aggressively, allowing longer sessions to fit in memory, but at the cost of losing fine-grained detail. Larger summaries preserve more nuance but consume more tokens, reducing overall efficiency. The trade-off must be tuned based on application needs: customer support might favor efficiency, while medical or financial advice must err on the side of precision.

If we treat each memory object (e.g., a user query and agent reply pair) as a document, then summarizing a session becomes a *multi-document* summarization problem. We examined those types of problems in *Chapter 4*.

Finally, even with summarization, limits remain. Compressing memory 3–5x may significantly delay hitting the ceiling, but eventually, long enough interactions will still exceed the effective context window.

Advanced hybrid approach

As we have seen, both cutting by token size and summarization have their own limitations. Cutting is simple but risks losing critical information, while summarization preserves content but can distort or omit details depending on how "importance" is defined. For this reason, many real-world systems adopt hybrid approaches that combine the two strategies.

A key insight from research on long-context use is:

In multi-turn conversations, the most recent interactions carry the highest priority.

This is sometimes referred to as the **recency effect** or **recency bias**: humans and models alike tend to weigh the latest exchanges more heavily than earlier ones. Because of this, a practical hybrid strategy is to keep recent dialog turns intact, while progressively summarizing and eventually trimming older ones.

So, the process of managing STM can look like this:

- *Preserve the last N turns* (e.g., 5–20) exactly as they occurred and include them directly in the LLM context.
- *Summarize older turns* into concise representations. The agent stores short summaries that capture the essence of past discussions.
- *Apply truncation to very old history* once it exceeds a predefined size.

The following diagram illustrates this hybrid strategy to manage STM:

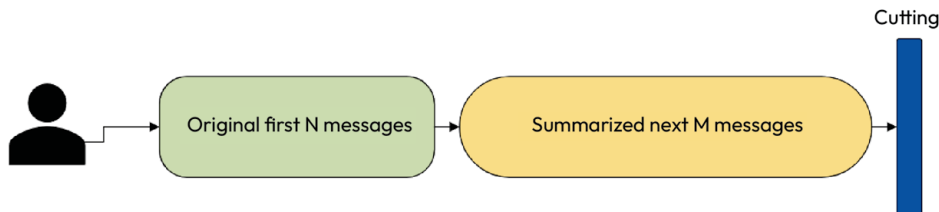


Figure 8.3: Hybrid memory pipeline with summarization and truncation

This hybrid approach combines the strengths of both strategies and establishes a memory pipeline, where messages naturally transition from raw interaction → to summarized form → to eventual trimming if they become too old.

To implement such a hybrid memory strategy effectively, we need to define several key parameters:

- *How much original context to preserve?* Decide the number of most recent messages (or total token budget) to keep intact. In practice, the last 5–10 turns of dialog are considered effective. Alternatively, a fixed token size can be used.
- *How much context to summarize before cutting?* Define the window of memory that should be summarized instead of being stored raw. For example, conversations older than 10 turns but newer than 50 turns must be summarized.
- *Which summarization strategy to use,* what summarization parameters to use: summary length, summarization focus (what is important), and multi-document summarization strategies (see *Chapter 4*).

It is important to note that the recency effect does not always apply. While conversational agents naturally prioritize the latest exchanges to maintain coherence, background agents often operate under very different conditions.

This highlights an important design principle: while conversational agents can benefit from recency-focused hybrid memory, background agents may require special handling.

Context relevance

Another major challenge for conversational agents is ensuring that *only relevant* past interactions are carried forward into the context. When a user changes topics mid-session, older exchanges may become irrelevant but still be included in the LLM's prompt. This leads to two key problems:

- **Quality degradation:** Irrelevant or outdated context can bias the model's reasoning and produce incorrect or confusing replies
- **Efficiency and cost impact:** Supplying unnecessary context inflates the token count, which increases computational cost and latency without adding value

For example, consider the following chat interaction:

- **User:** *"What's the best way to reduce cholesterol?"*
- **Agent:** *"A heart-healthy diet with more vegetables, lean proteins, and less saturated fat is usually recommended. Regular exercise also helps"*
- **User:** *"Are there specific foods I should avoid?"*
- **Agent:** *"Yes — foods high in saturated fat, such as processed meats, fried foods, and full-fat dairy, can raise cholesterol levels"*

Now the user changes topic abruptly:

- **User:** *"Can you help me plan a trip to Italy?"*

Because the memory still contains references to cholesterol and diet, the LLM might respond like this:

- **Agent:** *"Sure, I can plan your trip to Italy and provide suggestions for a healthy meal plan for your trip to avoid fried foods and high-fat cheese"*

This mixes the previous context (health/diet) with the new context (travel planning), producing an irrelevant and slightly confusing answer. In some specific areas, such a situation can lead to completely incorrect answers. For example, if we have a financial agent and the user is talking in the context of one financial document, and then the user starts asking about some other document, the numbers from the previous document can come into the answer, making it completely incorrect.

One of the approaches to improve relevance is to use **topic-aware memory (TAM)**. This is a strategy where every piece of stored content (like a user's question or conversation snippet) is tagged with one or more topics. Imagine you have tags like "travel," "finance," "gaming," and so on. The agent can refer to these tags to quickly filter which parts of the memory are relevant when a new question or request comes in.

To implement TAM, the following steps are typically required:

1. *Classify* each conversation or message into one or more topics
2. *Group* stored content by topic
3. *Retrieve* only the subset of memory entries relevant to the new request

This process requires a text classification component (see *Chapter 4*). In practice, both multi-class classification (one label per memory item) and multi-label classification (multiple labels per item) may be used.

Once the classification mechanism is in place, the next key design decision is topic granularity. We need to define how broad or specific each category should be:

- **Broad topics** (e.g., Finance, Sports, Education) reduce complexity but may group together too many unrelated conversations
- **Fine-grained topics** (e.g., Corporate Finance, Personal Finance, Sports Analytics) increase precision but create more categories, each with fewer items

The right level of granularity depends on the agent's domain. A customer support bot may benefit from fine-grained, product-level tags, whereas a general-purpose assistant may prefer broader groupings to avoid fragmentation.

Another fact we can notice is that conversations often span multiple domains. For example, a question "*What's the best way to pay for a flight to Alaska in euros?*" crosses both travel and finance. In such cases, multi-label tagging is needed. A single memory entry can be tagged with multiple topics, and retrieval can merge overlapping groups so that relevant details are not lost. So, in this case, we will need to perform multi-label classification.

As we mentioned already before, conversations can also evolve over time. To address these *topic shifts*, a *topic drift classifier* can be introduced to monitor whether the user's latest query remains aligned with the current session's theme. When a significant drift is detected, the agent can automatically start a new session instead of carrying forward irrelevant past interactions.

This approach offers several benefits: clearer context boundaries, higher efficiency, and an overall better user experience. The agent automatically starts a new session whenever the topic shifts, keeping interactions focused and relevant. However, this method relies on having a high-quality topic classifier. Errors can result in important context being lost.

In the GitHub materials accompanying this book, we provide an STM implementation that combines both efficient use of the LLM's context window and context relevance filtering. We encourage you to explore and experiment with this implementation as a practical foundation for your own systems.

Long-term memory

Long-term memory (LTM) in AI agents enables knowledge retention beyond a single session, supporting persistent context, user preference tracking, and adaptive learning over time.

LTM requires persistent storage, such as databases or external storage systems, to maintain data for extended periods. Because memory often has a dynamic and heterogeneous structure, document-oriented databases are frequently a practical choice (e.g., MongoDB, Azure Cosmos DB, Amazon DynamoDB). However, relational databases such as PostgreSQL, MySQL, or SQL Server can also be used.

For conversational agents, LTM provides continuity by remembering user preferences, prior interactions, and contextual facts, thereby enabling more personalized, coherent, and human-like conversations across sessions. For background agents, LTM is also important. It can store historical states, logs, and execution traces that allow agents to make decisions informed by long-term patterns.

Additionally, LTM can be used to support adaptive learning. By leveraging stored interactions, agents can gradually refine their knowledge, recognize user-specific patterns, and adjust behavior without retraining (*fine-tuning*) the base model. This adaptation can be scoped either per-user (*personalized memory*) or globally (*collective learning across many users*).

Some common items typically stored in LTM include:

- User preferences
- Profile data
- Previous conversations

Let's consider these in detail.

User preferences

User preferences can be thought of as a structured set of attributes extracted from conversations or background tasks that help personalize future interactions. For example, in the case of an email draft generation agent (such as Microsoft Outlook Copilot), which produces drafts based on user prompts and instructions, user preferences might include:

Preference	Description	Example value
Preferred email tone	The tone of emails the user prefers	"Formal", "Casual", "Friendly", "Professional"
Signature style	Custom email signature format	"Best Regards, [Name]"

Preference	Description	Example value
Default greeting	Preferred way to start an email	"Hi [Name]", "Hello [Name]", "Dear [Name]"
Closing preference	How the user prefers to end emails	"Thanks", "Sincerely", "Best wishes"
Frequently used phrases	Repeated phrases in emails	"Let me know if you have any questions."

Table 8.1: Example of a user preferences table

So, an email draft generation agent can automatically extract user preferences from previously written emails, store them in persistent storage, and then reuse this information for future draft generation. Over time, this creates a personalized memory of how the user prefers to communicate.

Extracting such preferences can be framed as either:

- **Classification tasks** (see *Chapter 4*): Assigning predefined labels to user emails, such as "formal" vs. "casual" tone, or "short" vs. "detailed" style.
- **Information extraction tasks** (see *Chapter 4*): Identifying specific patterns or attributes, like signature format, common greetings, or preferred structuring (e.g., bullet points vs. prose).

For instance, our email draft generation agent noticed the following:

- A user consistently corrects generated draft emails with endings like "Cheers, Alex." The agent can detect and store this signature style in long-term memory. Next time, when drafting an email, the system will automatically apply this closing without being asked.
- If the user often rephrases generated drafts to shorten them, the agent can classify this as a "conciseness preference" and adapt future drafts accordingly.

Profile data

Profile data in long-term memory refers to structured, persistent, user-specific information that an AI agent stores to enable personalized and adaptive interactions over time. Typical types of profile data include:

- Languages used (e.g., English, Spanish)
- Location and time zone (e.g., Tokyo, Japan, UTC+9)

- Occupation and profession (e.g., Software Engineer, AI Researcher)
- Topics of interest (e.g., finance, travel, machine learning)

This information can be extracted from conversations or explicitly provided by the user and stored as part of a user profile. The agent can then incorporate these details when constructing user prompts.

Previous conversations

Previous conversations can also be valuable for LTM. It may contain thousands or even millions of past conversations, many of which may span different topics and lose relevance over time.

One effective way to manage this is by applying the TAM we covered earlier in the STM section. Each conversation snippet can be classified and tagged with one or more topics. When retrieving content for a new query, only the most relevant topics are surfaced into context.

For example, consider a customer support agent who has handled past conversations across various topics, including billing issues. If a user asks, "Why was I charged twice last month?", the agent can classify this query as billing-related and retrieve only previous conversations tagged with billing. Conversations on unrelated topics are excluded. This allows the agent to focus on the most relevant context and provide a more accurate and efficient response.

LTM can also include structured forms of memory, such as *episodic* and *semantic* memory. Episodic memory captures specific experiences or events tied to particular contexts, preserving detailed records of past interactions, decisions, and outcomes.

For example, an agent may remember that a user rejected a certain recommendation or preferred a specific communication style in a prior interaction. These episodic traces allow the agent to reason about past behavior, reflect on previous outcomes, and adapt its future actions.

As agents accumulate more experiences, semantic memory emerges as a complementary component of LTM. Instead of storing individual events, semantic memory captures generalized knowledge and patterns derived from multiple episodes.

For instance, an agent may learn user preferences, recurring behaviors, or domain-specific insights by abstracting across past interactions. This enables more efficient reasoning, as the agent no longer needs to revisit every individual experience, but can rely on distilled knowledge that is reusable across contexts.

Semantic memory also closely aligns with concepts such as knowledge graphs (see *Chapter 6*), where relationships between entities and concepts are structured and interconnected. By organizing knowledge in this way, LTM supports scalable and transferable reasoning, allowing agents to move beyond isolated experiences and operate with a more coherent and generalized understanding of users and domains.

So, we have considered some of the most common types of information stored in long-term memory. In practice, however, agents often need to store a much broader range of data, including system state, execution traces, logs, intermediate reasoning artifacts, and so on.

We have now explored agent memory, including both short-term and long-term memory, and discussed the key challenges of scaling memory for long and complex interactions. We have seen that memory plays a fundamental role in enabling agents to maintain context, improve response quality, and operate effectively over time.

With this understanding, we can now move to the next critical component of agent systems: tools and orchestration, which allow agents to extend beyond internal reasoning and interact with external systems.

Tools and orchestration

As we've seen, a large language model can serve as the 'brain' of an agent, enabling it to reason and make decisions. However, an LLM cannot independently perform actions in the real world. To act, the agent needs "hands" and "visors" (actuators and sensors), mechanisms that let it interact with the external "world". For LLM-based agents, these mechanisms are implemented as **external tools**.

Tools can include APIs, services, databases, computational functions, retrieval systems (like RAG), or anything outside LLM. For example, a search API can be a tool for fetching internet knowledge, while a calendar API is a tool for scheduling events.

Selecting and executing external tools is called **function calling**. LLM makes decisions about which function to call and which parameters to pass. "Function" here means an abstract capability, not a software code method: it can be any API, a database query, a web request, or a device command.

Note

The terms *external tools* and *function calling* might seem unclear at first. **External tools** are anything the agent can reach beyond the LLM's own reasoning and pretrained knowledge. **Function calling** is the process of selecting, invoking, and integrating those tools' results into the ongoing plan.

Function calling has two primary use cases:

- **Fetching data:** Retrieve some information to incorporate into the LLM context. Examples: get current weather, stock, inventory, policy rules.
- **Taking action:** Perform operations that change states, such as submitting forms, calling transactional APIs, modifying application state, triggering workflows, or sending commands to devices.

All forms of RAG and agent memory can be viewed as mechanisms for fetching data. Both extract specific information and add it to the LLM's context (*prompts*). Moreover, both memory and RAG are external tools for LLMs and are typically exposed via APIs.

External tools and function calling define an agent's *skills*, extending LLM capabilities. These skills let agents retrieve and process external information, perform multi-step reasoning, or execute automated tasks.

Since a typical agent has multiple skills, a fundamental *orchestration* problem arises in deciding which tool to use, when to invoke it, with what parameters, in what sequence, and when to stop.

Agent orchestration

Agent orchestration is the process by which an LLM decides which tools to call, when to call them, what parameters to pass, how to sequence these calls, and how to incorporate their outputs.

The agent orchestration problem is one of the most complex challenges in building AI agents. It sits at the intersection of NLP and systems engineering. At its core, orchestration boils down to two fundamental NLP tasks we discussed in *Chapter 4*:

- **Classification:** Deciding which function to call
- **Information extraction:** Identifying the parameters that should be passed into that function

For example, a weather-forecasting chatbot might have a single function to call an online weather service (such as OpenWeather or WeatherAPI). To use this function, the agent must extract the relevant parameters from the user's request and pass them to the service to retrieve the forecast. Such parameters often are the location and the date.

To detect which function to call, we first need to give every function a clear specification. This specification typically includes three attributes:

- **Name:** A short identifier for the function (or skill). For example, `get_weather` or `search_flights`.

- **Description:** A short natural language explanation of what the function actually does and is responsible for. For example: *"Retrieve the weather forecast for a given location and date"*.
- **Parameters:** The structured list of inputs to the function. Each parameter has a name, a type (e.g., string, integer, Boolean, object), and a description of what the parameter represents.

By providing these attributes, we give the LLM enough context to perform both tasks of orchestration: it can classify the correct function to use (based on the function description) and extract the required arguments (based on the parameter definitions).

The process of defining the list of functions and their parameters with the proper descriptions is called *function registration*. For example, when registering function calling in the OpenAI API, we need to define functions as JSON objects with their names, descriptions, and parameter schemas. Here is an example of such a JSON object:

```
tools = [{
  "type": "function",
  "function": {
    "name": "send_email",
    "description": "Send an email to a given recipient with a subject and message.",
    "parameters": {
      "type": "object",
      "properties": {
        "to": {
          "type": "string",
          "description": "The recipient email address."
        },
        "subject": {
          "type": "string",
          "description": "Email subject line."
        },
        "body": {
          "type": "string",
          "description": "Body of the email message."
        }
      }
    },
    "required": [
      "to",
      "subject",
```

```
        "body"  
      ],  
      "additionalProperties": False  
    },  
    "strict": True  
  }  
}]
```

Once this registration is in place, the orchestration loop works as follows:

- The LLM reads the user query and classifies it against the available functions (tools)
- If a function is selected, the LLM extracts and fills in the required parameters
- The external tool is invoked, and its result is fed back into the LLM's context
- The LLM reasons again: it can either stop with a final answer or continue the loop by calling another function

The following figure shows a sequence of orchestration steps:

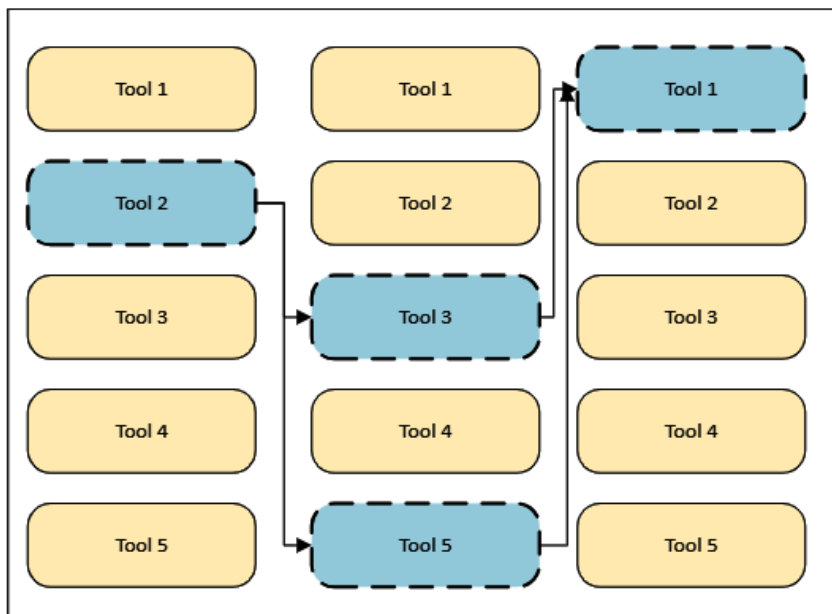


Figure 8.4: Agent tools orchestration

At the first stage, the LLM selects Tool 2 for execution. Next, in the second stage, the LLM decides to call Tool 3 and Tool 5. Finally, at the third stage, the LLM invokes Tool 1 and then stops execution.

It is important to notice that not all tools can run in parallel. For instance, the results of Tool 2 can be used as inputs to Tool 3 and Tool 5, and their outputs can then flow back into Tool 1. But tools 3 and 5 can be executed in parallel since they are independent. This dependency forces some tool calls to be sequential, while others can safely execute in parallel. The orchestration process must reason about these dependencies to optimize execution order.

Orchestration as a sequence of multi-label classification

So, the agent orchestration process can result in either selecting a **single function** (tool) or **multiple functions** (tools) to be called in parallel. This brings us to a more formal view of the agent orchestration problem from an NLP perspective:

Agent orchestration can be framed as a sequence of multi-class or multi-label classification steps.

Parallel execution is not just a design detail. It directly impacts the performance of the orchestration process. Minimizing the overall length of the tool execution sequence while maximizing parallelism is key to building efficient agents.

Formally, we can say that:

- At each orchestration step, the agent solves a classification problem to determine the most relevant tool(s)
- It simultaneously solves an information extraction problem to populate tool parameters
- It must also solve a meta-decision problem: whether to proceed with another step or stop

The termination step can also be integrated directly into the tool selection process. For example, if the agent can choose between three tools, the classification task at each step can include a fourth class – "stop". In this setup, the agent effectively performs a four-class classification: tool 1, tool 2, tool 3, or stop.

To make things clearer, let's consider a simplified orchestration example.

Imagine a weather forecast chatbot that accepts user questions like: "*What will the weather be in New York next Friday?*" Let's assume this agent has only one function it can call: `get_weather(location, date)`. Since users may ask a wide variety of questions, not all of them weather-related, the first step in orchestration is to perform a binary classification:

- If the question is about weather, the agent calls `get_weather`
- If not, the agent responds directly (e.g., "*Sorry, I can only answer weather-related questions*")

Once the classification confirms that the query is indeed about weather, the orchestration continues with information extraction. The model must extract the two required parameters: location (New York) and date (next Friday).

In this case, orchestration is quite straightforward:

- Perform a binary classification (weather vs. non-weather query).
- If positive, extract parameters from the question.
- Call the `get_weather` function with the extracted values.
- Form the final answer and return it to the user.

This example shows the simplest form of orchestration: a single-step classification + extraction + execution loop. Now, let's imagine the user asks a more advanced question: "I want to travel to Paris. Will it be hotter than in Madrid next weekend?"

In this case, the agent must:

- Classify that the request is still a weather-related question and should use the `get_weather` function
- Detect that two locations are mentioned: *Paris* and *Madrid*
- Extract parameters for each city along with the date range ("next weekend")
- Call the function twice (in parallel):

```
get_weather(location="Paris", date="next weekend")
get_weather(location="Madrid", date="next weekend")
```

- Perform reasoning over the results, comparing the predicted temperatures for both cities
- Generate the final answer: e.g., *"Yes, forecasts suggest Paris will be hotter than Madrid next weekend, with an average of 28°C compared to 25°C."*

This illustrates how orchestration can become quite complex in practice.

Orchestration plan

Another important aspect we can notice from the weather forecast example is that the list of functions to be called, their order, and their dependencies can often be determined at the very beginning, before any function is actually invoked. Such a list, together with its execution order, is called an **orchestration plan**.

An *orchestration plan* is essentially a structured algorithm or flow that specifies:

- Which functions need to be executed?
- In what order should they be called?
- Which calls can run in parallel and which must be sequential (due to data dependencies)?
- Where should the agent stop once the request is fulfilled?

Orchestration plan is defined by the user request, and for the preceding question, "Will it be hotter in Paris than in Madrid next weekend?", it will look as follows:

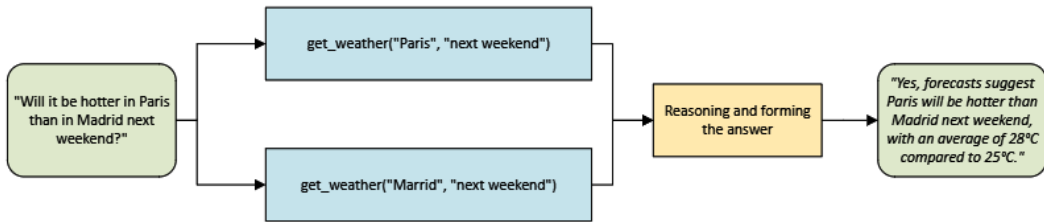


Figure 8.5: Execution plan for weather forecast chatbot

Specifically, the orchestration plan can also be represented as a structured JSON or XML object. This machine-readable format makes it easy for the agent to execute it step by step.

An orchestration plan is considered static when it can be fully generated in advance to satisfy the user's request, with the entire sequence of actions determined upfront. Orchestration static execution plans can be implemented as a two-step process:

- *Generate* the orchestration plan
- *Execute* the plan

Multi-plan orchestration

In some cases, multiple plans may satisfy the same user request. In such situations, the agent must not only generate candidate plans but also select the most efficient one.

To illustrate this, let's consider a personal travel planning agent that organizes trips based on user requests. One such request might be:

"Plan me a drive from San Francisco to Los Angeles this weekend, considering sunny weather and gas costs."

The agent could approach this task in different ways, resulting in distinct orchestration plans:

- **Plan A:** Get possible routes from San Francisco to Los Angeles, then optimize for weather conditions first (select the sunniest route) and refine the choice by gas cost
- **Plan B:** Get possible routes, then optimize gas costs first (select the cheapest option), and refine the choice by weather conditions

Both plans could lead to a valid answer, but they may differ in efficiency, cost of tool usage, and latency. For example, Plan A might avoid unnecessary gas cost calculations for routes that are already ruled out due to bad weather, while Plan B might minimize API calls if gas cost is the dominant factor.

This example shows why orchestration sometimes requires *multi-plan reasoning*: the system must generate several candidate plans, evaluate them against constraints (e.g., latency, reliability, or API budgets), and choose the one most aligned with the user's request and efficiency goals.

Dynamic orchestration plans

Up to this point, we have mostly considered static orchestration plans. Those are cases where the full plan can be generated in advance, before any functions are executed. However, in more advanced settings, this may not be possible. The reason is that the agent often cannot know what steps to take until it has already seen the results of earlier function calls. In other words, the plan must be adapted dynamically during execution, based on intermediate outputs.

This is similar to algorithms that search for an optimal path in an unknown environment. For example, algorithms like A* can compute the best route if the map of the terrain is fully known in advance, much like generating a static orchestration plan when all the steps and data dependencies are clear.

But if the map is incomplete or hidden, the agent cannot precompute the whole path. Instead, it must explore step by step, making decisions based on what it discovers along the way. Each move reveals new information, and the path is adjusted dynamically until the destination is reached.

The following diagram illustrates how an agent can make decisions dynamically, similar to a game bot navigating an unknown environment and determining each step as it progresses.

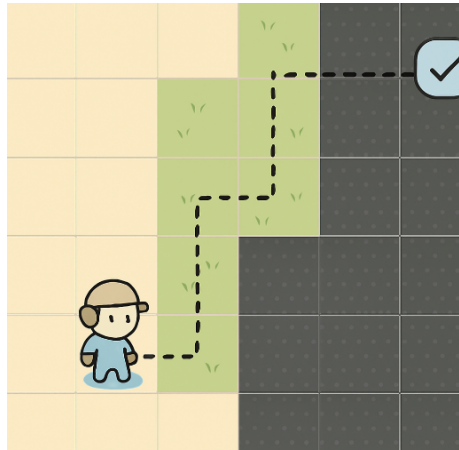


Figure 8.6: A game bot navigating a partially known environment, discovering the path step by step

Agent orchestration works the same way:

- If the "map" (all functions, dependencies, and required data) is fully known, a static plan can be built upfront
- If not, the agent must act more like a search algorithm in an unknown environment, deciding on the fly which function to call next, based on the outputs it observes

This approach is known as a *dynamic orchestration plan*. Instead of precomputing the entire sequence, the agent decides step by step:

- Select a tool (or tools) to call
- Execute it and observe the result
- Decide whether to continue, change direction, or stop

This can often be implemented as a **reason–act–observe loop**, where the LLM reasons about the next step, takes an action (function call), observes the output, and then repeats the cycle until the task is solved or a stopping condition is reached.

To illustrate this situation, imagine a user submits the following support request for customer support agents:

"My internet is slow, and sometimes it disconnects. Can you fix this?"

At first glance, the agent doesn't know exactly what's wrong. It can't generate a full static plan upfront because the next steps depend on what it discovers during the investigation. Instead, the agent must proceed dynamically:

- **Step 1:** Identify customer context and similar issues:
 - Call `get_customer_profile` (CRM) -> to check what service plan the customer has and whether they are under a support contract
 - Call `get_recent_tickets` to see if there's an ongoing or similar issue already in the same region
- **Step 2:** Investigate connectivity:
 - If the CRM shows the customer uses home broadband, the agent calls `run_line_diagnostics`
 - If the diagnostics report "line is healthy", the agent must decide dynamically to try another angle: `check_router_logs` or `query_network_outages`
- **Step 3:** Decide resolution path:
 - If diagnostics confirm a line problem -> escalate to technician
 - If router logs show repeated crashes -> trigger `send_replacement_router`
 - If an outage is detected -> notify the customer and link the ticket to the known issue

Here, the orchestration plan cannot be fully defined at the start. Each tool call provides new information that shapes the next step. Sometimes the agent stops early (if an outage is already reported); other times, it continues deeper (running multiple diagnostics).

This is the essence of dynamic orchestration:

- The system doesn't just execute a predefined script
- It explores step by step, making branching decisions based on uncertain intermediate results
- The flow resembles real-world troubleshooting, where each test either rules out or confirms possible causes

So, as we can see, orchestration can be quite complex. It is not just about picking a tool but about deciding the right sequence of actions, when to execute calls in parallel, how to handle dependencies, and when to stop. In simpler cases, we can rely on static orchestration plans, but many real-world scenarios require dynamic, step-by-step decision-making under uncertainty.

Orchestration challenges

As we've seen, orchestration is complex, and many things can go wrong. One common challenge is ambiguity in descriptions. When function or parameter names and descriptions are too similar or too vague, the LLM may struggle to select the correct tool and provide appropriate parameters.

To illustrate ambiguity issues, let's consider the following function names for a weather forecast agent:

```
get_weather(location: str) -> str
```

Description: *"Retrieves the current weather conditions for a specified location"*

```
get_forecast(location: str, days: int) -> str
```

Description: *"Provides a weather forecast for a given location over a specified number of days"*

The name descriptions are very similar, and therefore an LLM might misinterpret them, selecting the incorrect function. However, with clearer and more explicit definitions like in the example below, the distinction becomes easier:

```
get_weather(location: str) -> str
```

Description: *"Returns the real-time temperature, humidity, and conditions for a specific city or region"*

```
get_forecast(location: str, days: int) -> str
```

Description: *"Predicts temperature and weather conditions for a city over a future period (up to 7 days)"*

With these refined descriptions, the LLM has a much higher chance of selecting the correct function. To improve function classification, consider the following best practices:

- Use explicit and unique descriptions: Avoid vague or overlapping wording that blurs the distinction between functions
- Define strict function parameters: Ensure that parameters are specific and distinct enough to clearly separate usage scenarios

- **Limit unnecessary functions:** Where possible, merge overlapping functions by introducing optional parameters to reduce ambiguity
- **Include examples in descriptions (built-in few-shot prompting):** Supplement descriptions with example inputs/outputs to demonstrate when each function should be applied

By refining function descriptions and structuring them carefully, we can significantly improve the accuracy and reliability of LLM-driven function calling.

Similarly, the clarity and precision of parameter descriptions are critically important, since they directly affect how well the LLM performs the information extraction task when filling function arguments. Even if the correct function is classified, poor parameter definitions can lead to missing, incomplete, or incorrect values, resulting in failed or inaccurate execution.

To improve parameter extraction quality, consider:

- **Clear parameter definitions:** Each parameter should have a precise description and, when possible, example values. For instance, a date could be defined with specific formatting like *"calendar date in YYYY-MM-DD format, e.g., 2025-08-26"*
- **Schema constraints and validation:** Enforce type-checking, regex validation, or API-side verification to ensure extracted values match expectations
- **Entity resolution techniques:** Integrate **named entity recognition (NER)** or entity linking modules to disambiguate locations, times, or names when user input is messy
- **Dynamic handling of missing parameters:** If a required parameter is absent, the agent should either infer it from memory or proactively ask the user for clarification
- **Contextual disambiguation:** Use dialog history, user profiles, or previous tool outputs to resolve parameters that are underspecified

To illustrate these cases, consider the following example when a user asks: *"Can you check tomorrow's weather?"* The function is correctly classified as `get_weather(location: str, date: date)`. However, the required parameter `location` is missing. The agent should either:

- Infer the location from memory (e.g., if the user previously mentioned that),
- Ask for clarification: *"Which location would you like the weather for?"*

By designing precise parameter definitions, validating extracted values, and handling ambiguity or missing data dynamically, we can make the information extraction process more robust. This directly improves the reliability of function calling and ensures that agents interact with external tools more accurately.

Another challenge an agent may face is the *large number* of functions to orchestrate. From *Chapter 4*, we remember that the more classes we have in a multi-class classification, the

harder it is to achieve high accuracy. This is because, as the number of classes (*functions*) grows, it becomes increasingly difficult for the model to distinguish between edge cases and resolve ambiguities.

The challenge becomes even more complex when we consider that an agent does not perform classification just once. It often executes multiple multi-class classification steps in sequence. Errors can easily accumulate across these steps, leading to cascading failures. In practice, experience shows that today modern LLMs can manage relatively reliably around 20–30 functions, if we write clear, precise, and well-differentiated descriptions for them.

If an agent requires working with more than 20–30 functions, one practical solution is to use *multi-layered orchestration* (sometimes called *hierarchical routing*). In this approach, functions are grouped by higher-level categories, and the orchestration proceeds in layers:

- **First layer (coarse classification):** Decide which group of functions to consider
- **Second layer (fine classification):** Within the selected group, choose the exact function

For example, if an agent has 100 functions, we might cluster them into 10 groups (e.g., "weather," "travel booking," "customer support," "finance," etc.). The first classification step selects the relevant group, reducing the candidate space from 100 to 10. The second classification step then selects the precise function within that group.

This hierarchical design can reduce ambiguity, improve accuracy, and keep the classification manageable at each step. However, it also introduces its own complexity: designing meaningful groupings, ensuring balanced function distribution across groups, and minimizing overlaps between categories. Still, in practice, this approach makes it possible to scale orchestration to hundreds of functions while maintaining reliability.

Another challenge arises in defining the *stopping criteria* for dynamic orchestration plans and determining how many functions should be executed in response to a single user request. Unlike static plans, which have a fixed sequence, dynamic plans can keep branching and expanding as new information becomes available. Without a well-defined stopping rule, the agent risks either stopping too early (returning incomplete results) or continuing indefinitely (wasting time, cost, or API quotas).

For instance, consider a deep web-search agent designed to gather and summarize information from multiple sources. The agent may decide to follow links recursively, issue refined search queries, or fetch related documents. In this case, we might need to explicitly set:

- A maximum number of *search layers* (e.g., no more than three levels of link-following)

- A *cap* on the number of API calls (e.g., 10 search queries per request)
- A *confidence threshold* for stopping once consistent evidence has been gathered to formulate an answer

This problem is not unique to web search. Customer support agents, diagnostic agents, and planning agents face similar challenges in determining when to stop investigating, stop gathering additional evidence, and instead provide the best possible answer with the available information.

Stopping criteria are therefore a balance between accuracy, efficiency, and cost. Too few steps may yield shallow answers, while too many may result in unnecessary latency or expense. Designing robust heuristics, confidence thresholds, or even reinforcement learning policies to decide when to stop remains one of the open challenges in agent orchestration research.

So, we have now explored some of the most common orchestration challenges, but this is by no means a complete list. In fact, orchestration remains one of the most difficult and fascinating open problems in AI and LLM-based agents. Challenges such as ambiguous function selection, scaling across many tools, and deciding when to stop under uncertainty highlight the complexity of orchestration. They show how difficult it is to move from simple text generation to truly intelligent, action-taking systems.

What we covered here should give you a strong foundation, but the field is evolving rapidly, and there is much more to discover. If this topic sparks your curiosity, we strongly encourage you to dive deeper into current research and cutting-edge frameworks because orchestration is where some of the most exciting breakthroughs in AI are still waiting to happen.

Multi-agent flows

Up until now, we have focused on single-agent flows, scenarios where a single AI agent is responsible for solving a specific problem or executing a particular task. In these cases, the agent independently processes inputs, reasons through the problem, executes actions (such as calling APIs or performing computations), and delivers an output.

Today, we live in a world where software systems constantly exchange information, enabling seamless interaction between applications, services, and people. These exchanges can take many forms: from direct human-to-human communication (e.g., emails, messaging apps, video calls) to automated system-driven interactions (e.g., API calls, webhook triggers, event-driven processing).

With the rise of AI agents, we are rapidly moving toward a future where autonomous software entities (*agents*) communicate with each other without direct human involvement. These agents will collaborate to solve problems, coordinate tasks, and make decisions dynamically, much like human teams do today. Instead of simply reacting to human inputs, agents will be

able to plan, reason, and execute workflows autonomously by interacting with other agents in real time.

Many experts and industry leaders predict that in the near future, we will be surrounded by swarms of specialized agents. That will be an army of digital helpers, each designed to handle specific tasks. Imagine hiring dozens of diligent workers: one manages your calendar, another books your travel, another monitors your finances, while others search research papers, draft reports, or even negotiate deals on your behalf.

Just like human teams, these agents will coordinate with each other, share information, and delegate subtasks, all while you oversee the high-level goals. In this vision, you are less of a micromanager and more of a conductor of an intelligent workforce, directing a dynamic ensemble of agents that operate tirelessly in the background.

To make this vision possible, agents must be able to communicate and collaborate with one another. This is exactly what brings us into the realm of *multi-agent* environments, ecosystems where agents exchange information, coordinate actions, and work together to achieve goals that would be impossible for a single agent alone.

At the beginning of this chapter, we defined that agents could exist on two levels: business-level agents and low-level (code-level) agents. Business-level agents operate at the application or organizational layer, performing tasks such as financial analysis, customer support, or sales assistance. They are entities that are visible to end users and perform specific business tasks end to end.

Low-level or code-level agents, on the other hand, function inside these higher-level systems, handling more granular operations like parsing data, performing reasoning, or coordinating sub-tasks. Both types of agents share the ability to operate independently and autonomously, typically powered by AI.

We will begin with low-level (code-level) multi-agent communication, as it provides a clean way to demonstrate the overall concept. By looking at their interactions first, we can better understand the fundamental mechanics of multi-agent systems.

Code-level multi-agent flows

As we learned in *Chapter 3*, there are two main ways to organize multi-prompt flows: static and dynamic.

In a static schema, the flow of prompts is predefined. We know in advance the exact sequence of steps, how information will be passed from one prompt to the next, and what each prompt is responsible for. Such flows have clear advantages: they are reliable, stable, and relatively easy to test and evaluate. However, static flows can also be rigid. They don't perform well when

the user's request falls outside the predefined paths or when unexpected intermediate results require adapting the plan.

In contrast, dynamic flows allow building and executing plans "on the fly". Instead of following a fixed script, the orchestration adapts based on intermediate outputs and changing context. For example, an agent might decide to query a knowledge base only if the first reasoning step reveals missing information, or it may decide to call an additional helper prompt. Dynamic flows are therefore more flexible and adaptive, but they are also harder to control and test. Their behavior may vary significantly across runs.

One key idea behind multi-agent (code-level) flows was the benefit of executing prompts dynamically, rather than through a predefined sequence. Microsoft's AutoGen (<https://github.com/microsoft/autogen>) framework embodies this principle. The framework allows prompts (code-level agents) to converse with each other and make decisions on the fly based on intermediate outcomes.

AutoGen conceptualizes each prompt as an independent agent, complete with its own role, goals, and policy. Agents may also represent human proxies or tool connectors in addition to LLM prompts. The framework establishes communication channels among agents, enabling them to message one another, coordinate reasoning, critique outputs, and invoke tools, essentially creating a *conversational multi-agent flow*.

Note

Note that AutoGen was not the first to work on multi-agent dialog frameworks. Prior work, such as MetaGPT (<https://github.com/FoundationAgents/MetaGPT>) and OpenAI's Swarm (<https://github.com/openai/swarm>), has already introduced similar principles of agent communication and coordination. But the AutoGen team has proved that multi-prompt flows organized in a dynamic way can perform better than single prompts or static ones. The team proved that fact on different benchmarks.

Research shows that organizing multi-prompt flows in this dynamic way leads to better performance in complex tasks. For example, experiments (<https://arxiv.org/abs/2308.08155>) using AutoGen demonstrated enhanced effectiveness in mathematics, question answering, supply-chain optimization, and other domains compared to what single-agent systems can achieve.

This idea of using dynamic multi-prompt flows has gained significant traction in AI research. In essence, such a framework allows an LLM to "talk with itself" through different roles or "voices."

The following image illustrates this concept as a metaphor, showing how the model can internally explore different viewpoints to arrive at a solution.



Figure 8.7: Visual metaphor of an LLM "talking to itself"

For example, imagine two prompts acting as distinct agents: one takes the role of a financial analyst preparing a financial projection, while the other plays the role of a supervisor who reviews and validates the work. The analyst agent produces an initial projection, and the supervisor agent analyzes it, requests clarification, and suggests adjustments. Through this back-and-forth dialog, the system can refine its reasoning and arrive at a more accurate and reliable result than if the model were reasoning in a single voice.

To illustrate how such a conversation might unfold, consider the following exchange:

- **Analyst:** *"Based on Q1 revenue trends, I project a 7% growth in Q2, primarily driven by higher subscription renewals".*
- **Supervisor:** *"That's a good start, but have you factored in the potential impact of rising operational costs?"*
- **Analyst:** *"You're right, I missed that. Adjusting for a 5% increase in costs reduces the growth estimate to 4.5%".*
- **Supervisor:** *"Better. Now, please include a sensitivity analysis – show the projection under both optimistic and conservative scenarios".*
- **Analyst:** *"Done. Under optimistic assumptions, growth reaches 6%; under conservative assumptions, it drops to 3%. I'll add both cases to the final report."*

This simple interaction demonstrates how two specialized roles can collaborate to produce higher-quality outputs through multi-agent communication. Why can such a straightforward

idea (an LLM "talking to itself") lead to better results? The key is that an LLM, when given multiple roles or "voices", is effectively encouraged to externalize its reasoning rather than compress everything into a single response. By splitting the task into dialog turns, the model generates intermediate explanations, critiques, and refinements that expose hidden assumptions and potential mistakes. This creates a feedback loop where one "voice" generates content and another evaluates or challenges it, mimicking human peer review.

This mechanism of "reasoning better over more tokens" is similar to what we observed in **Chain-of-Thought (CoT)** prompting and in reasoning models such as GPT-5.4 or Claude Opus, as discussed in *Chapters 2 and 3*.

But just as multi-prompt flows introduce new complexities, multi-agent communication comes with a similar set of challenges:

- **Conversation drift:** Agent dialogues can grow excessively long diverging dialogs into multiply directions
- **Endless loops:** Dialogs can end up in endless loops without reaching a conclusion asking the same questions in a circle
- **High computational cost:** Every additional exchange consumes tokens (new LLM request), driving up processing expenses
- **Increased latency:** Longer back-and-forth communication slows down the overall response time compared to single-agent reasoning

Moreover, as the number of interacting agents increases, the risk of redundant exchanges, circular reasoning, and escalating token usage also grows. These trade-offs mean that, although multi-agent systems can provide richer reasoning and stronger error correction, they still require effective mechanisms for dialog management, clear stopping criteria, and efficiency optimization to remain practical.

To manage multi-agent interactions efficiently, several practical recommendations can be applied:

- **Number of agents:** Keep the number of agents relatively low. In practice, having more than 3–5 agents actively communicating in the same session is already considered complex and may lead to inefficiency.
- **Interaction safeguards:** Limit the number of exchanges per agent. Typically, 3–4 interactions are enough to reach meaningful results without drifting into repetitive loops. This ensures good interaction quality while keeping costs and latency under control.
- **Stop strategies:** In addition to setting hard limits on the number of turns, design intelligent stop conditions so agents can end conversations early when the goal has

already been achieved. This prevents wasted computation and avoids unnecessary dialog.

- **Role clarity:** Clearly define the roles and responsibilities of each agent. Ambiguous or overlapping roles often lead to redundant work, circular debates, or dropped tasks.
- **Summarization and compression:** Instruct agents to periodically summarize progress or compress context before continuing. This can reduce token usage and help keep the dialog focused.
- **Error handling and arbitration:** Provide a fallback or "arbiter" mechanism (often another supervisory agent) to resolve conflicts or deadlocks when agents disagree or repeat themselves.

So, as we can see, effectively managing multi-agent interactions requires careful control over the number of agents, their roles, communication patterns, and stopping conditions to ensure efficiency, reliability, and high-quality outcomes.

In the next chapter, we will dive deeper into multi-agent frameworks and platforms, exploring how they implement low-level (code-level) multi-agent interactions.

Multi-agent communication patterns

Similarly, just as real-world organizations have defined structures and hierarchies, code-level agents can also be organized in various ways to tackle more complex problems. Just as communication patterns guide human teams, agent interactions can be systematically structured to enable efficient coordination and collaboration.

In a multi-agent system, agents interact by sending and receiving messages, making requests, and providing replies similar to how humans communicate in business environments. For example, in human interactions, we have different communication modes:

- **Direct messaging:** One person sends a message to another and expects a response
- **Broadcasting:** A message is sent to multiple recipients at once (e.g., company-wide announcements)
- **Group discussions:** Multiple participants exchange information in a shared conversation
- **Hierarchical communication:** A manager assigns tasks to subordinates, who report back progress

The same can be applied to agents as well implementing the following patterns:

- **One-to-one messaging:** An agent sends a request to another agent and waits for a response

- **One-to-many broadcasting:** An agent sends a message to a group of agents, allowing multiple recipients to process it
- **Multi-agent group conversations:** Agents exchange messages in a shared workspace, where they collaboratively refine a decision
- **Hierarchical coordination:** A top-level orchestration agent delegates work to multiple specialized code-level agents, which execute subtasks and report back

Therefore, we can select several of the most frequently used agent communication patterns:

- Worker and supervisor
- Sequential chats
- Roundtable discussion
- Hierarchical task delegation model
- Competitive agent model

Let's consider them in detail.

Worker and supervisor

This is a two-agent schema where the worker agent performs the task, and the supervisor agent validates and reviews the worker's output, ensuring correctness. Examples are code generation agent and code review agent: one agent writes the code, and the other reviews it for errors or improvements. Another example is a data processing agent and quality control agent: one processes raw data, while the other verifies accuracy and correctness.

Sequential chat

In this pattern, agents collaborate in a sequence, where each step builds upon the previous agent's output. The following diagram illustrates a sequential chat pattern involving three agents:



Figure 8.8: Sequential chat flow

Instead of working in parallel, agents communicate in a structured pipeline, passing refined data along the chain. For instance, an AI research workflow might have three agents: a data collection agent that retrieves relevant sources, an analysis agent that processes and interprets the data, and a report generation agent that produces a structured research summary.

Roundtable discussion

In this approach, multiple agents participate in an open discussion, where they exchange arguments, validate each other's reasoning, and reach a final decision through consensus.

Example: an AI financial advisory agent with the following sub-agents—a risk analysis agent assesses investment risks, a market trend agent evaluates real-time stock market conditions, a portfolio optimization agent suggests an investment strategy, and a final decision agent consolidates all insights and presents the best recommendation.

Hierarchical task delegation model

A central orchestrator agent assigns work to multiple specialized agents, each handling a specific sub-task. This pattern is useful in complex workflows requiring parallel execution. For example, customer support agents are created from the following sub-agents: an intent classification agent detects the type of request, a knowledge retrieval agent fetches relevant information, and a response generation agent formulates the final reply.

Competitive agent model

In this approach, multiple agents independently generate competing solutions, and the best outcome is selected based on predefined criteria. This method mirrors peer review and competitive problem-solving models. For example, an algorithm optimization system with two code-level agents: agent A tests approach X, while agent B tests approach Y; a comparison agent evaluates both and selects the most efficient method.

So, here we have highlighted just a few of the common patterns of agent communication. In practice, many other variations can be applied when designing multi-agent flows. The choice of pattern depends on the problem, the roles of the agents, and the trade-offs between efficiency, accuracy, and control.

The choice of communication pattern depends on the task's complexity, the level of validation required, and the balance between efficiency and control:

- For accuracy-driven tasks, use worker and supervisor
- For multi-step workflows, use sequential chats
- For collaborative decision-making, use roundtable discussion
- For large-scale parallel processing, use hierarchical task delegation
- For competitive idea generation, use competitive agent model

In the following chapter, we will dive deeper into practical examples of using agent communication flows.

Agent goals and control mechanisms

When designing an agent or multi-agent flows, one of the fundamental goals is to establish a mechanism for validating and confirming that the task was successfully completed with high quality. Without a well-defined validation process, an agent's output may lack reliability, correctness, or user satisfaction.

For some tasks, success criteria are clear and objective. For example:

- **Code generation agent:** The goal is to generate code that compiles, executes, and produces correct results based on provided test cases
- **Math solver agent:** The solution is considered valid if it matches the expected mathematical proof or numerical answer
- **Data extraction agent:** The accuracy of the extracted information can be measured against a known structured dataset or ground truth

For other tasks, success may be more subjective, requiring user evaluation. In such cases, validation can depend on qualitative feedback, such as:

- **Conversational AI agent:** Success can be measured through user satisfaction scores, response coherence, and lack of repeated or re-phrased questions
- **Customer support agent:** Quality can be assessed based on resolved user issues, positive feedback, and escalation rate to human support

However, defining success criteria can be challenging for more complex, creative, or open-ended tasks. For example:

- **Content generation agent:** How do we define "good" writing? Success could be measured based on readability scores, engagement metrics, or human editorial approval.
- **Legal document analysis agent:** Determining whether the analysis is legitimate requires expert review or comparison with historical legal precedents.
- **Scientific research assistant:** The validity of a generated hypothesis might not be immediately verifiable without real-world experimentation or peer review.

To ensure that an agent's outputs meet the desired quality standards, different validation mechanisms can be used:

- **Self-validation and iterative refinement:** The agent evaluates its own output using logical reasoning, function verification, or multiple trials (e.g., ReAct framework for self-correction, see next chapter)

- **Multi-agent review flow:** One agent produces an output, while another agent critiques and improves it (e.g., Code generation agent & Code review agent)
- **Human-in-the-loop (HITL) confirmation:** When objective validation is difficult, human reviewers can assess the quality (e.g., User satisfaction surveys for AI-generated responses)
- **Automated benchmarking and scoring:** Comparing results to a known dataset, standard test cases, or pre-defined evaluation metrics (e.g., BLEU score for translation models)

So, by implementing effective control mechanisms, multi-agent systems can stay efficient, avoid unnecessary resource consumption, and ensure that agent interactions remain focused and productive. In the following chapter, we will explore how these mechanisms are realized in practice through different agent frameworks.

Summary

Great, we have just completed one of the most important chapters of this book, where many of the concepts introduced earlier come together. Up to this point, we explored LLMs, prompt engineering, core NLP tasks, RAG, and fine-tuning largely in isolation. In this chapter, we combined these building blocks to understand how AI agents are designed and implemented.

We covered the foundations of agent architecture and terminology, explored different types of agents such as conversational and background agents, and examined practical examples. We also looked in depth at agent memory, including both short-term and long-term memory, and discussed how agents manage context over time. A significant part of this chapter focused on tools and orchestration as well as multi-agent systems, including communication patterns and coordination strategies. We also introduced agent goals and control mechanisms, which are essential for guiding behavior and ensuring reliable outcomes.

Throughout the chapter, we saw that building effective agent systems requires solving complex challenges that span both AI and software engineering, particularly in areas such as memory design, orchestration, and system coordination.

With this foundation, you are now well prepared to explore patterns, frameworks, tools, and platforms for building AI agents. More importantly, you are equipped to design systems that go beyond simple prototypes and move toward robust, scalable, and production-ready solutions.

In the next chapter, we will build on this foundation and shift to the practical side, exploring how to implement agent systems using modern tools, frameworks, and protocols.

Subscribe for a free eBook

New frameworks, evolving architectures, research drops, production breakdowns – *AI_Distilled* filters the noise into a weekly briefing for engineers and researchers working hands-on with LLMs and generative AI systems. Subscribe now and receive a free eBook, along with weekly insights that help you stay focused and informed.

Subscribe at <https://packt.link/80z6Y> or scan the QR code below.



9

Building AI Agents

In the previous chapter, we explored the fundamentals of AI agents. Now, it's time to talk about the practical aspects of building AI agents.

The AI agent ecosystem is evolving rapidly. New frameworks, platforms, and tools are emerging almost monthly, and best practices are still taking shape. We are at the early stage of a new paradigm in software development. Because of this rapid pace, some specific technologies discussed here may change over time. However, we believe the underlying principles and design patterns will remain relevant and transferable. This chapter focuses on those practical foundations.

You will learn how to build AI agents from scratch and explore how to use existing frameworks and platforms to accelerate development. Throughout the chapter, we will reinforce all concepts with practical examples and case studies across different domains. By the end of this chapter, you will have a clear, actionable understanding of how to design, build, and deploy AI agents that operate reliably in real-world environments.

We will cover the following key topics:

- Agent design
- Agent memory implementation
- Tools and orchestration
- Agent decision-making and planning
- Multi-agent frameworks and protocols
- Implementing AI agents in practice

Let's get started!

Technical requirements

The code for this chapter is available on the book's accompanying GitHub repo: <https://github.com/PacktPublishing/Building-Agent-Powered-Applications/tree/main/Chapter9>

Agent design

In *Chapter 8*, we explored the architecture of LLM-based agents, including their core components. Now, we are shifting to the practical aspects of building real-world AI agents. The first step in this process is agent design.

Designing an AI agent shares many similarities with designing classical software systems. You still need to define requirements, design system components, and consider performance, reliability, and scalability. However, AI agents introduce several important differences.

Unlike traditional software, agents are not fully deterministic. They rely on probabilistic models, make decisions, use tools, and adapt their behavior based on context. As a result, the design process is less about defining exact logic and more about shaping behavior through constraints, instructions, and system boundaries. This requires a different engineering mindset.

At a high level, designing an AI agent involves defining the following aspects:

- **Functional requirements:** What should the agent do? What tasks should it solve? What level of quality, accuracy, and reliability is expected?
- **System requirements:** What constraints must the system satisfy in terms of scale, latency, cost, security, and compliance?
- **Interaction mode:** How will the agent interact with users or other agents? Is it conversational, background, event-driven, or part of a multi-agent workflow?
- **Implementation approach:** How will the agent be built? Will it be a custom code-first solution, a framework-based system, or a platform-based implementation?
- **Skills, tools, and capabilities:** What actions must the agent be able to perform to meet its goals? Which external systems and APIs need to be integrated?
- **Memory, state, and learning:** What information should the agent retain, and for how long? What learning capabilities should the agent have?
- **Reasoning and control mechanisms:** How will the agent make decisions? Will it follow reactive patterns, planning-based approaches, or structured reasoning workflows?

- **Safety and control boundaries:** What must the agent not do? What constraints, permissions, and validation mechanisms are required?
- **Evaluation and success metrics:** How will you measure whether the agent behaves correctly and meets quality expectations?
- **Deployment and operations:** Where and how will the agent run? How will it scale, be monitored, and be updated over time?

Let's briefly expand each of these design dimensions. They represent key aspects of building LLM-based agents. Later in this chapter, we focus on the most important practical areas of agent development.

Functional requirements

As with classical software design, the first step in building an AI agent is to clearly define its functional requirements. At a basic level, this means answering a simple question: what should the agent do? What tasks should it perform? What problems should it solve? What inputs will it receive, and what outputs is it expected to produce?

AI agents can take many forms and serve a wide range of purposes. Some are conversational assistants that interact with users in natural language (or speech), while others operate in the background, automate workflows, retrieve information, or coordinate actions across other agents. Because of this diversity, it is essential to clearly define the agent's scope and responsibilities early in the design process.

However, unlike traditional software, defining functionality alone is not sufficient. AI agents are built on probabilistic models such as LLMs, which means their behavior is not fully deterministic. As a result, functional requirements must be defined together with quality expectations. It is not enough to specify what the agent should do. You must also define how well it should do it.

This includes questions such as: What level of accuracy is acceptable? How often can the agent make mistakes? What types of errors are tolerable, and which are not? How consistent should the agent's behavior be?

In practice, an agent that performs the right tasks but fails to meet a minimum quality threshold is often not useful. Many AI systems only become viable once they cross a certain quality bar. For example, early language models could generate text, but the output was often incoherent or unreliable. Only after reaching sufficient levels of quality (in the GPT-3.5 model) did they become practical for real-world applications.

The same principle applies to AI agents. When defining functional requirements, it is therefore critical to pair each capability with a clear quality expectation. This forms the foundation for later evaluation, iteration, and deployment and release decisions.

Later in this chapter, and in more detail in *Chapter 10*, we will explore how to evaluate AI agents and define the criteria needed to determine whether their quality meets the specified requirements.

System requirements

Although AI agents represent a new type of software, they are still software applications. They consist of code running on compute infrastructure (such as CPUs and GPUs), process inputs, and produce outputs. Because of this, defining system requirements is just as important as it is for classical software.

At a high level, the key system requirements include:

- **Scale:** How many users, requests, or data volumes should the agent handle?
- **Latency:** How quickly should the agent respond?
- **Cost:** What operational cost is acceptable per operation or overall?
- **Security and compliance:** What data protection, privacy, and regulatory requirements must be satisfied?

As we learned, agents are composed of multiple components, including LLMs, memory, and tools. When defining system requirements, it is important to consider the impact of each of these components. For example, tool calls may introduce additional latency, memory may increase storage and retrieval costs, and orchestration logic may affect overall throughput.

As we learned in *Chapter 2*, large language models are powerful but can also be relatively slow and expensive to run. In addition, modern LLM deployments are often constrained by GPU availability and infrastructure limits. These factors directly influence how agents should be designed, especially when targeting production environments on a large scale.

Different use cases impose different system constraints. Some agents are designed for low-volume, high-value interactions, where higher latency, and cost may be acceptable. Others must handle large-scale traffic from the start, requiring strict optimization of performance and cost. For example, an internal research assistant may tolerate slower responses, while a customer-facing support agent is expected to respond quickly and consistently under heavy load.

In many cases, trade-offs between cost, latency, and privacy lead to the use of **smaller language models (SLMs)** instead of large ones. This often requires additional techniques such as fine-tuning or optimization to maintain acceptable quality levels (see *Chapter 7*).

So, system requirements play a critical role in shaping architectural decisions and directly influence model selection, tool design, and overall agent behavior. Throughout this chapter, we will return to system requirements multiple times, exploring them in greater depth as we dive into practical implementation.

Interaction mode

At this stage, it is important to define the interaction model of the agent. In other words, how will the agent be triggered, and how will it interact with users or other agents or systems?

As we already learned in *Chapter 8*, agents can take different forms depending on the use case. In practice, defining the interaction mode means identifying:

- What triggers the agent (user input, events, schedules, or other agents)?
- How does the agent process inputs?
- How and when does the agent produce outputs?

As we learned, modern agents can also be hybrid systems. They can both interact with users conversationally and perform background actions. In such systems, a chat interface is just one possible entry point. The same agent may also be triggered by events, workflows, or other services or agents, allowing it to operate across multiple interaction channels.

Because of this, it is important to think of the interaction mode not just as a user interface choice, but as a broader system design decision that defines how the agent integrates into real-world processes and environments.

Implementation approach

At this stage, we need to decide how the agent will be implemented. Similar to classical software development, there are multiple ways to build applications and services, ranging from fully custom code-based solutions to low-code or no-code platforms with prebuilt components and visual interfaces. Each approach comes with its own trade-offs in terms of flexibility, development speed, and control.

A code-first approach provides maximum flexibility and customization. In this case, the agent is implemented directly using programming languages (like Python, Java, or C#), APIs, and software libraries. This allows full control over orchestration logic, model selection, reasoning flows, memory handling, and integrations with external systems. However, it typically requires more development effort and deeper engineering expertise.

A code-first approach can be optimized by using existing agent frameworks and tools. Such frameworks provide reusable abstractions for common tasks such as tool usage, memory, and orchestration, allowing developers to build agents faster while still maintaining a good level of control. These frameworks help standardize development but may introduce some constraints based on their design.

Finally, a platform-based approach focuses on rapid development using managed environments, prebuilt components, and visual tools. Platforms such as Microsoft Copilot Studio or Google Vertex AI allow building agents with no or minimal code by leveraging built-

in integrations, UI items, and cloud-based infrastructure. This approach is well-suited for enterprise scenarios and rapid prototyping, but may significantly limit customization.

In practice, these approaches are not mutually exclusive. Many real-world systems combine them, for example, by using a platform for orchestration while implementing critical components in custom code.

Choosing the right implementation approach depends on the specific requirements of the agent, including complexity, scalability, required level of control, and available development resources. This decision will influence many aspects of the system, including tool integration, memory design, and deployment strategy.

Later in this chapter, we will explore frameworks and tools for implementing agent memory and orchestration and conclude by examining and comparing a range of agent development frameworks.

Skills, tools, and capabilities

After defining the functional and system requirements, as well as the interaction model, the next step is to determine the skills, tools, and capabilities the agent must have to perform its tasks effectively.

As we learned in *Chapter 8*, at a high level, a skill represents what the agent is able to do, while tools, functions, and APIs define how those skills are implemented. Clearly identifying these capabilities helps translate high-level requirements into concrete system components.

For example, a simple weather agent that answers user questions about current weather needs the ability to retrieve current and forecast weather data and generate a clear, structured response. A more advanced customer support agent requires a broader set of capabilities, such as accessing customer information, searching internal knowledge bases or similar cases, retrieving order status, and taking actions like running troubleshooting tools (like internet network repair tools) or triggering workflows.

By identifying the required skills, we can determine which tools, APIs, and data sources must be made available to the agent. This step is critical, as it defines the boundary between what the agent can reason about internally and what it must obtain or execute externally.

In practice, tools may include:

- External APIs and services (like weather forecast services)
- Databases, knowledge systems, and retrieval systems such as RAG
- Internal business logic and workflows

In this chapter, we will explore in more detail how tools are integrated and orchestrated, including the use of modern frameworks and protocols such as **Model Context Protocol (MCP)**, which standardizes how agents interact with external capabilities and data sources.

Memory, state, and learning

As discussed earlier, large language models are typically stateless. However, most real-world agents must maintain state and remember information over time to be useful.

A helpful analogy is a human employee working on tasks such as processing contracts, assisting customers, or managing deals. To perform effectively, the employee must track the current state of their work, remember past interactions, and carry that information forward across multiple steps. In addition, employees need to improve and adapt over time by learning from experience and acquiring new capabilities.

AI agents require similar capabilities, and this must be explicitly designed. When defining memory and state for an agent, several key questions need to be answered:

- What state must the agent maintain? What should the agent remember?
- How long should specific information be retained, and what type of memory should be used?
- Where and how is memory stored? How is memory retrieved and updated?
- Does the agent need to learn over time? Should the agent adapt based on past interactions or feedback?

So, designing memory, and state management is a critical aspect of building reliable agents. In this chapter, we will explore practical approaches to implementing these capabilities. We will also discuss techniques such as memory-augmented learning and how they can be applied in real-world systems.

Reasoning and control mechanisms

One of the core characteristics of AI agents is their ability to make decisions. This is what makes them "smart" systems. Because of this, designing how the agent reasons and makes decisions is a critical step.

In many practical applications, it is not enough to rely on the raw capabilities of a language model. The decision-making process must be structured and guided. When designing reasoning and control mechanisms, several key questions need to be addressed:

- How should the agent reason about tasks?
- How are decisions controlled and constrained? What instructions, rules, or system prompts guide the agent's behavior?
- How do we ensure consistent and predictable outcomes (no hallucinations)?

- How are multi-step tasks handled? Should the agent plan ahead, or operate in a reactive loop?
- When should human intervention be required? For critical or high-risk actions, should the agent request confirmation or escalate to a human?

In practice, these mechanisms are implemented through a combination of prompt design, structured workflows, and, in some cases, fine-tuning. Later in the chapter, we will explore these concepts in more detail, including reasoning patterns such as ReAct, planning strategies, and techniques for incorporating human-in-the-loop control into agent workflows.

Safety and control boundaries

As discussed earlier, LLMs are highly general-purpose models capable of handling a wide range of tasks. However, this flexibility also introduces risks, such as generating incorrect, inappropriate, or unsafe outputs. Since AI agents are built on top of LLMs, they inherit these risks.

In addition, agents often have access to tools and can perform real actions, which significantly increases the potential impact of errors. Unlike standalone models that only generate text, agents may trigger workflows, modify data, or execute transactions. For example, a financial agent capable of performing transactions must be carefully controlled to prevent incorrect or unintended actions.

Because of this, defining clear safety and control boundaries is a critical design step. When designing these boundaries, key questions include:

- What actions are the agent allowed or not allowed to perform?
- What validations are required before executing critical operations?
- When should human approval be required?
- How should failures and unsafe situations be handled?

Well-designed safety mechanisms ensure that agents operate within defined limits, reduce the risk of harmful behavior, and make them suitable for real-world use. In this chapter and especially *Chapter 10*, we will explore practical approaches to implementing these controls, including validation strategies, permission boundaries, and human-in-the-loop mechanisms.

Evaluation and success metrics

As discussed in the functional requirements, agent quality is one of the most critical aspects to define from the start. However, this naturally leads to the question: how do we measure and validate that quality?

Designing an agent requires clearly defining success metrics, such as task success rate, accuracy, performance, and user satisfaction. These metrics provide a concrete way to assess whether the agent meets its intended goals and quality expectations.

In *Chapter 10*, we will explore in detail how to evaluate AI agents, including how to design effective evaluation criteria and validate agent behavior in real-world scenarios.

Deployment and operations

Finally, it is important to define how the agent will be deployed, hosted, released, and operated in a real-world environment. In a code-first approach, deploying an agent is similar to deploying a traditional software service. This typically involves cloud infrastructure, APIs, scaling, and monitoring strategies. In contrast, low or no-code platforms often provide built-in support for the full agent lifecycle, including deployment, monitoring, debugging, and staging. These platforms simplify operations but limit flexibility in certain scenarios.

Now that we have covered the key questions for high-level agent design, let's dive deeper into the implementation of specific components, including memory, tool orchestration, agent platforms, and multi-agent collaboration.

Agent memory implementation

In this section, we will explore three main approaches to developing agent memory: custom memory development, the use of memory frameworks, and memory capabilities provided by low-code or no-code platforms.

Custom memory development

Custom memory development involves implementing memory mechanisms entirely from scratch. This approach provides maximum flexibility and control over all aspects of how memory is stored, updated, and used by the agent. However, it also requires more engineering effort and careful design to handle common challenges effectively.

In *Chapter 8*, we introduced a simple and naïve implementation of **short-term memory (STM)**, where all past interactions are appended to each new LLM request (see proper code example). While this approach is easy to implement,

As we learned before, agent memory introduces several challenges, including LLM's limited context windows, increased latency and cost, and the risk of topic drift as conversations grow longer.

To address these issues, more advanced memory strategies are required. A common approach for conversational agents is to maintain a sliding window of recent interactions, summarize older exchanges, and discard very old or irrelevant context. Additionally, for scenarios

involving multiple topics, *topic-aware memory* (see *Chapter 8*) can be used to organize and retrieve context more effectively.

When implementing STM in a custom way, you gain full control over key design parameters:

- **Recent interactions:** How many and what size of recent user–agent exchanges are included directly in the prompt?
- **Summarization of older interactions:** How and when the older context is summarized. You can choose the model, settings, prompt, and level of detail for summarization, allowing you to retain only the most relevant and important information.
- **Cutoff strategy:** When and how older interactions are removed entirely to control context size and cost.
- **Topic-aware memory (TAM):** How conversations are segmented into topics, whether single or multiple topics are tracked, and which classification or clustering approach is used.

These controls allow you to optimize memory for specific use cases. For example, a customer support agent may prioritize recent interactions and key facts, while a research assistant may retain more historical context with detailed summaries.

In more advanced scenarios, separate models or pipelines may be used for summarization and topic classification tasks. In some cases, fine-tuning these models can further improve performance, especially for domain-specific applications.

As we also learned in *Chapter 8*, **long-term memory (LTM)** is highly use-case dependent and often requires custom design. Designing LTM requires answering several key questions:

- What information should be stored? For example, user preferences, historical decisions, or important facts.
- How long should memory persist? Should it expire after a certain time, be updated continuously, or remain permanent?
- Where is memory stored? Options include databases, cloud, or application-specific storage systems.
- When and how is memory retrieved? Under what conditions should the information be injected into the LLM context?

For example, in an email drafting agent, LTM may store user preferences such as tone (formal vs informal), preferred structure, or commonly used phrases. When generating a new email, these preferences can be retrieved and incorporated into the prompt to produce more personalized outputs. Similarly, in a customer support agent, LTM may store account-specific information, past issues, or user interaction history.

As we can see, custom memory implementation requires significant engineering effort, but it provides the highest level of control and flexibility.

Memory tools and frameworks

While custom memory implementations provide maximum flexibility, they are not always necessary. In many cases, existing tools and frameworks can significantly simplify agent memory development. These solutions help accelerate development while addressing common memory challenges. Now, let's look at one representative memory framework, while noting that other tools follow similar design patterns and approaches.

Mem0

Mem0 (pronounced "mem-zero") is an open-source memory for AI agents, created by Taranjeet Singh and Deshraj Yadav, and launched after their work on Embedchain [50]. It is designed to provide a structured way to manage both STM and LTM for agents. Key features and capabilities include the following:

- **Multi-type memory support:** Supports working (**STM**), factual (**LTM**), episodic, and semantic memory types (see *Chapter 8*)
- **Intelligent filtering and summarization:** To reduce token consumption, latency and fit into the LLM effective context window, Mem0 employs memory summarization and dynamic forgetting (cutting by token size)
- **Graph-based memory:** Store LTM by using a knowledge graph, capturing relationships between entities and conversational elements
- **Efficiency and accuracy:** Achieves high scores in reasoning accuracy, token savings, and reduction in latency compared to full-context approaches on **long conversation memory (LoCoMo)** benchmark
- **Deployment:** Offers both open-source and managed cloud versions, with SDKs in Python and JavaScript, REST APIs, and integrations with agent frameworks like LangChain, AutoGen, and CrewAI

The following is a code example showing how to add, retrieve, search, and update memory using Mem0 in Python:

```
from mem0 import Memory
m = Memory()

# store a memory from a chat turn (Mem0 can infer "facts/preferences" from
messages)
messages = [
    {"role": "user", "content": "I travel a lot for work, prefer aisle seats. ...
```

```

    },
    {"role": "assistant", "content": "Noted. I'll prioritize aisle seats in
bookings. ... "}
]
m.add(messages, user_id="alice", agent_id="travel-assistant", run_id="sess-001",
metadata={"category": "travel_prefs"})

# retrieve all memories for a user (or scope by agent_id / run_id)
all_user_memories = m.get_all(user_id="alice")

# semantic search over memories for context building
relevant = m.search(query="What seat does Alice prefer?", user_id="alice")

# update or inspect history for a specific memory
mem_id = relevant[0]["id"]
m.update(memory_id=mem_id, data="User prefers aisle seats and avoids red-eye
flights.")
history = m.history(memory_id=mem_id)

# build an LLM prompt with retrieved memory
def build_prompt(user_query: str, memories: list[str]) -> str:
    memtext = "\n".join(f"- {m['memory']}" for m in memories)
    return f"""You are a travel assistant.
        Known user memory:
        {memtext}
        User: {user_query}
        Answer with these memories in mind."""

prompt = build_prompt("Book me to NYC next week", relevant)
print(prompt)

# delete (single / all for user)
m.delete(memory_id=mem_id)
m.delete_all(user_id="alice")

```

There are several other memory frameworks with similar goals, such as Zep, Claude-mem and MemEngine, all of which are actively evolving. Due to the scope of this book, we encourage readers to explore these frameworks in more detail.

As we can see, modern memory frameworks can significantly reduce the complexity of implementing agent memory, making them a practical choice for many real-world applications. However, this convenience often comes at the cost of reduced flexibility and customization.

Agent platforms memory implementation

As discussed earlier in this chapter, there are many platforms available today that support building AI agents. Examples include low-code and no-code platforms such as Microsoft Copilot Studio and Google Vertex AI. These platforms enable rapid agent development and typically provide built-in memory capabilities, especially for conversational agents.

Such built-in memory features eliminate the need to implement memory from scratch or rely on code-level frameworks like Mem0. Instead, developers can configure memory behavior through graphical user interfaces, significantly reducing development effort and time to deployment.

In most cases, these platforms offer standardized memory implementations designed to cover common scenarios, such as maintaining conversational context or storing basic user information. For many applications, especially those that do not require highly specialized behavior, these built-in capabilities are sufficient and allow teams to move quickly.

However, this convenience comes with trade-offs. Platform-based memory solutions often provide limited configurability and control. For example, while Microsoft Copilot Studio supports certain memory features, the ability to customize STM behavior is restricted and LTM options are very constrained.

As a result, while platform-based memory implementations are well-suited for rapid development and standard use cases, they may not be sufficient for more complex scenarios that require advanced memory strategies or precise control over agent behavior.

Memory-augmented learning

Traditionally, "learning" for large language models has meant retraining or fine-tuning. As we learned in *Chapter 7*, this process is expensive, slow, and requires extensive computational resources. However, adaptive behavior does not always require updating the model weights. Instead, **memory-augmented learning (MAL)** enables agents to adapt and improve by storing, organizing, and reusing experiences from past interactions without retraining the underlying LLM.

This approach builds on recent research in episodic memory for LLMs and *experience replay* techniques in reinforcement learning. MAL provides a lightweight way to let agents "learn" at runtime, similar to how humans use memory to guide future actions.

To implement MAL effectively, three structured components must be tracked in LTM:

- **Inputs (user queries):** What the user asked or instructed the system to do. This provides the *anchor* for retrieving similar past cases in future interactions.
- **Action plans (experience):** The reasoning steps or workflows the system followed in response. This can include tool usage (execution plan), intermediate decisions, or conversation strategies.
- **Achieved results (outcomes):** The measured success or failure of the action plan. This may be explicit (user feedback: thumb-up or thumb-down) or implicit (user ignored the result, escalated, or asked for refinement). The outcomes can also be system or API errors, external service failures, uncompiled code, and so on.

This triad (*query* → *plan* → *result*) allows the system to index experiences, retrieve similar cases when facing new queries, and adapt responses accordingly. Unlike fine-tuning, which learns globally, MAL supports incremental, case-based learning at runtime.

While MAL can be quite efficient, it has limitations as well:

- **Ambiguous or missing feedback:** In many cases, the outcome of an agent's action can be unclear (e.g., the user never replies, or the task ends without explicit evaluation).
- **Dynamic action plans:** Not all experiences can be directly reused. In some situations, action plans must be regenerated even if the user query is the same or very similar.
- **Generalization vs. memorization:** MAL must strike a balance between case-specific recall and higher-level generalization. Simply replaying past experiences risks overfitting to isolated situations.

To illustrate how MAL can be implemented, let's consider the following two examples:

Personal assistant:

- **User query:** "Generate a vacation tour optimizing my budget"
- **Action plan:** Retrieve the user's past travel preferences, check recent itineraries, and generate a new vacation plan
- **Achieved result (positive):** User accepts the plan – successful case
- **Achieved result (negative):** User rejects plan and asks for "beach only, fewer transfers" – updated plan and marked first attempt as unsuccessful
- **Learning effect:** Next time, the agent retrieves this case and prioritizes beach destinations with fewer transfers for budget-focused queries

Customer-support chatbot:

- **User query:** "I can't log in to my account"

- **Action plan:** Suggest password reset -> offer MFA support -> escalate if unresolved
- **Achieved result:** If escalation happens too often, the system logs a negative outcome for the first two steps

So, MAL can enable LLM agents to adapt quickly without LLM retraining and using fine-tuned models, personalize responses based on accumulated user-specific experience, and generalize from small data (case-based learning rather than massive datasets).

Tools and orchestration

After defining and implementing memory, the next step is to design tools and orchestration. As with memory, tool orchestration can be implemented in several ways: by building custom orchestration logic from scratch, by using frameworks and built-in function-calling LLM capabilities, or by leveraging orchestration features in agentic platforms such as Microsoft Copilot Studio or Google Vertex AI platform.

Custom tools orchestration

As discussed in *Chapter 8*, tool orchestration relies on two fundamental AI tasks:

- *Classification*, to select the appropriate tools
- *Information extraction*, to determine the required parameters

With a custom approach, you have full control over how both of these tasks are implemented. In many practical scenarios, agents operate with a fixed set of tools and well-defined parameters, which makes it possible to design efficient orchestration logic. Depending on the complexity of the use case, different approaches can be applied:

- **Single-prompt approach:** A single LLM call is used to both select the tool and extract its parameters. This is simple and can be efficient, but may be less reliable for complex scenarios.
- **Multi-step orchestration:** Tool selection and parameter extraction are handled in separate prompts and models. This can improve tool selection and parameter extraction quality, control, and interpretability.
- **Planning-based orchestration:** For more advanced agents, the agent may generate multi-step execution plan(s), deciding which tools to call and in what order.

For agents with a fixed set of tools, it is often also possible to build high-quality datasets for tool selection and parameter extraction and apply supervised fine-tuning (see *Chapter 7*). This can improve orchestration accuracy, latency, and costs. So, fine-tuning opens the possibility of using smaller, faster, and more cost-efficient models.

However, it introduces additional complexity, including the need for high-quality training data, machine learning expertise, and infrastructure for training and running models. As the system evolves, periodic updates and retraining may also be required.

Beyond tool selection and parameter extractions, custom orchestration must also handle tool execution issues, such as external API failures, slowness, and timeouts. Therefore, the agent must also implement retry and fallback mechanisms for tool executions to be reliable.

In summary, custom tools orchestration usually requires significant engineering efforts but provides maximum flexibility, enabling precise control over customization, latency, cost, and the ability to achieve high reliability and quality.

Function calling frameworks

Another approach to tool orchestration is to leverage existing frameworks and built-in LLM capabilities. These solutions can significantly simplify development while still providing strong orchestration quality and a reasonable level of customization.

Modern frameworks rely on the LLM's ability to perform tool selection and parameter extraction, while the framework itself handles the surrounding infrastructure. By abstracting much of the orchestration logic, these frameworks allow developers to focus on defining tools and high-level behavior rather than implementing low-level decision-making pipelines.

Figure 9.1 illustrates a typical data flow for tool orchestration using an LLM together with a function-calling framework:

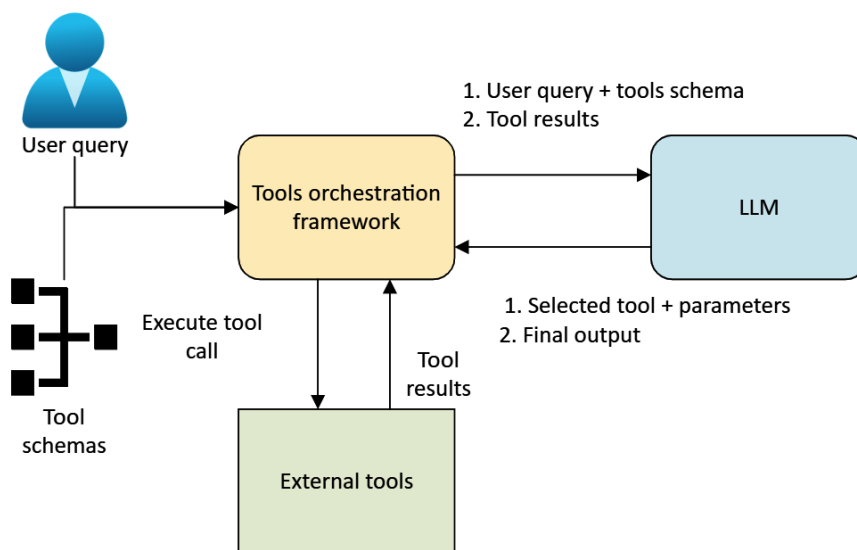


Figure 9.1: Data flow of tool orchestration using an LLM and a function-calling framework

As shown in the preceding figure, the process begins with a user query, which is passed to the tools orchestration framework. Along with the query, the framework provides the LLM with tool schema definitions, describing the available tools and their parameters.

The LLM then analyzes the request and decides whether tool execution is required. If so, it returns the selected tools along with the extracted parameters. The orchestration framework interprets this response, validates it, and executes the corresponding external tool. Once the tool is executed, the result can be passed back through the framework to the LLM, allowing it to generate a final response based on both the original query and the tool output.

Many frameworks support, completely or partially, the following paradigm. Let's consider some of them.

OpenAI function calling framework

OpenAI's platform supports function calling through both the Chat Completions API and the Assistants API (will soon be replaced by the Responses API). To use it, you first define the available functions, including their descriptions and parameter schemas, and then supply them as part of the model's tool configuration.

Here is an example of code in Python that registers tool definitions and runs the OpenAI chat completion API:

```
from openai import OpenAI
client = OpenAI()
tools = [{
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "Get current temperature for a given location.",
        "parameters": {
            "type": "object",
            "properties": {
                "location": {
                    "type": "string",
                    "description": "City and country e.g. Bogotá, Colombia"
                }
            }
        },
        "required": [
            "location"
        ],
        "additionalProperties": False
    },
}
```

```
        "strict": True
    }
}]

completion = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "What is the weather like in Paris
today?"}],
    tools=tools
)
print(completion.choices[0].message.tool_calls)
```

In this code example, we defined a single function, `get_weather`, with one required parameter (location), and passed that definition into the GPT chat completion request. When the user asks a question like *"What is the weather in Paris today?"*, the model produces an output in JSON format that specifies which function to call and what parameters to provide. It can look like this one:

```
[
  {
    "id": "call_12345xyz",
    "type": "function",
    "function": {
      "name": "get_weather",
      "arguments": "{\"location\":\"Paris, France\"}"
    }
  }
]
```

This output is essentially the orchestration plan (see *Chapter 8*). It contains the list of functions to be executed, their order, and the arguments to pass. In this case, the plan is very simple. One function call with a single parameter. But in more complex scenarios, the plan could involve multiple tools or sequential dependencies. Importantly, this is a static plan, generated before any function execution actually occurs.

Once the plan is generated, the application is responsible for carrying it out. That means calling the appropriate functions with the extracted parameters, collecting the results, and then sending those results back to GPT in a follow-up message (with role set to "tool"). GPT then integrates those tool outputs into its reasoning and produces the final response for the user.

So, we have the following flow for the OpenAI function calling framework:

- Call GPT to generate the orchestration plan
- Execute the plan by calling the specified functions with the required parameters
- Pass the results back to GPT in a second call so it can complete the user request with a final answer

So, as we can notice, the OpenAI framework does not execute the functions itself. It only generates the execution plan and then incorporates the returned results into the final reasoning. The actual function execution must be carried out by the application code.

While OpenAI's function calling is powerful, it also comes with limitations. The framework works best with a limited number of functions, does not allow customization of plan generation logic, and supports only static plans. It also lacks built-in stopping criteria, error handling, and retries, all of which must be implemented externally in the code.

We also need to mention that the model is fully responsible for tool selection and parameter extraction in one LLM call. In this regard, this flow is similar to the single-prompt flow we explored in the custom tools orchestration approach above. All hints to select the proper tools need to be provided inside the schema definitions.

LangChain function calling

Now let's take a look at how function calling can be implemented in the LangChain framework. Below is a code example to illustrate function calling in Python:

```
from langchain.chat_models import ChatOpenAI
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain.prompts import ChatPromptTemplate
from langchain.tools import tool

# Define a tool with defined schema.
@tool
def get_weather(location: str, day: str) -> str:
    """Return real-time temperature, humidity, and conditions for the given
    location and day """
    pass

tools = [get_weather]
llm = ChatOpenAI(model="gpt-4o", temperature=0.5)
SYSTEM_PROMPT = """You are a helpful weather assistant.
If the user asks about weather, decide whether to call the `get_weather` tool.
Only call tools when needed; otherwise, answer directly."""
```

```
prompt = ChatPromptTemplate.from_messages([
    ("system", SYSTEM_PROMPT),
    ("human", "{input}")
])

# Create a tool-calling agent and an executor.
agent = create_tool_calling_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

# Run: the agent decides if/when to call the tool and with what args.
user_q = "What is the weather like in Paris today?"
result = executor.invoke({"input": user_q})
print("\n=== Final Answer ===")
print(result["output"])
```

First, we defined the function to be called as a tool using the `@tool` decorator. Next, we created the LLM object by specifying a prompt and instructions about when to call tools. Then we set up an agent and executor to register tools in LLM. Finally, we executed a user request and obtained the answer.

As we can see from the code example, LangChain's framework automatically selected the appropriate tool, executed it with the extracted parameters, and incorporated the output into the final response. This framework provides more support for automating tools execution compared to OpenAI.

So, we explored function-calling frameworks provided by OpenAI and LangChain, which delegate tool selection and parameter extraction to the LLM. As discussed in *Chapter 8*, the quality of tool definitions plays a critical role in enabling the model to make correct orchestration decisions. At the same time, these frameworks have certain limitations, such as a restricted number of tools available for selection and limited support for fallback mechanisms.

Tools orchestration in an agentic platform

Now consider a scenario where agents are developed using agentic platforms such as Microsoft Copilot Studio or Google Vertex AI. In these environments, tool orchestration and execution are core built-in capabilities.

Such platforms allow developers to define external tools by specifying their schemas, parameters, and connections to external systems. Once configured, the platform automatically handles tool selection, parameter extraction, execution, and integration of results into the agent's response.

This approach provides a high level of convenience. It significantly reduces development effort, eliminates the need to implement custom orchestration logic, and enables rapid prototyping and deployment of agents.

However, this simplicity comes with limitations. Platform-based orchestration typically offers limited visibility and control over how decisions are made. Developers cannot fully customize how tools are selected, how parameters are extracted, or how execution flows are managed. Instead, they rely on the platform's internal logic, with configuration primarily limited to defining tool schemas and input/output parameters.

As a result, while agentic platforms are highly effective for many use cases and fast development, they may not be suitable for scenarios that require fine-grained control, advanced orchestration strategies, or highly optimized performance.

Model context protocol

As we can see, it is possible to integrate tools with different LLMs through different frameworks like OpenAI's function calling or LangChain frameworks and agentic platforms like Microsoft Copilot Studio. The challenge is that each framework, LLM, and platform has its own way of defining tools, passing parameters, and handling execution. This creates fragmentation: developers must learn and maintain multiple integration patterns, and tools often need to be rewritten for each ecosystem.

To address this, Anthropic introduced the **MCP**, an open standard now being adopted across the industry. MCP defines a universal protocol for exposing tools and exchanging context. This makes it possible to build a tool once and use it across different models, frameworks, and platforms, much like how USB provides a common interface for hardware devices.

MCP protocol opens up a wide range of possibilities for agents to access and utilize external functionality through a standardized, plug-in-like approach. Instead of building custom integrations for each tool, LLM, and data source, developers can expose capabilities in a consistent format that agents can discover and use with minimal additional effort.

The MCP is implemented as a **client-server architecture**. In this setup:

- *MCP server* exposes tools, data sources, or services through a standardized interface
- *MCP client* (typically an LLM application or agent framework) consumes tools

The client can call these tools as part of its orchestration process, without needing any custom integration code for each tool.

The following figure illustrates an example of an MCP server that exposes financial services and database functionality for use by an external LLM-based agent:

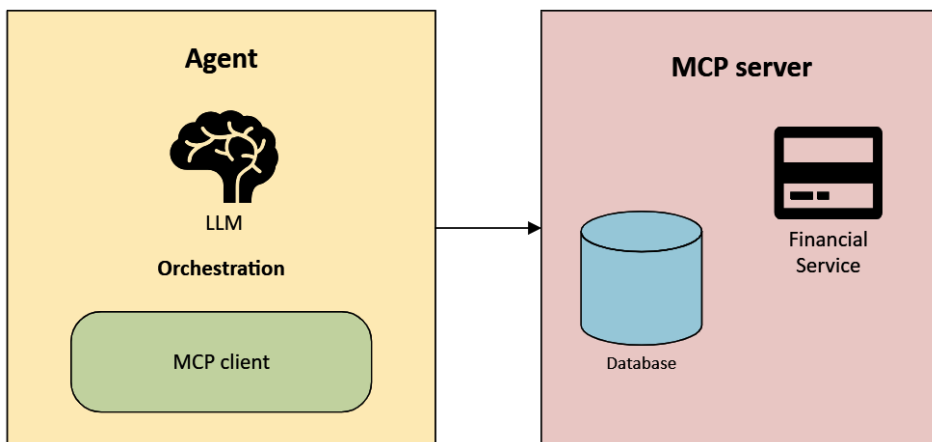


Figure 9.2: Interaction between an agent and an MCP server exposing financial services and database functionality

This client-server design provides several important benefits:

- **Decoupling:** Tools are defined once, on the server side, and can be reused across multiple LLMs frameworks or agents without rewriting
- **Interoperability:** Any MCP-compliant client can access any MCP-compliant server, creating a plug-and-play ecosystem for tools
- **Scalability:** Because tools are exposed as standardized endpoints, an orchestration layer can easily scale to dozens or hundreds of tools without collapsing into custom code

MCP uses JSON-RPC 2.0 as its transport messaging standard. So, it can run over local sockets, HTTP, or other transports, making it adaptable for both local development and cloud-scale deployments.

In practice, this means you can run an MCP server that exposes, for example, database access, a file system, or even APIs such as Slack, GitHub, or Google Drive. Any LLM framework or platform acting as an MCP client can then orchestrate these tools directly, integrating them into its reasoning loop as if they were native functions.

MCP can be seen as a new layer of interoperability, much like how RESTful APIs or SOAP services (legacy today) standardized access to web services over the past two decades. Today, REST APIs power everything from simple weather forecast services (e.g., OpenWeather) to global financial data providers (e.g., Bloomberg, Reuters).

Similarly, MCP aims to become the universal interface layer for AI-driven agents, allowing the creation of an ecosystem of MCP servers that expose nearly every kind of service on the

internet. So, we believe, MCP has the potential to power a unified marketplace of tools and services for LLM-based agents, making it possible to automate tasks across diverse domains with the same simplicity that APIs brought to web applications.

Despite these vast possibilities, we are still in the very early stages of MCP adoption. The vision is that almost everything available today via HTTP, REST, or SOAP will one day also be accessible through MCP, making AI agents truly powerful and versatile.

However, the protocol still faces several important technical challenges:

- **Security:** the protocol does not provide built-in, default authentication or integrity enforcement, leaving these responsibilities to implementations [51]. Security reviews have shown that weaknesses in current MCP implementations can be exploited to execute malicious code, exfiltrate sensitive data such as API keys or credentials, and compromise connected systems [52].
- **Protocol maturity:** The MCP ecosystem is still young. Official SDKs exist in multiple languages (Python, TypeScript, Java, C#, Go, etc.), and integrations have begun appearing in major IDEs. However, compared to mature web-based protocols, the ecosystem is not yet as stable or widely battle-tested.

So, we believe that future versions of the MCP protocol will gradually address many of these challenges, improving security, compliance readiness, and orchestration maturity.

Agent decision-making and planning

As we explored in the previous sections, agents require memory to retain context and tools (function calling) to interact with external systems and execute actions. However, having memory and tools alone is not enough. Agents must also be able to think, plan, and make decisions effectively using information from tools or memory. In this section, we will dive deeper into the agent's cognitive process, exploring how it reasons through complex tasks and adapts its approach based on context.

To do this, we will leverage the prompt engineering techniques we learned in *Chapter 3*, the LLM reasoning capabilities we discovered in *Chapter 5*, and the model fine-tuning explored in *Chapter 7*. These techniques, when integrated into agentic workflows, enable agents to break down complex problems, explore multiple reasoning paths, self-correct errors, and dynamically adjust their actions. By understanding how agents think, we can design more capable and autonomous systems.

It's important to note that while AI agents have made major strides, their reasoning capabilities generally do not yet match human-level performance when moving into tasks involving many decision steps, deep contextual or abstract understanding, long-term strategy, or intuitive judgment [35,33,87].

This limitation is especially evident in domains requiring precise calculations or real-world physical interactions, where even minor errors can lead to significant consequences. As AI research advances, new models and architectures, improved approaches, and technologies are gradually enhancing agent reasoning. But for now, human oversight and validation usually remain important and even critical in high-stakes applications.

Agent reasoning techniques

Many of the prompt engineering techniques introduced in *Chapters 3* and *5* can be directly applied to enhance agent reasoning capabilities. Agents can be guided to follow more structured reasoning processes by using **Chain-of-Thought (CoT)**, Tree-of-Thought, self-consistency, and similar techniques.

In practice, reasoning can be shaped through prompt design, multi-step workflows, and control logic that guide how the agent interprets inputs, plans actions, and produces outputs. In addition, modern reasoning-focused models like GPT-5.4 or Claude Sonnet (see *Chapter 5*) can further simplify this process by handling more of the reasoning internally, reducing the need for complex prompt engineering.

However, these approaches come with trade-offs. More advanced reasoning often increases latency and cost, and may reduce transparency, making debugging and control more challenging.

Fine-tuning (see *Chapter 7*) can further improve reasoning by exposing the model to task-specific patterns and domain-relevant decision processes. This allows the agent to learn consistent reasoning strategies and better align with the requirements of a particular application.

Overall, combining prompt engineering techniques, reasoning models, and fine-tuning provides a powerful toolkit for designing effective agent reasoning.

Planning strategies

Effective agents not only reason about problems but also plan and execute multi-step workflows. Complex real-world tasks often require structured decision-making, task decomposition, and the ability to chain multiple skills together. Several strategies can be used for multi-step decision-making:

- Sequential decision-making
- Hierarchical task decomposition
- Skill chaining

Let's consider them in more detail.

Sequential decision-making

For sequential decision-making, agents must decide the order of actions required to complete a complex task. This involves analyzing dependencies between steps and determining the most efficient execution path.

For example, a personal finance assistant may need to create a monthly budget plan. In this case, the agent must first determine the correct sequence of actions before generating the final output. The following prompt illustrates how such step-by-step reasoning and planning can be guided within the agent:

Prompt

You are a financial planning assistant that follows a structured decision-making process. When a user asks for budget advice, do NOT provide an immediate response. Instead, follow these sequential steps:

1. Gather the user's income and expense details.
2. Categorize spending into fixed and variable costs.
3. Identify any overspending categories.
4. Suggest personalized savings strategies.
5. Provide a detailed budget breakdown with recommendations.

Ensure that each step is followed in order and validated before proceeding to the next. If user data is missing, ask follow-up questions before continuing.

Such a prompt will direct LLM first to analyze different aspects of the personal budget, and only after that analysis will it generate the final plan. To get the information #1-5, the agent will need to execute external tools (function calling) or skills.

Such prompts (agent instructions) can also generate a dynamic list of analysis steps rather than a fixed, pre-defined sequence as we showed. This allows the agent to adapt its reasoning flow to the specific context of the query and the available data.

Hierarchical task decomposition

For large or complex goals, agents can break tasks into smaller, manageable sub-tasks, solving each sub-task before integrating them into a final response.

For example, consider a travel planner agent that needs to handle requests such as: "Plan a two-week Europe trip with a \$5,000 budget." In this case, the agent can decompose the high-level request into structured sub-goals and address them step by step. This approach can be illustrated with the following prompt:

Prompt

You are a travel planning assistant that organizes complex trips by breaking them

```
into smaller tasks. For any user request involving trip planning, follow a
hierarchical task decomposition approach:
- First, determine the user's travel preferences (dates, destinations, budget).
- Next, divide the total budget into major categories: flights, hotels, food, and
activities.
- For each destination, suggest flight and hotel options while maintaining budget
constraints.
- Plan a detailed daily itinerary, optimizing for cost and travel time.
- Present the final plan along with any trade-offs and alternative
recommendations.
If a sub-task depends on user input, pause and request clarification before
proceeding.
```

Such a prompt will define hierarchical decision making and execute the proper tools to get user preferences, search flights and hotels, and finally optimize the plan for the entire trip.

Skill chaining

Agents often need to call multiple external functions (*skills*) in sequence to complete a task. This approach is known as **skill chaining**, where an agent orchestrates different API calls, database queries, or computations in a structured workflow.

For example, we have a data analyst agent that must solve queries like this one: "Analyze sales data and generate a market trend report." Such an agent can be implemented with the following system prompt:

Prompt

```
You are an AI data analyst that processes sales reports by executing multiple
functions in sequence. For any sales analysis request, follow these steps:
- Retrieve relevant sales data using the `get_sales_data()` function.
- Apply statistical trend analysis using `analyze_sales_trends()`.
- Generate data visualizations using `generate_report()`.
- Summarize key findings and return a structured market analysis.
If any function fails, retry with adjusted parameters or request additional user
input.
```

Such a prompt, on the first step, queries the database for sales data (`get_sales_data()`), next applies statistical analysis to detect trends (`analyze_sales_trends()`), generates charts and summary insights (`generate_report()`), and finally returns a final market report to the user. Such a prompt can be executed with a framework that supports function calling, as we mentioned before.

So, here we have outlined only a few possible strategies, along with examples of how prompts can be used to implement them. In practice, planning for agents is a rich and evolving area, with many variations and hybrid approaches. The strategies we covered should give you a foundation, but real-world applications often require experimenting, adapting, and combining different methods to balance efficiency, accuracy, and robustness.

Error handling and recovery strategies

Even the most advanced LLM-powered agents inevitably encounter errors during reasoning, function execution, or user interaction. These failures can arise from invalid inputs, ambiguous queries, tool malfunctions, or model hallucinations.

To build robust, production-ready agents, we need mechanisms for error detection, classification, and recovery, allowing the agent to recognize problems, apply fallbacks, retry intelligently, or ask the user for clarification when needed.

Common categories of agent errors include:

- **Tools execution failures:** External APIs or system errors (e.g., failures, timeouts, rate limits, authentication issues)
- **Ambiguous or incomplete user input:** Missing or unclear details in user requests
- **Hallucination errors:** The model may fabricate information, generating execution plans that reference non-existent tools or parameters

One of the most common strategies for handling function call errors is implementing *retries*. In software engineering, retry mechanisms are a well-established reliability pattern, allowing a system to automatically reattempt a failed operation after a short delay.

Another option is to use cached data, if available, to serve the request without repeating the call. In some cases, agents can also fall back to alternative functions when predefined equivalents or backups exist.

To address ambiguous user inputs, agents should detect when a user's query is incomplete or underspecified and respond with clarifying questions instead of making incorrect assumptions. This can be guided through explicit prompt instructions such as:

Prompt

```
You are an AI travel assistant. If a
user's request is missing key details, do NOT assume answers.
Instead, ask clarifying questions to gather all necessary information before
proceeding.
```

Hallucination errors can occur when an agent calls a non-existent function or provides invalid parameters. These errors can be reduced by adding validation layers that check tool names, argument types, and schemas before execution.

If validation fails, the agent can retry the operation with error context to self-correct. Many modern frameworks, such as AutoGen and LangChain, perform this kind of automated retry and error propagation internally.

As we can see, effective error handling and recovery strategies are essential for building reliable agents, enabling them to gracefully handle failures and maintain robust operation in real-world scenarios.

Human-in-the-loop

Human-in-the-loop (HITL) decision-making is a hybrid approach where AI agents collaborate with humans to achieve more informed and reliable outcomes. Unlike fully autonomous systems, HITL ensures that a human can supervise, validate, or override AI decisions. HITL introduces human oversight to mitigate the following risks:

- **High-risk domains:** Medical, legal, and financial decision-making
- **Ethical concerns:** Preventing AI bias, misinformation, or unethical actions
- **Uncertain or ambiguous tasks:** Where human intuition and expertise are required
- **System training and learning:** Humans help AI improve by providing real-time feedback

Not all tasks require human involvement. HITL can be applied selectively based on risk level or task complexity. In practice, the agent can be prompted or programmed to assess their own confidence and decide whether human confirmation is needed. This can be achieved through the following:

- Prompt-based instructions, directing the agent to detect risk and escalate when necessary
- Rule-based logic, implemented in the orchestration layer to determine when human approval is required

For example, for a resume screening, an agent reviews candidate resumes and ranks applicants:

- If confidence > 90%, it automatically shortlists the candidate
- If confidence < 90%, it escalates the decision for human review

This ensures qualified candidates are not rejected due to low-confidence model judgments.

So, HITL can enhance error handling and AI safety by adding manual approval checkpoints in critical workflows.

Multi-agent frameworks and protocols

As discussed in *Chapter 8*, agents can communicate not only with humans but also with other agents. This enables the emergence of multi-agent systems, where multiple agents collaborate to solve tasks, coordinate actions, and share information.

In this section, we will cover a multi-agent framework: AutoGen. We will also introduce a common "language" for agents that serves as a standardized way to communicate, exchange information, and operate in multi-agent environments.

AutoGen framework (Microsoft Agent Framework)

In *Chapter 8*, we briefly introduced AutoGen. It is an open-source framework for building multi-agent workflows at the code level.

Note

AutoGen is being gradually phased out in favor of the **Microsoft Agent Framework**, a newer SDK developed by Microsoft as the next evolution of agent development tooling. This transition reflects lessons learned from real-world usage and combines ideas from both AutoGen and Semantic Kernel into a unified, more production-oriented framework.

While the tooling is evolving, the core concepts remain the same. Patterns such as multi-agent communication, orchestration, tool usage, and role-based collaboration are directly transferable. In fact, the Microsoft Agent Framework builds on these foundations rather than replacing them.

As a result, you should not be concerned about learning AutoGen in this chapter. The principles and patterns you gain here will apply to modern frameworks going forward, including the Microsoft Agent Framework and others in the ecosystem.

In AutoGen, every agent is treated as a *conversable entity*: it can receive, process, and respond to messages. Agents can be powered in different ways, depending on their role in the system:

- **LLM-powered agents (prompt-based)**: Agents driven by large language models. They can reason, generate responses, and interact with other agents based on pre-trained knowledge and ongoing conversational context.

- **HITL agents:** Agents that act as proxies for humans, incorporating human oversight, feedback, or direct input into the workflow. These agents ensure that critical decisions or validations can involve a person when needed.
- **Tool agents:** Agents that extend their capabilities by calling external resources such as APIs, databases, or search engines. Tool agents often rely on function calling, RAG, or other integrations to fetch real-time data, perform computations, or execute operations in external systems.

AutoGen can implement a wide variety of conversation patterns and schemas like the ones we explored in *Chapter 8*. To illustrate this, let's look at how the worker–supervisor pattern can be implemented with two agents: a code generator that writes Python code and a code reviewer that critiques and validates the output:

```
from autogen.agentchat import AssistantAgent, GroupChat, GroupChatManager

llm_config = {
    "cache_seed": 42, "temperature": 0.5,
    "config_list": [
        { "provider": "openai", "model": "gpt-4o", "api_key": "<OPENAI_API_KEY>" }
    ],
}

# define two LLM-powered agents
coder = AssistantAgent(
    name="Coder",
    system_message=("You are a senior software engineer. Generate clean,
production-quality Python code"),
    llm_config=llm_config,
)
reviewer = AssistantAgent(
    name="Reviewer",
    system_message=("You are a code reviewer. Review code for correctness,
performance, and tests."),
    llm_config=llm_config,
)

# put agents in a group chat and let a manager route the conversation
group_chat = GroupChat(
    agents=[coder, reviewer], max_round=4,
    speaker_selection_method="auto")
manager = GroupChatManager(groupchat=group_chat, llm_config=llm_config)
```

```

# start the conversation with a task for the Coder
task = (
    "Write a Python function `slugify(text: str) -> str` that converts arbitrary  

    text "  

    "into a URL-friendly slug (lowercase, hyphen-separated, strip punctuation/  

    accents). "  

    "Also include a small pytest file with 3-4 tests."  

)
result = manager.initiate_chat(coder, message=task)

# print results
print("\n--- Final Message ---\n")
print(result.summary or result.chat_history[-1]["content"])

```

In this example, the code begins by defining the LLM configuration. AutoGen supports a wide range of LLM providers, so the configuration can easily be adapted to OpenAI, Anthropic, Azure, or local models. Next, we define two agents: the code generator agent, which produces code, and the code reviewer agent, which checks correctness, style, and quality. These agents are LLM-powered and essentially correspond to constructed prompts bound to distinct roles and a proper LLM.

After defining the agents, we set up a group chat and a group chat manager. The manager is responsible for orchestrating the dialog, deciding which agent should speak next, and enforcing conversation rules. We then define a task, in this case, asking the generator to implement a Python function with accompanying tests. Once the chat is initiated and the conversation finishes, we print the result.

Finally, notice that when creating the group chat, we specified `max_round=4`. This parameter sets a hard limit on the number of turns (question-answers) between the agents, ensuring the conversation remains bounded and does not drift into endless back-and-forth exchanges.

To illustrate the type of dialog such code can generate, consider the following example:

- **Coder:** "I've implemented a `slugify(text: str)` function that converts text into lowercase, removes punctuation, and replaces spaces with hyphens. I've also added a small `pytest` file with three test cases".
- **Reviewer:** "Your implementation works for basic cases, but it doesn't handle accented characters like `café`. Consider normalizing Unicode input before stripping accents. Also, add a test case to verify this behavior."

- **Coder:** "Good point. I updated the function to use `unicodedata.normalize` to handle accents and extended the test file with a case for `café -> cafe`. The function now passes all tests."
- **Reviewer:** "LGTM. The function covers the requirements and the tests validate both normal and edge cases".

This simple interaction demonstrates how AutoGen can orchestrate a worker–supervisor loop: one agent produces output while another reviews and validates it. The back-and-forth interaction continues until the reviewer is satisfied or the maximum number of turns (`max_round`) is reached.

AutoGen provides several powerful capabilities that go beyond simple multi-agent conversations, making it highly flexible and production-ready:

- **Conversation termination controls:** In addition to `max_turns`, AutoGen supports parameters such as `max_consecutive_auto_reply` (to stop if an agent replies too many times in a row) and `is_termination_msg` (to stop if a particular message satisfies a given condition). This ensures conversations don't loop endlessly and can gracefully end when the goal is achieved.
- **HITL support:** AutoGen allows human intervention at any point in the conversation. This enables workflows where an agent can escalate decisions, ask for confirmation, or transfer control to a human operator, enabling hybrid AI-human systems.
- **Code execution:** AutoGen includes a special execution component that can take input messages as code, execute them, and return results as output messages. Supported environments include UNIX shells and Jupyter kernels. This allows agents to run scripts, test code snippets, and even perform live data analysis during a conversation.
- **Function calling (tools):** Agents can call external functions or tools using a function-calling interface. Tools are pre-defined functions that extend the agents' capabilities, allowing them to query APIs, fetch data, or perform actions outside the conversation context.
- **Rich conversation patterns:** AutoGen supports a variety of conversation setups: two-agent chats, sequential multi-agent workflows, group chats, and even nested (*hierarchical*) chats where sub-conversations are created and managed within the main thread.

Note that AutoGen can execute code in a secure, containerized environment as well. Running LLM-generated code inside a sandboxed container improves safety and reduces the risk of malicious or accidental system modifications.

AutoGen (Microsoft Agent Framework) is not the only agent framework available. Alternatives such as LangChain, LlamaIndex, Semantic Kernel, and Haystack also provide abstractions for orchestrating agents, managing tools, and chaining LLM calls. Each framework has its own strengths and limitations. For example, LangChain focuses heavily on composable chains and tool integration, while LlamaIndex is optimized for retrieval-augmented workflows. Choosing the right framework depends on your use case, ecosystem preferences, and the level of control you need over agent behaviors.

So far, we have explored how low-level or code-based agents can collaborate with each other. They live within a single application and form higher-level or business-oriented agents. Now, let's look at how business-level agents can collaborate. These agents may be written in different languages, deployed on different platforms, and run on different infrastructures.

Agent-to-agent protocol

In April 2025, Google and a group of partners introduced a standard for multi-agent collaboration. It was called the **agent-to-agent (A2A)** protocol. A2A is an open protocol designed to let AI agents communicate directly with one another, securely share context, and coordinate task execution.

The A2A protocol complements the MCP we explored earlier by extending interoperability from agent-to-tool communication to agent-to-agent communication. That enables business-level agents from different providers to connect and collaborate.

The protocol is built around several guiding principles:

- **Structured communication:** A2A relies on structured, machine-readable messages rather than free-form natural language. This ensures reliable, predictable, and interoperable communication between agents across different systems.
- **Based on existing standards:** A2A uses widely adopted protocols like HTTP, **server-sent events (SSE)**, and JSON-RPC, which makes it straightforward to integrate into existing enterprise infrastructure.
- **Enterprise-grade security:** From the start, A2A includes support for strong authentication and authorization.
- **Support for long-running tasks:** Agents can manage tasks that last over minutes, hours, or even days, while keeping each other (and the user) updated with progress notifications, state changes, and intermediate outputs.
- **Modality agnostic:** Communication isn't limited to text. A2A is designed to handle multiple modalities, such as images, audio, or even video streams.

At its core, the A2A protocol defines how a **client agent** (the one that issues a task) and a **remote agent** (the one that executes it) interact. In practical terms, A2A adopts the familiar client–server architecture that has long been used in the software industry.

The remote agent acts as the server: it exposes its capabilities by publishing an **agent card**, a structured JSON document that describes what the agent can do, the types of inputs it accepts, and the modalities (text, images, audio) it supports. The client agent then reads this definition to discover the available skills, select the most appropriate skill for a given task, and initiate communication. From that point, the two agents exchange messages, status updates, and artifacts until the task is completed.

The following diagram depicts the A2A client–server interaction between a client agent and a remote agent via an agent card:

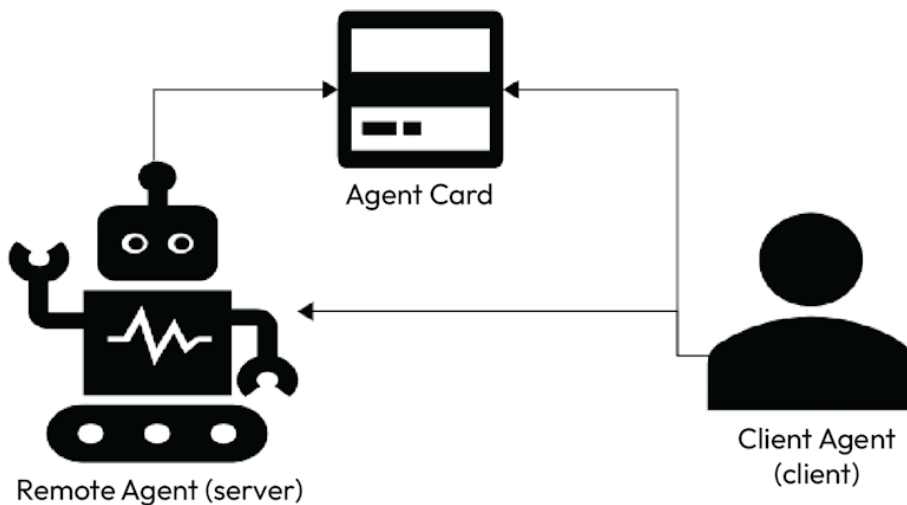


Figure 9.3: A2A design diagram

The A2A protocol includes several key capabilities:

- **Discovery:** Agents can tell what they can do through an *agent card* (a JSON description). This allows a client agent to identify which remote agent is best suited for a given task.
- **Task lifecycle management:** Communication revolves around tasks, represented as structured objects. Tasks may be completed immediately or span longer periods.

- **Collaboration and messaging:** Agents can exchange contextual messages, intermediate results, and artifacts to refine and complete tasks collaboratively.
- **User experience negotiation:** Messages can include structured "parts" (e.g., an image, a video snippet, or an interactive form). These parts let agents negotiate the most appropriate format and presentation for the end user's interface.

In short, A2A provides a common communication layer that enables agents from different vendors and ecosystems to work together securely and productively, making large-scale multi-agent collaboration practical for real enterprise use cases.

Let's now see how to define an A2A collaboration using the Google ADK A2A toolkit in Python. The first step is to create an A2A-enabled agent:

```
from google.adk import Agent
from google.adk.a2a.utils.agent_to_a2a import to_a2a

# Define the agent
root_agent = Agent(
    model="gemini-2.0-flash",
    name="hello_world_agent",
    instructions="You are a helpful assistant that responds with 'Hello, World!'
to any input"
)

# Expose the agent via A2A
a2a_app = to_a2a(root_agent, port=8001)

# Start the HTTP server (run with uvicorn):
uvicorn path.to.module:a2a_app --host localhost --port 8001
```

In the example code, we first define a simple "Hello World" agent with a name and basic instructions. Next, we expose this agent via the A2A protocol and launch it from the command line (last code line). Once started, the agent runs as a dedicated process on the local machine and automatically publishes its agent card at the following endpoint: `http://localhost:8001/.well-known/agent-card.json`

The agent card is a JSON object that describes the agent's identity and capabilities. A simplified example might look like this:

```
{
  "name": "hello_world_agent",
```

```
"description": "You are a helpful assistant that responds with 'Hello, World!'  
to any input.",  
"url": "http://localhost:8001",  
"protocolVersion": "0.2.6",  
"capabilities": { "streaming": true, "pushNotifications": false },  
"skills": [  
  { "id": "hello_world_agent-ask_question", "name": "ask_question", "tags":  
    ["llm", "tools"] },  
]  
}
```

This JSON structure defines the agent's name, a description of its purpose, the HTTP endpoint through which it can be reached, the supported protocol version, and its skills (in this case, the `ask_question` skill). Other agents acting as clients can read this card to discover what the agent can do and how to interact with it. So, in A2A, the agent card serves a role similar to a service contract in web services, such as REST or SOAP APIs.

Once discovery is complete, the client can initiate communication using JSON-RPC calls. Here's a simplified example of a message where the client sends a greeting message:

```
{  
  "jsonrpc": "2.0",  
  "id": "req-1",  
  "method": "tasks/send",  
  "params": {  
    "id": "task-123",  
    "sessionId": "session-abc",  
    "message": {  
      "role": "user",  
      "parts": [{ "type": "text", "text": "Hello agent" }]  
    }  
  }  
}
```

In this request, the client calls the remote agent's endpoint and sends a task containing the simple message "Hello agent." The response comes back in the same structured JSON-RPC format. In practice, the message could be any kind of instruction, like asking the agent to perform a computation, retrieve data, or initiate a long-running workflow.

This simple example illustrates the fundamental flow of A2A: an agent advertises its skills through an agent card, a client agent discovers and selects it, and the two communicate via

structured JSON messages. From here, the same mechanism can be extended to more complex, multi-step collaborations between agents across different providers.

The full A2A protocol specification is far too detailed to cover in this book. Instead, our goal here is to highlight the core concepts and show how they fit into the broader picture of multi-agent communication. For those interested in the finer details, such as the complete message schema, task lifecycle states, or advanced streaming patterns, we encourage you to explore the official A2A documentation and reference implementations.

Implementing AI agents in practice

We are now approaching the end of our journey in building LLM-based agents. Throughout these eight chapters, we have covered both the theoretical foundations and practical aspects of designing and developing modern AI agents. Now it's time to bring everything together.

In this final section, we will walk through an end-to-end example of building an AI agent: a *personal email filtering agent*. Step by step, we will implement this agent and explore the key design decisions, trade-offs, and challenges that arise in real-world scenarios.

Email filtering agent – requirements

In *Chapter 8*, we briefly introduced a personal email filtering agent. Now, let's apply the design principles from this chapter and walk through the key steps to define and build this agent.

Due to the scope of this book, we will focus on the most important aspects of this agent. However, in practice, production-ready agents would require more detailed design and additional considerations. At the end of this section, we will provide the full code implementation of this agent as well.

Functional requirements

We want to develop an agent that processes incoming user emails and performs the following actions:

- **Ignore:** If an email is not important, the agent takes no action
- **Escalate:** If an email is urgent and requires immediate attention, the agent sends a mobile push notification to the user
- **Summarize and log:** If an email contains important updates or status information, the agent generates a summary and stores it for inclusion in a daily report
- **Open dialog:** If an email contains open questions or missing information, the agent initiates a follow-up interaction to gather the required details and then classifies the email into one of the categories above

So, overall, this agent improves the efficiency of email management by automatically categorizing, prioritizing, and summarizing incoming messages, while initiating follow-up actions when needed. This reduces the user's cognitive load, ensures urgent email matters are addressed promptly, and minimizes the need for constant manual inbox monitoring.

In addition, the agent should support natural language interaction. Users should be able to ask questions about current or recent emails, as well as historical data within a defined retention window (for example, 6–12 months).

Since the goal is to reduce user effort, the agent should be accessible via a mobile application. This includes sending push notifications for urgent emails, allowing users to query their email activity, and presenting daily summaries.

However, mobile devices cannot efficiently host LLMs (except for very small models), and certain data must be stored and processed externally. Therefore, a server-side component is required to support model inference, data storage, and overall system functionality.

At this stage, we also need to define quality thresholds for the agent. To be useful, the agent should maintain a high level of accuracy in email classification, with no more than 3–5% misclassifications. Similarly, daily reports should capture the vast majority of important information, with no more than 3–5% of key details missed.

In addition, generated summaries must be factually grounded, containing only information derived from emails and avoiding hallucinations.

System requirements

Since this agent is designed for a large number of individual users, it must support high scale and variable load. At the same time, operational costs must remain low to ensure the solution is affordable for a broad user base.

Users may receive dozens or even hundreds of emails per day, and these emails can vary in size and format, including long text and embedded content such as images. Therefore, the agent must efficiently process high volumes of data while maintaining acceptable latency and cost. This requires careful design of model usage and data processing pipelines to ensure scalability and performance in real-world conditions.

Interaction mode

At first glance, this agent may seem to be a background agent, as it processes incoming emails autonomously in the background. However, it also needs to interact with users or other agents, for example, by initiating follow-up conversations, sending urgent notifications, presenting daily reports, and answering user questions.

Based on these requirements, this agent is best described as a hybrid one that supports both background processing and conversational interactions.

In this setup, the agent can be triggered by events such as:

- Incoming emails
- User requests, such as asking for daily reports or querying past emails

As a result, the agent produces outputs such as:

- Urgent email notifications
- Daily summaries and reports
- Interactive responses to user queries

Implementation approach

Since this agent is relatively complex and requires both a mobile application, backend storage, and processing capabilities, we will use a code-first approach. The mobile application will be implemented in TypeScript, while the backend service will be developed in Python to handle model inference, data storage, and orchestration.

Skills, tools, and capabilities

Based on the functional requirements, the agent needs the following skills, tools, and capabilities:

- Monitor the user's inbox and detect new incoming emails
- Read and process email content
- Send push notifications for urgent emails
- Classify emails into categories such as urgent, important, requires follow-up, or ignore
- Generate and present daily reports summarizing email activity
- Answer user questions about current and historical email reports

Identifying these capabilities helps determine which external tools, APIs, and systems the agent must integrate with in order to operate effectively.

Memory, state, and learning

This agent operates in two main modes: background processing and conversational interaction, each with different memory requirements.

For background processing, the agent analyzes incoming emails, classifies them, and prepares summaries. This part requires only minimal STM, primarily to process the current email and temporarily store intermediate results before updating the daily report.

However, the agent also requires LTM to persist information across sessions. Specifically, it needs to store:

- Draft daily reports that accumulate summaries of important (but not urgent) emails throughout the day
- Historical daily reports retained over a defined period (e.g., 6–12 months)

In addition, the agent supports conversational functionality, including email follow-ups and answering user questions about reports. These interactions will involve multi-turn conversations, requiring efficient STM to maintain context and ensure coherent dialogue.

Designing memory for this agent, therefore, involves combining STM for real-time processing with structured LTM for persistence and retrieval across interactions.

Reasoning and control mechanisms

In this step, we define how the agent makes decisions and controls its behavior. The agent must support the following decision-making tasks:

- Classify emails (urgent, important, requires follow-up, or ignore)
- Answer user questions about daily activity and reports
- Initiate and manage follow-ups when additional information is required

The first two tasks are relatively constrained. Email classification operates on a single input and produces a single label, while answering user questions typically results in a single response.

In contrast, managing follow-ups is more complex and requires explicit control mechanisms to avoid excessive or unnecessary communication. Without proper constraints, the agent may generate too many follow-up messages or create loops. To address this, the agent should enforce rules such as:

- **Single reply per email:** Do not send multiple follow-ups without receiving a response
- **Limit the number of follow-ups:** For example, no more than 3–5 attempts
- **Stop when sufficient information is obtained:** Avoid continuing the conversation unnecessarily

These control mechanisms ensure that the agent behaves predictably, avoids over-communication, and maintains a good user experience.

Safety and control boundaries

Since the agent supports two natural language interfaces, it is essential to implement robust safety mechanisms to ensure appropriate and secure behavior. We will cover safety and responsible AI in more detail in *Chapter 10*; here, we highlight the most important risks.

In particular, follow-up interactions must be carefully controlled to prevent the exposure of sensitive information. The agent should only engage in focused, task-specific communication and must enforce strict limits on what information can be shared with external users or other agents.

This is especially important because such interfaces can be exploited by malicious actors attempting to extract personal or confidential data. Therefore, the agent must include safeguards such as input validation, response filtering, and clear permission boundaries to protect user data and maintain secure interactions.

Evaluation and success metrics

We will explore agent evaluation in more detail in *Chapter 10*. At this stage, it is important to recognize that the overall quality of the agent depends on the quality of the individual tasks it performs. These include the following:

- Email classification
- Email summarization
- Follow-up question answering
- User question answering

Each of these tasks must meet a high-quality bar as we mentioned in the functional requirements above. For example, incorrect email classification may result in urgent or important messages being missed. Similarly, poor summarization can lead to incomplete or inaccurate daily reports.

Therefore, evaluating and optimizing each component is essential to ensure the reliability and usefulness of the agent as a whole.

Deployment and operations

Since the agent consists of both a client-side mobile application and a server-side backend, an appropriate deployment and release strategy is required.

The mobile application can be distributed through platforms such as the Apple App Store or Google Play. The backend service, which handles model inference, data processing, and storage, should be deployed in the cloud using providers such as Microsoft Azure, Amazon Web Services, or Google Cloud.

This setup enables scalable, reliable operation while supporting continuous updates and maintenance of both the client and server components.

Email filtering agent – design

As we can see, this agent is relatively sophisticated. Taking into account the requirements outlined above, we can now derive its high-level architecture, as shown in the following figure:

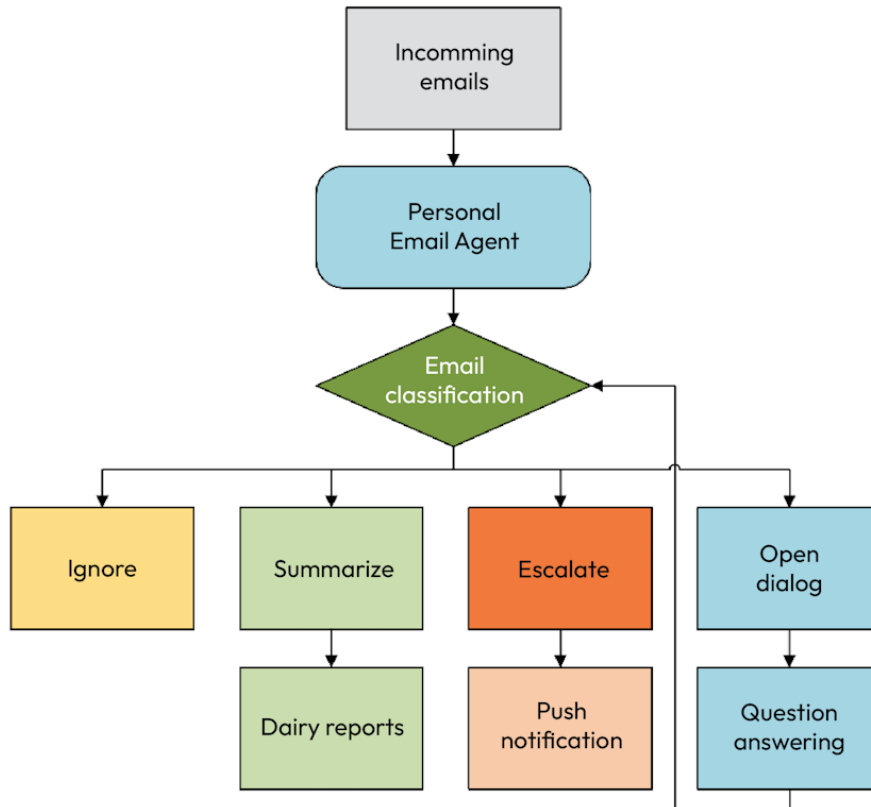


Figure 9.4: High-level architecture diagram of the personal email filtering agent

In the preceding diagram, incoming emails are first processed by the personal email agent and then classified into one of several categories. Based on the classification result, the agent takes the appropriate action: ignoring non-relevant emails, summarizing important ones for inclusion in daily reports, escalating urgent messages via push notifications, or initiating a dialog when follow-up is required.

Since the agent consists of both a backend service (*server*) and a mobile application (*client*), its functionality is distributed across these components. The backend is responsible for tasks such as email processing, classification, summarization, and data storage, while the mobile application handles user interaction, notifications, and presentation of results.

The following diagram illustrates the high-level client–server architecture of the system:

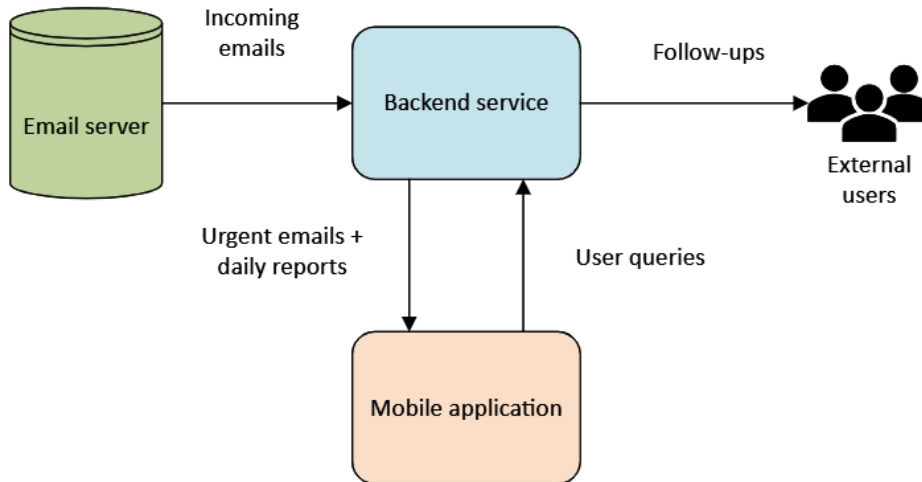


Figure 9.5: Client-server architecture of the personal email filtering agent

In this diagram, incoming emails are received from the email server and processed by the backend service. The backend performs classification, summarization, and decision-making, and then sends urgent notifications and daily reports to the mobile application.

The mobile application serves as the primary interface for the user, allowing them to receive notifications, view reports, and submit queries. User requests are sent back to the backend service for processing.

In addition, the backend service can initiate follow-up communications with external users when additional information is required. This architecture separates processing and interaction responsibilities, enabling scalable and efficient operation.

So, we now have a clear picture of the agent's design and can move on to the implementation phase.

Email filtering agent – implementation

Now, let's move to the implementation of the email filtering agent. Since most of the core functionality resides in the backend service, we will focus primarily on that component. The mobile application serves mainly as the user interface, providing access to urgent notifications, daily reports, and conversational interactions.

The agent backend service can be decomposed into the following core components:

- **Email classifier:** Responsible for email classification. The input includes the email content and user-defined preferences (e.g., what is considered urgent, important, or ignorable). The output is the assigned category. This component can be implemented using an LLM with prompt engineering and, optionally, optimized using smaller models with fine-tuning.
- **Email summarizer:** Responsible for generating a daily summary of important (but not urgent) emails. Since emails arrive continuously, the summary must be built incrementally. We will implement this component with an LLM and prompting.
- **Follow-up service:** Responsible for initiating follow-up communications when additional information is needed. Inputs include the original email, prior follow-ups, and constraints on what information can be shared. The output is a follow-up message. This component will be implemented with LLMs, prompts, and STM to maintain context.
- **User question responder:** Responsible for answering user queries about emails and reports. It takes user questions and historical reports as input and produces context-aware responses. This component supports multi-turn conversations and will be implemented using RAG, LLMs, and STM.
- **STM:** Supports multi-turn interactions for both follow-ups and user queries. A hybrid strategy can be used, such as keeping the most recent interactions in full, summarizing older ones, and truncating long histories to control context size.
- **Email report storage:** Responsible for storing daily reports and enabling retrieval for future queries.

These components form the foundation of the agent. The system can be further enhanced with additional capabilities, such as:

- **HITL** mechanisms for handling critical or ambiguous situations.
- **Learning from feedback** to improve the agent over time. Here, we can use approaches like MAL.
- MCP and A2A interfaces to expose the agent's capabilities to other agents and systems.

At this point, you have everything needed to start implementing such an agent. We strongly encourage you to build your own version from scratch. You can use tools such as Claude Code or GitHub Copilot to speed up development. You have all the requirements in place. You can then deploy it locally or in the cloud and test it with a simulated mobile application.

If you encounter challenges, you can also explore our reference implementation available on GitHub. We encourage you to go beyond this example and improve upon it by handling additional edge cases and real-world scenarios.

To explore the full implementation of the email filtering agent, refer to the GitHub repository linked earlier in this chapter.

Summary

Great, we have now completed the chapter on building AI agents, where we moved from high-level architecture to practical implementation. In the previous chapter, we focused on understanding how agents are structured. In this chapter, we explored how to design, build, and deploy them in real-world scenarios.

The AI agent ecosystem is evolving rapidly, and we are still at the early stages of a new class of applications with capabilities approaching human-level tasks. In this chapter, we examined some of the most common frameworks, platforms, and approaches for building AI agents. While many of these tools will continue to evolve, the core principles and design patterns are likely to remain stable.

Finally, we brought these concepts together by designing a personal email filtering agent and encouraging you to implement it yourself, supported by a reference implementation available on GitHub.

With this foundation, you are now equipped to build practical AI agents, choose appropriate tools and platforms, and design systems that operate effectively in real-world environments. The final topic we will explore is how to evaluate such agents and ensure they meet the required quality standards.

In the final chapter, we will focus on one of the most critical aspects of AI systems, evaluation, and learn how to measure and ensure the quality of agents.

Get this book's PDF version and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.

10

Evaluating LLM Applications and Agents

So, we've almost reached the end of our exciting journey through the world of AI, LLMs, and agents. Now it's time to turn to one of the most critical, yet often underestimated, aspects of the entire AI development lifecycle: *evaluation*.

You can design the most sophisticated LLM application or orchestrate the cleverest AI agent, but without proper evaluation, you're essentially flying "blind". You won't know if your agent is producing useful results, making harmful mistakes, or simply failing silently. Evaluation is how we gain visibility into the quality, reliability, and impact of what we've built and how we iterate and improve it.

In this final chapter, we will define what evaluation means in AI. We will examine how to design evaluation workflows and how to build and use evaluation datasets. You will learn about different types of evaluation metrics, from closed-ended task evaluation to similarity-based approaches for open-ended tasks, as well as human-as-a-judge and LLM-as-a-judge methods.

We will also compare offline and online evaluation techniques and dive into agent-specific evaluation, including **retrieval-augmented generation (RAG)**-based, conversational, and background agents.

Finally, we will cover responsible AI considerations, including safety, bias, and adversarial testing, and review tools and frameworks that support reliable and scalable evaluation.

This chapter focuses on the following key topics:

- Evaluation challenges
- Evaluation datasets
- Evaluation metrics
- Agent evaluation
- Responsible AI
- Evaluation tools and frameworks

Technical requirements

The code for this chapter is available on the book's accompanying GitHub repo: <https://github.com/PacktPublishing/Building-Agent-Powered-Applications/tree/main/Chapter10>

Evaluation challenges

Traditional software is typically rule-based, with clearly defined logic and deterministic behavior. As a result, it can be tested case by case, scenario by scenario, because the expected outputs are known and can be precisely asserted. This deterministic nature contrasts sharply with the *probabilistic* and *generative* behavior of AI systems, especially LLM-based ones, where evaluation is much less straightforward.

The key challenge in evaluating AI lies in the fact that these systems must handle an almost unlimited range of input variations and produce equally diverse outputs. If we have a model that detects whether an image contains a cat, the inputs can vary widely in breed, pose, lighting, fur color, and countless other factors. Evaluating such models on just a few test cases isn't enough.

This challenge becomes even more complex with LLM-based applications, where the outputs themselves are free-form text. LLMs take in arbitrary input text (a prompt) and generate open-ended responses. That raises a fundamental question: how do we know if the output is correct, useful, or aligned with the task?

Take an example of a chatbot (conversational agent). It may respond fluently and confidently, but how can we be sure its answers are accurate, complete, and aligned with the user's intent? There's often no single "correct" answer, and quality depends on nuanced dimensions like coherence, factuality, helpfulness, personalization, and tone.

The probabilistic nature of AI models adds yet another layer of complexity. All applications built on top of LLMs inherit this property. This means that even with the exact same input, an LLM might produce slightly different outputs across runs.

All of this makes evaluation an essential part of any AI development process, including LLM-based applications and agents. Without evaluation, improvements become guesswork rather than informed iteration. In the upcoming sections, we will explore how to approach evaluation in a structured and practical way.

Evaluation strategy

To dive deeper into how to approach evaluation and what steps we have to take, let's explore a simple example of an AI agent. We will use that example to walk through the main concepts and ideas behind evaluation.

Imagine we need to evaluate an AI chatbot that provides weather forecasts based on user queries, such as: *"What will the weather be in Bangkok next Monday?"* The expected output might be: *"The weather in Bangkok, Thailand, on Monday, 25th July 2025 will be cloudy, no rain, with a temperature range from 25°C to 31°C."*

First, we can observe that the input is a user question, essentially a piece of text, and the output is a weather response, also in text form. Next, it's important to recognize that we're dealing with a text generation problem (see *Chapter 4*) or more accurate question answering (*Chapter 5*). We can also notice that LLM by itself cannot know the current weather. It needs to be grounded with real-time weather forecast data. This means our task is not purely open-ended text generation; instead, it's a grounded generation problem where the output is constrained by factual, up-to-date information.

Understanding the exact type of AI problem we are dealing with is crucial because it shapes the entire evaluation process. It informs what scenarios and edge cases we need to test, which metrics are most relevant, and how to design an evaluation dataset that truly reflects the problem's requirements.

The next step is *input–output analysis*, examining in detail what kind of data goes in and what comes out. This step guides the design of the evaluation dataset and informs which assessment methods and metrics will be most appropriate for measuring performance.

From *Chapter 1*, we know that a dataset is a collection of examples (or samples) used as inputs to our AI model or application. Each example may also include the expected or desired output corresponding to the given input. For evaluation, such a dataset acts as our *benchmark*, allowing us to compare the AI's actual outputs with the correct ones.

In our weather chatbot example, the evaluation dataset might look like this:

Id	Question	Expected answer
1	What will the weather be like in Bangkok next Monday?	The weather in Bangkok, Thailand, on Monday, 25 July 2025, will be cloudy, with no rain, and a temperature range from 25°C to 31°C.
2	Will it rain in Tokyo tomorrow?	No, it will be mostly sunny in Tokyo, Japan, on 12 August 2025, with temperatures between 27°C and 33°C.

Table 10.1: An overview of the evaluation dataset of the weather chatbot

We can also notice that for the first question, the correct answer could also be phrased as *"Expect cloudy skies and no rain in Bangkok, Thailand on Monday, 25 July 2025, with temperatures ranging from 25°C to 31°C"* or some other form. All those forms are valid, factually correct responses.

This reveals an important insight: in many AI applications, there is rarely a single 'perfect' or correct answer. Instead, there may be many correct variations that differ in wording, style, or level of detail but still convey the same factual meaning.

For other closed-ended problems, such as text classification or information extraction (see *Chapter 4*), the situation is quite the opposite. Here, there is only one or at most a few correct answers, and the evaluation focuses on whether the output matches the expected result exactly. For example, in sentiment analysis, a message can be classified as either positive or negative, but not both. This contrast highlights an important point: open-ended and closed-ended problems require fundamentally different evaluation approaches.

Now, a natural question arises: how can we evaluate open-ended outputs when there is no single correct answer?

To understand this challenge better, let's relate it to the example we discussed earlier with the weather forecast problem:

Question	Correct answers
What will the weather be like in Bangkok next Monday?	The weather in Bangkok, Thailand, on Monday, 25 July 2025, will be cloudy, with no rain, and a temperature range from 25°C to 31°C.
	Expect cloudy skies and no rain in Bangkok, Thailand, on Monday, 25 July 2025, with temperatures ranging from 25°C to 31°C.
	On Monday, 25 July 2025, Bangkok will be cloudy with no rain and temperatures between 25°C and 31°C.

Table 10.2: Sample outputs for a weather forecast agent query

Looking at the correct answers, we can notice a few key points:

- **Inclusion of requested information:** The answer contains the key details requested in the query. In this case, the location (*Bangkok, Thailand*) and the date (*25 July 2025*).
- **Accurate weather attributes:** The answer correctly provides the requested weather details: cloudy, no rain, temperature 25°C to 31°C.
- **Appropriate formatting and presentation:** The information is presented clearly, with consistent structure and readable phrasing.

These observations give us a clear intuition for how to evaluate our responses. For factual and structured parts of the answer, we can treat the task as a form of information extraction (*Chapter 4*). The evaluation then becomes a matter of checking whether the correct entities were extracted and presented: location entity (e.g., Bangkok, Thailand), date entity (e.g., 25 July 2025), weather condition (e.g., cloudy, no rain), temperature values (e.g., 25°C to 31°C, or correctly converted to °F for relevant audiences).

So, if all such attributes are present and correct, we have a good, correct output. So, such an evaluation is a closed-ended problem evaluation because the entities in the output should be exactly as expected.

However, accuracy alone isn't enough. Since this is also a text generation problem, we must assess how well the information is communicated, and how well the text is well-formed, clear, and concise. So, for this open-ended text, we need to introduce specific metrics that evaluate

fluency (naturalness and grammatical correctness of the text) and *coherence* (logical flow and consistency of the answer). We will dive into that a little bit later in detail.

In summary, for a grounded generation task like this weather chatbot, evaluation involves two complementary perspectives:

- **Content accuracy:** Verifying that all required facts and entities are present and correct
- **Presentation quality:** Ensuring that the response is clear, coherent, and easy to read for the target user

Now that we understand the AI problem for our weather chatbot, what kind of dataset we need, and how to evaluate its outputs, the next question is: *How do we organize the evaluation process?*

In classical software engineering, we have different kinds of tests, such as unit, integration, and end-to-end tests. AI evaluation can follow a similar structure. When it runs on a predefined, well-specified dataset in a non-production environment, it's called **offline evaluation**. Offline evaluation is essential for validating AI functionality before it reaches real users. In our case, we could prepare a set of questions along with the expected outputs and use them to test the chatbot. An example of an offline evaluation dataset might look like this:

Id	Question	Expected entities	Expected text quality
1	What will the weather be like in Bangkok next Monday?	Location: Bangkok, Thailand Date: 25 July 2025 Weather condition: cloudy, no rain Temperature: 25–31°C	Fluent, coherent, concise, and relevant
2	Will it rain in Tokyo tomorrow?	Location: Tokyo, Japan Date: 12 August 2025 Weather condition: mostly sunny Temperature: 27–33°C	

Table 10.3: An offline evaluation dataset

In offline evaluation, we take each question, pass it to the chatbot (via an API call), collect the output, extract the required entities, check them against the "Expected entities" column, and then evaluate the quality of the text using the criteria in the "Expected text quality" column. The correct expected outputs are often referred to as the *ground truth*.

In contrast, **online evaluation** is performed in a production environment (real-time product usage), using real customer queries instead of predefined ones. This type of evaluation is not always possible because we often don't have a clear ground truth for arbitrary user inputs.

So, now, let's summarize the core concepts of evaluation strategies and outline the key steps required to perform an effective evaluation:

1. **Identify the AI problem type:** Understand whether you are dealing with open-ended generation, closed-ended prediction, classification, information extraction, reasoning, or a hybrid task
2. **Perform input–output analysis:** Clearly define what data goes into the system and what comes out. This helps understand the data flow, formats, anticipate cases, scenarios, and edge cases, and determine the structure of your evaluation dataset.
3. **Define the evaluation dataset and ground truth:** For offline evaluation, prepare a representative set of inputs and the corresponding correct outputs (ground truth). For open-ended tasks, ground truth may include multiple valid variations.
4. **Determine evaluation criteria and metrics:** For our weather forecast example, we considered content accuracy and presentation quality. For other tasks, it could be another type of validation; we will explore that later.
5. **Choose evaluation mode:** *Offline evaluation* to test on predefined datasets in a non-production environment or *online evaluation* to test in production using real user queries

To perform step #1 above, we can refer back to *Chapters 4* and *5*, where we explored in depth the main AI and NLP problems solved by LLMs. Steps #2, #3, and #5 focus on building evaluation datasets, while steps #4 and #6 relate to defining and applying evaluation metrics. We will cover all these aspects in more detail in the following sections.

Multi-task AI agents' evaluation

Above, we considered the relatively straightforward example of a weather forecast agent that simply responds to user questions. However, in real-world applications, AI agents often can follow much more sophisticated workflows, and their evaluation becomes correspondingly more complex.

To illustrate this, let's revisit the personal email filtering agent we introduced in *Chapter 8* and designed in *Chapter 9*. As a reminder, this agent effectively performs the following AI tasks:

- **Email classification:** Deciding whether to *Ignore*, *Escalate*, *Summarize and Log*, or *Open Dialog*
- **Custom summarization:** Summarizing selected emails into concise summaries for the daily activity report
- **Question Answering (QA):** Conducting follow-up conversations and user questions for past email activities

Since these tasks are independent, they can (and should) be evaluated separately, each with its own datasets, metrics, and methodologies:

- For classification, we need a labeled dataset containing diverse email examples tagged with the correct action category
- For summarization, we need a dataset of emails paired with high-quality human-written summaries or points that should be included in the summary
- For QA, we need multi-turn conversation datasets covering realistic follow-up scenarios

The evaluation strategies for these tasks differ significantly. As we will explore later, text classification, summarization, and QA each require distinct metrics, datasets, and evaluation approaches.

Evaluation datasets

While working with AI, you've probably noticed that we often talk about datasets. Data is the fundamental and central element of AI. In traditional software development, we explicitly define logic in code. In AI, however, much of that *logic* resides in the data itself.

Most datasets are designed to serve both purposes: training models and evaluation. In many ways, evaluation datasets closely resemble training datasets. The main distinction is usually size: evaluation typically requires far less data than training.

For example, if we have a dataset with 10,000 samples, we might allocate 8,000 for training and 2,000 for evaluation. While the evaluation set can be significantly smaller, it must still be large and diverse enough to cover all relevant aspects of the AI's functionality so that the results are reliable and representative.

Another fundamental principle in AI is that evaluation data must never be used for training in any form. This ensures that the evaluation remains unbiased and gives an accurate picture of how well the model can generalize to truly unseen data.

This principle applies not only to model training or fine-tuning but also to in-context learning (see *Chapter 3*). So, the few-shot examples provided in prompts must be separated from the evaluation dataset. The same applies to any prompt engineering technique that uses examples, templates, or other prompt components derived from data. These must not overlap with evaluation data.

As defined earlier, a dataset is a set of inputs and, optionally, expected correct outputs. For closed-ended problems like text classification, regression, or information extraction, a dataset with expected outputs is called a **supervised dataset**. Without expected outputs, it is considered an **unsupervised dataset**.

For example, the dataset below for sentiment analysis is supervised because it contains expected outputs (ground truth or labels) for each input.

Post	Sentiment label
I absolutely love this product, it works perfectly!	Positive
The service was terrible and I will never use it again.	Negative

Table 10.4: Dataset for sentiment analysis

As we mentioned earlier, for open-ended tasks like text summarization or generative question answering, multiple outputs can be correct. In such cases, one or more *reference outputs* serve as benchmarks for evaluating the correctness and quality of other valid responses. For example, in summarization tasks, evaluation datasets often include human-written reference summaries against which generated summaries are compared. For example, consider a case like this one:

Document text	Reference summary
The city council met on Tuesday to discuss the proposed budget for the upcoming fiscal year. The mayor emphasized the need for increased funding for public transportation and affordable housing. Several council members raised concerns about potential tax increases, while others supported reallocating funds from less critical projects. The final vote on the budget will take place next month.	The city council debated the proposed budget, focusing on funding for public transport and housing, with a final vote set for next month.

Table 10.5: Example of a document–reference summary pair

Now that we've reviewed the basics of evaluation datasets, let's explore public datasets and the different approaches to creating custom ones.

Public datasets

Public datasets have played an essential role in AI development as a whole. Without them, we would not have AI in the form we know today. They have served as the foundation for training, testing, and benchmarking countless models.

Public datasets also help create *shared benchmarks*, so that improvements in models or algorithms can be measured objectively. For example, leaderboards on platforms like Kaggle or Hugging Face Datasets give a clear view of which models perform best on a given task at a given time. This open competition has accelerated innovation in AI dramatically.

Public datasets can sometimes be used for evaluation. For example, if you are developing a specialized sentiment analysis system and find an existing dataset that closely matches your problem domain, you can use it to benchmark your application. This approach can significantly reduce the development overhead compared to building an evaluation dataset entirely from scratch.

However, public datasets often don't match your specific task. That's why, in practice, public datasets are often used in combination with custom evaluation datasets. This means that if you are developing a highly specific functionality, you are unlikely to find a perfect dataset ready to use. Next, we will explore how to build custom datasets.

Building custom evaluation datasets

Now, let's dive deeper into how to build a custom evaluation dataset when no suitable public one exists. Datasets can vary dramatically from one problem to another, as each task often has its own specific requirements, domain constraints, and subtle nuances.

For example, imagine you are developing a customer support assistant designed to answer user questions about issues with a specific product. In this case, the evaluation dataset would need to include a variety of realistic initial questions, follow-up (multi-turn) interactions, and potentially even attached screenshots or files containing problem details. All of these elements should be part of the evaluation dataset to accurately reflect real usage scenarios.

Another example could be a financial agent that analyzes personal expenses, flags unhealthy financial behaviors, and generates periodic financial reports. Here, the inputs might include historical transaction data and contextual parameters (e.g., spending limits or categories), while the outputs would consist of timely alerts, actionable recommendations, and well-structured reports.

So, building evaluation datasets is inherently a task-specific challenge. However, we can still identify several common characteristics that every high-quality evaluation dataset should possess:

- **Diversity:** The dataset should cover a wide range of examples, including common cases, varied scenarios, edge cases, and challenging inputs, to thoroughly stress-test the AI's capabilities
- **Representativeness:** The dataset should reflect the actual diversity and distribution of real-world inputs the AI will encounter in runtime (production)
- **Data quality:** All examples should be accurate, consistently annotated, and free from noise, ambiguity, duplication, and conflicts

While diversity and representativeness may overlap, they capture different ideas. Diversity ensures breadth, covering as many different cases and situations as possible.

Representativeness ensures relevance, meaning the dataset aligns closely with the types of inputs real users will provide during live interactions. A dataset can be highly diverse yet still unrepresentative if it contains many unusual or unrealistic examples that do not reflect actual user behavior. Similarly, the dataset can be representative but not diverse.

To have a better understanding of the difference between diversity and representativeness, let's consider the following examples:

- **Diverse but not representative:** A dataset for a customer support assistant for an online clothing store includes questions about airline ticket refunds, hotel reservations, cooking recipes, and car repairs in addition to clothing-related ones. While this dataset is highly diverse, most of the examples are irrelevant to the store's actual customer interactions, not representative.
- **Representative but not diverse:** You collect 1,000 real customer queries from the clothing store's live chat logs. All the queries are realistic and relevant, but they are heavily skewed toward a few common scenarios while rarely including edge cases and unusual product questions. This dataset mirrors real usage (is representative) but doesn't test the chatbot's ability to handle rare or challenging cases (not diverse).

Now let's look at the data quality aspect. Data should be properly formatted and ready to use. A serious data quality issue arises from conflicting data.

For instance, in a sentiment analysis dataset, you might find:

Post	Sentiment label
I absolutely love this product, it works perfectly!	Positive
The service was terrible and I will never use it again.	Positive

Table 10.6: An example of conflicting data

Here, the second sample is clearly negative, yet it is labeled as positive. Evaluating such incorrectly labeled data will produce misleading results, just as training on it will lead to an inaccurate model.

Another common issue is **duplicates**. While duplicates might not be as damaging as missing or incorrect labels, they still consume unnecessary resources and can distort evaluation results. For example, imagine a dataset where one-third of the cases are exact duplicates of another third, with the remaining third being unique. If the model performs correctly on the first third, it will also appear correct on the duplicated third, artificially inflating performance to 66%.

For open-ended inputs and outputs, duplication can also appear in the form of *semi-duplicates*. Those are cases that are not identical but are so similar that they test the same capability repeatedly without adding much new evaluation value. An example would be multiple user questions that are only slightly rephrased:

- "What's the weather in New York tomorrow?"
- "Can you tell me the weather forecast for New York for tomorrow?"

While technically different, these queries are so close in meaning that they may overrepresent certain patterns, again skewing evaluation results.

Note that in practice, it's almost impossible to cover all possible cases, scenarios, and edge cases. Therefore, in practice, we focus on including a practically achievable set of examples that still represent the diversity and representativeness we need. Each sample can be grouped into a specific category. Categorizing samples this way helps ensure coverage across all important input types. This approach strikes a balance between completeness and practicality, ensuring the evaluation dataset is both manageable and meaningful.

To illustrate this principle, let's dive deeper into how we might develop an evaluation dataset for our weather forecast assistant.

First, we can observe that user questions can vary widely in terms of geographic location and date references. Locations could be cities, regions, landmarks, or even entire countries. Dates might be specific (e.g., "25 July 2025") or relative (e.g., "today," "tomorrow," "next Monday"). Our chatbot might also be expected to handle queries about past weather, the current day, or forecasts far into the future. This gives us an initial understanding of the range of valid weather-related questions we need to cover.

Second, we must recognize that the assistant is a domain-specific one. It should answer only weather-related questions. Because the input is free-form text, users may ask anything, including questions outside the weather domain. This introduces an important evaluation category: *out-of-scope queries*. For example, if a user asks, "Tell me a joke" or "What's the capital of France?", the assistant should respond appropriately with something like, "I can only answer weather-related questions." Including such cases in the evaluation dataset ensures the system handles irrelevant inputs as well.

Third, ambiguities and missing context are common in natural user interactions. For example, the question "Tell me the weather for tomorrow" doesn't specify a location. If the chatbot has access to the user's current location, it can use that; otherwise, it should prompt for clarification. Including these cases in the dataset ensures we test the assistant's ability to request and handle missing information.

Finally, the assistant may support *multi-turn* conversations (multi-turn QA). A user might start with "What's the weather in Paris tomorrow?" followed by "And what about the day after?" The second question depends on the context of the first. Therefore, the evaluation dataset should include not only isolated single-turn questions but also multi-turn dialogues that test the system's ability to maintain and correctly interpret conversational context.

By identifying these dimensions, we can design a dataset that reflects realistic usage patterns and thoroughly tests the assistant's capabilities.

Synthetic and human-generated datasets

When building a custom evaluation dataset, there are two broad data sources to consider:

- Human-generated data
- AI-generated synthetic data (see *Chapter 5*)

Both can be valuable, and in practice, they are often used together.

Human-generated datasets come directly from people, either real users interacting with the system, domain experts creating examples, or annotators labeling and categorizing data. They are often the most representative and reliable, especially for nuanced or domain-specific tasks.

Human-generated data has its limitations as well. It can be difficult, time-consuming, and costly to obtain. Even when collected, it may lack sufficient diversity, omit important edge

cases or over-represent certain patterns due to natural human bias or the sampling method used.

Human data can also suffer from labeling inconsistencies, conflicting annotations, and duplicates, all of which can reduce the reliability of evaluation results. Additionally, annotator subjectivity can introduce subtle biases, particularly in tasks with ambiguous or subjective labels such as sentiment, relevance, or helpfulness.

To illustrate challenges with human-generated data, consider the following example: we need to develop a dataset for evaluating a customer support summarization agent. We decided to collect human judgments on generated summaries. We show multiple annotators the same chat transcript between a customer and a support agent, along with the model's generated summary.

One annotator might rate the summary as excellent. Another might rate it as poor because it omits a minor but, in their view, important detail. A third annotator might give a medium score, arguing that the summary is factually correct but overly verbose.

In this case, all annotators are evaluating the same piece of content but bringing different priorities and subjective interpretations of what "good" means. This disagreement introduces label noise into the dataset, which can complicate evaluation.

In cases when human data suffers, *synthetic datasets* can help. Synthetic datasets are created artificially, often with the help of AI models like LLMs or automated scripts. They can be used to:

- Generate variations of existing examples (for example, by rephrasing existing text)
- Cover rare or edge cases
- Quickly scale up dataset size when human data are limited
- Bypass privacy issues

While synthetic data can speed up dataset creation and improve coverage, it should be reviewed and validated to ensure quality and avoid introducing systematic errors. In many evaluation setups, a hybrid approach works best: start with a core set of high-quality, human-generated examples, then expand with carefully verified synthetic data to increase coverage and diversity.

Synthetic text data has its own limitations, as discussed in *Chapter 5* when exploring text generation tasks. Such data often carries biases and tends to be over-idealized, with very correct grammar and syntax. In reality, people rarely write text in a perfectly polished style. Language models also tend to be biased toward specific structures, formats, or phrasing patterns (skills acquired during RLHF), whereas human writing is highly varied.

Despite such limitations, synthetic data brings substantial value, especially when human-generated data is expensive, scarce, or impractical to collect. Synthetic examples can be generated rapidly, at scale, and at low cost using LLMs or automation. This speed and flexibility make synthetic data a powerful tool for filling gaps in evaluation datasets when human data is limited.

Evaluation metrics

At this point, we understand what type of AI problem we are addressing, the nature of its inputs and outputs, and we have already prepared an evaluation dataset. The next step is to use this dataset to feed inputs into the AI system and collect its outputs.

This raises a fundamental question: how do we assess the quality of these outputs and determine whether the AI functionality is performing as intended? That's where evaluation metrics come in.

Evaluation metrics are quantitative or qualitative measures used to assess the performance of an AI system against a predefined goal. They translate raw outputs into interpretable values that allow us to compare quality, track improvements, and identify weaknesses. For example, an accuracy metric measures how correct the AI's responses are, while a usefulness metric reflects how helpful those responses are to the user.

The choice of evaluation metrics depends heavily on the type of problem we are solving. In general, AI tasks can be grouped into two broad categories:

- Closed-ended problems
- Open-ended problems

Closed-ended task evaluation

As we already briefly mentioned before, closed-ended problems involve tasks where the set of valid outputs is finite (often unique), well-defined, and can be explicitly enumerated.

Examples include classification, regression, information extraction, closed-ended question answering, and certain types of reasoning tasks.

Closed-ended evaluations are relatively straightforward because they require having correct answers or ground truth to compare against the model's outputs. In these tasks, evaluation often comes down to checking whether the predicted output exactly matches the expected one.

To illustrate, let's consider a spam detection problem where we need to classify an email as either spam or not spam (ham). For such classification, we would need a labeled dataset, for example, the Enron email dataset [<https://www.kaggle.com/datasets/wcukierski/enron-email-dataset>], which could look like this:

Email content (subject + body)	Expected label
Email 1	Spam
Email 2	Ham
...	...
Email N	<label N>

Table 10.7: Sample email spam dataset

Here, the *Email content* is the input to our spam detector, and the *Label* column contains the expected (ground truth) classification.

Now, imagine we run all *N* emails through our spam detector and obtain *N* predicted labels. We then compare these predicted labels against the ground truth in the *Expected Label* column, getting a table like this one:

Email content (subject + body)	Expected label	Actual prediction (Label)	Match
Email 1	Spam	Spam	True
Email 2	Ham	Spam	False
...	...		
Email N	<label N>	<actual label N>	

Table 10.8: A comparison of predicted and expected labels

So, we have explored what evaluation datasets can look like. Now, let's move on to how to use these datasets to measure the performance of LLM-based applications.

Accuracy and exact match

Exact match is a comparison method that checks whether each prediction exactly matches the correct label. The metric derived from this process is sometimes also referred to as the **exact match (EM)** metric.

In many cases, *accuracy* for closed-ended problems can be seen as a direct extension of the exact match concept. It is essentially the proportion of predictions that exactly match the correct answers. So, accuracy is defined by the formula:

$$\text{Accuracy} = \frac{\text{Number of correct outputs}}{\text{Total numbers of outputs}}$$

For example, if we have 100 samples ($N = 100$) and got 75 correct answers and 25 incorrect, then our accuracy will be

$$\text{Accuracy} = \frac{75}{100} = 0.75 \text{ or } 75\%$$

Accuracy tells us how often AI gets the answer right, and it plays a fundamental role in AI evaluation. It's one of the most widely used metrics, but at the same time, it's not perfect and has its own limitations. That's why other metrics are often used in addition to, or instead of, accuracy.

To illustrate a situation where accuracy may not provide a reliable measurement, let's consider a sentiment analysis problem for product reviews where the evaluation dataset contains significantly more positive posts than negative ones (typical case). For example, among 100 reviews, we might have 90 positives and only 10 negatives. Such datasets are called *imbalanced* because one category dominates significantly over the others.

In this case, accuracy can give a misleading impression of the classifier's quality. Imagine our sentiment analysis detector always predicts "positive" for every review, without actually analyzing the content. Out of the 100 reviews, it would correctly classify all 90 positive ones but misclassify all 10 negative ones. This results in an accuracy of 90%.

At first glance, 90% accuracy might seem good, but in reality, our system completely fails to detect negative reviews. This means it's useless for the intended purpose, despite the high score. If our dataset were more balanced, containing 50 positive and 50 negative reviews, the same model would achieve only 50% accuracy instead of 90%. This shows that accuracy is heavily influenced by the label distribution in the dataset.

Precision, recall, and F1 for binary classification

When dealing with imbalanced datasets, we need metrics that give a more complete picture of how well each class is being detected. Two of the most common metrics are precision and recall, often used together with their harmonic mean, the F1 score.

Precision measures the proportion of predicted positives that are actually correct. It answers the question: "Of all the reviews the system marked as positive, how many were really positive?"

Precision is defined by the formula:

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

where **true positive (TP)** is when the detector correctly detected reviews as positive, and **false positive (FP)** is when the detector incorrectly detected reviews as positive (but they were actually negative ones).

Example: The sentiment detector predicted 50 reviews as positive; 45 were truly positive (**TP**), and 5 were actually negative (**FP**).

$$\text{Precision} = \frac{45}{45+5} = 0.9 \text{ or } 90\%.$$

High precision means few false alarms. The system is usually correct when it labels something as positive.

Recall measures the proportion of actual positives that were correctly identified. It answers the question: "Of all the truly positive reviews in the dataset, how many did we catch?" Recall is defined by the following formula:

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

Here, **false negatives (FN)** are when the detector predicted a review as negative, but actually, it was positive.

Example: There are actually 90 positive reviews in the dataset. The system caught 45 of them (**TP**) but missed the other 45 (**FN**).

$$\text{Recall} = \frac{45}{45+45} = 0.5 \text{ or } 50\%.$$

High recall means few misses; the system catches most of the positives. So, depending on the use case, you might prioritize one over the other:

- High precision is preferred when false positives are costly (e.g., wrongly marking a bad review as positive)
- High recall is preferred when missing positives is costly (e.g., failing to catch genuine positive reviews for marketing insights)

When both precision and recall are important, we use the F1 score, which is the harmonic mean of precision and recall:

$$F1 = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

Example: if our precision is 0.9 and recall is 0.5, then F1 will be:

$$F1 = 2 * \frac{0.9 * 0.5}{0.9 + 0.5} \approx 0.64$$

So, we considered that accuracy, precision, and recall are for binary classification. Now, let's consider a case where we have a multi-class classifier, such as sentiment analysis with three classes: positive, neutral, and negative.

Multi-class classification metrics

In this case, accuracy and exact match are still defined in the same way as for binary classification:

- If the predicted label matches the expected label (class), it counts as a *match*
- If it doesn't match, it counts as a *mismatch*

However, precision, recall, and F1 score need to be adapted because in a multi-class setting, we no longer have a single, clear definition of true positives, false positives, and false negatives. Instead, we have more than two classes, and these metrics must be computed per class.

The most common approach is to treat each class in turn as the "positive" class and all other classes as "negative," effectively breaking the multi-class problem into multiple *one-vs-all* binary problems. From there, precision, recall, and F1 can be calculated for each class individually and then combined into a single number in one of two main ways:

- **Macro-averaging:** Calculates the metric for each class independently and then takes the simple average. This treats all classes equally, regardless of how many examples each class has.
- **Micro-averaging:** Aggregates the counts of true positives, false positives, and false negatives across all classes before calculating the metric. This weights the result by the class distribution and is more influenced by larger classes.

For the sake of this book, we encourage you, the reader, to experiment with these metrics on multi-class classification problems. Try applying accuracy, precision, recall, and F1 using publicly available datasets and observe how the results differ when the class distribution changes.

Similarity metrics for open-ended tasks

For open-ended tasks, where the output is free-form text, evaluation becomes significantly more challenging than in closed-ended cases. This is primarily because there can be many equally valid answers. The same input may produce multiple different yet correct responses, making it difficult to define a single ground truth or an exact correctness criterion.

Over time, researchers and practitioners have developed a variety of strategies to tackle this complexity. Let's explore them in detail.

Lexical similarity-based metrics

One common approach to evaluating open-ended tasks involves comparing the model's output to one or more preferred outputs using similarity metrics. These preferred outputs are often referred to as **reference outputs** or **golden answers**.

To illustrate this, consider the following reference summary (golden output):

```
The conference focused on advances in renewable energy, featuring talks on solar, wind, and battery technologies. Industry leaders discussed challenges in scaling and integration.
```

Now, compare it with two possible summary outputs:

```
Candidate 1: Speakers at the conference discussed renewable energy developments, including solar, wind, and batteries, and addressed integration challenges.
Candidate 2: A conference was held to explore sustainability, highlighting climate change and the need for cleaner technologies.
```

There are several ways to compare these candidates to the reference. Let's begin with lexical similarity metrics, which focus on word overlap between the texts. Some of the most widely used lexical metrics include **BLEU**, **ROUGE**, and **METEOR**:

- **Bilingual Evaluation Understudy (BLEU)**: measures the overlap of n-grams (sequences of words) between the output and the reference
- **Recall-Oriented Understudy for Gisting Evaluation (ROUGE)**: measures how much of the reference's content appears in the output
- **Metric for Evaluation of Translation with Explicit Ordering (METEOR)**: Builds on basic word overlap by incorporating stemming (e.g., matching "run" and "running") and synonym matching, providing a more flexible lexical comparison

To demonstrate how these metrics work in practice, the following code example compares the two candidate summaries to the reference using BLEU, ROUGE, and METEOR:

```
import sacrebleu
from rouge_score import rouge_scorer
from nltk.translate.meteor_score import meteor_score
import nltk
nltk.download('wordnet', quiet=True) # for METEOR synonyms/stemming
nltk.download('punkt', quiet=True)
```

```

reference = "The conference focused on advances in renewable energy, featuring
talks on solar, wind, and battery technologies. Industry leaders discussed
challenges in scaling and integration."

candidates = [

    "Speakers at the conference discussed renewable energy developments, including
    solar, wind, and batteries, and addressed integration challenges.",
    "A conference was held to explore sustainability, highlighting climate change
    and the need for cleaner technologies."
]

# BLEU (sacreBLEU expects list of refs; can pass multiple refs)
for i, cand in enumerate(candidates, 1):
    bleu = sacrebleu.corpus_bleu([cand], [[reference]]).score
    print(f"\nCandidate {i} - sacreBLEU: {bleu:.2f}")

# ROUGE (rouge-score)
scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'], use_stemmer=True)
for i, cand in enumerate(candidates, 1):
    scores = scorer.score(reference, cand)
    print(f"\nCandidate {i} - ROUGE:")
    print("ROUGE-1 F:", scores['rouge1'].fmeasure)
    print("ROUGE-2 F:", scores['rouge2'].fmeasure)
    print("ROUGE-L F:", scores['rougeL'].fmeasure)

# METEOR (NLTK)
for i, cand in enumerate(candidates, 1):
    m = meteor_score([reference], cand)
    print(f"\nCandidate {i} - METEOR: {m:.4f}")

```

The code first defines the reference and candidate summaries. Then it calculates BLEU scores for both candidates. ROUGE scores are computed for unigram (**ROUGE-1**), bigram (**ROUGE-2**), and longest common subsequence (**ROUGE-L**) overlaps. Finally, METEOR is applied.

Running this code gives us the following results:

```

Candidate 1 - sacreBLEU: 16.09
Candidate 2 - sacreBLEU: 2.93

```

```
Candidate 1 – ROUGE:  
ROUGE-1 F: 0.5853658536585366  
ROUGE-2 F: 0.2564102564102564  
ROUGE-L F: 0.48780487804878053
```

```
Candidate 2 – ROUGE:  
ROUGE-1 F: 0.2  
ROUGE-2 F: 0.0  
ROUGE-L F: 0.15  
Candidate 1 – METEOR: 0.3014  
Candidate 2 – METEOR: 0.0862
```

This outcome reflects the fact that Candidate 1 has a greater lexical overlap with the reference. In particular, both the reference and Candidate 1 share the same words, such as:

```
The, Conference, renewable, energy, solar, wind, battery, technology, and,  
industry, leaders, challenges, integration.
```

In contrast, Candidate 2 shares only a few words with the reference, such as:

```
The, conference, and, technology
```

As we can see, lexical similarity metrics like BLEU, ROUGE, and METEOR provide a fast way to evaluate model outputs by measuring surface-level word overlap. However, they have well-known limitations.

These metrics often fail to reward valid paraphrases, nuanced synonyms, or alternative but semantically correct answers that diverge from the reference text. This can lead to misleading evaluations, especially for tasks like summarization, dialog, or open-ended question answering, where diverse phrasing is not only expected but desirable.

Semantic similarity-based metrics

Semantic similarity metrics have emerged as a more robust alternative. They rely on sentence embeddings (see *Chapter 1*), allowing for comparisons based on semantic closeness. Semantic metrics are more tolerant of paraphrasing and rewording.

However, this benefit comes at a cost. Embedding-based evaluation methods tend to be:

- Slower and more computationally expensive
- Require GPUs or specialized hardware for large-scale runs
- Results may vary depending on the embedding model used

One simple approach to perform semantic comparison is to encode both the reference and candidate sentences using a transformer model (e.g., MiniLM) and compare them using cosine similarity or L2/L1 distances as shown in the following code snippet:

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

# Load a pretrained sentence embedding model
model = SentenceTransformer('all-MiniLM-L6-v2') # Fast and lightweight

# Define reference and candidate summaries
reference = "The conference focused on advances in renewable energy, featuring talks on solar, wind, and battery technologies. Industry leaders discussed challenges in scaling and integration."
candidates = [

    "Speakers at the conference discussed renewable energy developments, including solar, wind, and batteries, and addressed integration challenges.",
    "A conference was held to explore sustainability, highlighting climate change and the need for cleaner technologies."
]

# Compute embeddings
reference_embedding = model.encode([reference])
candidate_embeddings = model.encode(candidates)

# Compute cosine similarity
cosine_scores = cosine_similarity(candidate_embeddings, reference_embedding)

# Display results
for i, score in enumerate(cosine_scores):
    print(f"Candidate {i+1} Cosine Similarity: {score[0]:.4f}")
```

This code computes embeddings for the reference and candidates, then calculates cosine similarity:

- Candidate 1 Cosine Similarity: **0.9387**
- Candidate 2 Cosine Similarity: **0.7724**

This shows that Candidate 1 is much closer to the reference. However, unlike lexical metrics, the gap between Candidate 2 and the reference is not as extreme because this method recognizes shared meaning even with different wording.

Another popular approach to measuring semantic similarity is BERTScore. It leverages contextual embeddings from transformer-based models (such as BERT or RoBERTa) to compare tokens between the candidate and reference texts. It computes a precision-, recall-, and F1-style similarity score based on how well the embeddings of corresponding tokens match.

Despite their popularity, both lexical and semantic similarity metrics have several critical limitations that affect their reliability in practice:

- **Dependence on reference outputs:** Require one or more reference ("golden") answers
- **Similarity thresholds selection:** Complexity to define when two texts are "similar enough"
- **Blindness to critical semantic errors:** Can be misleading in cases where small but important details change the meaning entirely

As we can see, lexical and semantic similarity metrics can help evaluate open-ended tasks. However, these approaches have important limitations and are not suitable for all scenarios. Let's now explore additional metrics designed to better handle the complexity of open-ended evaluation.

Human as a judge

As we've seen, similarity-based metrics have limitations. In many cases, especially where meaning, nuance, or factual precision matter, human evaluation remains the gold standard.

After all, it's humans who ultimately interact with the product and judge the quality of its outputs. Moreover, one of the key strengths of human evaluation is that it can be conducted without requiring reference (golden) answers.

This *reference-free* setup greatly simplifies the evaluation process, especially for tasks with high variability or creativity. Human judges can directly assess whether the model's response is fluent, accurate, relevant, or helpful. This makes human evaluation more flexible, domain-adaptable, and often more reliable than automatic metrics in open-ended settings.

It's also worth noting that human evaluation was the earliest form of assessment in NLP and still remains the most trusted. Long before automatic metrics were introduced, researchers relied on manual judgments to assess translation quality, summarization, and dialogue.

To perform a human-based evaluation, we must first define what aspects of the output we expect human judges to assess. These aspects are often called **evaluation criteria** or **evaluation dimensions**. The following are some of the most common and widely accepted criteria used in evaluating outputs from language models and generative systems:

- **Fluency and coherence:** Is the text grammatically correct, well-structured, and easy to read? Are the sentences logically connected, and does the content flow naturally?
- **Accuracy and relevance:** Does the output provide factually correct information? Is it directly relevant to the input prompt, query, or source text?
- **Clarity and readability:** Is the message clearly expressed and easy to understand? Are there any ambiguities or confusing formulations?
- **Usefulness and completeness:** Does the output provide a complete and helpful response? Does it fully address the user's need?
- **Personalization or context awareness:** In personalized applications (e.g., recommendations, outreach, or chat), does the output reflect the user's preferences, context, or background?
- **Groundedness and faithfulness:** Does the output remain consistent with the provided source material, evidence, or context? Does it avoid introducing unsupported claims, hallucinations, or fabricated details?

The criteria listed above are among the most commonly used, but they are not exhaustive. Each task, domain, or application may require its own custom evaluation dimensions, depending on what defines "quality" in that specific context.

These criteria can and should be adapted to the task at hand. For example:

- In summary, criteria like completeness (does it cover key points?) and coherence (does it read logically?) are critical.
- In dialog systems (chatbots), relevance, helpfulness, and human-likeness tend to be prioritized to ensure natural and effective interactions.
- In creative text generation, such as storytelling or copywriting, fluency, style, and originality may take precedence over strict factuality.

Some evaluation criteria can be assessed based solely on the model's output, while others require additional context or reference information. For example:

Criteria are based only on the output text	Criteria that require additional context
Fluency	Accuracy (requires source or ground truth)
Coherence	Relevance (depends on the prompt or user query)
Clarity and readability	Personalization (depends on user profile or context)

Table 10.9: Evaluation criteria for generative model outputs

Once evaluation criteria are defined, they can be used to create *human-scored metrics*, which are structured ways for annotators to assess model outputs using numerical scores or categorical labels. In most setups, this means asking human judges to assign a score to each criterion for a given output.

For example, a common approach is to use a *Likert scale*, such as:

- 1 to 5 (e.g., 1 = very poor, 5 = excellent)
- 0 to 3 (low granularity for faster evaluation)
- 0 to 7 or 1 to 10 (for finer granularity when needed)

This turns qualitative judgment into quantifiable metrics that can be averaged across examples, compared across models, and used to track improvements over time. The following is an example of how such customized criteria (metrics) might look for a summarization task:

Criteria (Metric)	Score	Objections
Relevance	1-5	Relevance checks whether the summary captures the most important points from the original text. Relevance focuses on what to include in the summary and what to exclude. Grading: 1 - includes irrelevant details or topics from source; fails to mention any significant topics; 5 – includes the most relevant, most important summary; doesn't include unimportant points.
Accuracy	1-5	Accuracy assesses whether the summary precisely matches what is in the original text. It measures how closely the summary reflects the content of the original text. Gratings: 1 – includes incorrect facts: statements, dates, numbers, and so on. 5 – all facts and statements are correct.

Table 10.10: Definitions of relevance and accuracy metrics

It's important to note that many evaluation criteria overlap or influence one another. They're not always fully independent. For example, accuracy and relevance are closely related: a summary that includes irrelevant or unimportant details may also reduce factual precision by emphasizing the wrong points.

So, after conducting the human evaluation, we might collect results for a single output using a 5-point scoring scale across multiple criteria. A sample result might look like this:

Criterion	Score (1–5)	Description
Fluency	5	Reads naturally and is grammatically correct
Relevance	4	Mostly on-topic but slightly off in one detail
Helpfulness and completeness	3	Provides some value but lacks completeness

Table 10.11: Sample results for fluency, relevance, and helpfulness metrics

These scores can then be aggregated by taking the mean, median or weighted average across multiple raters or criteria. So, after performing the human survey we can receive the following aggregated table:

QA responses	Fluency (avg)	Relevance (avg)	Helpfulness (avg)	Overall Score
Response 1	4.8	4.5	4.4	4.47
Response 2	3.9	4.7	3.2	3.65
Response 3	4.2	4.0	4.6	4.33

Table 10.12: Examples of human evaluation results

But the challenge here arises when human raters assess the same output very differently. Variability in scores can reduce the reliability of the evaluation and make it harder to draw meaningful conclusions.

Let's imagine we have 5 raters evaluating the Helpfulness of responses to a question in a QA task. Here's how the ratings might look:

QA responses	Rater 1	Rater 2	Rater 3	Rater 4	Rater 5	Mean	Std dev*
Response 1	5	5	5	5	5	5	0.0
Response 2	4	4	5	3	4	4	0.63
Response 3	1	5	2	3	4	3	1.41

* - Standard deviation

Table 10.13: Examples of human evaluation results from five raters

The table illustrates how ratings can vary across raters for the same response, affecting the overall reliability of the evaluation. The key observations are the following:

- *Response 1* shows strong consensus. All raters gave the same score, suggesting the output was clearly helpful and unambiguous.
- *Response 2* shows moderate agreement with a small amount of noise. Most raters scored it a 4, but one gave it a 3 and another a 5. This might still be acceptable. The final rate could be selected as 4 (mean or mode)
- *Response 3* demonstrates high rater variance. Scores range from 1 to 5. This likely reflects ambiguity, subjectivity, or disagreement about the output's value.

When there are such disagreements as in *Response 3* above, additional steps may be necessary:

- Reviewing ambiguous outputs manually
- Refining the evaluation instructions
- Disqualifying responses with high disagreement (high standard deviation)

In this way, we can fix the human misalignment and improve evaluation quality.

LLM as a judge

As we've seen, human evaluations are powerful, reliable, and closely aligned with real-world user expectations. However, they also come with a few significant limitations:

- **Cost and scalability:** Running human evaluations is expensive and time-consuming, especially at scale
- **High inter-rater disagreement:** Even with clear instructions, different annotators may interpret criteria differently, especially for subjective dimensions like helpfulness, tone, or creativity
- **Limited speed and throughput:** Humans can't evaluate thousands of examples in minutes

Because of these limitations, there's growing interest in using *LLMs as a judge*. This is the approach that simulates human evaluation and offers better scalability, consistency, and speed.

To illustrate how an LLM can serve as a substitute for human raters, let's consider the following scenario: An email is sent by a company's CEO to employees celebrating a recent achievement; for example, the successful launch of a product, hitting a revenue milestone, or closing a major deal.

In such messages, personalization is important. A well-crafted email not only shares success but also acknowledges the contributions of specific individuals or teams, highlights their roles, and connects their efforts to the overall result. This makes the message more engaging, motivating, and credible.

The goal is to have the LLM evaluate how well the email meets these expectations. Here's a structured evaluation prompt you can use to instruct the LLM to act as a judge:

Prompt

You are acting as a communication evaluator. Your task is to rate how well the following email demonstrates personalization – specifically, how effectively it highlights individual contributions and connects them to the company's success.

Instructions:

- Read the email.
- Evaluate how clearly and specifically it acknowledges people by name, team, or role.
- Consider whether the impact of their work is described in a meaningful way.
- Identify whether the message feels tailored to the recipients or is too generic.
- Provide both a numeric score and a short explanation.

Scoring:

- 5 = Strong personalization: multiple individuals or roles are named, their contributions are clearly linked to the success.
- 4 = Good personalization: at least one person or team is mentioned meaningfully.
- 3 = Moderate: some general appreciation is given, but it's vague or lacks specificity.
- 2 = Weak: minimal personalization; generic praise without details.
- 1 = No personalization at all; feels templated or impersonal.

Email:

[Insert the email text here]

Your response format:

Personalization Score: 1-5

Explanation: [Short paragraph explaining the score]

This prompt can be embedded in automated pipelines where thousands of company-personalized emails are evaluated for quality, tone, and personal impact, making it scalable without needing human reviewers for each one.

This example illustrates how an LLM can be used as a judge, replacing human evaluation. Such replacement offers significant benefits:

- **Scalability:** LLMs can evaluate thousands of outputs in minutes
- **Consistency:** LLMs provide more consistent evaluations than human annotators
- **Speed and cost-efficiency:** Removes the need for recruiting, training, and paying human raters

But using LLM-as-a-judge also comes with important limitations and challenges:

- **Bias toward known patterns:** LLMs often favor outputs that match their own training distribution or phrasing, which can result in inflated scores.
- **Prompt sensitivity:** Evaluations can vary significantly depending on how the task is framed. Small changes in wording or formatting may produce different ratings.
- **Randomness:** Unless explicitly controlled (e.g., deterministic or low randomness decoding), LLMs may yield non-reproducible results across runs.

Despite these limitations, the use of LLMs as evaluators is becoming increasingly widespread. Recent studies and industry benchmarks have shown that state-of-the-art LLMs can often produce judgments that closely align with human ratings, particularly for subjective criteria such as fluency, coherence, and tone [60].

It's important to emphasize that everything we've discussed about evaluation criteria applies equally to LLM-based evaluation metrics as it does to human-annotated ones. In other words, when we use LLMs as evaluators, we are still fundamentally working with the same evaluation dimensions: fluency, accuracy, helpfulness, personalization, tone, and so on. These criteria don't change because the judge is a model rather than a human.

At the same time, using LLMs as evaluators doesn't eliminate the need for using humans as judges.

Online vs. offline evaluations

Nearly everything we've discussed so far falls under the category of *offline evaluation*. These evaluations are performed on pre-collected, static datasets before the product is deployed to real users. Offline evaluation helps us assess how well our product performs under controlled conditions and gives us early insights into quality, reliability, and potential failures before release.

However, once the product is deployed, offline metrics are no longer enough. A useful analogy can be made to drug development:

- Offline evaluation is like the *preclinical* and *clinical trial phases* (phases 1-3) of a new drug. These tests are carefully controlled and use predefined protocols (metrics) to measure safety and efficacy before the drug reaches the market.
- Online evaluation, on the other hand, is like the *post-market surveillance phase* (also called *phase 4*). Once the drug is released to the public, it is used by a diverse population in real-world conditions. Unexpected side effects, rare interactions, or changes in effectiveness may surface, and ongoing monitoring is required to ensure safety and value.

Similarly, in AI systems:

- Offline evaluations give us early confidence in the system's capabilities, but they are inherently limited to the scenarios we've anticipated and tested
- Online evaluations help us understand how the system performs in real-time, capturing issues that only emerge at scale, with real users, in complex and evolving contexts

That's why combining both offline and online evaluations is essential for responsible, effective AI product development.

However, online evaluation differs from offline evaluation and comes with its own set of challenges:

- We can't pre-label real-time data, so traditional ground-truth comparisons are no longer possible
- Evaluation must be passive or implicit, often relying on user signals like click-through rates, drop-off points, thumbs-up/down feedback, or task completion rates
- User satisfaction and behavior shifts become key indicators of success or failure

We can also notice that some of the metrics used in offline evaluation can also be repurposed for online setups, particularly when they do not rely on ground truth or reference labels. For example, the following metrics (criteria) can be reused for online evaluation as well:

- **Fluency:** Is the generated text grammatically correct and natural-sounding?
- **Readability:** Is it easy to read and understand?
- **Clarity:** Is the message unambiguous and well-expressed?
- **Human-likeness:** Does it feel like the output was written by a person rather than a machine?

In addition, some context-aware metrics that require both the input (e.g., user query or source document) and the generated output can also be used online. These include:

- **Relevance:** Does the generated content stay on topic and align with the user's input?
- **Groundedness:** Are the facts in the output consistent with the input context or source data?

For instance, in a question-answering or summarization system where the generated response is derived from a user prompt or a retrieved document, we can evaluate relevance and factual alignment by comparing the output with the context it was generated from without needing predefined "correct" answers.

Using online evaluation metrics comes with several important challenges:

- **Scalability and cost:** Evaluating every output in real time can be computationally expensive and increase infrastructure costs
- **Latency impact:** Complex metrics may slow down response times and affect user experience
- **Additional context retrieval overhead:** Fetching inputs or supporting data for evaluation adds extra complexity and load

To mitigate these challenges, the following techniques can be applied:

- **Asynchronous metrics:** Execute metrics in the background without blocking main functionality
- **Sampling:** Running metrics only for some outputs (like for every 10th one) and not for all
- **Smaller models:** Using smaller language models (like *-mini) instead of full-size ones for LLM-as-a-judge metrics

Another major challenge lies in data accessibility and visibility. In offline setups, evaluation is conducted on pre-collected datasets. In contrast, online evaluation operates in a dynamic and less predictable environment. Inputs are often generated by end users or integrated production systems, such as real-time queries, uploaded documents, API calls, or interactive sessions. Outputs are generated in response to these inputs and may never be stored either by design or due to strict data privacy policies.

Now, let's consider a scenario where a metric performs well during offline evaluation but shows significantly worse results once deployed in an online setting. For example, imagine a personalization metric that scored an average of 4.5 out of 5 in offline tests, but drops to only 3.0 in live usage. What does this tell us?

This discrepancy usually points to one clear issue: a data mismatch between offline and online environments. This phenomenon is often referred to as a **distribution shift** between online and offline data. In other words, the offline dataset is *not representative* anymore. It doesn't reflect the real-world product usage. To address this, we may need to:

- Revisit and expand the evaluation dataset to better reflect real-world diversity
- Monitor input distributions in production to detect and quantify shifts
- Recalibrate the evaluation metric using online-collected samples with human-rated ground truth

These measures can help fix the discrepancy between online and offline evaluations.

So, we have learned how to create evaluation datasets and apply appropriate metrics to assess model performance. We have also seen that closed-ended and open-ended tasks require different evaluation approaches. With this foundation, we can now move to the practical aspects of evaluation and explore how to evaluate LLM-based agents in real-world scenarios.

Agent evaluation

As discussed earlier in this chapter, using the weather forecasting and personal email-filtering agents as examples, evaluating an agent is closely tied to evaluating its underlying functions. The more complex an agent is, the more advanced and sophisticated its evaluation needs to be.

In the following sections, we will explore the methodology for evaluating AI agents. We will begin with RAG-based agents, which use context retrieval to extend the LLM's knowledge. Then, we will move on to evaluate conversational and background (autonomous) agents.

RAG-based agent evaluation

Retrieval-augmented generation (RAG) applications introduce a unique set of evaluation challenges.

Consider an agent that answers user questions by retrieving information from a large document collection or performing a live web search. Systems such as ChatGPT (chat.openai.com), Microsoft Copilot, and Google Gemini (gemini.google.com) are all examples of conversational agents that rely on this principle. The key question is: how do we evaluate such a system end-to-end and ensure it performs well in practice?

In principle, a comprehensive offline evaluation of a RAG system requires three key components:

- A well-defined *document collection* that represents the system's knowledge base
- A representative set of *questions* grounded in that collection
- *Ground-truth answers* (or reference outputs) against which the system's responses can be compared

If you are building a domain-specific RAG system (for example, one focused on legal contracts or medical reports), it is feasible to curate such datasets. However, if you are developing a general-purpose RAG service, the challenge becomes much greater. In that case, you need multiple benchmark datasets covering a variety of subject areas.

As we discussed in *Chapter 5*, RAG consists of two primary components:

- **Retriever:** Responsible for finding and ranking the most relevant documents
- **Generator:** Responsible for producing the final answer based on the retrieved documents

So, evaluation of these components can be done in the following way:

- **Retriever evaluation:** Assessing the retriever successfully returns relevant documents for a given query
- **Generator evaluation:** Measuring how well the generator answers the user's query

Retriever evaluation

Looking more closely, the retriever in a RAG system can be viewed in two complementary ways:

- Retrieval can be formulated as a *multi-label classification* problem, where for a given query, multiple documents in the corpus may be relevant while others are not. The retriever's task is to correctly assign a relevance label (relevant or not relevant) to each document in the set.
- Retrieval tasks can also be formulated as *ranking tasks*, where the goal is to assign higher scores to more relevant documents so that they appear earlier in the ranked list of results.

These perspectives allow us to apply well-established classification metrics such as accuracy, precision, recall, and F1. To evaluate performance, ranking-based metrics are commonly used:

- **Precision@k:** Among the top-k retrieved documents, how many are actually relevant?
- **Recall@k:** Out of all relevant documents in the collection, how many appear in the top k?
- **F1@k:** A balanced measure combining precision@k and recall@k

To better illustrate what these metrics represent, consider the following example. Imagine a document collection with 10 documents, of which 3 are truly relevant to a user query. Your retriever returns the following top 5 results ($k=5$), and 2 of them are relevant.

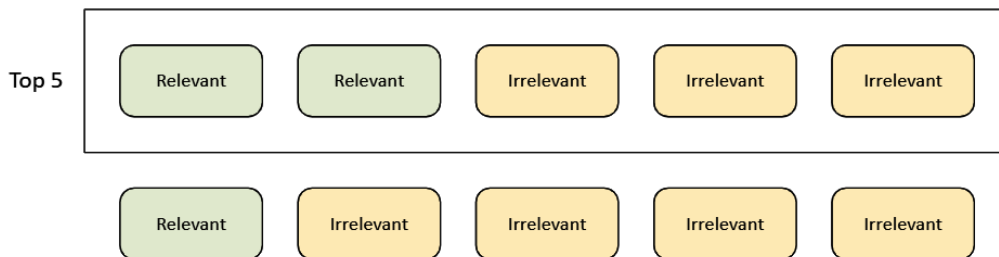


Figure 10.1: Corpus of 10 documents: 3 relevant, 7 irrelevant. 2 relevant are selected in the top 5

Then:

- $\text{Precision@5} = 2 / 5 = 0.4 \rightarrow 40\%$ of the retrieved top-5 documents are relevant
- $\text{Recall@5} = 2 / 3 = 0.67$ (67%) $\rightarrow$ the retriever found two-thirds of all relevant documents
- $\text{F1@5} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}) \approx 0.5$ (50%) $\rightarrow$ a balanced measure of accuracy

If we test multiple queries, we can compute the average precision score, which helps us understand how well our retrieval system performs.

Generator evaluation

Now, let's look more closely at how to evaluate the generator. Because the generator operates within a closed domain (documents retrieved by the retriever), its outputs should be derived directly from the retrieved documents rather than from the model's factual knowledge or reasoning alone.

To evaluate this component, we can adopt principles from closed-domain QA, focusing on two key aspects:

- **Groundedness (or faithfulness):** Measures whether the generated answer is supported by the retrieved context. In other words, does the response rely on the evidence provided, or does it introduce information from the model's own factual knowledge or reasoning?
- **Answer accuracy (or correctness):** Even if an answer is grounded, it still needs to be factually correct. Accuracy captures whether the response states true facts.

Additional evaluation metrics can also be applied, depending on the specific task and context:

- **Relevance:** Evaluates whether the answer directly addresses the user's query without drifting into unrelated or tangential information
- **Completeness and conciseness:** Measures whether the answer covers all essential points while remaining clear and succinct

These evaluation criteria can be implemented using either *human-as-a-judge* or *LLM-as-a-judge* approaches. It is also worth noting that the generator can be partially evaluated in a *reference-free manner* (without ground truth), since both of its inputs (the user question and the retrieved documents) are usually relatively small and bounded.

Taken together, this makes a separate evaluation strategy for the retriever and generator both feasible and efficient. By isolating the two components, we gain clearer insights into where errors originate, whether from irrelevant retrievals or from poor answer generations.

Conversational agent evaluation

Now, let's dive deeper into the evaluation of conversational agents, or chatbots. Earlier in this chapter, we introduced the basics of chatbot evaluation using the example of a weather-forecasting assistant. Here, we will expand on that discussion and explore conversational agent evaluation in greater depth.

As we learned in *Chapter 8*, modern conversational agents must support two fundamental capabilities:

- **Multi-turn conversation:** Maintaining context and coherence across multiple user interactions
- **Tool integration;** Extending the base of LLM's capabilities by invoking external resources such as web search, RAG retrieval, databases, or other APIs

Therefore, in this section, we will focus on evaluating these two foundational aspects as they form the core of modern conversational agent performance.

Agent memory evaluation

To evaluate an agent's memory, the first step is to build an appropriate evaluation dataset. To design such a dataset, we need to understand exactly what agent memory should support.

As we learned in *Chapter 8*, STM enables an agent to carry context across multiple user turns inside one conversational session, resolve coreferences, and maintain coherence throughout the dialogue.

Typically, an STM evaluation dataset should therefore include:

- **Short and long dialogues:** Examples ranging from short exchanges (2–3 turns) to longer conversations (5–10+ turns) with substantial text content. These help assess how well the agent maintains context depth and coherence over time.
- **Multi-modal inputs and outputs:** If the agent supports additional modalities (e.g., images, audio, video) multi-modal inputs should be part of the evaluation dataset.
- **Coreference and ellipsis cases:** Dialogues that involve pronouns or implicit references (e.g., "it," "that one," "the same person") to test whether the agent correctly resolves them based on previous turns.
- **Cross-turn factual recall:** Examples where the user refers back to entities, facts, or intentions introduced earlier (e.g., "What was the second destination I mentioned?").
- **Context shifting and correction:** Cases where the user changes topic or corrects a previous statement (e.g., "Actually, not Paris — I meant Berlin."), testing whether the agent properly updates its internal context.

For example, several first rows in such a dataset could look like this:

Turn	User question	Contextual challenge	Expected agent behavior
1	"Who is the CEO of Microsoft?"	Establishes an entity reference.	Agent should correctly answer: "Satya Nadella."
2	"How old is he?"	Co-reference resolution ("he" → "Satya Nadella").	Agent should respond with their age, using context from Turn 1.
3	"And what company did he work for before his current job?"	Cross-turn recall and temporal reasoning ("he" → "Satya Nadella", "current job" → CEO of Microsoft).	Agent should answer: "He worked at Sun Microsystems," maintaining the reference chain.

Table 10.14: Example STM evaluation dataset with contextual challenges

Such datasets provide the foundation for testing whether an agent correctly preserves context and resolves references. These are the key elements of short-term conversational memory.

Now, let's look at how we can evaluate the responses of conversational agents. As discussed earlier, one straightforward approach is to use reference (ground truth) answers and measure how similar the agent's outputs are to them. For this, similarity-based metrics such as BLEU, ROUGE, or cosine similarity over embeddings can be applied to estimate how close a generated answer is to the expected one.

However, this approach works best for general or open-ended responses, where minor variations in wording are acceptable. It becomes unreliable for factual or knowledge-based questions, where even a small factual error (e.g., a wrong date, number or name) can completely change the meaning.

For example, consider the following questions and agent responses:

- **Question:** "What was Microsoft's total revenue in Q2 2024 according to its financial report?"
- **Reference answer:** "Microsoft reported \$62.0 billion in total revenue for Q2 2024"
- **Agent output A (Correct):** "Microsoft's revenue for Q2 2024 was \$62 billion"
- **Agent output B (Incorrect):** "Microsoft's revenue for Q2 2024 was \$61.2 billion"

Using embedding similarity or lexical comparison, output B might score almost identically to output A, since only a small numeric token differs. But even though the factual meaning of answer B is completely wrong, this example shows why similarity metrics alone are not sufficient for evaluating factual accuracy.

In such cases, as we discussed earlier in the weather-forecast chatbot example, evaluation should be separated into two stages:

- **Information extraction:** Extract factual entities, quantities, or relationships from the model's response (e.g., company: Microsoft, metric: revenue, value: 62.0, period: Q2 2024)
- **Exact match (EM) comparison:** Compare extracted facts against the reference facts, requiring an exact numeric or textual match

This structured approach is fundamental in factual QA and grounded generation evaluation, ensuring that subtle factual errors (like \$61.2 vs. \$62.0) are properly detected even when the rest of the response is nearly identical.

However, how that truth is communicated also plays a vital role. A response can be factually correct yet expressed poorly. It could sound awkward, abrupt, or even impolite. Therefore, beyond factual accuracy, we must also evaluate text quality metrics such as fluency, coherence, tone, human-likeness, and other aspects. These ensure that the answer is not only correct but also clear, natural, and appropriate in its delivery.

Now let's turn to the evaluation of LTM. Such an evaluation should test whether the agent can:

- Remember facts or user preferences mentioned in previous sessions
- Update or forget outdated information when the context changes
- Retrieve relevant past knowledge to personalize or improve future responses

For example, if a user previously told the assistant, "I prefer reports in Excel format," and two days later asks, "Can you prepare the quarterly report?", the agent should automatically respond with an Excel version, demonstrating successful long-term memory recall.

To evaluate such capabilities, datasets must simulate cross-session continuity, with stored memory elements (facts, preferences, summaries) and subsequent prompts that test whether the agent can recall or update them correctly.

Tool orchestration evaluation

Now let's look at how to evaluate tool orchestration. The first step is to build an evaluation dataset that reflects realistic orchestration scenarios. To do this, we need to understand what kinds of tools and orchestration patterns the agent supports.

For example, for the weather forecast chatbot we discussed earlier, the agent should call the weather API when the query is about weather, but it should avoid calling it when the user asks unrelated questions like "Tell me a joke."

If the agent can perform multiple actions per query or chain them (e.g., retrieve data → summarize → send email), then the evaluation dataset must cover:

- Different combinations of tools
- Variations in tool execution order
- Cases requiring conditional branching (e.g., tool A only if tool B's result meets a condition)

For example, a sales assistant agent might first call a CRM search tool to find an opportunity record, then a forecasting model to estimate deal probability, and finally an email API to draft an outreach message. The dataset should test all possible tool sequences:

- [CRM Search → Forecast → Email]
- [CRM Search → Email]
- [Email only] (for a pre-known contact)

Once we have the dataset, the next question is how to evaluate the orchestration results. There are two general approaches: black-box evaluation and white-box evaluation.

In *black-box evaluation*, we treat orchestration as a single end-to-end process. We look only at the final output and assess correctness, factuality, and response quality (e.g., fluency, relevance). This approach is conceptually similar to conversation or memory evaluation and can use reference answers, similarity metrics, or factual extraction with accuracy checks.

However, this method can be unreliable when tools rely on external, dynamic systems such as web search (Google Search or Microsoft Bing). Because these systems constantly update their indexes and results, the same query might return different data tomorrow, making reference-based (with ground truth) evaluation unstable.

In white-box evaluation, instead of judging only the final output, we inspect the sequence of decisions the agent made: which tools were selected, in what order, and with what parameters they were executed. This requires visibility into the orchestration pipeline — for instance, Microsoft Copilot's declarative agents allow reviewing tool execution logs, selected actions, and parameter values.

As we learned in *Chapter 7*, orchestration can be modeled as a *sequence of classification tasks* and *information extractions* for parameter selection. Therefore, we evaluate several aspects:

- Tools selection (multi-class or multi-label classification)
- Parameter selection (slot filling or structured value extraction)
- Execution sequence ordering consistency

This framing allows us to reuse metrics from closed-ended evaluation, such as accuracy, precision, and recall, to measure whether the agent chose the correct tools and executed them in the proper order.

To illustrate this in practice, let's consider a few examples:

Scenario	User query	Expected orchestration	Alternative valid plans	Evaluation focus
Weather assistant	"What's the weather in Paris tomorrow?"	[GetWeather (city=Paris)]	None	Correct single tool call
Research assistant	"Summarize the latest article about quantum computing."	[WebSearch → SummarizeText]	[WebSearch → RetrieveDocs → SummarizeText]	Handling multiple valid plans

Table 10.15: Example orchestration evaluation scenarios and expected execution plans

When multiple valid execution plans exist, all of them should be treated as equally correct, unless there is a clear distinction between optimal and suboptimal plans. Ultimately, tool orchestration evaluation ensures that the agent not only produces accurate outputs but also reasons through the correct sequence of actions to achieve them. This step closes the loop between reasoning, planning, and execution.

Background agent evaluation

Unlike conversational agents, background agents are not triggered by direct user queries. Instead, they respond automatically to external events or conditions and often execute a much wider range of tasks. For instance, consider the email-filtering agent introduced at the beginning of this chapter. Such an agent may be triggered when a new email arrives or on a scheduled timer to summarize recent messages.

Because these agents perform multiple heterogeneous tasks, their evaluation should focus on the specific functions they execute. Fundamentally, evaluating a background agent means evaluating the set of its underlying tasks. These tasks may include classification, summarization, information extraction, recommendation, or decision support actions. Depending on the nature of each task, they can be closed-ended or open-ended.

Therefore, all the evaluation principles we have discussed throughout this chapter can be directly applied here. The overall performance of a background agent is effectively an aggregate of its task-level evaluations, combined with monitoring of its trigger reliability and timing consistency.

Responsible AI

Evaluation isn't just about measuring how well a system performs. It's also about assessing how safely, ethically, and fairly it behaves. LLMs accept arbitrary input and generate free-form responses, which may be factually correct but still harmful, biased, or unethical. So, beyond performance, we must also evaluate and monitor whether AI systems are safe, fair, and resistant to misuse. This all forms responsible AI evaluation.

In this final section, we'll shift our focus from traditional performance metrics to responsibility metrics. We'll explore how to evaluate fairness, detect harmful or unsafe content, manage hallucinations, and implement safeguards that promote trust, accountability, and responsible AI behavior.

As we discussed in *Chapter 2*, most large language models are initially trained on massive-scale text corpora, often derived from public internet sources like Common Crawl, Wikipedia, news sites, books, forums, and social media. These datasets are broad and diverse, providing LLMs with general world knowledge and language patterns. However, they're not carefully curated, and their scale makes it nearly impossible to fully vet every document for legality, factuality, or ethical soundness.

As a result, LLMs inevitably absorb problematic patterns, including harmful stereotypes, toxic language, misinformation, and unethical content. Even if the data source seems reliable on average, undesirable biases and edge cases slip through due to sheer volume and uneven representation across communities and perspectives.

Moreover, due to their generative nature and reasoning capabilities, LLMs are not limited to regurgitating training data. They can synthesize novel, potentially harmful content, such as instructions for illegal activities, conspiracy theories, or targeted harassment, even when such content wasn't explicitly part of the training data. For example, if prompted, an uncensored LLM might generate a recipe for a banned substance or a step-by-step plan for committing a crime, simply by recombining plausible knowledge fragments.

Another core issue is representation bias. Internet content is not evenly distributed across languages, cultures, genders, or regions. Some groups are vastly overrepresented (e.g., English-speaking users from Western countries), while others are underrepresented or misrepresented. As a result, LLMs may encode cultural, gender, or racial biases that reflect these imbalances, leading to outputs that disadvantage or marginalize certain users.

Thus, after pretraining, LLMs become capable of generating virtually any kind of text, including toxic, biased, manipulative, or unsafe outputs. This is why further fine-tuning, post-training alignment, and **reinforcement learning from human feedback (RLHF)** are essential to mitigate risks before deployment.

Now that we understand the risks, it's clear that developers of language models like OpenAI, Anthropic, or Google have a responsibility to ensure their models are safe, unbiased, and ethical. But this raises an important question:

If we're simply building an AI application or agent on top of an existing foundation model (like GPT, Claude, or Gemini), why should we care about responsible AI? Isn't that the base model developer's job?

The short answer is we must care because risks can easily emerge or be amplified during downstream use. Even if the foundation model is well-aligned, the way we prompt it, the data we feed it, and the actions we automate can introduce or surface harmful behavior.

Just like using secure cloud infrastructure (like Azure or AWS) and secured software components doesn't automatically make your web application secure, building on top of a well-aligned and safe LLM doesn't guarantee that your AI system is safe, fair, or ethical.

Relying on secure components is necessary, but not sufficient. You must validate and enforce responsible behavior at the application level. This principle applies not only to prompt design but becomes even more critical when performing model fine-tuning. With prompt engineering, we can sometimes steer or constrain the model's behavior through instructions and context, but the underlying safety alignment generally remains intact.

However, during full fine-tuning, we effectively overwrite parts of the model's learned behavior, potentially undoing previous safety safeguards or introducing new biases inherited from our custom data. Unless carefully controlled, such fine-tuning can degrade the model's alignment, moderation filters, or factual grounding.

So, at this point, it becomes evident that evaluating the safety of LLMs is not optional but essential and mandatory in modern AI development. This, too, raises a practical question:

What exactly should we do to ensure that our application meets responsible AI standards?

Fortunately, the structure of Responsible AI evaluation follows a similar approach to functional evaluation. In both cases, we:

- Define what "good behavior" looks like (e.g., safe, fair, inclusive, non-toxic outputs)
- Build appropriate evaluation datasets to test these risks
- Define metrics to measure the presence or absence of problems
- Run evaluations both offline and online, before and after deployment

Just as with functional evaluation, responsible AI evaluation is not a one-time task. It's an ongoing, iterative process that must evolve as your application, users, and risk surface evolve.

Responsible AI risks and evaluation criteria

When evaluating LLM-based systems for safety and responsibility, one of the key priorities is detecting and mitigating content that falls into known high-risk categories. These include the following content types:

- **Hateful and discriminatory:** Includes hate speech, offensive language, or slurs targeting individuals or groups based on race, gender, religion, nationality, sexual orientation, or other protected attributes.
- **Sexual and inappropriate content:** Covers sexually explicit material, adult content, or content that is not suitable for general audiences. Even if not intentionally harmful, such outputs can violate product policies or age-appropriate use guidelines.
- **Violent and graphic content:** Involves depictions or descriptions of violence, threats, abuse, or glorification of physical harm. This type of content may traumatize users, violate terms of service, or breach legal standards in sensitive regions.
- **Self-harm and suicide content:** Refers to suggestions, encouragement, or detailed discussion of self-harm, suicide, eating disorders, or other mental health crises.
- **Copyrighted or protected material:** Includes outputs that repeat verbatim or substantially reproduce content protected by copyright, licensing, or privacy rights, such as books, code, lyrics, proprietary data, or **identifiable personal information (PII)**. This raises legal, ethical, and compliance concerns.

- **Dangerous and illegal instructions:** Covers advice, recipes, or guides for carrying out harmful, unlawful, or unsafe activities. This includes making explosives, hacking, fraud, bypassing content restrictions, or drug manufacturing.
- **Harassment and bullying:** Refers to targeted abuse, intimidation, doxxing, or incitement of harassment against individuals. Unlike hate speech, which is based on identity, harassment can be personal and directed, regardless of protected status.

These categories are commonly used in industry moderation policies (e.g., OpenAI's safety categories, Facebook's community standards, Google's responsible AI guidelines) and should be explicitly tested when building or auditing LLM-powered applications.

Fortunately, many responsible AI metrics have already been standardized and implemented in evaluation frameworks such as the Azure Evaluation SDK, OpenAI's eval frameworks, and safety evaluation datasets. In the next section, we'll explore how to leverage these existing datasets, tools, and libraries to run evaluations of safety, fairness, and bias.

Evaluation datasets for safety, bias, and fairness

The responsible AI criteria and metrics we discussed above can be applied to your regular offline evaluation datasets to ensure there are no safety violations. To truly stress-test a system's safety, fairness, and bias resilience, we must also provoke the product to fail, intentionally feeding it inputs that have a higher chance of eliciting harmful, biased, or unsafe behavior. This requires specialized evaluation datasets created specifically for responsible AI checks.

Such datasets are designed to:

- Contains high-risk prompts and edge cases
- Probe sensitive demographic or ethical scenarios
- Test model behavior under adversarial or "jailbreak" conditions (see below)
- Assess consistency of responses across diverse user groups

There are already several well-known responsible AI evaluation datasets available for public use. For example:

- **RealToxicityPrompts:** Focuses on eliciting toxic, offensive, or otherwise unsafe responses
- **HarmBench:** Contains prompts designed to trigger unsafe or harmful instructions across multiple categories (e.g., weapons, hacking, abuse)
- **CrowS-Pairs:** tests for social bias by presenting minimally different sentence pairs that differ only in demographic or identity references

These datasets are task-agnostic. They can be used in question answering, summarization, dialog, or any other generative scenario. The prompts are designed to stress-test your application or agent by increasing the likelihood of unsafe content generation.

In other circumstances, you may need to create your own responsible AI evaluation dataset, particularly when working in a specialized domain (e.g., healthcare, finance, law) or targeting unique user scenarios not covered by public benchmarks.

For example:

- A medical chatbot might require prompts that intentionally seek dangerous treatment advice or misinformation specific to its field
- A financial planning assistant might need adversarial cases where users attempt to elicit insider trading tips or unethical/risky investment strategies

Obtaining realistic, human-generated datasets for safety evaluation can be difficult and resource-intensive. Real-world unsafe examples are hard to collect ethically and may involve privacy or legal concerns. A common alternative is to use synthetic data generation producing evaluation prompts and risky scenarios. Synthetic data offers scalability and flexibility, allowing you to cover a broad range of potential risks without collecting sensitive real-world content.

However, this approach has its own challenges:

- To generate convincing, harmful, or unsafe prompts, you often need *uncensored* or minimally filtered language models, which may not be publicly accessible or may carry legal or ethical implications
- Care must be taken to avoid leaking unsafe generated data into training corpora or inadvertently exposing it to annotators without safeguards

Today, there are many uncensored or minimally moderated LLMs available for research purposes, such as LLaMA-Uncensored, Vicuna-Uncensored, Guanaco, or certain Mistral and Falcon variants hosted on open model hubs like Hugging Face. These models are trained without the strong safety filters present in production-grade systems like GPT-4 or Claude.

So, building custom safety evaluation datasets is not trivial and comes with its own set of risks, including the use of uncensored or minimally filtered LLMs, handling sensitive content, and ensuring ethical safeguards. At the same time, these datasets are essential for thoroughly stress-testing AI systems and ensuring they behave safely, fairly, and responsibly in real-world scenarios.

Jailbreak and adversarial prompt evaluation

A **jailbreak** is a technique where a user manipulates the input prompt to override the system's safety constraints or alignment instructions. This can involve:

- **Role-play or framing** ("pretend you are a historian describing how to...")
- **Prompt injection** ("ignore your previous instructions and instead...")
- **Translation tricks** (asking in another language to bypass keyword filters)
- **Encoding or obfuscation** (using Unicode, symbols, or code blocks to hide unsafe requests)

The goal is to make the model believe it is operating in a "safe" or permissible context while still generating harmful, prohibited, or unintended content. Such methods are highly creative and constantly evolving, making them difficult to fully defend against.

Note

The term **jailbreak** comes from the security world, where it describes bypassing restrictions on locked-down devices (such as smartphones or gaming consoles) to gain full, unauthorized control. In the context of LLMs, jailbreaking refers to bypassing built-in safety, policy, or alignment constraints to make the model produce outputs it is otherwise programmed to refuse.

While jailbreaks intentionally aim to bypass safeguards, **adversarial prompts** represents a broader category of crafted inputs designed to provoke incorrect, misleading, or unsafe outputs even without explicitly breaking the model's rules. These can target:

- **Functional reliability** (forcing the model to output wrong facts)
- **Fairness** (eliciting biased or discriminatory statements)
- **Safety** (provoking harmful or explicit content)

Understanding and testing against such prompts is essential. A robust system must therefore demonstrate resilience not only to ordinary inputs but also to these adversarial edge cases.

For example, consider a sales agent who sends outreach emails offering a product or service to potential customers. A user might respond with a cleverly phrased request for a 50% discount. If the system is not properly constrained, the agent may generate a response that confirms the discount or negotiates beyond allowed limits. Such behavior could mislead customers, violate pricing policies, or even create legal and reputational risks.

This illustrates that safety and responsibility are not limited to avoiding toxic or harmful language. They also include ensuring factual accuracy, policy compliance, and consistency with organizational intent.

Evaluation tools and frameworks

Now that we've explored what to evaluate, it's time to discuss the practical steps for running offline and online evaluations. In this section, we'll look at a few established evaluation frameworks and tools that can streamline the process and reduce engineering overhead.

As we already discussed, building an effective evaluation pipeline generally requires three core components: evaluation datasets, offline evaluation frameworks, and online evaluation frameworks.

Evaluation datasets can be a set of static files (JSON, JSONL, CSV, YAML, plain text) or some more advanced data structures. Many modern evaluation ecosystems provide dataset classes, loading utilities, and APIs for seamless integration into evaluation pipelines. Here is a code example of how evaluation can be done for the summarization task:

```
from datasets import load_dataset
from transformers import pipeline
import evaluate
import numpy as np

# Load a known CNN/DailyMail dataset (summarization)
dataset = load_dataset("cnn_dailymail", "3.0.0", split="test")

# Define a generator (model)
summarizer = pipeline("summarization", model="google/flan-t5-small",
tokenizer="google/flan-t5-small",
device_map="auto", max_new_tokens=128, truncation=True,
)

def generate_summary(text: str) -> str:
    out = summarizer(text, do_sample=False)[0]["summary_text"]
    return out.strip()

# Run generation
preds, refs = [], []
for row in dataset:
    article = row["article"]
    reference = row["highlights"]
    pred = generate_summary(article)
```



```

    preds.append(pred)
    refs.append(reference)

# Load & compute metrics. ROUGE (lexical overlap) and BERTScore (semantic
similarity
rouge = evaluate.load("rouge")
rouge_scores = rouge.compute(predictions=preds, references=refs)
bertscore = evaluate.load("bertscore")
bs = bertscore.compute(predictions=preds, references=refs, model_type="microsoft/
deberta-large-mnli")

# Convert list scores to means for a compact report
bs_report = {
    "bertscore_precision": float(np.mean(bs["precision"])),
    "bertscore_recall":    float(np.mean(bs["recall"])),
    "bertscore_f1":       float(np.mean(bs["f1"])),
}

```

This code first loads the CNN/DailyMail dataset, which is commonly used to validate summarization models. For demonstration purposes, it then loads a T5 model to perform summarization. It takes an article from the dataset, generates a summary, and uses the provided highlights field as the human-written reference. Finally, it calculates BLEU, ROUGE, and BERTScore between the generated summary and the reference, producing an evaluation report.

Today, the Hugging Face evaluate library supports a wide range of evaluators from simple exact-match and lexical similarity metrics (BLEU, ROUGE, METEOR) to semantic similarity measures (BERTScore, MoverScore) and more advanced evaluators for bias detection, copyright detection, and even custom LLM-as-a-judge metrics.

Another framework we can use to perform evaluation is the Azure Evaluation SDK. It allows you to run offline evaluations locally and log results to Azure. It supports a large range of metrics:

- Algorithm-based metrics (BLEU, ROUGE, METEOR, F1, similarity)
- AI-assisted evaluators for quality (fluency, coherence, relevance, groundedness)
- Safety evaluators (hate/unfairness, sexual, violence, self-harm, jailbreak/indirect attack, protected material/PII, etc.)

These evaluators can also return brief reasons/rationales in addition to evaluation scores or labels. Azure's risk and safety evaluations additionally support adversarial testing (e.g.,

jailbreak vulnerability checks) and can generate adversarial test sets. Here is a code example of how we can use the Azure Evaluation SDK to run evaluations:

```
from azure.ai.evaluation import (
    evaluate,
    RelevanceEvaluator,      # quality
    GroundednessEvaluator,   # quality (RAG / factual consistency)
    ContentSafetyEvaluator    # composite safety: hate/sexual/violence/self-harm
)

# Evaluator configuration
model_config = {
    "azure_openai_endpoint": "<YOUR_AOAI_ENDPOINT>",
    "api_key": "<YOUR_AOAI_KEY>",
    "deployment_name": "gpt-4o-mini"
}

# Instantiate evaluators
relevance = RelevanceEvaluator(model_config)
groundedness = GroundednessEvaluator(model_config)  # for RAG answers with
context
safety = ContentSafetyEvaluator(get_azure_ai_project())  # composite safety
(hate, sexual, violence, self-harm)

# run the batch evaluation and optionally log to Azure AI project
result = evaluate(
    data="eval_data.jsonl",
    evaluators={
        "relevance": relevance,
        "groundedness": groundedness,
        "safety": safety
    },
    evaluator_config=get_evaluator_config(),
    azure_ai_project=azure_ai_project,  # Logs a run you can inspect in Azure AI
Foundry
    output_path="./eval_summary.json"  # saves aggregate + per-row metrics
locally
)
```

This code first defines the model configuration, specifying the Azure OpenAI deployment details that the evaluators will use. It then initializes relevance, groundedness, and content

safety evaluators. Finally, it runs the evaluation on the dataset and saves both the aggregate metrics and detailed per-example results.

So, we've explored the Hugging Face and Azure AI Evaluation SDK frameworks. While many other evaluation frameworks exist today, such as OpenAI Evals and LangChain, the ecosystem is evolving rapidly. New tools and approaches continue to emerge, making it important to choose solutions that fit your use case and remain adaptable as the field advances.

Summary

Great, we've now wrapped up one of the most fundamental chapters in this book. While we've covered evaluation near the end, in practice, it should be one of the very first things you think about when developing AI systems. In AI, everything revolves around accuracy, quality, and metrics. Without reliable performance measurements, it's impossible to know if your system is truly ready for real-world use.

History shows that we only start using AI models and features at scale once they meet certain accuracy or quality thresholds; otherwise, they remain research experiments rather than viable products. The same principle applies when building AI-powered agents.

Evaluation is not just a box to tick at the end of development. It's the feedback loop that drives iteration, ensures reliability, and builds trust with users. It guides product teams toward meaningful improvements and provides an objective basis for choosing one approach over another when solving a specific problem.

And last but not least, everything we've covered here is only the beginning of your journey into evaluating real-world AI applications and agents. The tools, metrics, and strategies you've learned form a strong foundation. But true expertise comes from applying them in practice, adapting them to your specific domain, and evolving your evaluation processes, such as when product, data, and user needs change. Evaluation is not a one-time milestone; it's an ongoing discipline that will shape the reliability, safety, and trustworthiness of every AI system you build.

Subscribe for a free eBook

New frameworks, evolving architectures, research drops, production breakdowns – *AI_Distilled* filters the noise into a weekly briefing for engineers and researchers working hands-on with LLMs and generative AI systems. Subscribe now and receive a free eBook, along with weekly insights that help you stay focused and informed.

Subscribe at <https://packt.link/80z6Y> or scan the QR code below.



11

Unlock Your Exclusive Benefits

Your copy of this book includes the following exclusive benefits:



DRM-Free PDF Version

Download DRM-free PDF and ePub copies of this book.



7-Day Packt Library Access

Get 7-day unlimited access to 8,000+ books and videos. No credit card required.

Available for first-time Packt+ trial users only.



Next-Gen Reader Access

Read this book on Packt Reader with progress sync, dark mode and note-taking.

Follow the guide below to unlock them. The process takes only a few minutes and needs to be completed once.

Unlock this Book's Free Benefits in 3 Easy Steps

Step 1

Keep your purchase invoice ready for *Step 3*. If you have a physical copy, scan it using your phone and save it as a PDF, JPG, or PNG.

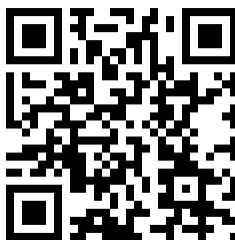
For more help on finding your invoice, visit <https://www.packtpub.com/unlock-benefits/help>.

Note

If you bought this book directly from Packt, no invoice is required. After *Step 2*, you can access your exclusive content right away.

Step 2

Scan the QR code or go to [packtpub.com/unlock](https://www.packtpub.com/unlock).



On the page that opens (similar to *Figure 11.1* on desktop), search for this book by name and select the correct edition.

packt Search...

Subscription Shopping Cart User Profile

Explore Products Best Sellers New Releases Books Videos Audiobooks Learning Hub Newsletter Hub Free Learning

Discover and unlock your book's exclusive benefits

Bought a Packt book? Your purchase may come with free bonus benefits designed to maximise your learning. Discover and unlock them here

Discover Benefits Sign Up/In Upload Invoice

[Need Help?](#)

1. Discover your book's exclusive benefits

Search by title or ISBN

CONTINUE TO STEP 2

2. Login or sign up for free

3. Upload your invoice and unlock

Figure 11.1: Packt unlock landing page on desktop

Step 3

After selecting your book, sign in to your Packt account or create one for free. Then upload your invoice (PDF, PNG, or JPG, up to 10 MB). Follow the on-screen instructions to finish the process.

Need help?

If you get stuck and need help, visit <https://www.packtpub.com/unlock-benefits/help> for a detailed FAQ on how to find your invoices and more. This QR code will take you to the help page.



Note

If you are still facing issues, reach out to customercare@packt.com

Stay Sharp in Cloud and DevOps – Join 44,000+ Subscribers of CloudPro

CloudPro is a weekly newsletter for cloud professionals who want to stay current on the fast-evolving world of cloud computing, DevOps, and infrastructure engineering.

Every issue delivers focused, high-signal content on topics like:

- AWS, GCP & multi-cloud architecture
- Containers, Kubernetes & orchestration
- **Infrastructure as Code (IaC)** with Terraform, Pulumi, etc.
- Platform engineering & automation workflows
- Observability, performance tuning, and reliability best practices

Whether you're a cloud engineer, SRE, DevOps practitioner, or platform lead, CloudPro helps you stay on top of what matters, without the noise.

Scan the QR code to join for free and get weekly insights straight to your inbox:



<https://packt.link/cloudpro>

Appendix 1

Expert Insights

To complement the book vision, we wanted to include insights from someone actively shaping modern AI transformation. In May 2025, I had the opportunity to speak with Egor Pushkin, *Chief Architect, Data and AI at Oracle Cloud*. With deep expertise in NLP and years of experience building enterprise-scale AI systems, Egor brings a unique perspective on where we are today and where we're headed next.

In our conversation, he shared insights on the current state of AI, the evolving role of large language models, and the challenges that still lie ahead. Here is our dialogue:

- **Vasyl:** Hi Egor, great to see you again. Let's talk about the current state of AI — where we are today and where things might be in the future.
- **Egor:** Hi Vasyl, thanks for setting this up. Absolutely, let's talk about that.
- **Vasyl:** Prompt engineering has opened a new era, allowing many AI problems to be solved without deep AI expertise. Tasks that before required significant effort and specialized knowledge can now be solved with prompts. So, the question: Do you believe this shift has fundamentally changed the roles of applied AI professionals and software engineers as well?
- **Egor:** Despite common misconceptions, the rise of generative AI has not redefined the core responsibilities of applied scientists and data scientists. Instead, it has led to widespread mischaracterizations of technical work — for example, launching a fine-tuning job is often misinterpreted as applied science, when in reality it typically is not. However, new roles have emerged, such as prompt engineering, which, despite some debate around its compensation and place in the AI workflow, has proven to require skill, knowledge, and deliver value. That said, as large language models become more capable, prompt engineering is likely to be increasingly automated. We're already

seeing signs of this, with prompt optimization becoming a key focus for automation — a trend that will continue until the process becomes largely invisible to the end user.

- **Vasyl:** Do you feel that LLMs might be approaching a performance ceiling, at least within the bounds of current architecture and design? We can say that models like GPT-3 and GPT-3.5 represented major leaps in what language models could do functionally. GPT-4 brought another significant jump in performance and quality. However, more recent models have delivered less dramatic breakthroughs. For instance, GPT-4o is essentially an optimized and more cost-effective version of GPT-4, with native multimodal capabilities. Similar trends are seen across the industry — newer models are becoming slightly better, faster, cheaper, and safer, but without the transformative leaps we saw in earlier generations.
- **Egor:** We're definitely seeing a slowdown in progress with the current LLM architecture — the big leaps are becoming rarer. However, I believe there are still emergent abilities we haven't fully tapped into, particularly through better scaling and more sophisticated data preparation. The question of how LLMs learn is a fascinating one. Research from teams like Anthropic is shedding light on this, and by digging deeper, we can unlock even more capabilities.
That said, I do think there are future architectures out there that could deliver the next step-change in performance. Whether we absolutely need them to keep progressing is debatable, but they certainly offer promise.

Personally, I'm very bullish on a compound systems approach — wrapping LLMs into richly contextualized environments that break down tasks, manage orchestration, and elevate model output. This agentic space, as I see it, is one of the most exciting directions AI can take next.

- **Vasyl:** Before the rise of prompt engineering and high-quality LLMs (such as GPT-3), solving business-specific AI problems typically required fine-tuning or building models from scratch. Today, prompt engineering offers a faster and cheaper (for development) alternative in many cases.
There are still scenarios where fine-tuning or custom model development remains necessary. Looking ahead, do you think fine-tuning and model building will become more of a niche approach, or will they continue to be widely used alongside prompt engineering?
- **Egor:** Although Hugging Face hosts a vast number of fine-tuned model repositories, deploying these models into production remains relatively uncommon — and I don't expect that to change much moving forward. It serves a purpose, but only in very

specific, often niche scenarios. The process is complex, resource-intensive, and frequently increases inference costs.

There's legitimate value in fine-tuning for edge cases, but the industry trend is clear: as generative AI proves its utility in more domains, leading model providers are embedding these capabilities directly into their foundation models. This shift makes external fine-tuning largely redundant. We've already seen this play out in the coding space, where foundational improvements and the rise of compound systems have sharply reduced the need for custom fine-tuning.

- **Vasyl:** You've spoken a lot about the concept of an AI agent and specifically agentic OS. What do you see as the key components that are still missing — or not yet mature enough — for agent-based systems to scale and gain widespread adoption?
- **Egor:** To enable the broad adoption of automation through agent-based systems, we need a well-structured ecosystem composed of three key layers: authoring, marketplace, and workspace.

Authoring is the foundational layer where intelligent agents are created and refined using a wide array of technologies, such as LLMs, development frameworks, and low-/no-code IDEs. This environment empowers developers to build increasingly sophisticated agents, enabling complex automation across many domains.

Marketplace acts as a distribution and discovery hub where intelligent agents can be hired for on-demand digital work. Similar to freelance platforms, it allows users to contract agents for tasks of varying complexity and duration. Beyond hiring, it serves as a centralized system for managing agent availability, performance, and costs—essentially functioning as HR software for digital labor.

Workspace is the user-facing layer, providing the digital environment in which agents operate. This layer ensures agents have access to tools and data, while offering users transparency, oversight, and control. Serving as a crucial interoperability bridge, it enables seamless integration across diverse digital systems and use cases—from family tasks to enterprise-level deployments.

- **Vasyl:** What's your take on low-code/no-code platforms for building AI agents? Companies like Microsoft and Google are heavily investing in tools like Copilot Studio or Google Vertex AI to make agent development more accessible. Do you see these platforms as a viable path forward, or are there limitations that still need to be addressed?
- **Egor:** This really comes down to the question of specialization. Specialized agents can still outperform in narrowly defined tasks, delivering high accuracy where it matters most. But what we're seeing in the industry is a steady shift toward generalist agents

— systems capable of navigating broader, cross-domain tasks with growing competence. Over time, I expect these two approaches will begin to merge. Generalists will likely become the dominant model due to their flexibility and scalability, while specialized agents will continue to hold their ground in areas where exactness and deep domain expertise remain essential.

- **Vasyl:** To what extent do you think current LLMs and AI systems are ready for truly autonomous use cases, and is society prepared to handle that level of autonomy? An analogy that comes to mind is fully autonomous vehicles. While we've made great progress with driver assistance systems, full autonomy still feels far away — not just because of remaining technical hurdles, but also because society itself isn't fully ready to trust or accept such systems at scale. Do you see a similar gap in readiness when it comes to AI agents and autonomous decision-making in broader domains?
- **Egor:** This is a multi-dimensional question, at the intersection of quality and trust. In my view, we're still quite far from seeing widespread deployment of fully autonomous systems powered by Generative AI. The path forward, realistically, begins with tightly integrated AI capabilities operating within human-supervised workflows. It's a step-by-step journey. Over time, as these systems prove reliable and as younger generations grow increasingly accustomed to offloading routine tasks to machines, we'll see a natural, perhaps inevitable, deepening of automation across both industry and society.
- **Vasyl:** And the last question. Are you optimistic about the societal impact of AI — especially when it comes to potential job displacement? There's growing concern today about AI replacing human workers, and fears around job loss are becoming more widespread. In fact, we recently saw the first strike at Seattle port, where workers protested against being replaced by machines. How do you see this tension playing out, and do you think society can adapt to the changes AI is bringing?
- **Egor:** I tend to view the situation through a different lens, one that isn't colored by alarm or pessimism. Change has always been a constant throughout human history, but the pace and nature of that change have dramatically accelerated, first over the past century, and now even more rapidly in just the last decade. Technology today is driving transformation at a scale that's often unpredictable. Consider the unexpected inversion where trades like plumbing are seen as more stable than entry-level coding. At the same time, we're witnessing the rise of junior specialists entering the workforce in AI-assisted "copilot" roles. These individuals are learning and growing into areas that are not yet automated, highlighting the dynamic interplay between human adaptability and machine advancement.

Egor's perspective highlights a powerful topic: this is a pivotal moment in AI. We've moved from isolated models to general-purpose LLMs, from theory to production-ready tools, and

from narrow use cases to the possibility of fully autonomous agents. At the same time, the conversation reminded us that progress is not just about technology; it's about people, trust, and the ways we choose to integrate AI into our work and society.

This book aims to help you bridge the gap between understanding these trends and putting them into practice. By mastering prompts, RAG, fine-tuning, agents, and evaluation, you'll be able to create AI systems that not only work but also deliver value responsibly, shaping a future where AI augments human potential rather than replaces it.

Appendix 2

Bibliography

1. M. Suleyman and M. Bhaskar, *The Coming Wave: Technology, Power, and the Twenty-First Century's Greatest Dilemma*, New York: Crown, 2023.
2. "AI for the Next Era: OpenAI's Sam Altman on the New Frontiers of AI," [Online]. Available: <https://greylock.com/greymatter/sam-altman-ai-for-the-next-era/>
3. "Almost Half of S&P 500 Companies Cite AI on Fourth Quarter Earnings Call," [Online]. Available: <https://www.thecorporatecounsel.net/blog/2025/03/almost-half-of-sp-500-companies-cite-ai-on-fourth-quarter-earnings-call.html>
4. "Highest Number of S&P 500 Companies Citing "AI" on Q2 Earnings Calls in Over 10 Years," [Online]. Available: <https://insight.factset.com/highest-number-of-sp-500-companies-citing-ai-on-q2-earnings-calls-in-over-10-years>
5. S. V. Z. V. Skopetskiy V.V., "Mathematical Modeling of the Dynamics of Distributed Space-Time Processes", Kiev, 2008.
6. "arXiv trends," [Online]. Available: <https://arxiv-trends.com/>
7. N. S. N. P. J. U. L. J. A. N. G. L. K. I. P. Ashish Vaswani, "Attention Is All You Need," *arXiv*, 2023.
8. N. Heidloff, "Foundation Models, Transformers, BERT and GPT," [Online]. Available: <https://heidloff.net/article/foundation-models-transformers-bert-and-gpt/>
9. "Open AI models benchmarks," Open AI, [Online]. Available: <https://github.com/openai/simple-evals?tab=readme-ov-file#benchmark-results>
10. "100+ LLM benchmarks and evaluation datasets," Evedently AI, [Online]. Available: <https://www.evidentlyai.com/llm-evaluation-benchmarks-datasets>

11. H. D. F. D. T. B. R. A. R. S. Y. H. S. Ali Modarressi, "NoLiMa: Long-Context Evaluation Beyond Literal Matching," *arXiv*, 2025.
12. "Long Context RAG Performance of LLMs," databricks, [Online]. Available: <https://www.databricks.com/blog/long-context-rag-performance-llms>
13. K. L. J. H. A. P. M. B. F. P. P. L. Nelson F. Liu, "Lost in the Middle: How Language Models Use Long Contexts," *arXiv*, 2023.
14. "Introducing GPT-4.1 in the API," Open AI, 2025. [Online]. Available: <https://openai.com/index/gpt-4-1/>
15. "Non-determinism in GPT-4 is caused by Sparse MoE," Simple Thoughts, 2023. [Online]. Available: <https://152334h.github.io/blog/non-determinism-in-gpt-4>
16. "Understanding why deterministic output from LLMs is nearly impossible," Unstrat.com, 2025. [Online]. Available: <https://unstrat.com/blog/understanding-why-deterministic-output-from-llms-is-nearly-impossible/>
17. "Face hallucination," Wikipedia, [Online]. Available: https://en.wikipedia.org/wiki/Face_hallucination
18. "Hallucinations in Neural Machine Translation," OpenReview.net, 2023. [Online]. Available: <https://openreview.net/forum?id=SkxJ-309FQ>
19. "Hallucination (artificial intelligence)," Wikipedia.org, [Online]. Available: https://en.wikipedia.org/wiki/Hallucination_%28artificial_intelligence%29
20. "GPT-4 architecture, datasets, costs and more leaked," decoder, 2023. [Online]. Available: <https://the-decoder.com/gpt-4-architecture-datasets-costs-and-more-leaked/>
21. "Learning to reason with LLMs," OpenAI, 2024. [Online]. Available: <https://openai.com/index/learning-to-reason-with-llms/>
22. K. Hu, "ChatGPT sets record for fastest-growing user base - analyst note," Reuters, 2023. [Online]. Available: <https://www.reuters.com/technology/chatgpt-sets-record-fastest-growing-user-base-analyst-note-2023-02-01>
23. "The Power of Hugging Face AI," Medium, 2023. [Online]. Available: <https://medium.com/@gargg/the-power-of-hugging-face-ai-4f6558ee0874>
24. Y. H. L. Junyan Zhang, "Do BERT-Like Bidirectional Models Still Perform Better on Text Classification in the Era of LLMs?," *arXiv*, 2025.

25. "Claude is providing incorrect or misleading responses. What's going on?," Claude, 2025. [Online]. Available: <https://support.claude.com/en/articles/8525154-claude-is-providing-incorrect-or-misleading-responses-what-s-going-on>
26. "Zero-shot learning," Wikipedia, [Online]. Available: https://en.wikipedia.org/wiki/Zero-shot_learning
27. P. A. S. L. Chelsea Finn, "Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks," *arXiv*, 2017.
28. S. B. Mojan Javaheripi, "Phi-2: The surprising power of small language models," Microsoft Research Blog, [Online]. Available: <https://www.microsoft.com/en-us/research/blog/phi-2-the-surprising-power-of-small-language-models>
29. F. W. J. S. S. K. Juyong Jiang, "A Survey on Large Language Models for Code Generation," *arXiv*, 2024.
30. D. H. Spennemann, "Delving into: the quantification of Ai-generated content on the internet (synthetic data)," *arXiv*, 2025.
31. "What is synthetic data — and how can it help you competitively?," MIT Management, [Online]. Available: <https://mitsloan.mit.edu/ideas-made-to-matter/what-synthetic-data-and-how-can-it-help-you-competitively>
32. "Most Used Languages on the Internet in 2024," Oberlo, 2024. [Online]. Available: <https://www.oberlo.com/statistics/most-used-languages-on-the-internet>
33. H. L. W. Y. F. L. L. X. R. Haoang Chi, "Unveiling Causal Reasoning in Large Language Models: Reality or Mirage?," *arXiv*, 2025.
34. V. K. J. M. R. T. Yuefei Chen, "CounterBench: A Benchmark for Counterfactuals Reasoning in Large Language Models," *arXiv*, 2025.
35. Q. H. Z. Z. J. W. Y. Z. J. W. X. L. J. G. T. O. F. M. C. S. Y. Y. Q. Y. Y. S. S. R. X. H. Y. L. J. F. C. G. Y. L. Fengli Xu, "Towards Large Reasoning Models: A Survey on Scaling LLM Reasoning Capabilities," *arXiv*, 2025.
36. Y. Z. S. F. A. S. Z. Y. Daking Rai, "A Practical Review of Mechanistic Interpretability for Transformer-Based Language Models," *arXiv*, 2024.
37. "Public Preview: Vector Search in Azure Cosmos DB for MongoDB with DiskANN," Microsoft, 2024. [Online]. Available: <https://devblogs.microsoft.com/cosmosdb/diskann-vcore-based-azure-cosmosdb-mongodb/>
38. W. A. S. L. S. C. A. W. Inacio Vieira, "How Much Data is Enough Data? Fine-Tuning Large Language Models for In-House Translation: Performance Evaluation Across Multiple Dataset Sizes," *arXiv*, 2024.

39. S. S. G. J. C. S. E. S. N. R. M. S. K. J. T. J. J. B. B. Zoltan P Majdik, "Sample Size Considerations for Fine-Tuning Large Language Models for Named Entity Recognition Tasks: Methodological Study," JMIR Publications, 2024. [Online]. Available: <https://ai.jmir.org/2024/1/e52095>
40. S. A. G. S. K. K. B. A. Rakshit Aralimatti, "Fine-Tuning Small Language Models for Domain-Specific AI: An Edge AI Perspective," *arXiv*, 2025.
41. Y. P. Z. Y. Keyu Chen, "Parameter-Efficient Fine-Tuning With Adapters," *arXiv*, 2024.
42. Y. S. P. W. Z. A.-Z. Y. L. S. W. L. W. W. C. Edward J. Hu, "LoRA: Low-Rank Adaptation of Large Language Models," *arXiv*, 2021.
43. J. W. X. J. D. A. C. L. W. P. M. C. Z. S. A. K. S. A. R. J. S. J. H. F. K. L. M. M. S. A. A. P. W. P. C. Long Ouyang, "Training language models to follow instructions with human feedback," *arXiv*, 2022.
44. A. S. E. M. S. E. C. D. M. C. F. Rafael Rafailov, "Direct Preference Optimization: Your Language Model is Secretly a Reward Model," *arXiv*, 2023.
45. J. S. G. S. S. D. G. A. R. N. Lovedeep Gondara, "Small or Large? Zero-Shot or Finetuned? Guiding Language Model Choice for Specialized Applications in Healthcare," *arXiv*, 2025.
46. M. M. Martin Juan José Bucher, "Fine-Tuned 'Small' LLMs (Still) Significantly Outperform Zero-Shot Generative AI Models in Text Classification," *arXiv*, 2024.
47. J. A. H. A. A. A. A. N. B. A. B. A. B. J. B. H. B. A. B. M. B. J. B. S. B. M. C. Q. C. V. C. Marah Abdin, "Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone," *arXiv*, 2024.
48. B. H. Sandali Goonatilke, "Past, Present and Future Trends in Multi-Agent System Technology," Research Gate, [Online]. Available: <https://www.researchgate.net/publication/367228899>
49. "Rational agent," Wikipedia, [Online]. Available: https://en.wikipedia.org/wiki/Rational_agent
50. D. K. S. A. T. S. D. Y. Prateek Chhikara, "Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory," *arXiv*, 2025.
51. S. Farber, "Model Context Protocol (MCP): A Security Overview," Paloalto Networks, 2025. [Online]. Available: <https://www.paloaltonetworks.com/blog/cloud-security/model-context-protocol-mcp-a-security-overview/>
52. S. B. G. W. Dolev Moshe Attiya, "Cato CTRL™ Threat Research: Exploiting Model Context Protocol (MCP) – Demonstrating Risks and Mitigating GenAI Threats," Cato

- Networks, 2025. [Online]. Available: <https://www.catonetworks.com/blog/cato-ctrl-exploiting-model-context-protocol-mcp>
53. H. L. E. F. G. K. R. B. A. A. E. H. Mohammed Mehedi Hasan, "Model Context Protocol (MCP) at First Glance: Studying the Security and Maintainability of MCP Servers," *arXiv*, 2025.
 54. R. Z. Y. M. B. Z. S. L. Y. L. P. K. Wenlong Liang, "Large Model Empowered Embodied AI: A Survey on Decision-Making and Embodied Learning," *arXiv*, 2025.
 55. J. Z. D. Y. N. D. I. S. K. N. Y. C. Shunyu Yao, "ReAct: Synergizing Reasoning and Acting in Language Models," *arXiv*, 2022.
 56. L. D. A. T. Mihai Nadas, "Synthetic Data Generation Using Large Language Models: Advances in Text and Code," *arXiv*, 2025.
 57. X. G. T. S. Ryan Law, "74% of New Webpages Include AI Content (Study of 900k Pages)," Ahrefs, 2025. [Online]. Available: <https://ahrefs.com/blog/what-percentage-of-new-content-is-ai-generated>
 58. "The State of AI In Marketing 2025," Coschedule, 2025. [Online]. Available: <https://coschedule.com/ai-marketing-statistics>
 59. A. Borji, "A Note on Shumailov et al. (2024): 'AI Models Collapse When Trained on Recursively Generated Data'," *arXiv*, 2024.
 60. Q. D. J. C. H. S. Y. Z. Q. A. Z. Y. Y. L. Haitao Li, "LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Methods," *arXiv*, 2024.
 61. R. R. R. D. Nitay Calderon, "The Alternative Annotator Test for LLM-as-a-Judge," *arXiv*, 2025.
 62. "Housing Prices Dataset," Kaggle, [Online]. Available: <https://www.kaggle.com/datasets/yasserh/housing-prices-dataset>
 63. N. Sharma, "DeepSeek V3 vs GPT-4o: Which is Better?," 2025. [Online]. Available: https://www.analyticsvidhya.com/blog/2024/12/gpt-4o-vs-deepseek-v3/?utm_source=chatgpt.com
 64. C. L. X. Z. S. J. J. S. G. L. D. S. X. E. W. Kaiwen Zhou, "The Hidden Risks of Large Reasoning Models: A Safety Assessment of R1," *arXiv*, 2025.
 65. M. Sellman, "DeepSeek fails truth test by repeating Beijing talking points," The Times, [Online].
 66. "Why do Large Language Models Hallucinate?," Medium, Jan 2025. [Online]. Available: <https://medium.com/%40tahirbalarabe2/large-language-model-hallucinations-14aad4ccc78e>

67. "Understanding LLM hallucinations: causes, examples, and strategies for reliable AI-generated content," Cybernews, March 2025. [Online]. Available: <https://cybernews.com/ai-news/llm-hallucinations-causes-examples-strategies/>
68. I. L. T. S. S. L. D. R. R. Alexander R. Fabbri, "Multi-News: a Large-Scale Multi-Document Summarization Dataset," *arXiv*, 2019.
69. H. Z. D. Z. Y. L. S. H. Jiahao Zhang, "End-to-End Beam Retrieval for Multi-Hop Question Answering," *arXiv*, 2024.
70. "Deterministic Results Impossible for GPT-4o," OpenAI, [Online]. Available: <https://community.openai.com/t/deterministic-results-impossible-for-gpt-4o/1059584>
71. "Why Open AI's API models can't be forced to behave deterministically," ML6, [Online]. Available: <https://www.ml6.eu/blogpost/openais-api-models-not-deterministic>
72. G. Z. Q. D. D. †. Y. W. C. Tianle Li, "LongICLBench: Long-context LLMs Struggle with Long In-context Learning," *arXiv*, 2024.
73. S. S. S. K. S. A. D. R. F. J. Y. Z. B. G. Cheng-Ping Hsieh, "RULER: What's the Real Context Size of Your Long-Context Language Models?," *arXiv*, 2024.
74. M. D. C. K. S. L. M. T. R. M. W. Harsha Nori, "Sequential Diagnosis with Language Models," *arXiv*, 2025.
75. J. Z. M. L. L. S. G. Y. L. J. X. L. K. Chenxin An, "WHY DOES THE EFFECTIVE CONTEXT LENGTH OF," *arXiv*, 2024.
76. F. W. J. S. S. K. S. JUYONG JIANG, "ASurvey on Large Language Models for Code Generation," *arXiv*, 2024.
77. H. B. R. R. C. S. A. J. Rafael-Michael Karampatsis, "Big Code != Big Vocabulary: Open-Vocabulary Models for Source Code," *arXiv*, 2020.
78. F. W. J. S. S. K. S. K. Juyong Jiang, "A Survey on Large Language Models for Code Generation," *arXiv*, 2024.
79. "Context Degradation Syndrome: When Large Language Models Lose the Plot," James Howard, 2024. [Online]. Available: <https://jameshoward.us/2024/11/26/context-degradation-syndrome-when-large-language-models-lose-the-plot>.
80. J. P. S. H. M. Z. a. M. C. Quinn Leng, "Long Context RAG Performance of LLMs," Databricks, 2024. [Online]. Available: <https://www.databricks.com/blog/long-context-rag-performance-llms>.
81. A. B. P. A. I. R. D. S. A. S. M. B. Yuri Kuratov, "BABILong," *arXiv*, 2024.
82. H. R. D. C. R. L. B. Y. C. Maarten Sap, "SOCIAL IQA," *arXiv*, 2019.

83. J. W. X. J. D. A. C. L. W. P. M. C. Z. S. A. K. S. A. R. J. S. J. H. F. K. L. M. M. S. A. A. P. W. P. C. Long Ouyang, "Training language models to follow instructions with human feedback," *arXiv*, 2022.
84. S. Raschka, "Understanding Encoder And Decoder LLMs," 2023. [Online]. Available: <https://magazine.sebastianraschka.com/p/understanding-encoder-and-decoder>
85. K. E. P. S.-K. Dan Saattrup Nielsen, "Encoder vs Decoder," *arXiv*, 2025.
86. K. Naminas, "LLM Model Size: Parameters, Training, and Compute Needs," 2024. [Online]. Available: <https://labelyourdata.com/articles/llm-fine-tuning/llm-model-size>
87. H. H. Abulhair Saparov, "Language Models Are Greedy Reasoners," *arXiv*, 2023.



packtpub.com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

Why subscribe?

- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

At www.packtpub.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:

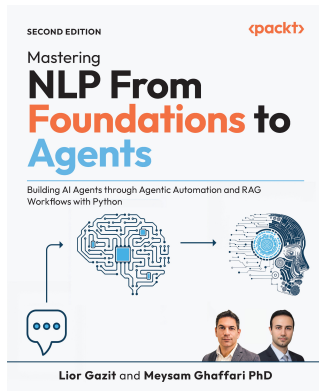


30 Agents Every AI Engineer Must Build

Imran Ahmad

ISBN: 9781806109012

- Deploy production-ready agent systems that scale securely and reliably
- Use LangChain and LangGraph to build autonomous agents with modular architectures
- Implement agents with sophisticated memory, planning, and reasoning capabilities
- Seamlessly integrate tools, APIs, and external data into agent workflows
- Establish robust evaluation frameworks to measure and optimize agent performance
- Implement guardrails and explainability features to ensure ethical and safe deployment
- Build multi-agent systems for complex, collaborative task orchestration
- Apply specific agent architectures across healthcare, finance, and legal domains



Mastering NLP From Foundations to Agents

Lior Gazit, Meysam Ghaffari

ISBN: 9781806106134

- Build strong NLP foundations in math and ML
- Engineer text classification and NLP pipelines
- Train and fine-tune modern LLM architectures
- Implement RAG systems with LangChain
- Orchestrate multiple AI agents and tools to solve complex tasks
- Evaluate NLP model performance and apply AI safety best practices
- Integrate external data and tools using Model Context Protocol (MCP)
- Fine-tune transformers with LoRA, QLoRA, and DPO techniques

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit authors.packt.com and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Share your thoughts

Once you've read *Building Agent-Powered Applications*, we'd love to hear your thoughts! Scan the QR code below to go straight to the Amazon review page for this book and share your feedback.



<https://packt.link/r/1807605167>

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Index

Symbols

32-bit floating-point number (FP32) 37 – 39

A

AI model

architecture 242
parameters 242

Amazon Web Services (AWS) 258

Annoy 230

AutoGen 313, 351 – 353
capabilities 354

Autoregressive Integrated Moving Average (ARIMA) 51

abstractive summarization 135, 136
challenges 136

accuracy metric 385

activation function 21

adapter layers 250

adaptive learning 99

adversarial prompts 416

agent card 356

agent communication patterns

competitive agent model 318
hierarchical task delegation model 318
roundtable discussion 318
sequential chat 317
worker and supervisor 317

agent design 324, 325

capabilities 328
control boundaries 330
control mechanisms 329, 330
deployment 331
evaluation metric 330
functional requirements 325
implementation approach 327, 328
interaction mode 327
learning 329

memory 329
operations 331
reasoning 329, 330
safety 330
skills 328
state 329
success metric 330
system requirements 326
tools 328

agent evaluation 403

background agent evaluation 411
conversational agent evaluation 406
RAG-based agent evaluation 403

agent memory 286

long-term memory (LTM) 287, 295
short-term memory (STM) 287 – 289
tools and frameworks 333 – 335
versus RAG 287

agent memory implementation 331

custom memory development 331, 332

agent orchestration 299 – 301

as sequence of multi-label 302, 303
classification
challenges 308 – 311
dynamic orchestration plans 305 – 307
multi-plan orchestration 304, 305
plan 303, 304

agent planning strategies 346

hierarchical task decomposition 347, 348
sequential decision-making 347
skill chaining 348

agent platforms

memory implementation 335

agent reasoning techniques 346

agent-based learning 278

agent-to-agent (A2A) protocol 355 – 359

capabilities 356
guiding principles 355

agents 278 – 280

business-level agents 280
control mechanisms 319

error handling	349	Chat Completions API	93
errors	349	ChatGPT	286
goals	319	Claude	87
low-level (code-level) agents	280	Copilot	280
analogical reasoning	194	catastrophic forgetting	246
annotation	16	causal language modeling (CLM)	64
approximate nearest neighbor (ANN)	222	causal reasoning	193
benefits	222	chain	126
graph-based methods	223 – 225	chain-of-thought (CoT)	85, 110, 183, 346
libraries	229	classical NLP pipelines	154
quantization-based methods	225	classification	23 – 25, 144
tree-based methods	223	binary classification	145
arithmetic reasoning	191	multi-class classification	145
assistant	279	multi-labeled classification	145
auto-regressive generation	55	classification tasks	296
auto-regressive models	55	client agent	356
autonomous inspection robot	280	closed-domain QA	180 – 182
B			
BERT-base (110M parameters) fine-tuning	250	closed-ended problem	383
BLIP (Bootstrapping Language-Image Pretraining)	139	closed-ended task evaluation	383, 384
Bilingual Evaluation Understudy (BLEU)	388	accuracy and exact match	384, 385
BioBERT	247	multi-class classification metrics	387
background agents	284	clustering	16, 25
evaluation	411	code generation agent	319
best matching (BM) algorithms	213	code-level multi-agent flows	312 – 316
best matching 25 (BM25)	213	code-switching	162
document length normalization	213	coefficient	7
implementing	215	commonsense reasoning	190, 191
term frequency saturation	213	constitutional AI	87
bias	7	content generation agent	319
binary classification	24, 145	context vector	52
branching chaining	126	context window	76 – 78
business-level agents	280	contextual embeddings	41
autonomous inspection robot	280	contrastive prompting	115
personal travel planning agent	280	conversational AI agent	319
sales leads generation agent	280	conversational QA	183
byte pair encoding (BPE)	29	conversational agent evaluation	406
C			
CLIP (Contrastive Language-Image Pretraining)	139	agent memory evaluation	406 – 408
		tool orchestration evaluation	409, 410
		conversational agents	283
		core machine learning problems	23
		classification	23
		clustering	25
		ranking	26
		recommendation	26

regression	23	system requirements	360
corpus	40	embedded agents	278
cosine similarity	43 – 45	embeddings	34 – 38
cue words	134	generating	40, 41
custom memory development	331, 332	encoder	52 – 55, 59
custom tools orchestration	337	entities	153
customer support agent	285, 319	epochs	253
cutting memory	289, 290	evaluation challenges	370
		strategy	371
D		evaluation criteria	393
DiskANN	224	evaluation datasets	376, 377
data extraction agent	319	public datasets	378
dataset	12	synthetic and human-generated datasets	381 – 383
decision trees	24	evaluation dimensions	393
decoder	55 – 59	evaluation metrics	383
dense neural network	20	closed-ended task evaluation	383
dialog summarization	133	human-as-a-judge	392 – 397
direct preference optimization (DPO)	252, 256, 257, 265	LLM, as judge	398, 399
distribution shift	402	online, versus offline evaluations	400 – 402
domain adaptation datasets	270	similarity metrics, for open-ended tasks	387
duplicates	380	evaluation strategy	371 – 375
dynamic few-shot prompting	103	multi-task AI agents' evaluation	375, 376
dynamic multi-prompt flows	119 – 121	evaluation tools	417 – 419
dynamic orchestration plans	305 – 307	event extraction	153, 158, 159
E		exact NNS	221, 222
Euclidean distance	45	min heap	221
email filtering agent		quickselect	221
control boundaries	362	exact match (EM) metric	384
control mechanisms	362	external tools	298
deployment	363	extractive summarization	133, 134
design	364, 365	limitations	135
evaluation metric	363	F	
functional requirements	359, 360	F1 score	386
implementation	365, 366	F1@k	404
interaction mode	360, 361	Facebook AI Research (FAIR)	162
learning	361	Facebook AI similarity search (FAISS)	229
memory	361	face hallucination	82
operations	363	faithfulness	405
reasoning	362	false negatives (FN)	386
requirements	359	false positive (FP)	386
safety	362	fastText	162
state	361		
success metric	363		

-
- feedforward neural networks** 20
 - few-shot prompting** 97, 98
 - embedding 103 – 106
 - example selection 100
 - example selection classifier 107, 108
 - similarity 103 – 106
 - financial summarization** 133
 - fine-tuned models**
 - hosting 271 – 273
 - fine-tuning** 22, 242 – 244, 266, 267
 - approaches 258
 - as managed feature 259, 260
 - datasets 269, 270
 - effect, on model 244 – 246
 - full fine-tuning 246 – 248
 - in cloud 258
 - model family, selecting 261, 262
 - model size, selecting 262 – 264
 - on-premises hardware 258
 - partial fine-tuning 248 – 250
 - preference-based fine-tuning 252 – 254
 - strategy, selecting 264, 265
 - supervised fine-tuning (SFT) 252, 253
 - forward() method** 55
 - foundation models** 6, 139
 - frameworks** 418
 - full fine-tuning** 246 – 248
 - fully connected neural network** 20
 - function calling** 298
 - use cases 299
 - function calling frameworks** 338, 339
 - LangChain 341, 342
 - OpenAI 339 – 341
- G**
- Gaussian error linear unit (GeLU)** 20
 - Gemini** 86
 - Gemini Chat** 286
 - Global Vectors (GloVe)** 40
 - Google Cloud Platform (GCP)** 273
 - Google Dataset Search** 12
 - gated recurrent unit (GRU)** 55
 - generalization** 13
 - generative pretrained transformer (GPT) models** 83 – 85
 - generator** 205, 206
 - geo-political entities (GPE)** 153
 - golden answers** 388
 - graph-based algorithms** 223
 - DiskANN 224
 - HNSW 224
 - K-nearest neighbor graph (KGraph) 224
 - NSW 224
 - greedy decoding** 78 – 80
 - ground truth** 375
 - groundedness** 405
 - grounding** 181
- H**
- HITL agents** 352
 - HNSWlib** 229
 - Hidden Markov Models (HMMs)** 51
 - Hugging Face** 12
 - hallucinations** 81, 82
 - factors 82, 83
 - hidden layers** 20
 - hierarchical navigable small world (HNSW)** 224
 - hierarchical summarization** 143
 - hierarchical task decomposition** 347, 348
 - human-in-the-loop (HITL) decision-making** 350
 - human-scored metric** 394
 - hybrid memory pipeline** 291, 292
 - hybrid summarization** 136
- I**
- imbalanced dataset** 385
 - in-context learning** 97 – 99
 - incremental summary aggregation** 144
 - inference** 15
 - information extraction** 27, 152, 153
 - event extraction 153, 158
 - named entity recognition (NER) 153, 154
 - relation extraction 153, 157
 - information extraction tasks** 296
 - input-output analysis** 371
 - instruction-following datasets** 270
 - intent detection** 150 – 152

inverse document frequency (IDF)	134	large document summarization	141
inverted index	215	large language models (LLMs)	63, 64
iterative self-refinement prompting	124, 125	benchmarking	70 – 73
		Claude	87
		context window	76 – 78
		families	83
		for NER	156, 157
		Gemini	86
		GPT models	83 – 85
		hallucinations	81, 82
		LLaMA	86
		model selection	87 – 89
		sampling and randomness	78 – 81
		training datasets	65 – 70
		training objectives	64, 65
		using	76
J		layers	20
jailbreak	416	lead generation agent	285
		learning from data	12
K		legal document analysis agent	319
KGraph	224	lemma	31
Kaggle	12	lemmatization	31
knowledge database	13	lexical retrieval	207
knowledge graphs (KGs)	157	BM25	213 – 215
		email chatbot	207, 208
		implementing	208, 209
		inverted indexes	215, 216
		TF-IDF	209 – 212
L		lexical similarity-based metrics	388
L1 distance	46	linear regression	9, 23
L2 distance	45	linear relationship	18
LLM agents	282 – 285	listwise models	26
customer support agent	285	logical reasoning	191, 192
external tools	282	logistic regression	24
lead generation agent	285	logits	57
LLM	282	long conversation memory (LoCoMo)	333
memory	282	long-term memory (LTM)	287, 295, 332
personal email filtering agent	285	previous conversations	297, 298
LLM reasoning	190	profile data	296, 297
analogical reasoning	194	user preferences	295, 296
arithmetic reasoning	191, 192	lossy compression	132
causal reasoning	193	low-level (code-level) agents	280
commonsense reasoning	190, 191	low-rank adaptation (LoRA)	244, 265
logical reasoning	191, 192		
multi-hop reasoning	195		
symbolic reasoning	191, 192		
temporal reasoning	193		
LLM-powered agents	351		
LLMs as a judge	398		
LLaMA 2	87		
LLaMA 2 (7B parameters) fine-tuning	250		
LangChain	127, 341, 342		
Large language model Meta AI (LLaMA)	86		
Likert scale	394		
labeling	16		
labels	16		
langdetect	162		
language detection	161, 162		

M

MCP (Model Context Protocol)	329, 343
client-server architecture	343, 344
technical challenges	345
MCP client	343
MCP server	343
Manhattan distance	46
Manhattan similarity	46
MapReduce	141, 142
Mem0	333
features	333
MetaGPT	313
Metric for Evaluation of Translation with Explicit Ordering (METEOR)	388
Microsoft 365 Copilot	286
Microsoft Agent Framework	351
machine learning (ML)	
basics	4 – 10
machine learning models	
architectures	10, 11
input-output flow	5
training	11 – 15
machine translation (MT)	163 – 165
macro-averaging	387
masked language modeling (MLM)	64, 65
massive multitask language understanding (MMLU)	72
math solver agent	319
memory-augmented learning (MAL)	335
examples	336
limitations	336
meta prompting	115, 178
micro-averaging	387
mixture-of-experts (MoE)	81, 85
model features	8
model parameters	8
model-agnostic meta-learning (MAML)	98
multi-agent chaining	126
multi-agent flows	311
code-level multi-agent flows	312 – 316
communication patterns	316, 317
multi-agent systems (MAS)	278
multi-class classification	24, 145

multi-hop QA	182
multi-hop reasoning	195
multi-labeled classification	24, 145
multi-level summarization	142, 143
multi-plan orchestration	304, 305
multi-prompt chain-of-thought (CoT)	121 – 124
multi-prompt engineering techniques	116 – 118
dynamic multi-prompt flows	119 – 121
iterative self-refinement	124, 125
multi-prompt chain-of-thought (CoT)	121 – 124
prompt chaining	126, 127
static multi-prompt flows	119
multilanguage support	159 – 161
language detection	161, 162
machine translation (MT)	163 – 165
multimodal models	6, 74, 75, 139, 140
multimodal summarization	138, 139
text-first multimodal summarization	139

N

named entity recognition (NER)	27, 65, 153, 247
challenges	154
classical NLP pipelines	154
large language model	156, 157
transformer-based fine-tuned models	155
natural language to SQL (NL2SQL)	176
natural language to code (NL2Code) problem	176
navigable small world (NSW)	224
nearest neighbor search (NNS)	221
approximate NNS (ANN)	221
exact NNS	221
neural machine translation (NMT)	163
neural networks (NNs)	18 – 21, 24
neurons	18 – 20
news summarization	133
nucleus sampling	81

O

OpenAI	339 – 341
offline evaluation	374, 400

clinical trial phases	400	contrastive	115
preclinical	400	meta	115
offline evaluations	400	persona-based	114
one-hot encoding	40	reflection	115
online evaluation	375, 400	self-ask	115
open relation extraction	158	self-consistency	112, 113
open-domain QA	182	template-based	115
open-ended problem	383	Tree-of-Thought (ToT)	111, 112
optical character recognition (OCR)	139	prompts	299
tools		public datasets	378
orchestration plan	303, 304	custom evaluation datasets,	378 – 381
out-of-vocabulary (OOV) words	29	building	
outlier	46		
overfitting	13, 71		
P		Q	
Precision@k	404	quantization-based methods	225
PyTorch	13	inverted file index (IVF)	225
pairwise models	26	product quantization (PQ)	225
parameter	7	query generation phase	188
parameter-efficient fine-tuning (PEFT)	250, 251, 265	question answering (QA)	27, 179, 283
parser	204	closed-domain QA	180 – 182
partial fine-tuning	248 – 250	conversational QA	183
pathways language model (PaLM)	86	for structured data	184 – 189
persona-based prompting	114	multi-hop QA	182
personal email filtering agent	285	open-domain QA	182
personal travel planning agent	280		
pointwise models	26		
policy	17, 278		
post-market surveillance phase	400		
precision	385		
preference-based fine-tuning	252 – 254	R	
pretrained models	67	RAG components	202, 203
pretraining stage	67	approximate nearest neighbor (ANN)	222
profile data	296	generator	205, 206
prompt	92	lexical retrieval	207, 208
system prompt	92, 93	parser	204
user prompt	92, 93	question answering (QA) component	207
prompt chaining	126, 127	semantic retrieval	216, 217
prompt engineering	265, 266	static retriever	203, 204
prompt techniques	109	RAG implementations	225
chain-of-thought (CoT)	110, 111	custom pipeline	226
		hybrid approach	232, 233
		lexical retriever	226, 227
		RAG-as-a-Service	238
		semantic retriever	228 – 231
		vector databases	233 – 237
		RAG-based agent evaluation	403
		generator evaluation	405
		retriever evaluation	404, 405
		Recall-Oriented Understudy for Gisting Evaluation (ROUGE)	388
		Recall@k	404

random forests	24	Support Vector Machines (SVM)	149
ranking	26	sales leads generation agent	280
rational agents	278	sampling	78, 79
reason-act-observe loop	306	scalable nearest neighbor (ScaNN)	230
reasoning	27, 189	scientific research assistant	319
LLM reasoning	190	self-ask prompting	115
reasoning models	85, 195 – 197	self-attention mechanism	59, 60
recall	386	self-consistency prompting	112, 113
recommendation	26	semantic compression	132
rectified linear unit (ReLU)	19	semantic retrieval	216
recurrent neural network (RNN)	53	document length, selecting	218, 219
recursive chaining	126	document preprocessing	217
reference output	377, 388	relevant documents, identifying	221, 222
reference-free evaluation	405	sentence-level embedding models	220
reflection	115	similarity metric, selecting	220
regression	9, 23	working	216
reinforcement learning (RL)	16, 17, 195, 254, 265, 278	semantic similarity-based metrics	390 – 392
reinforcement learning from human feedback (RLHF)	64, 69, 168, 252 – 256, 412	sentence position	134
relation extraction	153, 157	sentence-level embeddings	41, 42
open relation extraction	158	sentence-level tokenization	30, 31
supervised relation extraction	158	sentiment analysis	147 – 150
remote agent	356	sequence-to-sequence (Seq2Seq) models	50, 51
responsibility metrics	411	decoder	55 – 59
responsible AI	411, 412	encoder	52 – 55
adversarial prompt evaluation	416	limitations	59
evaluation datasets	414, 415	sequential chaining	126
jailbreak	416	sequential decision-making	347
risks and evaluation criteria	413	server-sent events (SSE)	355
retraining	242	short-term memory (STM)	287, 288, 331
retrieval-augmented generation (RAG)	182, 201, 265 – 268, 369, 403	advanced hybrid approach	291, 292
generator	403	context relevance	293, 294
retriever	403	cutting memory	289, 290
versus memory	287	LLM context window	289
reward model	254	summarizing memory	290, 291
S		sigmoid	19
Scikit-learn (sklearn) library	211	similarity metrics	43
Sentence-BERT (SBERT)	42	similarity metrics, for open-ended tasks	387
		lexical similarity-based metrics	388 – 390
		semantic similarity-based metrics	390 – 392
		simple linear regression	22
		skill chaining	348
		skills	281

small language models (SLMs)	74, 248	template-based prompting	115
smaller language models (SLMs)	326	temporal reasoning	193
speech-to-speech (S2S) models	286	tensor	15
static embeddings	40, 41	term frequency (TF)	134
static few-shot prompting	102	term frequency - inverse document frequency (TF-IDF)	209
static multi-prompt flows	119	implementing	211, 212
static retriever	203, 204	inverse document frequency (IDF)	210
stemming	31	term frequency (TF)	209
stop words	32	text classification	26, 144 – 146
summarization	27, 132	intent detection	150 – 152
abstractive summarization	135, 136	sentiment analysis	147 – 150
dialog summarization	133	text generation	27, 168 – 171
extractive summarization	133 – 135	text generation parameters	171
financial summarization	133	code generation	174 – 178
hybrid summarization	136	few-shot prompting	172
multimodal summarization	138, 139	instruction	171
news summarization	133	model selection	174
summarization parameters	137	output length	173
maximum tokens	137	prompt	172
model selection	138	randomness	173, 174
randomness settings	137	synthetic data generation	178
refocus	137	text-first multimodal summarization	139
summary length	137	text-to-text transfer transformer (T5)	86
summarizing memory	290, 291	token IDs	28
supervised dataset	377	token-level embeddings	41
supervised fine-tuning (SFT)	252, 253	tokenization	28 – 30
supervised learning	15	tokenizer	28
supervised relation extraction	158	tokenizer libraries	290
symbolic reasoning	191, 192	tokens	28
system instruction	92	tool agents	352
system message	92	tools orchestration	
system prompt	92, 93	in agentic platform	342
structure	95, 96	top-p sampling	81
system role technique	114	topic-aware memory (TAM)	293, 332
T		training	11
TensorFlow	13	training objectives	64, 65
Text Generation Inference (TGI)	273	transformer-based fine-tuned models	155
Tree-of-Thought (ToT)	111, 112	transformers	59 – 63
prompting		travel operations agent	281
table selection phase	188	travel planner agent	281
tabular data		travel supervisor agent	281
summarizing	140, 141	tree-based algorithms	223
task-specific datasets	270	true positive (TP)	386

U

unidirectional context	65
unknown reasoning	197
unsupervised dataset	377
unsupervised learning	16
user instruction	92
user message	92
user prompt	92, 93
structure	97

V

vector	34
---------------	-----------

vector databases	234 – 237
-------------------------	------------------

W

Word to Vector (Word2Vec)	40
weights	8
word-based approximation	290
word-level embeddings	41, 42

Z

zero-shot learning	97, 98
zero-shot prompting	97, 101, 102

